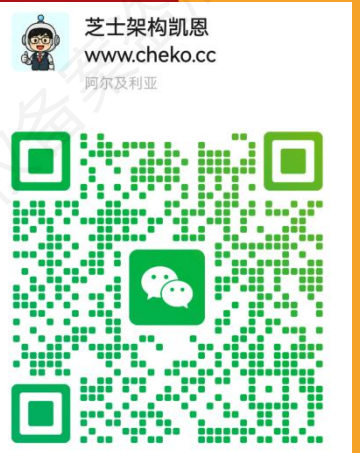


芝士架构案例冲刺宝典

2026 年 10 月-芝士架构凯恩编辑整理 v6.0.1



目录

芝士架构案例冲刺宝典	1
前言	15
考试规则	15
命题趋势	16
刷题形式	17
案例题型	19
关注范围	20
1. 上篇-需求分析（次重点★★★☆☆）	21
1.1. 结构化分析（重点★★★★★）	21
1.1.1. 数据流图 DFD（重点★★★★★）	21
1.1.2. 状态转换图（次重点★★★☆☆）	24
1.2. 面向对象分析方法（超级重点★★★★★）	27
1.2.1. UML 引入（重点★★★★★）	27
1.2.2. 用例图（超级重点★★★★★）	28
1.2.3. 类图（重点★★★★★）	34
1.2.4. 顺序图（重点★★★★★）	36
1.2.5. 通信图/协作图（重点★★★★★）	40
1.2.6. 活动图（重点★★★★★）	41
1.2.7. 常见图对比（重点★★★★★）	48
2. 上篇-系统设计/面向对象设计（重点★★★★★）	49
2.1. 类识别（次重点★★★☆☆）	49
2.1.1. 识别原则（次重点★★★☆☆）	49

2.1.2. 典型例题（重点★★★★★）	50
2.2. 面向对象设计原则（重点★★★★★）	51
2.2.1. 设计原则（重点★★★★★）	52
2.2.2. 典型例题（重点★★★★★）	52
3. 上篇-层次结构（次重点★★★☆☆）	53
3.1. 层次结构简介和问题（重点★★★★★）	53
3.1.1. 层次架构简介（重点★★★★★）	53
3.1.2. 污水池反模式（重点★★★★★）	53
3.2. 典型的层次架构（重点★★★★★）	53
3.2.1. 表现层层次架构（重点★★★★★）	54
3.2.2. 中间层架构设计（次重点★★★☆☆）	55
3.2.3. 数据访问层设计（重点★★★★★）	56
3.3. 物联网层次架构设计（次重点★★★☆☆）	60
3.3.1. 物联网设备接入协议:MQTT、CoAP、HTTP(重点★★★★★☆☆)	60
3.3.2. MQTT 协议（重点★★★★★）	62
3.3.3. 典型例题（重点★★★★★）	65
4. 上篇-云原生架构设计（超级重点★★★★★）	68
4.1. 云原生架构原则（重点★★★★★）	68
4.2. 云原生架构模式（重点★★★★★）	68
4.3. 不好的实践（次重点★★★☆☆）	70
4.4. 容器技术（重点★★★★★）	71
4.4.1. 容器和虚拟机对比（重点★★★★★）	71

4.4.2. 容器和容器编排技术（重点★★★★★）	72
4.4.3. 容器运维指令（重点★★★★★）	73
4.4.4. 典型例题（重点★★★★★）	74
5. 上篇-面向服务架构（超级重点★★★★★）	76
5.1. 什么是 SOA（重点★★★★★）	76
5.2. SOA/微服务/单体架构对比（重点★★★★★）	76
5.3. 服务注册模式（重点★★★★★）	78
5.3.1. 服务注册模式概念（重点★★★★★）	79
5.3.2. UDDI、WSDL 和 SOAP（重点★★★★★）	79
5.3.3. REST 规范/风格（重点★★★★★）	80
5.3.4. 典型例题（重点★★★★★）	81
5.4. ESB 模式（重点★★★★★）	82
5.4.1. ESB 概念（重点★★★★★）	83
5.4.2. 典型例题（重点★★★★★）	83
5.5. SOA 设计原则（重点★★★★★）	84
5.6. 典型架构（重点★★★★★）	84
5.6.1. 某 ESB 架构实现（重点★★★★★）	85
5.6.2. 微服务模式（重点★★★★★）	85
6. 上篇-系统架构设计（超级重点★★★★★）	88
6.1. 软件架构风格（重点★★★★★）	88
6.1.1. 概念辨析（重点★★★★★）	88
6.1.2. 数据流风格（重点★★★★★）	89

6.1.3. 调用/返回体系结构风格（重点★★★★★）	91
6.1.4. 以数据为中心的体系结构风格（重点★★★★★）	94
6.1.5. 虚拟机体系结构风格（重点★★★★★）	96
6.1.6. 独立构件体系结构风格（重点★★★★★）	98
6.1.7. 典型例题（重点★★★★★）	99
6.2. 面向架构评估的质量属性（超级重点★★★★★）	100
6.2.1. 九大必考质量属性（超级重点★★★★★）	101
6.2.2. 典型例题（超级重点★★★★★）	103
6.3. 质量属性场景描述（超级重点★★★★★）	105
6.3.1. 质量属性场景描述的语言（重点★★★★★）	105
6.3.2. 质量属性场景描述案例（超级重点★★★★★）	107
6.3.3. 典型例题	111
6.4. 系统架构评估（重点★★★★★）	111
6.4.1. 系统架构评估方法（重点★★★★★）	113
6.4.2. 基于场景的架构分析方法 SAAM（次重点★★★☆☆）	113
6.4.3. 架构权衡分析方法 ATAM（重点★★★★★）	113
6.5. ATAM 方法的架构评估实践（重点★★★★★）	116
7. 上篇-安全架构设计理论与实践（超级重点★★★★★）	118
7.1. 安全架构特性（次重点★★★☆☆）	118
7.2. 数据加密技术（重点★★★★★）	118
7.2.1. 对称加密概念（重点★★★★★）	118
7.2.2. 非对称加密概念（重点★★★★★）	119

7.2.3. 非对称加密的应用场景（重点★★★★★）	119
7.2.4. 对称加密和非对称加密的区别（重点★★★★★）	120
7.3. 认证技术（重点★★★★★）	120
7.3.1. 数字签名（重点★★★★★）	121
7.3.2. 数字证书（重点★★★★★）	122
7.3.3. 非对称公钥的使用场景（重点★★★★★）	123
7.3.4. 典型例题（重点★★★★★）	123
7.4. 网络安全体系架构设计（次重点★★★☆☆）	125
7.4.1. 安全服务和安全机制（次重点★★★☆☆）	125
7.4.2. 典型软件架构的脆弱性分析（次重点★★★☆☆）	125
8. 上篇-大数据架构（次重点★★★☆☆）	127
8.1. 系统架构原则（次重点★★★☆☆）	127
8.2. 批处理架构 VS 流处理架构（重点★★★★★）	128
8.3. Lambda 架构（重点★★★★★）	129
8.3.1. Lambda 分层介绍（重点★★★★★）	129
8.3.2. Lambda 架构实现（重点★★★★★）	131
8.3.3. Lambda 架构优缺点（重点★★★★★）	132
8.4. Kappa 架构（重点★★★★★）	132
8.4.1. Kappa 架构的优缺点（重点★★★★★）	134
8.4.2. Lambda VS Kappa（超级重点★★★★★）	135
8.5. IOTA 架构（次重点★★★☆☆）	135
8.6. 典型架构（重点★★★★★）	137

8.6.1. 某网奥运（重点★★★★★）	138
8.6.2. 某网络广告平台（重点★★★★★）	140
8.6.3. 某证券公司日志大数据系统（重点★★★★★）	141
8.6.4. 某电商智能决策大数据系统（重点★★★★★）	142
8.7. 典型例题（重点★★★★★）	143
9. 下篇-微服务架构（次重点★★★☆☆）	146
9.1. 微服务架构下的特定问题（重点★★★★★）	147
9.2. 基于 Spring Cloud 的微服务整体架构	148
9.3. 服务拆分与集群部署（重点★★★★★）	149
9.4. Nacos 服务注册与服务发现（重点★★★★★）	150
9.5. OpenFeign：声明式远程调用（重点★★★★★）	151
9.6. Gateway：微服务系统的统一入口（重点★★★★★）	153
9.7. Nacos 配置中心（重点★★★★★☆☆）	154
9.8. Sentinel 服务保护与流量控制（重点★★★★★）	156
9.8.1. Sentinel 流量控制（重点★★★★★）	157
9.8.2. Sentinel 熔断降级（重点★★★★★）	158
10. 下篇-领域驱动设计（次重点★★★☆☆）	163
10.1. 大的边界（业务版图边界）（次重点★★★☆☆）	163
10.1.1. 领域与子域（次重点★★★☆☆）	163
10.1.2. 子域详解（重点★★★★★）	164
10.2. 小的边界（团队协作边界）（重点★★★★★）	164
10.2.1. 统一语言（重点★★★★★）	164

10.2.2. 限界上下文（重点★★★★★）	165
10.2.3. 上下文映射与边界协作关系（次重点★★★☆☆）	165
10.3. 代码边界（重点★★★★★）	166
10.3.1. 实体（Entity）（重点★★★★★）	166
10.3.2. 聚合与聚合根（重点★★★★★）	166
10.3.3. 领域服务（重点★★★★★）	167
10.3.4. 领域事件（重点★★★★★）	167
10.3.5. 仓储与工厂（重点★★★★★）	167
10.3.6. 应用编排（重点★★★★★）	168
10.4. 小结	168
11. 下篇-分布式系统（重点★★★★★）	169
11.1. CAP 理论（重点★★★★★）	169
11.1.1. 数据一致性（重点★★★★★）	170
11.1.2. 可用性（重点★★★★★）	170
11.1.3. 分区容错性（重点★★★★★）	171
11.2. CAP 理论与系统实现（重点★★★★★）	172
11.3. BASE 理论（重点★★★★★）	173
11.4. 分布式事务理论解决方案	174
11.4.1. 刚性事务 2PC（重点★★★★★）	174
11.4.2. 柔性事务 TCC（重点★★★★★）	177
11.4.3. 柔性事务本地消息表（重点★★★★★）	179
12. 下篇-数据库系统（重点★★★★★）	181

12.1. 数据库性能优化（重点★★★★★）	181
12.1.1. 索引建立注意事项（重点★★★★★）	181
12.1.2. 数据分片（重点★★★★★）	182
12.1.3. 数据分区（重点★★★★★）	183
12.1.4. 分库分表（重点★★★★★）	185
12.2. 数据库高可用模式（重点★★★★★）	186
12.2.1. 读写分离（重点★★★★★）	186
12.2.2. 主从复制（重点★★★★★）	187
12.2.3. 一致性问题（重点★★★★★）	189
12.3. 数据库的选型（重点★★★★★）	191
13. 下篇-Redis（重点★★★★★）	192
13.1. 数据类型（重点★★★★★）	192
13.2. 持久化机制（重点★★★★★）	193
13.3. Redis 集群（重点★★★★★）	194
13.3.1. 主从复制模式（重点★★★★★）	194
13.3.2. 主从复制流程（重点★★★★★）	195
13.3.3. 哨兵模式（重点★★★★★）	197
13.3.4. Cluster 模式（重点★★★★★）	199
13.3.5. 哨兵模式和 Cluster 模式区别（重点★★★★★）	201
13.4. Redis 常见生产问题（重点★★★★★）	202
13.5. 缓存和数据库一致性问题（重点★★★★★）	202
13.6. 过期数据删除策略（重点★★★★★）	204

13.7. 内存淘汰策略（重点★★★★★）	204
13.8. 热 Key 和大 Key 问题（重点★★★★★）	205
13.8.1. 大 Key（重点★★★★★）	205
13.8.2. 热 Key（重点★★★★★）	206
13.9. 分布式锁（重点★★★★★）	207
13.10. 布隆过滤器（重点★★★★★）	208
13.11. Redis 事务（超级重点★★★★★）	209
13.12. Redis 指令集（重点★★★★★）	210
14. 下篇-ElasticSearch（重点★★★★★）	212
14.1. ES 概述（重点★★★★★）	212
14.1.1. 分词原理（重点★★★★★）	212
14.1.2. 倒排索引（重点★★★★★）	214
14.1.3. 相关性算法（重点★★★★★）	215
14.2. ES 核心概念（重点★★★★★）	216
14.2.1. Index 索引库（次重点★★★☆☆）	216
14.2.2. Document 文档（次重点★★★☆☆）	218
14.2.3. Field 字段（次重点★★★☆☆）	218
14.2.4. Mapping 映射（次重点★★★☆☆）	219
14.2.5. DSL（重点★★★★★）	219
14.2.6. Analyzer 分析器（重点★★★★★）	220
14.2.7. Shard 分片（重点★★★★★）	224
14.3. ES 与传统关系型数据库的区别（重点★★★★★）	225

14.4. MySQL 与 ES 数据同步（重点★★★★★）	226
15. 下篇-MongoDB（重点★★★★★）	227
15.1. 数据类型（重点★★★★★）	227
15.1.1. BSON（重点★★★★★）	227
15.1.2. GeoJSON（重点★★★★★）	228
15.2. 分片集群（重点★★★★★）	228
15.3. 高可用（副本复制）（重点★★★★★）	229
16. 下篇-大模型应用（重点★★★★★）	231
16.1. 检索增强生成（RAG）（重点★★★★★）	231
16.1.1. 数据准备（重点★★★★★）	232
16.1.2. 分块策略（重点★★★★★）	233
16.1.3. 向量化 Embedding（重点★★★★★）	234
16.1.4. 索引存储（重点★★★★★）	236
16.1.5. 检索增强（重点★★★★★）	237
16.1.6. 生成答案（重点★★★★★）	239
16.2. 模型上下文协议协议（MCP）（重点★★★★★）	239
16.2.1. 解决什么问题？（重点★★★★★）	240
16.2.2. MCP 工具的组成（重点★★★★★）	241
16.3. 智能体（AGENT）（次重点★★★☆☆）	242
16.3.1. 智能体简介（次重点★★★☆☆）	242
16.3.2. 智能体架构（次重点★★★☆☆）	243
16.3.3. ReAct 模式（次重点★★★☆☆）	244

16.3.4. 规划-执行模式（次重点★★★★☆）	244
16.3.5. 客服智能体架构探索（次重点★★★★☆）	245
16.3.6. Skill（重点★★★★★）	246
17. 下篇-锁机制（重点★★★★★）	251
17.1. 为什么要使用锁（重点★★★★★）	251
17.2. 常见锁类型（重点★★★★★）	252
17.3. 乐观锁与悲观锁（重点★★★★★）	252
17.3.1. 悲观锁（重点★★★★★）	252
17.3.2. 乐观锁（重点★★★★★）	253
17.4. 实际场景中的锁方案（重点★★★★★）	254
17.4.1. 使用 <code>synchronized</code> 本地锁（重点★★★★☆）	254
17.4.2. 使用数据库行锁 <code>select for update</code> （重点★★★★★）	255
17.4.3. 使用 <code>Redis</code> 分布式锁（重点★★★★★）	256
17.4.4. 使用数据库乐观锁（重点★★★★★）	257
17.4.5. 几种方案对比（重点★★★★★）	257
18. 下篇-消息队列与异步架构（重点★★★★★）	258
18.1. 为什么需要消息队列（重点★★★★★）	258
18.1.1. 先看一个同步调用的问题	258
18.2. 消息队列的作用（重点★★★★★）	260
18.2.1. 异步解耦（重点★★★★★）	260
18.2.2. 削峰填谷（重点★★★★★）	260
18.2.3. 引入 <code>MQ</code> 的代价（重点★★★★★）	260

18.3. 消息队列核心概念（重点★★★★★）	261
18.3.1. 消息流转过程（重点★★★★★）	261
18.3.2. Producer、Consumer、Message、Broker（重点★★★★★）	262
18.3.3. Topic、Queue、Partition（次重点★★★★☆☆）	264
18.3.4. 点对点模式与发布订阅模式（重点★★★★★）	265
18.3.5. Consumer Group（重点★★★★★）	266
18.3.6. 消息确认机制 ACK、NACK 和 offset（重点★★★★★）	266
18.3.7. Push 和 Pull（重点★★★★☆）	267
18.3.8. JMS、AMQP 的区别	268
18.4. MQ 常见生产问题（重点★★★★★）	268
18.4.1. 消息丢失（重点★★★★★）	269
18.4.2. 消息重复消费（重点★★★★★）	270
18.4.3. 死信队列（DLQ）（重点★★★★★）	271
18.4.4. 顺序性（重点★★★★★）	273
18.4.5. 消息积压（重点★★★★★）	273
18.5. MQ 与分布式事务（重点★★★★★）	274
18.5.1. 本地消息表（重点★★★★★）	274
18.6. 常见 MQ 对比（次重点★★★★☆）	275
19. 下篇-秒杀场景设计（重点★★★★★）	277
19.1. 秒杀系统的核心痛点（重点★★★★★）	277
19.2. 秒杀系统的整体架构（重点★★★★★）	277

19.3. 访问层设计（重点★★★★★）	278
19.4. 中间转发层设计（重点★★★★☆）	279
19.5. 服务层设计（重点★★★★☆）	280
19.6. Redis 扣库存（重点★★★★★）	281
19.7. 消息队列层设计（重点★★★★★）	283
19.8. 数据层设计（重点★★★★★）	283
19.9. 异常补偿与最终一致性（次重点★★★★☆☆）	285
19.10. 用户体验设计（次重点★★★★☆）	286
20. 下篇-知识图谱（次重点★★★☆☆）	288
20.1. 知识图谱的由来（次重点★★★☆☆）	288
20.2. 知识图谱相比关系型数据库的优势（重点★★★★★）	289
20.3. 知识图谱的基本属性与 Neo4j（重点★★★★★）	292
20.3.1. 实体、关系、属性、标签和路径（重点★★★★★） ..	292
20.3.2. Neo4j（重点★★★★★）	293
20.4. 医药知识图谱问答实例（重点★★★★★）	294
20.4.1. 使用 BERT 识别实体（次重点★★★☆☆）	296
20.4.2. 使用 BERT 识别关系（次重点★★★☆☆）	298
20.4.3. 医药问答例子：查询疾病症状（重点★★★★★）	300
20.4.4. 医药问答例子：查询药物相互作用（重点★★★★★）	301
20.4.5. 现实应用差距与落地问题（重点★★★★★）	302
21. 后记	305

前言

各位同学好，众所周知，案例分析是架构三个科目中最难的最容易翻车的科目。之所以难，一是因为案例出题范围不固定，导致我们准备起来很难，很难精准打击。二是因为需要记记背背的东西不少，假如你只是看，不去记忆，考试的时候遇到题目写不出一二三条还是会翻车。所以，针对这两个问题，凯恩根据自己对软考高级的理解，结合官方教材、历年真题和权威教材编制了这本案例冲刺红宝书。这本红宝书的目标就是帮你缩小准备范围，帮你节约 30% 的时间，让你在做题目的时候能够有话可说。

4.x 版本相对于 3.x 版本在领域驱动设计，AI 大模型应用部分也做了很多内容的新增，整体内容丰富度提升 10%，建议打印出来好好背，考场上绝对有知识点在本书中出现。

版权提醒：本资料已通过国家版权局登记（登记号：渝作登字-2025-A-00624260），（一旦发现资料恶意流出，直接封号不退款，请珍惜账号权益）。

考试规则

从 2024 年开始，案例考试和综合知识被放在上午一起考。这两门考试加起来是 240 分钟，其中综合知识最短作答时间是 120 分钟，最长 150 分钟。按照以往经验，综合题时间比较充裕，大部分同学都可以结余出 30 分钟左右的时间。按照新规你可以提前交卷，多出来的这 30 分钟可以和案例原本的 90 分钟累加，最多有 120 分钟的时间来做案例。

因为案例相对选择时间紧题目难，90 分钟搞定会比较吃力，所以凯恩在这里诚恳地建议各位同学选择题至少提前 15 分钟交卷，结余 15 分钟放到案例分析中。

综合和案例分析联考之后时间安排（合计 240 分钟）			
综合	最短 120 最长 150 分钟（120 分钟后可以交卷）	满分 75 分	75 题
案例分析	最短 90 分钟最长 120 分钟（选择节约的时间可以放到案例分析）	满分 75 分	5 题第一题必做题，后面 4 选 2

从表格里我们可以看到，系统架构设计师的案例分析和高项完全不同，它是有选做题的。考试的时候，机考作答系统会给你 5 道题目。其中第一题是必做题，后面四题四选二。这样必做一题加上选做两题等于三道题，每题的分值是 25 分，就可以凑成 75 分的总分。

之所以有选做题，是因为系统架构设计师考试设计之初是要照顾不同行业背景的软件从业者，所以考试主题涵盖数据库、大数据、嵌入式等等，让考生根据自己的行业背景来做题。这个出发点当然是好的，但是目前来看，实际上考察的内容还是偏向 JavaWeb 居多。

命题趋势

案例虽然离谱，但还是有一定的规律。实际上结合近两年来的考试情况可以发现，无论架构还是系分，案例第一题考察的知识点必定是书本上的内容。第一题 25 分只要你用心背，至少可以拿 20 分。而其余 4 道题目，确实极难预测，但基本分布在需求分析、数据库、缓存等主题范围内。对于这些主题，红宝书会尽可能深入且全面地呈现相关内容，力求押中题目，让你可以拿到剩余的 25 分，

45+通过考试。

所以结合上面的知识点分布形式，凯恩将案例红宝书分为上篇和下篇，上篇主要提炼书本上的案例相关考点，下篇主要聚焦书本之外的内容，力求提升你的技术视野，让您做选做题的时候有话可说。

刷题形式

案例一定要做真题，否则你不知道题目形式、答题方法和自己的真实水平。

凯恩建议每个同学都要去练真题做真题至少三遍。做真题有两个渠道，第一个去官网下载案例分类解析，这里的题目凯恩按照知识点大类对内容做了全面的解析，方便你对知识点进行重点突破。



第二是点击我们的官网首页 www.cheko.cc 的历年真题模块，下载历年试题打印出来进行练习（都是附带解析的）。

芝士架构 题库 统计 我的 定价 APP 红宝书

当前位置 - 历年真题

系统分析师 考试时间预计为 2025-05-24, 距今还有 80.5 天, 报名时间预计为 2025-03-25

备考红宝书 14万字 - 浓缩备考知识点
20万字 - 详细历年真题解析

学习互动群 无限次-备考详细答疑互动交流
累计3次-论文批改及写作指导

选择题 案例题 论文题

一键下载

2024年11月 (案例题真题) 导出 难度简单 掌握程度 最近得分 0分 / 满分 11分 去练习

2024年5月 (案例题真题) 导出 难度中等 掌握程度 最近得分 0分 / 满分 11分 去练习

2023年5月 (案例题真题) 导出 难度简单 掌握程度 最近得分 0分 / 满分 13分 去练习

当然假如你想在线练习也可以, 点击官网案例解析模块在线练习也非常容易。

芝士架构 题库 课程 统计 定价 下载客户端 红宝书 论文助手

错题爆破 所有错过的都值得重做

每日一练 每天打卡难度近似真题

我的收藏 所有的收藏都有目的

选择题 案例题 论文题

筛选

Web应用设计 不看必挂 暂无要点 掌握程度 刷题进度 0已做/55合计 去练习

数据库系统与缓存设计 不看必挂 暂无要点 掌握程度 刷题进度 0已做/30合计 去练习

系统架构设计与评估 不看必挂 暂无要点 掌握程度 刷题进度 0已做/36合计 去练习

系统设计与建模 不看必挂 暂无要点 掌握程度 刷题进度 0已做/36合计 去练习

项目管理 非重点 暂无要点 掌握程度 刷题进度 0已做/4合计 去练习

系统安全 非重点 暂无要点 掌握程度 刷题进度 0已做/3合计 去练习

嵌入式系统架构设计 非重点 暂无要点

案例题型

从形式角度上来看，虽然官方没有给出题型类别，但是根据真题的出题方式，目前可以分为选词填空型、简答填空型、概念对比型、概念简述型、解决方案简述型五大类别，如下表所示。

案例题型	真题问法	难易程度
选词填空型	胰岛素剂量传输不准会带来一些安全问题，请将下列内容填入下方的空格处，完善可能导致胰岛素剂量不准的原因。 备选选项：(a) 血糖传感器错误 (b) 传输系统异常 (c) 血糖计算不准 (d) 定时器失效 (e) 泵信号失效 (f) 错误时间推送预定的量 (g) 算法错误 (h) 计算错误 (i) 胰岛素计算错误	简单
简答填空型	根据题干中描述的基本功能需求，架构师王工通过对需求的分析和总结给出了无人直升机控制系统纵向控制状态图。请根据题干描述，提炼出相应状态及条件，并完善如图所示状态图中的 (1) ~ (5)，将答案填写在答题纸中。	较难
概念对比型	该系统需实现用户终端与服务端的双向可靠通信，请用 300 字以内的文字从数据传输可靠性的角度对比分析 TCP 和 UDP 通信协议的不同，并说明该系统应采用哪种通信协议。	较难
概念简述型	请用 300 字以内的文字，说明 Redis 分布式存储的两种常见方案，并解释说明 Redis 集群切片的几种常见方式。	最难
解决方案简述型	经过运维人员的深入分析，发现存在两种情况： (1) 用户请求的 key 值在系统中不存在时，会查询数据库系统，加大了数据库服务器的压力； (2) 系统运行期间，发生了黑客攻击，以大量系统不存在的随机 key 发起了查询请求，从而导致了数据库服务器的宕机。 经过研究，研发团队决定，当在数据库中也未查找到该 key 时，在缓存系统中为 key 设置空值，防止对数据库服务器发起重复查询。请用 100 字以内文字说明该设置空值方案存在的问题，并给出解决思路	最难

选词填空型是最简单的，因为给出的信息很多很直接，比较利于你做判断，对行业背景知识要求不高，这种题目是非后端类同学的最爱。

简答填空题是较难的，因为实际上这类题目也给你提供了一些信息，但是由于是开放的填

空，所以比选择填空信息要少，难度略有提升。

概念对比型是较难的，一来因为是简述题，需要你结合自己的理解来写，同时要对比两个或者多个不同的工具和技术的异同，对你的知识储备要求较高。但是因为一般来说题目中可能会给你一些提示，比如从哪些方面进行比较，因此在作答的时候范围还是受到了约束，可以通过题干的倾向推断出一部分来。

解决方案简述型和概念简述型是最难的，因为几乎不给你额外信息，就是考你知识储备，假如没有接触过，或者没有准备过，一点都写不出来。

针对后端经验不是特别丰富，八股卷的不是特别多的同学，在做题之前你是可以快速估算出每个大题的预测得分情况，选择一个得分最高的问题回答就行。

关注范围

从 23 年 11 月架构新教材改版以来，可以明显的发现出题者的规律，就是第一题必定是书上的知识点。所以我们必须要重点关注新教材案例篇出现的所有概念。这些内容凯恩都标记了超级重点。但是我们也会发现，很多同学所谓的“八大架构”（架构新教材第 12 章开始到第 19 章），并不都是考试重点。从出题角度来说，第 12 章信息系统架构案例设计过于理论化，很难结合一个具体案例来考察，所以基本不会考。第 16 章嵌入式系统架构设计理论与实践、第 17 章通信系统架构设计理论和实践，由于嵌入式、网络并不是架构考试主流方向，所以也不是重点。感兴趣的同学看一下教材了解即可。另外少部分真题出现过的概念比如区块链，消息队列等我们通过真题来进行补充。

1.上篇-需求分析（次重点★★★★☆☆）

这一章节的考察主要还是会以需求建模（填空）为主，加上对数据流图，状态转换图，数据字典，UML 图等相关概念的考察。建模题你做过真题就知道，需要一定的逻辑，通过文字说明给出业务流程，让你在对应的状态图，顺序图，活动图上填空。这种题目往往都有争议，毕竟题干描述的需求往往比较模糊，需要你结合自己的理解去推理补全拼图，拿分容易，高分难。架构这块考得不多，但是，只要出这个章节的题目就是送福利，送你过。

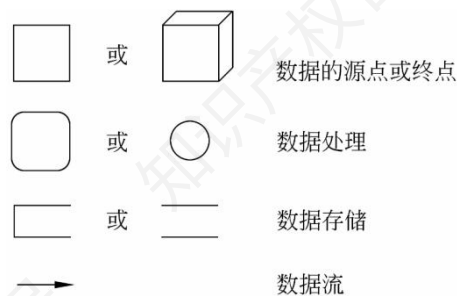
1.1.结构化分析（重点★★★★★★）

1.1.1.数据流图 DFD（重点★★★★★★）

1.1.1.1.基本符号（重点★★★★★★）

在 DFD 中，通常会出现 4 种基本符号，如下表所示（结合图片进行记忆）。这一块是看图的基础，必须掌握。在选择题一本全里，我们重点讲到过这个部分，但是一般来说案例不会直接考你这些数据元素的内容，而是结合实际案例考察你对数据流图的运用。

符号名称	表示内容	图形表示
数据流	具有名字和流向的数据	标有名字的箭头
加工（数据处理）	对数据流的变换	圆圈
数据存储	可访问的存储信息	直线段
外部实体（数据源及数据终点）	位于被建模的系统之外的信息生产者或消费者，表明数据处理过程的数据来源及数据去向	标有名字的方框



补充：数据存储要被外部实体使用必须要经过加工。有一年案例考察过这个知识点。

有的同学看到其他资料反馈图对不上，这里特此解释。数据流图模板有两种常见的记号集：Yourdon 和 Coad 方法以及 Gane 和 Sarson 方法。这两个系统都以创造它们的计算机科学家的名字命名。上图左侧的是 Gane 和 Sarson 方法，右侧是 Yourdon 和 Coad 方法。

1.1.1.2.平衡原则（重点★★★★★）

重点关注，数据流图的平衡原则，如下所示，要掌握父子图的平衡和子图的平衡，结合案例真题能够写出错误的理由或能写出基本的数据流图平衡办法。

（1）父图（上层数据流图）与子图（下层数据流图）平衡（也叫作分层细化过程平衡）。

个数一致：两层数据流图中的数据流个数一致。方向一致：两层数据流图中的数据流方向一致。

（2）子图内部的平衡表示如下。

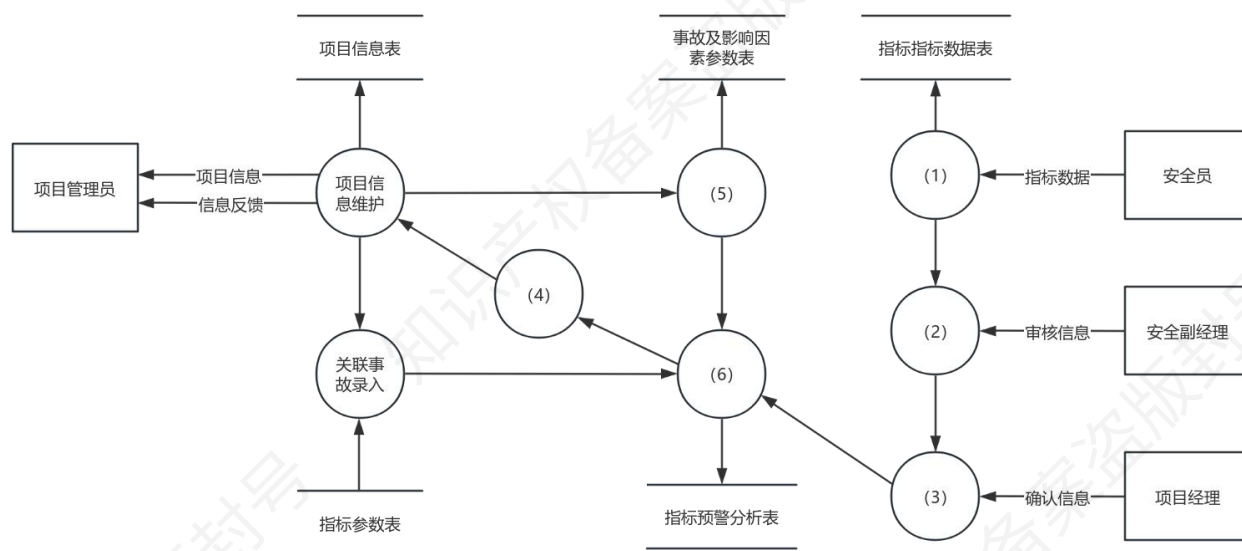
概念	描述
黑洞	加工只有输入没有输出（只进不出）
奇迹	加工只有输出没有输入（只出不进）
灰洞	加工中输入不足以产生输出（加工应该输出 A，实际给出 B，挂羊头卖狗肉的意思）
数据存储	一般来说这个点可以不提。正常情况下必须既有读的数据流，又有写的数据流；在某张子图中，可能只有读没有写，或者只有写没有读。

1.1.1.3.典型例题 (重点★★★★★)

（2022 年架构真题）煤炭生产是国民经济发展的重要领域之一，其煤矿的安全非常重要。某能源企业拟开发一套煤矿建设项目安全预警系统，以保护煤矿建设项目从业人员生命安全。本系统的主要功能包括如下（a）～（h）所述。

(a)项目信息维护(b)影响因素录入(c)关联事故录入(d)安全评价得分(e)项目指标预警分析(f)项目指标填报(g)项目指标审核(h)项目指标确认

王工根据煤矿建设项目安全预警系统的功能要求,设计完成了系统的数据流图,如图所示。请使用题干中描述的功能(a)~(h),补充完善空(1)~(6)处的内容,并简要介绍数据流图在分层细化过程中遵循的数据平衡原则。



这一题是凯恩说的典型的选词填空题，是最简单的案例题型。一般来说，这种题目即使题干给的信息不多，也不会影响答题。

我们先看数据流图的右侧，安全员提供“指标数据”，表明系统需对指标数据进行收集录入，对应功能(f)项目指标填报。安全副经理处理“审核信息”，表明是对已填报指标的审核操作，对应(g)项目指标审核。项目经理处理“确认信息”，表明最终确认指标数据，对应(h)项

目指标确认。

再看左侧。(5): 数据流向“事故及影响因素参数表”, 因此需录入影响因素数据, 对应(b)影响因素录入。(6): 最终输出到“指标预警分析表”, 因此这里的加工的对象是预警分析, 对应(e)项目指标预警分析。(4): 最后再看 4, 指标预警分析输出得分较为合理, 故填(d)安全评价得分。

答案 (1) f (2) g (3) h (4) d (5) b (6) e

分层细化的数据平衡原则首先要考虑分层数据流图中的数据平衡原则, 父类和子类之间的数据流必须保持一致, 包括数据流个数和方向一致。然后要考虑每张数据流图的数据平衡原则。加工的输入数据流和输出数据流要平衡, 保证每个加工都有对应的输入和输出数据流。避免三种常见错误: 黑洞: 只进不出。奇迹: 只出不进。灰洞: 加工中输入不足以产生输出 (加工应该输出 A, 实际给出 B, 挂羊头卖狗肉的意思)。

1.1.2.状态转换图 (次重点★★★★☆☆)

状态转换图/状态机图, 就是 UML1.x 中的状态图, 利用状态和事件描述对象本身的行为。在书本上它是被归类到结构化分析工具里头的, 实际上面向对象分析也用得到。它是一种非常重要的行为图, 强事件导致的对象状态的变化。状态机图中的主要概念包括以下 3 个。(1) 状态、初态、终态。(2) 事件、转移、动作。(3) 并发状态机。状态机图的推荐使用场合: 包括类设计场合。目前只在系分中考过一次, 架构案例还没考到过, 可以准备一下。和数据流图一样, 只会考你填空。

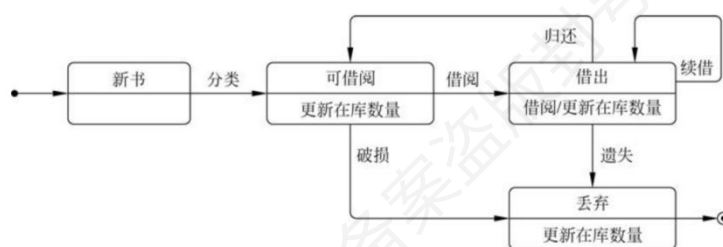
1.1.2.1.基本符号 (重点★★★★★★)

状态转换图是一种描述系统对内部或外部事件响应的行为模型。它描述系统状态、事件和

事件引发系统在状态之间的转换。在状态转换图中定义的状态主要有：初态（即初始状态）、终态（即最终状态）和中间状态。初态用黑圆点表示，终态用黑圆点外加一个圆表示，状态图中的状态用圆角四边形表示，状态之间状态转换用带箭头的线表示。如下表所示。

元 素	图 形 符 号	元 素	图 形 符 号
状态(State)		初态(Initial State)	
浅度历史(Shallow History)		终态(Final State)	
深度历史(Deep History)		转移(Transition)	
选择(Choice)		分叉/合并(Fork/Join)	

例子如下：



1.1.2.2.典型例题（重点★★★★★）

（2020 年 5 月系分真题）某公司长期从事嵌入式系统研制任务，面对机器人市场的蓬勃发展，公司领导决定自主研发一款通用的工业机器人。王工承担了此工作，他在广泛调研的基础上提出：公司要成功地完成工业机器人项目的研制，应采用实时结构化分析和设计（RTSAD）方法，该方法已被广泛应用于机器人顶层分析和设计中。

下图给出了机器人控制器的状态转换图，其中 T1T6 表示了状态转换过程中的触发事件，请将 T1-T6 填到图中的空（1）～（6）处，完善机器人控制器的状态转换图，并将正确答案填写在答题纸上。

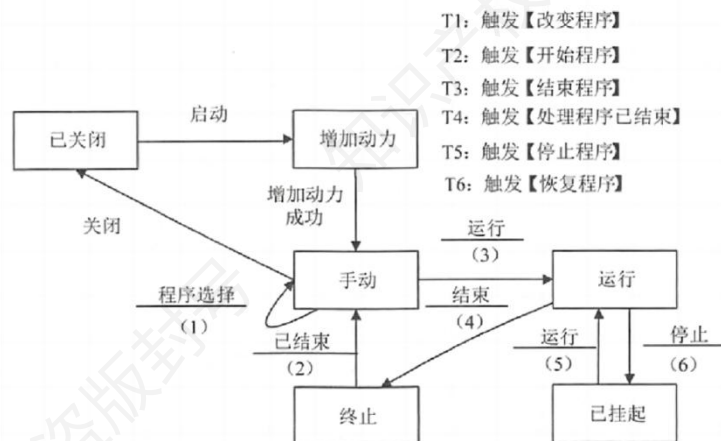


图 3-1 机器人控制器状态转换图

状态转换图，即 STD 图（StateTransformDiagram），表示行为模型。STD 通过描述系统的状态和引起系统状态转换的事件，来表示系统的行为，指出作为特定事件的结果将执行哪些动作（例如处理数据等）。STD 描述系统对外部事件如何响应，如何动作。在状态转换图中，每一个节点代表一个状态。有题干可知，机器人控制器设定了 6 种状态，即已关闭、增加动力、手动、运行、终止和已挂起，在 6 个状态相互转换时，设计了 6 个触发事件（T1~T6）。

当按下启动按键时，系统就会进入增加动力状态。在成功地完成了增加动力的过程之后，系统就会进入手动状态。系统手动状态时操作员按下运行按钮，就会启动当前选择程序的执行过程，然后系统就会过渡到运行状态，所以第三空应该为 T2：触发【开始程序】。

系统运行状态时操作员可以通过按下停止按钮来挂起程序的执行过程，然后系统就会进入已挂起状态，所以第六空应该为 T5：触发【停止程序】。

系统已挂起状态时操作员可以按下运行按钮来继续执行程序，系统则返回到运行状态，所以第五空应该为 T6：触发【恢复程序】。

系统运行状态时操作员可以按下结束按钮，系统进入终止状态，所以第四空应该为 T3：触发【结束程序】。

当程序终止执行时要想返回手动状态，就需要触发【处理程序已结束】，从而回到手动状

态。所以第二空应该为 T4：触发【处理程序已结束】。

系统手动状态时操作员现在可以使用程序选择旋钮开关来选择程序，所以应该触发【改变程序】，第一空应该为 T1：触发【改变程序】。

答案：（1）T1（2）T4（3）T2（4）T3（5）T6（6）T5

1.2.面向对象分析方法（超级重点★★★★★）

面向对象分析方法架构系分都爱考常考，概念对比类、简答类、选词填空类都考过。这一小节，凯恩把常考的概念对比贴出来，并且结合典型例题，让你掌握这里的重点难点。

1.2.1.UML 引入（重点★★★★★）

软考里面面向对象分析工具就是 UML。只要出题人家庭还和睦，情绪还稳定，那只会考类图、用例图、顺序图、协作图、活动图和状态图（之前结构化分析提到过）。别看内容少，但是这些图只要配合上任意场景就能摇身变成一道新的题目。另外这些图之间的对比异同也是它常考的内容，需要重点关注。下图是 UML2.0 包含的所有分析建模工具，但是常考的就是蓝色高亮标注的这些，其他工具不需要深入，知道它们大致能做什么就行。

有人是大纲信徒，说大纲没有 UML，我不准备我不背了。软考的大纲和厕纸没有区别，现在还有这样的人你们就把它自动归类为傻子，让他们老老实实做分母吧。

名称	描述
类图	描述类、接口、协作及它们之间的关系
对象图	描述对象及对象之间的关系
包图	描述包及包之间的相互依赖关系

组合结构图	描述系统某一部分（组合结构）的内部结构
构件图	描述构件及其相互依赖关系
部署图	展示构件在各节点上的部署
外廓图	展示构造型、元类等扩展机制的结构
顺序图	展示对象之间消息的交互，强调消息执行顺序的交互图
通信图/协作图	展示对象之间消息的交互，强调对象协作的交互图。协作图是 UML1 的说法。
时间图/定时图	展示对象之间消息的交互，强调真实时间信息的交互图
交互概览图	展示交互图之间的执行顺序
活动图	描述事物执行的控制流或数据流
状态机图	描述对象所经历的状态转移
用例图	描述一组用例、参与者及它们之间的相互关系

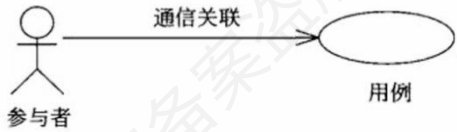
1.2.2.用例图（超级重点★★★★★）

用例模型中只会结合具体场景考你识别参与者、调整用例模型的基本能力。这里要知道面向对象分析的若干基本步骤，按顺序就是识别参与者、合并需求获得用例、细化用例描述和调整用例模型，需要记忆。

1.2.2.1.用例图的元素（重点★★★★★）

结构元素	图形符号	关系元素	图形符号
用例 (Use Case)		关联 (Association)	
参与者 (Actor)		扩展 (Extend)	
系统边界 (System Boundary)		包含 (Include)	
注释 (Note)		泛化 (Generalization)	
		注释链接 (Note Link)	

用例是一种描述系统需求的方法，使用例的方法来描述系统需求的过程就是用例建模。不考虑通信关联的类型，用例图可以简化成参与者、用例和通信关联三种元素组合，如图所示。这是做题的基础，案例不会直接让你默写这些图示的意义。



1.2.2.2.识别参与者（重点★★★★★）

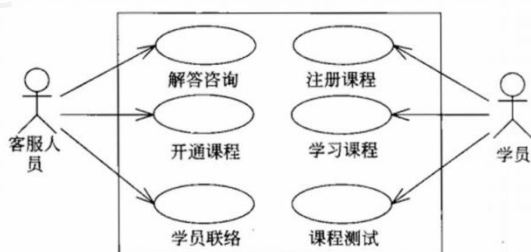
参与者是与系统交互的所有事物，该角色不仅可以由人承担，还可以是其他系统和硬件设备，甚至是系统时钟。案例真题让你分析题目中哪些属于参与者，这里要记住处理人，一些系统和设备都是。要注意的是，参与者一定在系统之外，不是系统的一部分。下面是三个具体的参与者识别的场景。

参与者类型	描述	示例
其他系统	当系统需要与其他系统交互时，该其他系统为参与者	对某企业在线教育平台系统，该企业的 OA 系统是参与者
硬件设备	若系统需与硬件设备交互，此硬件设备为参与者	开发集成电路（IC）卡门禁系统时，IC 卡读写器是参与者
时钟	当系统需定时触发时，时钟作为参与者	开发在线测试系统“定时交卷”功能时，引

	入时钟作为参与者
--	----------

1.2.2.3.合并需求获得用例（次重点★★★★☆☆）

将识别到的参与者和合并生成的用例，实际上就是合并同类项或者对于同一参与者的用例进行合并汇总。通过用例图的形式整理出来，以获得用例模型的框架，如图所示。



1.2.2.4.细化用例描述（次重点★★★★☆☆）

这一章节几乎没有考过，可以忽略，了解一下用例描述包含哪些部分即可。

用例描述部分	内容说明
用例名称	应与用例图相符，并标注相应编号
简要说明	用简洁自然语言，描述用例为参与者传递的价值结果
事件流	参与者和系统为达成目标所发生的一系列活动
非功能需求	单列描述用例涉及的非功能需求，需保证可度量和可验证
前置条件和后置条件	前置条件为执行用例前系统必须存在的状态，不满足则用例无法启动；后置条件是用例执行完毕系统可能处于的一组状态
扩展点	若有扩展（或包含）用例，写出其名称及使用情况
优先级	表明用户对该用例的期望，确定开发先后顺序

比如我们可以这样描述注册和登录的用例。

用例描述部分	内容说明
用例名称	注册/登录（UC01）

简要说明	为学员/老师/管理员提供安全便捷的账号接入，确保个性化学习与权限管控。
事件流	参与者输入手机号/邮箱或使用三方登录； 系统执行身份验证（密码/验证码/单点登录）； 成功后创建/加载用户会话与角色信息。
非功能需求	平均响应时间 $\leq 2s$ ；；密码与令牌加密存储（PBKDF2/BCrypt；传输 TLS1.2+）。
前置条件和后置条件	前置条件为系统可用，认证服务正常；后置条件为会话有效并带角色与权限。
扩展点	异常重试、找回密码、二次验证(2FA)
优先级	高

1.2.2.5.调整用例模型（重点★★★★★）

这个章节是必须要掌握的，死记硬背也要背下来，包括箭头形状，箭头指向含义，各个关系的内涵。用例之间的关系主要有包含、扩展和泛化，利用这些关系，把一些公共的信息抽取出来，以便于复用，使得用例模型更易于维护。

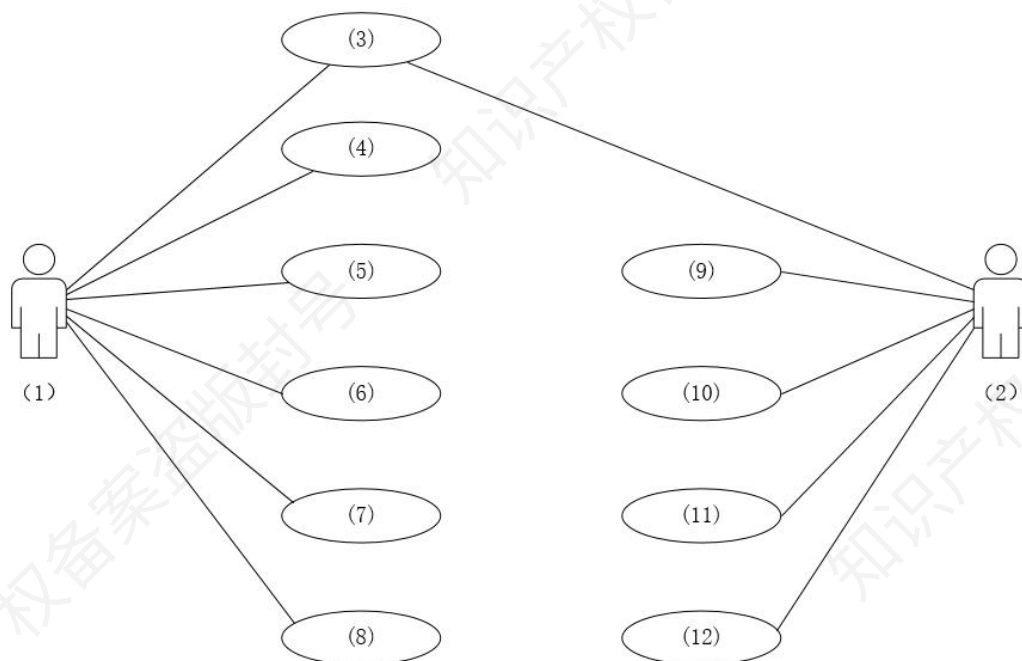
关系类型	描述	示例	构造型	箭头指向	图例
包含关系	从两个或多个用例提取公共行为,提取出的公共用例为抽象用例,原始用例为基本用例。	“学习课程”和“课程测试”都需检查学员权限，“检查权限”为抽象用例，“学习课程”和“课程测试”为基本用例。	<code><<include>></code>	指向抽象用例	
扩展关系	一个用例混合多种不同场景,可分为一个基本用例和多个扩展用例。	学员“课程测试”时，若测试次数超出限额需“充入学习币”，“课程测试”是基本用例，“充入学习币”是扩展用例。	<code><<extend>></code>	指向基本用例	

泛化关系	多个用例有类似结构和行为，将共性抽象为父用例，其他为子用例，子用例继承父用例所有结构、行为和关系。	学员课程注册可通过电话或网上注册，“注册课程”是“电话注册”和“网上注册”的父用例。	→	指向父用例	
------	---	--	---	-------	--

1.2.2.6.典型例题（重点★★★★★）

（2021 年架构真题）某医院拟委托软件公司开发一套预约挂号管理系统，以便为患者提供更好的就医体验，为医院提供更加科学的预约管理。本系统的主要功能描述如下：（a）注册登录，（b）信息浏览，（c）账号管理，（d）预约挂号，（e）查询与取消预约，（f）号源管理，（g）报告查询，（h）预约管理，（i）报表管理和（j）信用管理等。

若采用面向对象方法对预约挂号管理系统进行分析，得到如图所示的用例图。请将合适的参与者名称填入图中的（1）和（2）处，使用题干给出的功能描述（a）~（j），完善用例（3）~（12）的名称，将正确答案填在答题纸上。



这是一道套用例图壳的选词填空题。这里涉及到两个参与者，分别是患者和医生/医院管理人员/医院方都可以，两边可以看到他们对应的用例数量是不一样的。并且他们都有一个公共的用例。所以我们首先要把这用例进行分类。

患者作为预约挂号服务的直接使用者，用例围绕挂号流程展开，包括使用系统的前置操作（注册登录、信息浏览、账号管理），挂号核心操作（预约挂号、查询与取消预约），以及附加医疗服务需求（报告查询）。

医院方负责系统的运营与管理，聚焦对挂号资源及系统规则的管控。如号源管理（维护可挂号资源）、预约管理（制定预约规则与分配），以及报表管理、信用管理（从数据和信用维度优化系统运营）。

这里有一个最大的问题，就是账号管理到底属于谁的用例。假如考虑到医院方可以新增医生账号，那么属于医院方的用例，假如说账号管理包括修改信息，修改绑定手机号等功能，则属于患者的用例。这道题出得非常没水平，大家了解就行。

答案：（1）预约人员（患者）（2）医院管理人员（3）（a）注册登录

（4）-（8）（b）信息浏览（c）账号管理（d）预约挂号（e）查询与取消预约（g）报告

查询 (9) - (12) (f) 号源管理 (h) 预约管理 (i) 报表管理 (j) 信用管理

1.2.3.类图 (重点★★★★★)

类图可以用于面向对象分析也可以用于设计，软考案例中它只会考类之间的六大关系。

1.2.3.1.类关系 (重点★★★★★)

类之间的主要关系有关联、依赖、泛化、聚合、组合和实现等六大关系，它们在 UML 中的表示方式如图所示。其中，聚合和组合关系都是关联关系的一种。这些关系的强弱顺序不同，下面的他们关联强度的排序结果为：泛化=实现>组合>聚合>关联>依赖。

这个顺序需要记忆，口诀为饭（泛）食（实现）组聚关羽（依）。（吃饭小组相聚和关羽）。

类关系一般不会让你之间默写概念，而是和特定的场景结合，让你根据实际情况建立类之间的关联模型，考察你的运用能力。

类图实际上是 UML 里比较复杂的一个图，一般来说掌握这 6 个关系就已经非常不错，深挖没有必要，内容太多无法记忆。

关系类型	定义	举例	图例
关联关系	一种强语义联系的结构关系，表明两个事物间存在明确、稳定的语义联系	工人指向任务，表明工人与任务两者存在明确关联	<pre> classDiagram Task --> Worker class Worker { + task : Task } </pre>
依赖关系	两个类 A 和 B，若 B 的变化可能引起 A 的变化，A 依赖于 B	司机类指向汽车类，表示司机类依赖于汽车类。即司机要执行驾驶行为，必须依赖于汽车这个对象，没有汽车，司机就无法完成驾驶操作。	<pre> classDiagram Car ..> Driver class Driver { + Drive(Car car) } </pre>

泛化关系	描述一般事物与特殊种类的关系,即父类与子类的关系	狗和猫是动物的子类,继承动物属性和方法并扩展特有功能	<pre> classDiagram Animal < -- Dog Animal < -- Cat </pre>
聚合关系	表示类之间整体与部分的关系,“部分”可同时属于多个“整体”,“部分”与“整体”生命周期可不同,好聚好散,部分能单独存在	学生和班级,学生可以脱离班级存在。	<pre> classDiagram Class o-- Student </pre>
组合关系	同样表示类之间整体与部分的关系,“部分”只能属于一个“整体”,“部分”与“整体”生命周期相同,与聚合比,生命周期同步	汽车与发动机,发动机仅属于一辆汽车,发动机随汽车创建与消亡	<pre> classDiagram Car "1" *-- "1" Engine Car "1" *-- "1" Wheel </pre>
实现关系	将说明和实践联系起来,接口说明行为,类实现接口操作	音频播放器实现“播放器”接口	<pre> classDiagram class IPlayer { + Start() + Stop() } class AudioPlayer { + Start() + Stop() } AudioPlayer .. > IPlayer </pre>

1.2.3.2.典型例题（重点★★★★★）

(2016 年 11 月架构考试真题)某软件公司计划开发一套教学管理系统,用于为高校提供教学管理服务。该教学管理系统基本的需求包括:

- (1) 系统用户必须成功登录到系统后才能使用系统的各项功能服务;
- (2) 管理员 (Registrar) 使用该系统管理学校 (University)、系 (Department)、教师 (Lecturer)、学生 (Student) 和课程 (Course) 等教学基础信息;
- (3) 学生使用系统选择并注册课程,必须通过所选课程的考试才能获得学分;如果考试不及格,必须参加补考,通过后才能获得课程学分;

- (4) 教师使用该系统选择所要教的课程，并从系统获得选择该课程的学生名单；
- (5) 管理员使用系统生成课程课表，维护系统所需的有关课程、学生和教师的信息；
- (6) 每个月到了月底系统会通过打印机打印学生的考勤信息。

项目组经过分析和讨论，决定采用面向对象开发技术对系统各项需求建模。

问：类图主要用来描述系统的静态结构，是组件图和配置图的基础。请指出在面向对象系统建模中，类之间的关系有哪几种类型？对题目所述教学服务系统的需求建模时，类 University 与类 Student 之间、类 University 和类 Department 之间、类 Student 和类 Course 之间的关系分别属于哪种类型？

第一个小问就是直接默写概念。第二小问，我们首先来看，在题目要求中，类 University 与类 Student 之间的关系是整体与部分关系。并且我要换成其他业务系统的时候，不设计类 University，类 Student 可以单独设计，他们没有紧密关系，也就是我说的具有不同的生存周期，口诀好聚好散，所以是聚合关系。

类 University 和类 Department 之间的关系是整体与部分的关系，两者具有相同的生存周期，你不能说设计一个系统，类 Department 但是没有类 University，这个说不通。所以它们之间是组合关系。同理，类 Student 和类 Course 我认为也是聚合关系，但是官方是关联关系，这个说不太通。因为关联包含了聚合和组合，粒度太粗了。

答：类之间的关系包括：泛化=实现>组合>聚合>关联>依赖。类 University 与类 Student 之间的关系是聚合关系。类 University 与类 Department 之间的关系是组合关系。类 Student 与类 Course 之间的关系是关联关系。

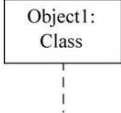
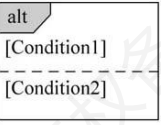
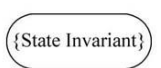
1.2.4.顺序图（重点★★★★★）

顺序图和活动图是案例常考的内容，放在这里作为补充。特别是它们各自对应的元素、图

形符号都要了解含义。顺序图是一种最常用的动态交互图，用于显示对象间的交互活动，关注对象之间消息传送的时间顺序。在软考案例中直接考察概念只有一次（组合片段），多数是简答填空的形式让你对业务场景进行建模。

1.2.4.1.顺序图的元素（重点★★★★★）

类别	详情
核心概念	对象、生命线、执行发生、消息，交互片段
推荐使用场合	用例分析、用例设计等场合

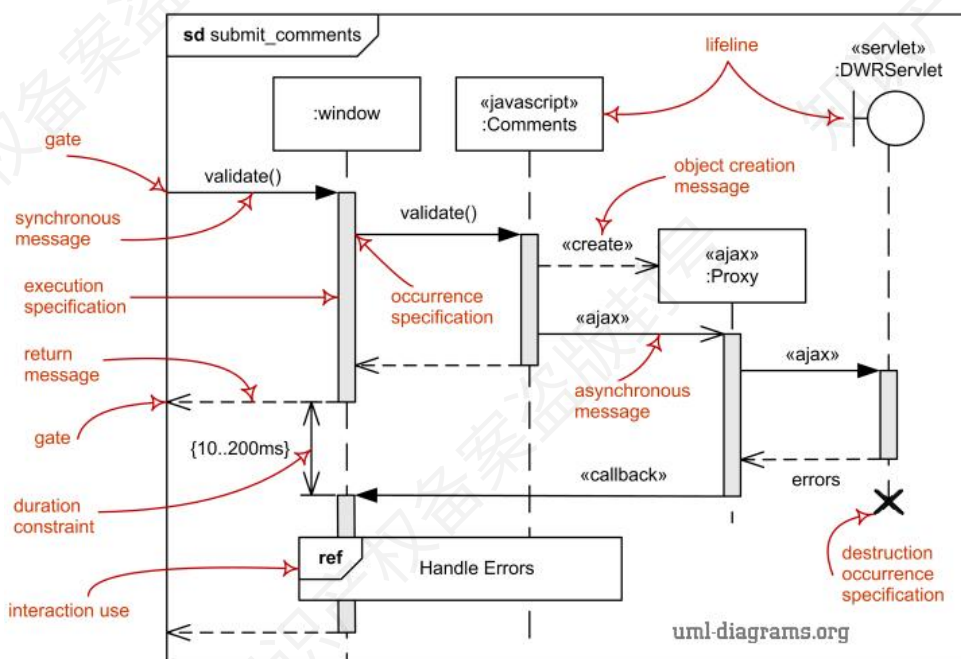
元 素	图 形 符 号	元 素	图 形 符 号
对象/生命线 (Object/Lifeline)		同步消息 (Synchronous Message)	
交互片段 (Interaction Frame)		异步消息 (Asynchronous Message)	
执行发生 (Execution Occurrence)		返回消息 (Return Message)	
状态不变式 (State Invariant)		创建消息 (Create Message)	

概念	解析
Lifeline（生命线）	代表对象或参与者生命周期的垂直虚线，贯穿其存在时间段，直观呈现对象在交互中的存在周期。
Message（消息）	对象间交互的载体，通过箭头表示，包含同步（实心三角箭头）、异步（开放箭头）等类型，标注消息名称或操作，驱动交互行为。
Endpoint（端点）	位于生命线底部的终止标记，用于标识对象生命周期结束（如对象销毁），明确对象存在的起止边界。

Gate (门)	连接不同交互片段 (如组合片段) 的接口, 标记消息流入/流出点, 在复杂交互中管理消息流向, 支持片段嵌套或引用。
CombinedFragment(组合片段)	容纳多个交互操作数的容器, 用于描述复杂逻辑。常见类型: alt (条件分支)、loop (循环)、par (并行)、opt (可选) 等

CombinedFragment (组合片段, 也可叫作交互片段), 上图和上表表述不一致这里提醒一下。

典型的顺序图如下所示。



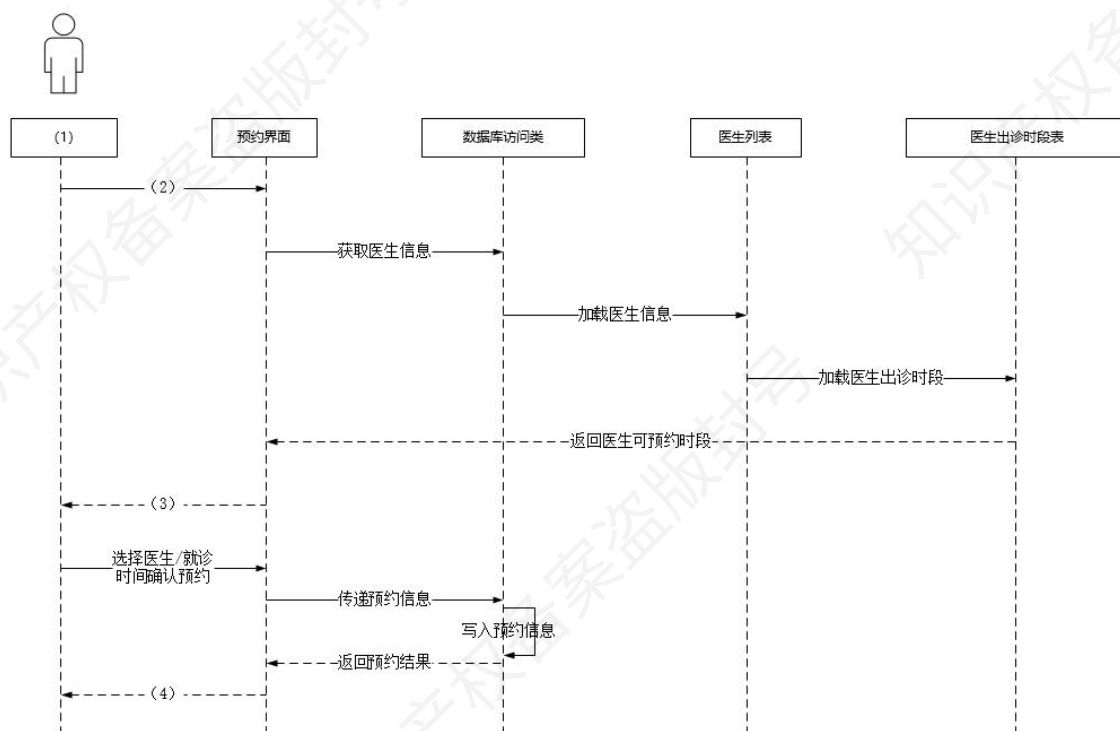
1.2.4.2.典型例题 (重点★★★★★)

(21 年架构真题) 某医院拟委托软件公司开发一套预约挂号管理系统, 以便为患者提供更好的就医体验, 为医院提供更加科学的预约管理。本系统的主要功能描述如下: (a) 注册登录, (b) 信息浏览, (c) 账号管理, (d) 预约挂号, (e) 查询与取消预约, (F) 号源管理, (g) 报告查询, (h) 预约管理, (i) 报表管理和 (j) 信用管理等。

问: 预约人员 (患者) 登录系统后发起预约挂号请求, 进入预约界面。进行预约挂号时使用数据库访问类获取医生的相关信息, 在数据库中调用医生列表, 并调取医生出诊时段表, 将

医生出诊时段反馈到预约界面,并显示给预约人员;预约人员选择医生及就诊时间后确认预约,系统反馈预约结果,并向用户显示是否预约成功。

采用面向对象方法对预约挂号过程进行分析,得到如图所示的顺序图,使用题干中给出的描述,完善图中对象(1),及消息(2)~(4)的名称,将正确答案填在答题纸上。



此题难度属于软考中最简单等级,该问考查 UML 中的顺序图,紧扣题目描述来组织内容即可,从题干中“预约人员(患者)登录系统后发起预约挂号请求,进入预约界面”的信息可知(1)应为预约人员(患者)(2)为预约挂号请求;从题干中“将医生出诊时段反馈到预约界面,并显示给预约人员”的信息可知(3)应为显示医生可预约时段;从题干中“系统反馈预约结果,并向用户显示是否预约成功”的信息可知(4)应为显示预约是否成功。

答案:(1) 预约人员(患者)(2) 预约挂号请求(3) 显示医生可预约时段(4) 显示预约是否成功

1.2.5.通信图/协作图（重点★★★★★）

通信图/协作图是 UML 交互图的一种，在 UML1.x 版本中被称为协作图。一个通信图显示了对象间的联系以及对象间发送和接收的消息。它和顺序图的最大区别就是它不反映时序信息。

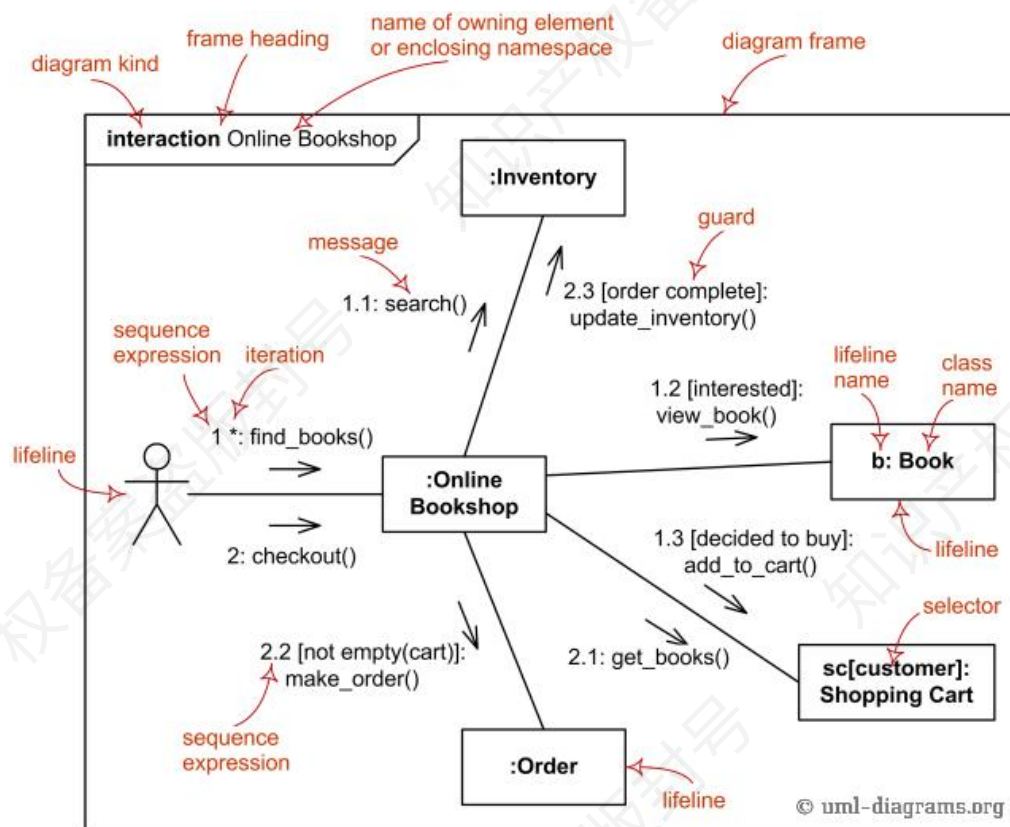
1.2.5.1.通信图/协作图的元素（重点★★★★★）

通信图常用的建模元素如下所示：包括对象（Object）、通信链接（Link）、消息（Message）。

概念	解析
对象	类的实例，也可代表协作、组件、节点等事物的实例
通信链接	对象间发送消息的路径，是通信图特有的连接元素
消息	对象间发送和接收的交互内容，体现动态行为

元 素	图 形 符 号	元 素	图 形 符 号
对象/生命线 (Object/Lifeline)		同步消息 (Synchronous Message)	
链接(Link)		异步消息 (Asynchronous Message)	
		返回消息 (Return Message)	

这里的同步消息、异步消息和返回消息的箭头和顺序图是一样的。典型的通信图如下所示：



1.2.5.2.典型例题（重点★★★★★）

（21 年架构真题）请简要说明在描述对象之间的动态交互关系时，协作图与顺序图存在哪些区别。

答：协作图与顺序图侧重点不同：顺序图以时间轴为核心，通过垂直生命线和消息箭头清晰展示消息传递的时序关系，适合体现复杂流程、并发或异步操作；协作图则以对象间的结构关系为中心，通过节点和链接呈现交互路径，强调对象协作的拓扑结构，适合分析静态关系紧密、消息路径简单的场景。二者语义等价可相互转换，实际应用中需根据需求选择——关注时序细节用顺序图，强调结构关系用协作图，也可结合使用以全面描述交互逻辑。

1.2.6.活动图（重点★★★★★）

将业务流程或其他计算的结构展示为内部一步步的控制流和数据流，主要用于描述某一方

法、机制或用例的内部行为，适用于业务建模、需求、类设计等场合。它的作用和流程图、数据流图、状态图非常相似，常被拿来一起比较。

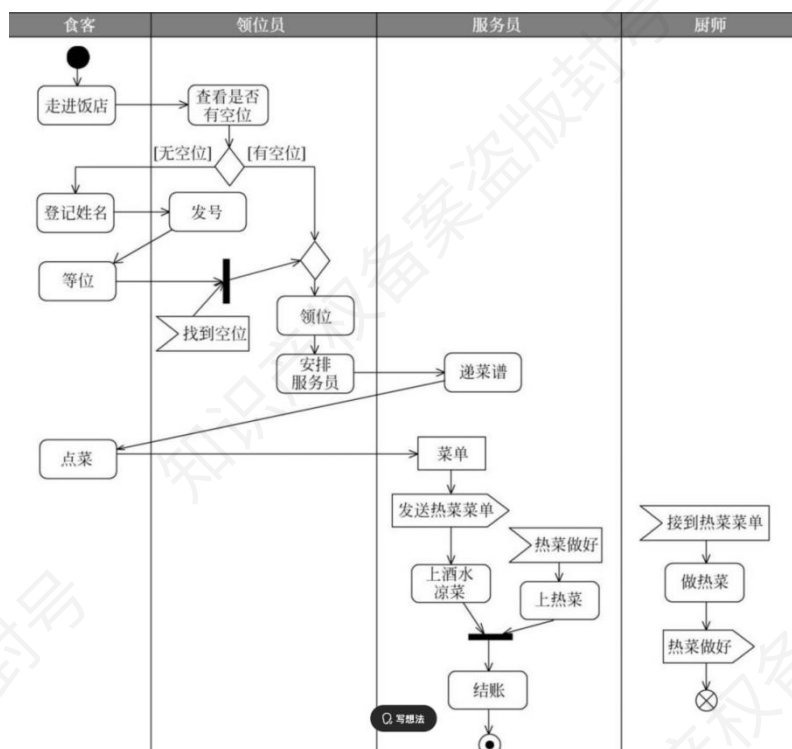
1.2.6.1.活动图的元素（重点★★★★★）

活动图和我们传统的流程图最大的区别在于它可以做并行（通过分叉/合并）的表示，同时还有事件发送和接收系统，相当于他可以做一个异步的表达，这是最大的不同。

概念	解析
初始节点	活动的起点，表示流程开始
终止节点	活动的终点，表示流程结束
动作状态	原子级操作或活动步骤
决策节点	用于流程分支，根据条件选择路径
合并节点	汇合多个分支流
分叉/合并	并行处理多个控制流（分叉）及汇合（合并）
泳道（分区）	按角色/部门划分的垂直区域，明确动作执行主体
对象节点	表示对象状态变化或数据流
控制流	连接动作节点的有向线段，表示执行顺序
对象流	连接对象节点的有向线段，表示对象传递
信号节点	异步通信的发送/接收事件

元 素	图 形 符 号	元 素	图 形 符 号
活动/动作 (Activity/Action)		起点 (InitialNode)	
对象 (Object)		终点 (FinalNode)	
发送事件 (SendEvent)		流结束 (FlowFinal)	
接收事件 (AcceptEvent)		分叉/合并 (Fork/Join)	
分区 (Partition)		控制流 (ControlFlow)	
决策点 (Decision)		对象流 (ObjectFlow) (在 UML 1. x 中为虚线)	

活用活动图是关键，大家看下图这个例子。这里体现了分叉合并和决策、发送接收事件等关键用法，把下面的解析一定要看懂。



这里列出了一个相对完整的活动图，该活动图描述了饭店业务中食客吃饭业务流程。图中首先通过分区机制将整个活动图划分为 4 个区域（食客、领位员、服务员和厨师），名称代表该区域的动作主体（即由指定的角色执行该区域的动作）。

具体的业务流程如下。首先，从起点开始，通过控制流进入第一个动，即食客走进饭店，饭店领位员帮忙查看是否有空位。该动作执行完成后，会产生有无空位的分支，通过决策节点

描述，两种分支情况分别放置在决策节点的输出控制流上。

如果有空位则直接领位，进入后续动作；反之，如果无空位，则食客登记姓名后，领位员发号表明等待的顺序，食客等位。在等位期间，领位员会频繁检查是否有空位（图中没有建模该动作），如果有空位，则发送找到空位的信号给对应的食客（这是一个领位员执行的独立控制流），这时这两个控制流汇合，并通过合并节点与有空位的分支流合并进入领位动作。

领位完成后，领位员安排服务员点菜，服务员递菜谱，食客根据菜谱点菜。点菜完成后，通过对象流形成菜单对象，服务员将菜单中的热菜发送给厨师（异步发送事件），从而通知厨师做热菜，之后安排上酒水凉菜。厨师接到热菜菜单后（异步接收事件），即启动独立地做热菜流程，热菜做好后发送信号通知服务员上热菜，当前厨师的控制流结束。服务员接收到热菜做好的信号后，为食客上主菜，这是与之前上酒水凉菜的控制流相对独立的一个新流程。

在酒水凉菜和主菜都上完后，汇合在一起进入后续的结账环节（省略了中间的食客吃饭的过程）。结完账后，进入终点表明整个流程结束。

1.2.6.2.典型例题（重点★★★★★）

（15 年架构真题）某公司拟研制一款高空监视无人直升机，该无人机采用遥控—自主复合型控制实现垂直升降。该直升机飞行控制系统由机上部分和地面部分组成，机上部分主要包括无线电传输设备、飞控计算机、导航设备等，地面部分包括遥控操纵设备、无线电传输设备以及地面综合控制计算机等。其主要工作原理是地面综合控制计算机负责发送相应指令，飞控计算机按照预定程序实现相应功能。经过需求分析，对该无人直升机控制系统纵向控制基本功能整理如下：

- （a）飞控计算机加电后，应完成系统初始化，飞机进入准备起飞状态；
- （b）在准备起飞状态中等待地面综合控制计算机发送起飞指令，飞控计算机接收到起飞指令

后，进入垂直起飞状态；

(c) 垂直起飞过程中如果飞控计算机发现飞机飞行异常，飞行控制系统应转入无线电遥控飞行状态，地面综合控制计算机发送遥控指令；

(d) 垂直起飞达到预定起飞高度后，飞机应进入高度保持状态；

(e) 飞控计算机在收到地面综合控制计算机发送的目标高度后，飞机应进入垂直升降状态，接近目标高度；垂直升降过程中出现飞机飞行异常，控制系统应转入无线电遥控飞行；

(f) 飞机到达目标高度后，应进入高度保持状态，完成相应的任务；

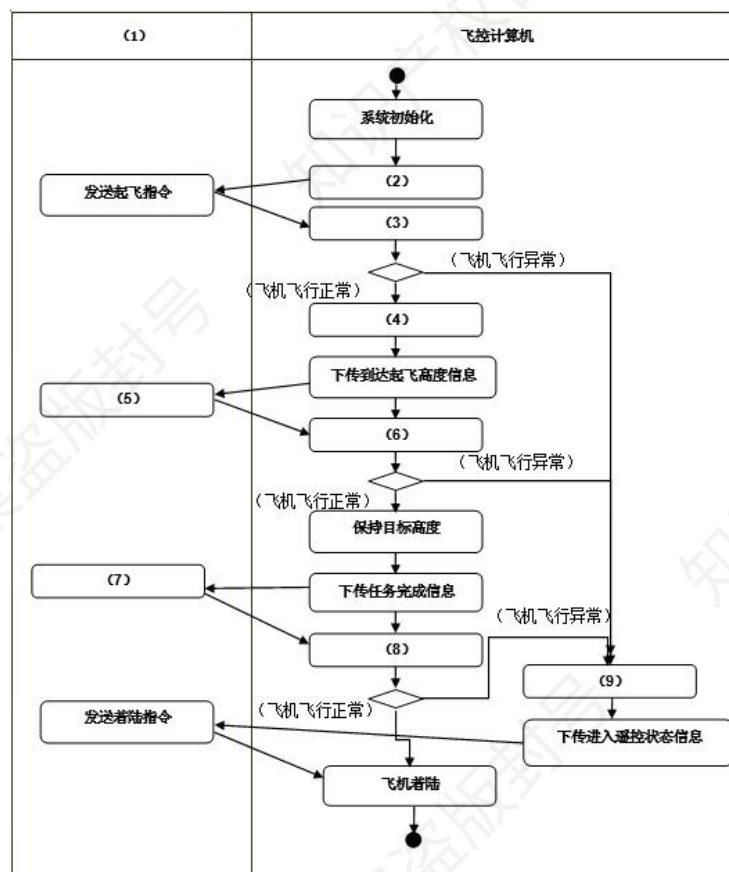
(g) 飞机在接到地面综合控制计算机发送的任务执行结束指令后，进入飞机降落状态；

(h) 飞机降落过程中如果出现飞机飞行异常，控制系统应转入无线电遥控飞行；

(i) 飞机降落到指定着陆高度后，进入飞机着陆状态，应按照预定着陆算法，进行着陆；

(j) 无线电遥控飞行中，地面综合控制计算机发送着陆指令，飞机进入着陆状态，应按照预定着陆算法，进行着陆。

问：根据题目中描述的基本功能需求，架构师王工给出了无人直升机控制系统纵向控制的顶层活动图。请根据题干描述，完善图活动图的（1）-（9），将答案填写在答题纸中



此题看似考察活动图，实际上考察你业务需求理解的能力。

(1) 地面综合控制计算机。题干中明确提到“地面综合控制计算机负责发送相应指令，飞控计算机按照预定程序实现相应功能”，在整个活动图中，与飞控计算机交互并发送指令的地面部分设备就是地面综合控制计算机，所以(1)处应填地面综合控制计算机。

(2) 下传起飞就绪信息。根据需求描述“飞控计算机加电后，应完成系统初始化，飞机进入准备起飞状态；在准备起飞状态中等待地面综合控制计算机发送起飞指令”，在进入准备起飞状态后，飞控计算机需要向地面综合控制计算机反馈一个状态信息，告知其已准备好，所以这里(2)是下传起飞就绪信息，以便地面综合控制计算机决定是否发送起飞指令。

(3) 垂直起飞。由“飞控计算机接收到起飞指令后，进入垂直起飞状态”可知，当飞控计算机接收到来自地面综合控制计算机的起飞指令后，下一步动作就是进入垂直起飞状态，所以(3)为垂直起飞。

(4) 高度保持。依据“垂直起飞达到预定起飞高度后，飞机应进入高度保持状态”，在垂直起飞完成并达到预定高度这个条件满足后，飞机接下来应进入的状态就是高度保持，故(4)填高度保持。

(5) 发送目标高度。从“飞控计算机在收到地面综合控制计算机发送的目标高度后，飞机应进入垂直升降状态”可知，在高度保持之后，地面综合控制计算机要向飞控计算机发送一个新的指令信息，即目标高度，从而让飞机进入下一阶段，所以(5)是发送目标高度。

(6) 垂直升降。按照“飞控计算机在收到地面综合控制计算机发送的目标高度后，飞机应进入垂直升降状态”，当接收到目标高度指令后，飞机对应的动作状态就是垂直升降，因此(6)为垂直升降。

(7) 发送任务结束指令。根据“飞机在接到地面综合控制计算机发送的任务执行结束指令后，进入飞机降落状态”，在飞机完成相应任务后，地面综合控制计算机需要发送一个指令来让飞机进入降落阶段，这个指令就是任务结束指令，所以(7)是发送任务结束指令。

(8) 飞机降落。由“飞机在接到地面综合控制计算机发送的任务执行结束指令后，进入飞机降落状态”可知，接收到任务结束指令后，飞机进入的状态是飞机降落状态，故(8)填飞机降落。

(9) 无线电遥控飞行。从“垂直起飞过程中如果飞控计算机发现飞机飞行异常，飞行控制系统应转入无线电遥控飞行状态”“垂直升降过程中出现飞机飞行异常，控制系统应转入无线电遥控飞行”“飞机降落过程中如果出现飞机飞行异常，控制系统应转入无线电遥控飞行”可知，当飞行出现异常时，控制系统会转入的状态是无线电遥控飞行，所以(9)为无线电遥控飞行。

答：(1) 地面综合控制计算机 (2) 下传起飞就绪信息 (3) 垂直起飞 (4) 高度保持 (5) 发送目标高度 (6) 垂直升降 (7) 发送任务结束指令 (8) 飞机降落 (9) 无线电遥控飞行

1.2.7.常见图对比（重点★★★★★）

讲了那么多图，实际上让你说对比也不太容易。为此，凯恩整理了历年真题中出现过的各个图之间概念的比较，学习的时候要重点关注对比结论。

图类型	主要关注点	典型用途	对比结论
用例图(Use Case Diagram)	系统外部用户（参与者）与系统之间的交互	捕获需求、确定系统边界、展示用户价值	抽象需求视角，描述“做什么”；不涉及内部实现
类图(Class Diagram)	系统内部类、属性、方法及类之间的关系	定义系统的结构设计、支撑代码实现	实现视角，描述“怎么做”；和用例图形成需求→设计的映射
数据流图(DFD)	数据的加工、流动、存储过程	需求分析阶段，建模系统数据处理逻辑	关注“数据在系统中如何流动”，忽略具体实现细节
活动图(Activity Diagram)	工作流、活动之间的顺序、分支、并行	表达业务流程或算法步骤	更偏向“任务执行步骤”，强调行为和流程控制
流程图(Flowchart)	操作步骤及逻辑判断	表达算法逻辑、简单过程	与活动图类似，但更传统，常用于说明程序逻辑
状态图(State Machine Diagram)	对象的状态变化及触发条件	建模复杂对象的生命周期	关注“对象随事件演化的状态”，区别于活动图的任务流
顺序图(Sequence Diagram)	对象间消息的时间顺序	展示场景交互流程，强调消息调用先后	强调“时序”，适合表现请求响应的详细过程
通信图(Communication Diagram)	对象之间的连接关系及消息传递	表达协作结构和对象间关系	强调“结构与协作”，和顺序图互补：内容相同，视角不同

2.上篇-系统设计/面向对象设计（重点★★★★★）

架构系统设计类考题只考面向对象设计方面内容，结构化设计基本不涉及，所以这里的标题取成系统设计/面向对象设计。这里主要考察两块内容，第一是类识别，一般来说会给你场景让你判断，某个类是边界类，控制类，还是实体类，这个方面架构考的少，系分考的多。第二类考点是类设计原则，架构案例还没考过。一般会以概念题的形态出现，需要好好记忆。

2.1.类识别（次重点★★★☆☆）

2.1.1.识别原则（次重点★★★☆☆）

一个类可以有多种职责，设计得好的类一般至少有一种职责，在定义类时，将类的职责分解为类的属性和方法，其中属性用于封装数据，方法用于封装行为。在系统设计过程中，类可以分为三种类型：实体类、边界类和控制类。它们的意义如下所示。我们重点关注类识别方法，也就是词性法。这里要强调的是边界类比较特殊，我们一般把窗口、通信协议、打印机接口、报表等内容当作边界类，其他都做实体类。

类型	识别方法（词性）	作用	示例
实体类	通常是名词	保存需永久存储的信息,属性和关系	在线教育平台系统的学员类、课程类
控制类	由动宾结构的短语（“动词+名词”或“名词+动词”）纯动词也可以	用于控制用例工作,对一个或几个用例特有的控制行为建模行为具有协调性	用例“身份验证”对应的控制类“身份验证器”
边界类	通常是名词(一般来说就例子所示的内容)	用于系统接口与外部交互,将系统与外部环境变更分隔开	窗口、通信协议、打印机接口、报表或其他可以连接外部和控制类的东西。

2.1.2.典型例题（重点★★★★★）

（21 年 5 月系分真题）某高校拟开发一套图书馆管理系统，在系统分析阶段，系统分析师整理的核心业务流程与需求如下：

系统为每个读者建立一个账户，并给读者发放读者证（包含读者证号、读者姓名），账户中存储读者的个人信息、借阅信息以及预订信息等，持有读者证可以借阅图书、返还图书、查询图书信息、预订图书、取消预订等。在借阅图书时，需要输入读者所借阅的图书名、ISBN 号，然后输入读者的读者证号，完成后提交系统，以进行读者验证。如果读者有效，借阅请求被接受，系统查询读者所借阅的图书是否存在，若存在，则读者可借出图书，系统记录借阅记录；如果读者所借阅的图书已被借出，读者还可预订该图书。读者如期还书后，系统清除借阅记录，否则需缴纳罚金，读者还可以选择续借图书。同时，以上部分操作还需要系统管理员和图书管理员参与。

问：设计类图的首要工作是进行类的识别与分类，该工作可分为两个阶段：首先，采用识别与筛选法，对需求分析文档进行分析，保留系统的重要概念与属性，删除不正确或冗余的内容；其次，将识别出来的类按照边界类、实体类和控制类等三种类型进行分类。请用 200 字以内的文字对边界类、实体类和控制类的作用进行简要解释，并对下面给出的候选项进行识别与筛选，将合适的候选项编号填入表中的（1）～（3）空白处，完成类的识别与分类工作。

类型	实例
边界类	(1)
实体类	(2)
控制类	(3)

候选项：a) 系统管理员 b) 图书管理员 c) 读者 d) 读者证 e) 账户 f) 图书 g) 借阅 h)

归还 i) 预订 j) 罚金 k) 续借 l) 借阅记录

第一小问纯概念不再展开，看看第二问。控制类前面说过一般是动宾结构或者动词，所以这里的 g（借阅）、h（归还）、i（预订）、k（续借）都是控制类。边界类这里不好判断，之前凯恩说过边界类窗口、通信协议、打印机接口、报表等，好像看起来都不像。那只能从定义判断，就是可以沟通外接和控制类的东西。这里读者证是最合适的。其余的都可以看作实体类。

答：边界类主要用于描述外部参与者与系统之间的交互。边界类是一种用于对系统外部环境与其内部运作之间的交互进行建模的类。这种交互包括转换事件，并记录系统表示方式（例如接口）中的变更。实体类主要是作为数据管理和业务逻辑处理层面上存在的类。实体类的主要职责是存储和管理系统内部的信息，它也可以有行为，甚至很复杂的行为，但这些行为必须与它所代表的实体对象密切相关。

控制类用于描述一个用例所具有的事件流控制行为，控制一个用例中的事件顺序。控制类是控制其他类工作的类。每个用例通常有一个控制类，控制用例中的事件顺序，控制类也可以在多个用例间共用。其他类通常并不向控制类发送消息，而是由控制类发出消息。

第二问官方的答案如下（说不通）

(1) j、l (2) a、b、c、f (c 可替换成 e) (3) g、h、i、k

凯恩给出的答案是

(1) d (2) a、b、c、e、f、j、l (3) g、h、i、k

2.2.面向对象设计原则（重点★★★★★）

在面向对象设计中，可维护性的复用是以设计原则为基础的。常用的面向对象设计原则包括单一职责原则（S）、开闭原则、里氏替换原则、依赖倒置原则、组合/聚合复用原则、接口隔离原则和最少知识原则等。每个原则具体是什么意思需要记忆。

2.2.1.设计原则（重点★★★★★）

设计原则	详细描述
单一职责原则（S）	单一职责原则要求类应该只有一个设计目的
开闭原则（O）	可扩展系统并提供新行为，通过添加抽象层实现，是其他原则的基础。
里氏替换原则（L）	软件实体使用基类对象时适用于子类对象，反之不一定，设计时应将变化类设计为抽象类或接口。
接口隔离原则（I）	分逻辑和狭义两种理解，前者将接口视为角色，后者要求接口提供客户端需要的行为，将大接口方法分至小接口。
依赖倒置原则（D）	抽象不依赖于细节，针对接口编程，是实现开闭原则的主要机制，与各种技术和框架相关。
组合/聚合复用原则	通过组合或聚合关系复用已有对象，比继承更灵活，耦合度低。
最少知识原则（迪米特法则）	软件实体应尽量少与其他实体相互作用

2.2.2.典型例题（重点★★★★★）

（24 年 5 月系分真题）某高校需开发一套在线论文管理系统，功能包括：学生用户通过界面登录、查看可选课题列表、提交选题；教师用户可发布课题、批阅论文；管理员管理用户权限；系统需支持论文上传下载、审核状态跟踪等功能。 问：请用 150 字简要说明什么是面向对象设计的开闭原则。
--

此题考察面向对象设计原则，这是反复让大家记忆背诵的内容。

答：面向对象设计的开闭原则（OpenClosedPrinciple, OCP）指的是一个软件实体（如类、模块、函数等）应该对扩展开放，对修改关闭。也就是说，在设计一个模块时，应当使该模块可以在不被修改源代码的前提下被扩展，即能够在不必修改原有代码的情况下改变这个模块的行为。

3.上篇-层次结构（次重点★★★★☆☆）

层次架构这一章很少直接考察，因为太简单了，东西不多。一般会以概念题或填空题的形式出现。凯恩这里只保留层次架构的一些重要概念，帮你节约时间。

3.1.层次结构简介和问题（重点★★★★★★）

3.1.1.层次架构简介（重点★★★★★★）

层次式体系结构设计是将系统组成一个层次结构，每一层为上层服务，并作为下层客户。在一些层次系统中，内部的层接口只对相邻的层可见。分层架构的一个特性就是关注分离。该层中的组件只负责本层的逻辑，组件的划分很容易明确组件的角色和职责，也比较容易开发、测试、管理和维护。在书本上重点从表现层、中间层和数据访问层展开讨论。

3.1.2.污水池反模式（重点★★★★★★）

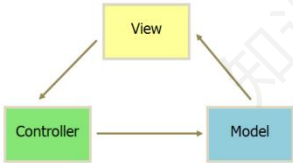
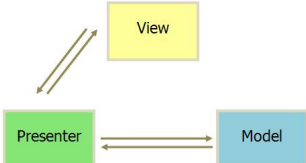
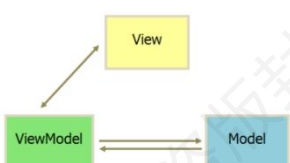
在软件架构设计里，不合理的层次架构会存在“污水池反模式（SinkholeAnti-Pattern）”。在这个模式中，请求流只是简单穿过各层次。比如说界面层响应了一个获得数据的请求，界面层把这个请求传递给了业务层，业务层也只是传递了这个请求到持久层，持久层对数据库做简单的 SQL 查询获得用户的数据。数据按照原路返回，不会有任何的二次处理。

这种做法会导致层次间隔离失效会使系统结构不清晰，业务规则变化时修改易出现遗漏或不一致、可维护性降低。**解决方法**是所谓的层次开放，就是有些数据处理可以直达核心层次，而不做层层穿越。

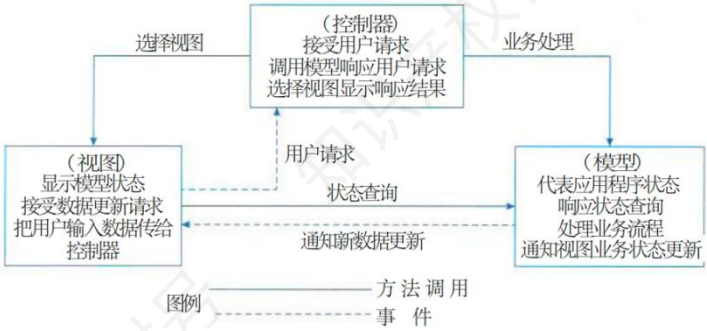
3.2.典型的层次架构（重点★★★★★★）

3.2.1.表现层层次架构（重点★★★★★）

表现层可以简单理解为各种前端展示的层次，这里可以分成三种架构，如下表所示。

维度	MVC	MVP	MVVM
核心组件	模型(Model)、视图(View)、 控制器(Controller)	模型(Model)、视图(View)、 呈现器(Presenter)	模型(Model)、视图(View)、 视图模型(ViewModel)
概念	模型：数据存储与业务逻辑 视图：用户界面展示 控制器：处理用户输入，协调模型与视图	模型：数据存储与业务逻辑 视图：用户界面交互接口 呈现器：处理逻辑，控制视图更新	模型：数据存储与业务逻辑 视图：绑定数据的 UI 视图模型：数据转换与双向绑定
数据流	单向：视图→控制器→模型 →视图更新（通过控制器）	双向：视图↔呈现器↔模型 （呈现器主动更新视图）	双向：视图↔视图模型↔模型 （自动同步，基于绑定）
数据交互	视图可直接调用控制器，可能访问模型	视图通过接口与呈现器通信，不直接访问模型	视图通过数据绑定与视图模型交互，无直接逻辑调用
			

这里需要注明的是书本上 MVC 模型会让人迷惑如下图所示，视图和模型是双向数据流，控制器和视图也有双向数据流。除非考试出到原题，否则都以凯恩给出的图例为准。



3.2.2.中间层架构设计（次重点★★★★☆☆）

中间架构层书本没有明说，这里实际上可以理解为是表现层和数据访问层中间的，控制数据请求逻辑处理的层次。这一节重点要掌握的是下面这张图。

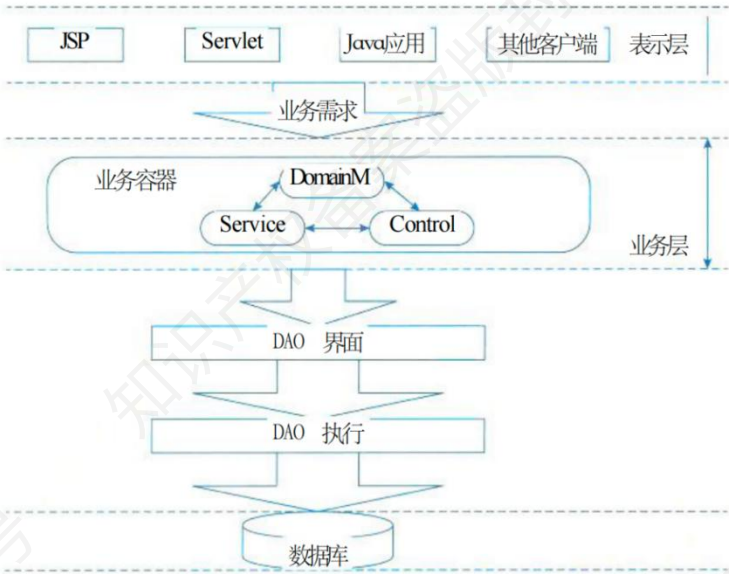


图13-10 业务框架在整个系统架构中的位置

在此图中，将业务系统划分为三层架构。分别是表示层，业务层和数据库层。这里业务层又通过 DomainModel—Service—Control 思想解耦职责、将服务和 Service 控制隔离，使程序具备高度的可重用性和灵活性。这里书本直接抛出领域模型（DomainModel），但是未做解释。凯恩补充梳理如下表所示。

组件	职责
----	----

DomainModel	封装业务实体及核心逻辑，如用户、订单等对象的属性和行为，确保业务规则的一致性。
Service	协调跨领域的业务操作（如事务管理、权限验证），调用多个 DomainModel 完成复杂功能。
Control	作为服务控制器，根据业务状态和参数决定 Service 的执行顺序与依赖关系，实现服务间的松耦合切换。

3.2.3.数据访问层设计（重点★★★★★）

这一层次是架构新教材新加的，23 年 11 月的架构案例考了一次 Hibernate 的架构图填空，不看不记是拿不到分的。数据访问模式 17 年考过，是两种访问模式的对比，值得关注。

3.2.3.1.数据访问模式（重点★★★★★）

这里没做过 Java 开发的同学可能不明白这些模式的含义，特别是 DAO 和 DTO。凯恩补充如下。

DAO 模式主要是为了把底层的数据访问操作和高层的业务逻辑隔离开。它把所有和特定数据源（如数据库）交互的具体逻辑封装在 DAO 类里。在一个图书管理系统中，有一个 Book 类表示图书信息。为了对图书数据进行操作，我们可以创建一个 BookDAO 接口，定义 addBook、deleteBook、getBookById 等方法。然后创建一个 BookDAOImpl 类来实现这个接口，在这个类中使用 JDBC（JavaDatabaseConnectivity）或者其他数据库访问技术来完成具体的数据库操作。

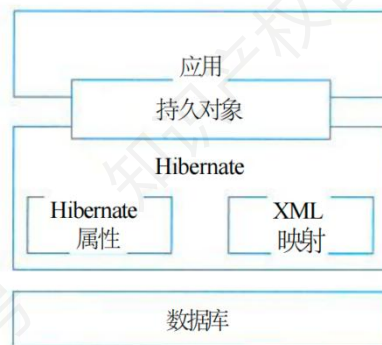
DTO 是一种用于在不同进程或者网络之间传输数据的对象容器，它里面只包含数据，不包含任何业务逻辑。可以把它想象成一个快递包裹，里面装着要传递的物品（数据），但包裹本身不具备任何处理物品的能力。比如，前端页面需要显示商品的信息。后端从数据库中查询出商品的详细信息后，可能包含很多不需要传递给前端的字段。这时，我们可以创建一个 ProductDTO 类，只包含前端需要的字段，如商品名称、价格、图片地址等，然后把这些数据封装到 ProductDTO 对象中传递给前端。

其余的内容比较好理解，见下表所示，各种数据模式会不会让你比较。比较的时候最简单的做法就是先把他们的各自核心思想默写出来。然后从性能、可维护性角度按照自己的理解进行分析即可，不需要非常准确。

模式名称	核心思想	特点
在线访问模式	应用程序与数据库保持持续连接，实时执行数据操作。	实时性强，直接操作数据库，但依赖网络和连接稳定性。
DAO（数据访问对象）	将数据访问逻辑与业务逻辑分离，通过接口封装数据库操作。	解耦数据层与业务层，支持不同数据库切换，提高代码复用性。
DTO（数据传输对象）	用于跨进程/网络传输数据的轻量级容器，无业务逻辑。	减少网络传输次数，可序列化，独立于数据源结构。
离线数据模式	数据从数据源获取后，按预定义结构存储在本地，与数据源断开连接。	离线处理数据，支持 XML/JSON 格式转换，不依赖数据库连接。
对象/关系映射（ORM）	ORM 在关系型数据库和对象之间建立映射，应用程序可以像操作对象一样操作数据库，它可以大幅减少数据访问层的开发工作，专注于实现业务逻辑。	面向对象编程，自动生成 SQL，支持复杂查询优化。

3.2.3.2.ORM 落地（Hibernate）（次重点★★★★☆☆）

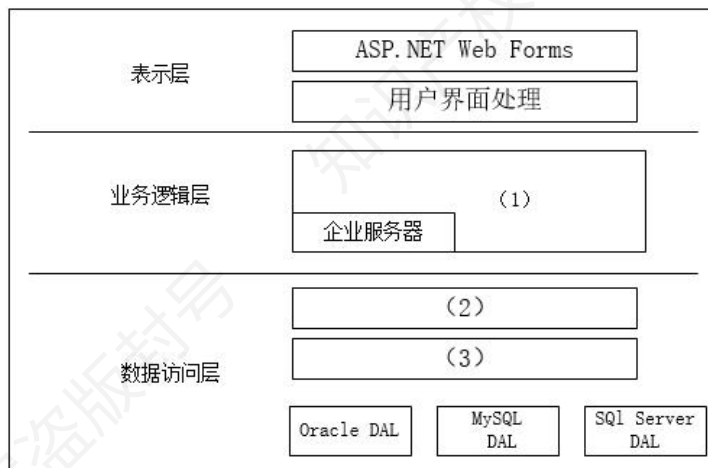
Hibernate 是一个功能强大的 ORM 框架，可以自动完成从 Java 对象到数据库表的映射。下面的架构图 23 年 11 月架构案例真题，题目形式是简答填空。



3.2.3.3.典型例题（重点★★★★★）

（17 年 11 月架构真题）某制造企业为拓展网上销售业务，委托某软件企业开发一套电子商务网站。初期仅解决基本的网上销售、订单等功能需求。该软件企业很快决定基于.NET 平台和 SQLServer 数据库进行开发，但在数据库访问方式上出现了争议。王工认为应该采用程序在线访问的方式访问数据库；而李工认为本企业内部程序员缺乏数据库开发经验，而且应用简单，应该采用 ORM（对象关系映射）方式。最终经过综合考虑，该软件企业采用了李工的建议。

随着业务的发展，该电子商务网站逐渐发展成一个通用的电子商务平台，销售多家制造企业的产品，电子商务平台的功能也日益复杂。目前急需对该电子商务网站进行改造，以支持对多种异构数据库平台的数据访问，同时满足复杂的数据管理需求。该软件企业针对上述需求，对电子商务网站的架构进行了重新设计，新增加了数据访问层，同时采用工厂设计模式解决异构数据库访问的问题。新设计的系统架构如图所示。



问：请用 300 字以内的文字分别说明数据库程序在线访问方式和 ORM 方式的优缺点，说明该软件企业采用 ORM 的原因。

此题直接考察数据访问方式的对比，前面的表格里提到过在线访问和 ORM，自行组织语言就可以作答。

答：数据库程序在线访问方式优点包括性能较优、能处理复杂查询；缺点是要求程序员懂 SQL 且修改维护相对困难。ORM 优点有降低学习和开发成本、无需编写 SQL、减少代码量和降低 SQL 质量带来的影响；缺点是处理复杂查询困难且性能不如直接 SQL。选择 ORM 的主要考虑是程序员缺乏数据库经验、避免 SQL 质量问题和降低学习成本，且不用担心性能影响。

问：请用 100 字以内的文字说明新体系架构中增加数据访问层的原因。请根据题干图片所示，填写图中空白处（1）-（3）。

第一问设置数据访问层的原因和设计层次架构的原因本质上是一样的，但是回答的时候要针对数据访问这一特定场景进行拓展。第二问别看就三空，但是因为简答填空，实际上答案会五花八门。第一空因为它处在业务逻辑层，所以业务规则处理容易写出，但是第二空和第三空容易写成数据访问接口数据访问实现。然而出题人的意思是想让你设计一个工厂类来产出不同数据库的访问模式（把原来的 ORM 进行改造支持更多的数据库类型）。

答：增加数据访问层的原因是为了实现数据与业务逻辑的分离，提高系统的可维护性、扩

展性和灵活性。数据访问层负责处理数据库操作，封装数据访问细节，使业务逻辑层与具体的数据存储技术解耦，简化开发流程，同时也方便对数据访问进行统一管理和优化，从而提升系统的整体性能和可靠性。

(1) 执行业务逻辑/业务组件/业务构件 (2) 数据访问接口层/DAL 接口 (3) 工厂层/DAL 工厂/数据访问工厂

3.3.物联网层次架构设计（次重点★★★★☆）

这一节案例主要以填空题形式出现，特别爱考察 MQTT 协议的相关内容。

首先，我们从层次架构的角度来看，物联网各个层次可以怎么设计。这个也是书本上的内容啊，需要有个初步的印象，简单可以分成感知层、网络层和应用层。

层次	介绍	设备
感知层	物联网的基础，用于识别物体、采集信息，解决人类世界和物理世界的获取问题。先通过传感器等设备采集外部物理世界的信息，再通过短距离传输技术传递数据。	传感器、RFID 标签和读写器、二维码标签和识读器、摄像头、GPS、M2M 终端、传感器网关等
网络层	作为纽带连接感知层和应用层，功能是传递信息和处理信息，负责将感知层获取的信息安全可靠地传输到应用层，并根据不同应用需求进行信息处理。	通信网、互联网、网络管理中心；接入网（光纤接入、无线接入等各类接入方式）、传输网（电信网、广电网等）；3G/4G 通信网络、IPv6 等新技术；路由器、交换机、网关等网络互联设备
应用层	位于物联网三层结构中的最顶层，实现广泛的行业智能化应用，与行业技术深度融合，通过对感知层采集数据进行计算、处理和知识挖掘，从而实现对物理世界的实时控制、精确管理和科学决策。	应用程序层相关应用（如智能操控、安防、远程医疗等）；终端设备层相关终端产品；物联网中间件、云计算

3.3.1.物联网设备接入协议：MQTT、CoAP、HTTP（重点★★★★☆）

物联网系统和普通 Web 系统不太一样。普通 Web 系统面对的主要是浏览器、小程序、App，用户点一下，系统响应一下，典型方式是 HTTP 请求。但是物联网系统面对的是大量

设备，比如温湿度传感器、水表、电表、摄像头、车载终端、工业设备等。这些设备的特点是：数量多、性能弱、网络不稳定、上报频率高、单条数据比较小。

所以物联网设备接入时，协议层要做特殊的设计。常见的物联网设备接入协议主要有 MQTT、CoAP 和 HTTP。

我们先看 HTTP 协议。它的优点是通用、简单、生态成熟，开发人员容易上手。比如设备注册、设备配置查询、固件下载、低频数据上传，都可以使用 HTTP。但是 HTTP 也有问题。它更适合请求一次响应一次。如果大量设备频繁上报数据，比如 10 万台设备每 5 秒上报一次温度、湿度、电量和运行状态，那么每次都走 HTTP 请求，连接、请求头、认证、服务端接口处理都会带来比较大的开销。

这时候 MQTT 就比较合适。MQTT 是一种轻量级的发布订阅协议。它的特点是设备和平台之间可以保持长连接，设备把数据发布到某个主题，平台侧订阅这个主题，就可以收到数据。设备端不需要关心后面到底有几个业务系统在处理数据，平台也可以方便地增加告警服务、监控服务、数据分析服务。所以看到“大量设备”“频繁上报”“弱网环境”“小数据包”“平台需要下发控制命令”这些关键词，就要优先想到 MQTT。

CoAP 则更偏向资源受限设备。比如一些低功耗传感器、嵌入式设备、受限网络环境下的终端，设备本身算力、内存、电量都比较有限，这类场景可以考虑 CoAP。它可以理解为面向受限设备的轻量级 Web 协议。

对比维度	HTTP	MQTT	CoAP
典型模型	请求/响应	发布/订阅	请求/响应
典型连接方式	短连接或长连接	长连接	通常更轻量
适合场景	普通接口调用、低频上传、配置查询	高频数据上报、设备状态监控、远程控制	低功耗、资源受限设备
主要优点	通用、简单、生态成熟	轻量、低带宽、适合海量设备	报文小、开销低
主要问题	高频上报时开销较大	需要 Broker，Topic 要设计好	生态和使用普及度不如 HTTP、MQTT
典型应用	设备注册、配置查询、管	智慧农业、车联网、工业物联	低功耗传感器、受限网络设

	理接口	网、设备监控	备
--	-----	--------	---

3.3.2.MQTT 协议（重点★★★★★）

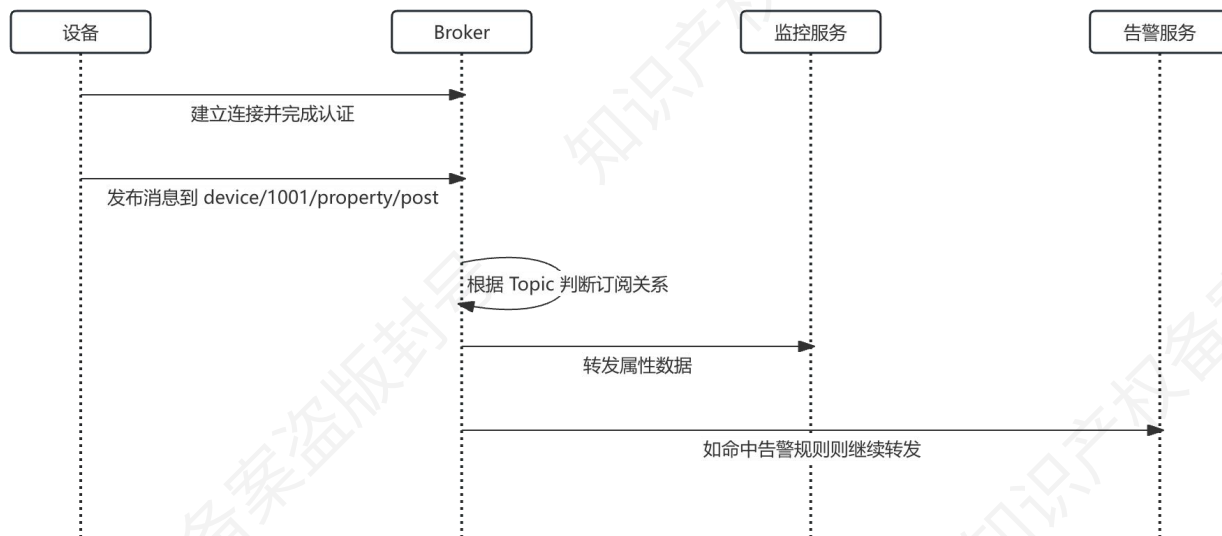
这个小节，我们深入探索一下 MQTT 协议。前面我们提到，传统 HTTP 调用是客户端直接请求服务端。比如设备要上传温度，就调用平台的上传接口。平台要返回什么，就直接返回。这个模型简单，但设备和平台接口之间关系比较直接。

而 MQTT 不是。在 MQTT 里，设备不直接调用某个业务服务，而是先连接到 MQTT Broker（消息代理服务器）。设备把消息发布到某个 Topic，平台侧服务订阅这个 Topic。只要 Topic 对上，Broker 就会把消息转发过去。比如一个温室大棚里有温度传感器，它每 5 秒上报一次温度。设备只管把温度数据发布到：`greenhouse/001/temperature`

监控服务可以订阅这个 Topic，用来展示实时温度。告警服务也可以订阅这个 Topic，用来判断温度是否过高。数据分析服务也可以订阅这个 Topic，用来做趋势分析。设备端不需要知道后面到底有几个系统在消费数据。以后平台新增一个大屏服务，也只要订阅这个 Topic，不需要修改设备程序。这就是发布订阅的价值。

3.3.2.1.MQTT 中的 Broker

Broker 可以理解为 MQTT 的消息服务器。在 MQTT 架构里，设备不会像 HTTP 那样，直接把请求发给某个业务接口；而是先与 Broker 建立连接，然后把消息发布到某个 Topic，再由 Broker 根据 Topic 把消息转发给对应的订阅者。



Broker 主要做 5 件事。

Broker 能力	说明	解决的问题
连接管理	物联网系统可能有几万、几十万甚至更多设备， Broker 需要维护这些设备的长连接	解决海量设备接入和在线连接维护问题
消息接收与转发	设备上报数据后， Broker 根据 Topic 将消息转发给对应的订阅服务	解决设备消息路由和业务系统解耦问题
认证鉴权	通过设备密钥、用户名密码、证书、 ACL 等机制，控制设备能否接入，以及能发布或订阅哪些 Topic	解决设备接入安全和 Topic 权限控制问题
会话管理	设备经常会断线重连， Broker 需要处理设备在线状态、订阅关系、离线消息等	解决设备离线、重连和订阅状态保持问题
集群扩展	当设备数量很大、单台 Broker 无法承载时，通过 Broker 集群提升并发连接数和消息吞吐能力	解决高并发连接和高吞吐扩展问题

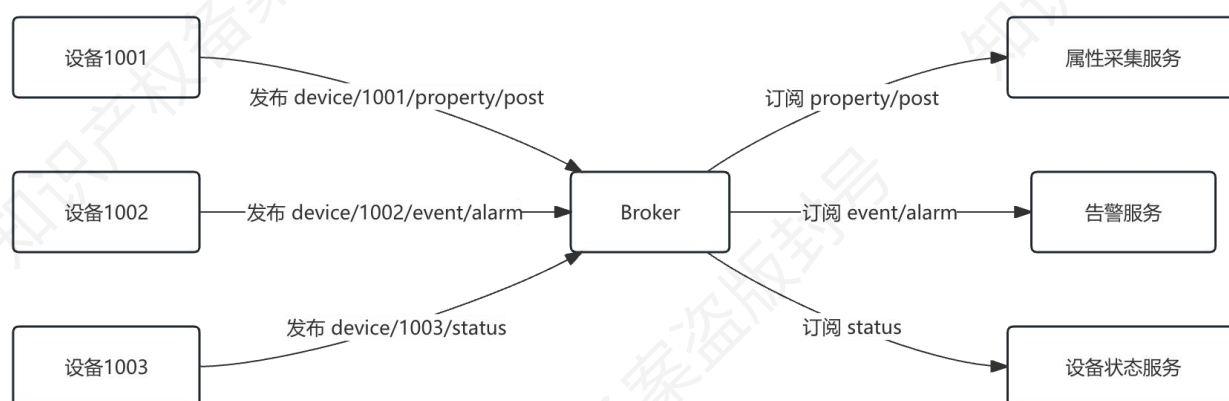
3.3.2.2.MQTT 中的 Topic

Topic 是 MQTT 中用来做消息分类和路由的主题。可以把它理解成 消息地址，也可以理解成消息标签。设备把消息发出来时，不是随便发，而是要发到某个 Topic 上。订阅者想接收消息，也不是接收全部，而是订阅自己关心的 Topic。前面也提到，Broker 正是根据 Topic

来判断：这条消息该转发给谁。可以把它理解成消息地址。比如：

device/1001/property/post, device/1001/event/alarm, device/1001/command/down

第一个表示设备 1001 上报属性数据。第二个表示设备 1001 上报告警事件。第三个表示平台给设备 1001 下发控制命令。如下图所示。可以看到，设备 1001 上报属性数据，会被转给属性采集服务，设备 1002 上报告警，会被转给告警服务，设备 1003 上报状态，会被转给状态服务。



3.3.2.3.MQTT 中的 QoS

QoS 是 MQTT 里非常重要的概念，表示消息服务质量。简单说，就是这条消息到底要不要保证送达，以及用多大代价保证送达。MQTT 常见 QoS 有三个等级。这个案例中考察过。

QoS 等级	含义	适合场景
QoS 0	最多一次，发了就算，不保证一定送达	高频采集数据，比如温度、湿度、心跳
QoS 1	至少一次，保证送到，但可能重复	告警事件、关键状态变化、执行结果
QoS 2	只有一次，可靠性最高，但开销也最大	极少量非常关键的消息

QoS 0 最好理解，就是“发了就算”。比如温度每 5 秒上报一次，偶尔丢一条问题不大，因为下一条马上又来了。QoS 1 是“至少送到一次”。它适合比较重要的数据，比如设备告警、

状态变化、控制命令执行结果。它的问题是可能重复，所以后端系统要做幂等处理。比如根据消息 ID、设备 ID、采集时间、业务流水号去重。QoS 2 可靠性最高，但协议交互更复杂，开销更大。高频数据全部用 QoS 2，系统压力反而会很大。一般只有少量非常关键、不能重复处理的消息才考虑 QoS 2。

3.3.3.典型例题（重点★★★★★）

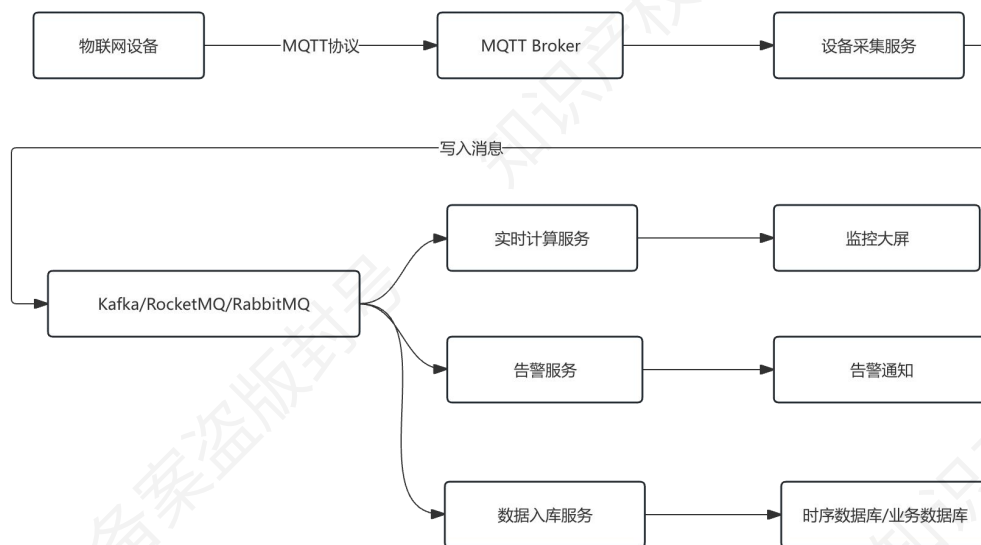
某市计划建设智慧农业物联网平台，接入温室大棚、灌溉设备、土壤传感器、气象站等终端设备。系统预计接入 10 万台设备，设备分布在多个乡镇，部分区域网络质量较差。设备需要每隔 5 秒上报温度、湿度、土壤墒情、电量和运行状态等数据。当温度过高或土壤缺水时，平台需要及时触发告警。运维人员还需要通过平台远程下发灌溉阀门开关指令。项目组提出三种设备接入方案：HTTP、CoAP 和 MQTT。

问：该场景下更适合采用哪种协议？请说明原因。

答：该场景更适合采用 MQTT。MQTT 是一种轻量级发布订阅协议，适合物联网设备接入和小数据频繁上报。设备端可以与 MQTT Broker 建立长连接，并按照约定 Topic 上报温度、湿度、土壤墒情、电量和运行状态等数据。平台侧服务订阅相关 Topic 后即可获取设备数据，实现设备和业务系统之间的解耦。相比 HTTP，MQTT 通信开销更小，更适合大量设备、高频上报、低带宽和弱网络场景。同时 MQTT 支持平台向设备下发控制命令，适合远程开关灌溉阀门等控制类业务。通过 QoS、心跳检测、断线重连、TLS 加密和 Topic 权限控制等机制，可以进一步提升消息可靠性和设备接入安全性。

问：请设计该智慧农业平台的典型物联网数据上报架构。

答：可以采用“设备/边缘网关 + MQTT Broker + 规则引擎 + 消息队列 + 时序数据库 + 监控告警平台”的架构。



温湿度传感器、土壤传感器、气象站等设备通过 MQTT 协议接入 Broker，并按照约定 Topic 上报采集数据。对于现场协议不统一或网络不稳定的区域，可以通过边缘网关完成协议转换、数据汇聚、本地缓存和断点续传。

MQTT Broker 负责设备连接管理、消息接收、Topic 路由和权限控制。后端采集服务订阅设备上报 Topic，将数据送入规则引擎和消息队列。规则引擎负责数据解析、格式转换和阈值判断；消息队列用于削峰填谷，避免瞬时大量数据直接冲击后端服务和数据库。

对于温度、湿度、土壤墒情等高频时序数据，可以写入时序数据库。设备档案、告警记录、命令流水等业务数据可以写入关系型数据库。设备最新状态可以写入 Redis，供监控大屏和移动端快速查询。

当平台需要下发灌溉控制命令时，由设备控制服务向设备订阅的命令 Topic 发布消息。设备收到命令后执行操作，并上报执行结果。平台根据执行结果更新命令状态，实现远程控制闭环。

问：该系统中不同类型的数据应该如何选择 MQTT 的 QoS 等级？

答：对于温度、湿度、土壤墒情等高频采集数据，可以采用 QoS 0。因为这类数据上报频率高，偶尔丢失一两条影响不大，下一次采集数据很快又会上报。采用 QoS 0 可以降低通信

开销，提高系统吞吐能力。

对于设备告警、关键状态变化、灌溉阀门执行结果等重要消息，可以采用 QoS 1。QoS 1 能保证消息至少送达一次，但可能出现重复消息，所以后端要通过消息 ID、业务流水号、设备编号加采集时间、数据库唯一约束等方式做好幂等处理。

对于极少量非常关键且不允许重复处理的控制类消息，可以考虑 QoS 2。但 QoS 2 开销较大，一般不适合大规模高频数据上报，只适合少量关键消息。

问：如果部分乡镇网络不稳定，经常出现设备离线和数据补传问题，系统应如何优化？

答：可以在现场部署边缘网关，由边缘网关接入本地传感器并统一上报平台。边缘网关负责协议转换、数据汇聚、本地缓存和断点续传。当外网中断时，网关先将采集数据缓存在本地；网络恢复后，再按照采集时间顺序补传到平台。设备端和网关端应支持断线重连机制，并通过心跳检测判断设备在线状态。平台侧对补传数据要携带设备编号、采集时间和消息 ID，入库时通过唯一约束或幂等逻辑避免重复写入。同时，平台需要对设备在线率、消息延迟、Broker 连接数、消息积压量、数据补传量等指标进行监控。一旦出现大面积设备离线或消息积压，应及时触发告警，通知运维人员处理。

问：HTTP、CoAP、MQTT 三种协议分别适合哪些场景？

答：HTTP 适合普通 Web API、设备注册、配置查询、低频数据上传等场景，优点是通用性好、开发简单、生态成熟。如果设备数量不大、上报频率不高，用 HTTP 完全可以满足需求。CoAP 适合低功耗、低资源、受限网络中的嵌入式设备，特别是设备算力、内存、电量都比较有限，并且希望通信报文开销尽量小的场景。MQTT 适合大量物联网设备接入、频繁小数据上报、设备状态监控和平台远程控制等场景，尤其适合弱网、低带宽和需要发布订阅解耦的系统。

4.上篇-云原生架构设计（超级重点★★★★★）

这一章属于比较重点的章节，云原生微服务也放在这里进行讨论。目前案例还没出过真题。特别这里的概念类的简述和对比值得关注。书上说了很多，其实用云原生的主要目的就是减少运维成本，用金钱换省事，所以它常常和运维联系在一起，历年真题论文中考过 Devops 相关内容，这个参考论文押题做练习。

4.1.云原生架构原则（重点★★★★★）

云原生架构的必要条件如下所示。这里唯一需要注意的是弹性原则和韧性原则的区别。乍看好像都是为了针对业务来做资源伸缩，保障系统稳定运行。实际上弹性侧重“资源动态适配业务需求”，韧性侧重“故障场景下的系统保护”，记住这一点即可，防止案例让你比较。

原则	描述
服务化原则	项目规模大时拆分服务，提升复用性与流量治理能力
弹性原则	系统自动伸缩，适配业务需求
可观测性原则	通过日志、链路跟踪等掌握系统状态
韧性原则	抵御软硬件异常，保障系统稳定
全流程自动化原则	自动化交付提升效率、降低风险
零信任原则	基于身份访问控制，替代传统网络边界安全
持续演进原则	架构适应技术与业务变化，渐进式升级

4.2.云原生架构模式（重点★★★★★）

这里主要提到 7 种云原生架构模式。这些架构在实际场景中往往组合使用。案例中一般会给出实际场景，让你判断属于哪种模式，不太会让你默写概念。看到名字你要想到它们的内涵。

这里多说一句，下表所列的架构模式，极有可能在论文中出现。实际上假如不严格定义的话，服务化架构，无服务架构，分布式事务，事件驱动架构都已经考过，那么剩下的这些主题都有可能考论文。

架构模式	核心定义（更完整说明）
服务化架构模式 （SOA/微服务/ 小服务）	按业务域或功能模块拆分应用，每个模块作为独立服务运行，通过接口契约（API/协议）进行交互。常结合 DDD（领域驱动设计）进行服务边界划分，用 TDD 提升质量，并通过容器化与 CI/CD 管理快速迭代。优点是灵活扩展、独立部署、降低耦合；缺点是分布式环境下治理成本高。典型场景：电商订单、支付、库存拆分为微服务。
Mesh 化架构模式 （ServiceMesh）	将中间件能力（RPC 通信、负载均衡、限流熔断、安全认证等）从业务进程中剥离，由独立的 Sidecar/Proxy 组件（Mesh 进程）统一处理。这样业务代码只需专注逻辑实现，而网络通信、服务治理、可观测性由 Mesh 负责。优点是统一治理、解耦复杂逻辑，缺点是引入额外性能开销。典型实现：Istio、Linkerd。
Serverless 模式	应用无需管理底层服务器，开发者只需编写业务逻辑代码，由云平台自动完成部署、运行、弹性伸缩和关闭。应用一般通过事件或流量触发执行，按调用次数或运行时间计费。优势是极大降低运维成本、按需伸缩，劣势是冷启动延迟和厂商锁定风险。典型场景：图片处理、日志分析、IoT 数据采集。
存储计算分离模式	将计算层与存储层解耦：计算节点保持无状态，数据全部存储在分布式存储/云存储中。有状态服务通过写前日志（WAL）+快照（Snapshot）机制实现恢复。优点是弹性伸缩、成本优化；缺点是计算与存储分离带来延迟问题。典型应用：Snowflake、TiDB、云数仓。

分布式事务模式	在跨数据库/跨服务的事务中保持数据一致性，常见模式有： XA 模式：两阶段提交，强一致，但性能差。 消息最终一致性：通过消息队列异步保证，性能好但一致性延迟。 TCC 模式：应用层显式 Try/Confirm/Cancel，灵活但侵入性强。 SAGA 模式：将长事务分解为本地事务+补偿操作，适合复杂业务流程。 SEATAAT 模式：通过代理数据库操作实现自动回滚，性能较好，但受限于数据库类型。
可观测架构	通过三大核心能力实现对系统运行状态的全面感知： Logging：多级日志，记录系统行为和错误。 Tracing：分布式调用链路追踪，快速定位性能瓶颈。 Metrics：系统指标采集与量化（QPS、响应时间、并发度等）。 结合 SLO/SLA 定义服务目标，用于监控、报警和运维优化。典型工具：Prometheus、Grafana、ELK、Jaeger。
事件驱动架构（EDA）	系统以事件为核心进行交互，事件作为独立数据单元，包含 Schema，可被生产者发布、消费者订阅。具备 QoS 保证和失败补偿机制。优点是服务解耦、异步处理、可扩展；适合高并发、异步业务场景，如订单完成触发发货、数据库变更触发缓存更新。 常用技术：Kafka、Pulsar、EventBridge。

4.3.不好的实践（次重点★★★★☆☆）

假如说云原生架构是正面教材，那么你不照着他的原则来就是反面教材，也就是所谓的反模式、不好的实践或者说糟糕的做法，书本提到了三点。这里最简单的考法就是列出下表，让你简答填空，做过简单了解即可。
--

云原生架构反模式	问题描述	解决方案
庞大的单体应用	缺乏依赖隔离，存在代码耦合、模块间接口缺乏治理、不同模块开发发布进度难以协调、单个模块不稳定影响整个应用等问题	通过服务化进行适度拆分，梳理聚合根，明确服务模块边界和模块间接口定义，使组织关系和架构关系匹配
单体应用“硬拆”为微服务	过度服务化拆分会致使新架构与组织能力不匹配，影响架构升级效果。具体表现有小规模软件过度拆分、服务间数据依赖、服务拆分导致性能下降等	合理评估拆分粒度，充分考量组织能力与业务实际需求，避免过度拆分，优化服务间的数据交互设计，提升整体性能

缺乏自动化能力的微服务	软件规模增大时，人工处理开发测试运维等工作会造成交付时间变长、风险提升、运维成本增加等问题	建立完善的自动化能力，涵盖自动化测试、发布、环境管理等，以适应复杂度提升的需求
-------------	---	---

4.4.容器技术（重点★★★★★）

容器作为标准化软件单元，它将应用及其所有依赖项打包，使应用不再受环境限制，在不同计算环境间快速、可靠地运行。容器技术只要出题肯定会和虚拟机进行比较。另外单纯的容器和容器编排的区别也是重点考察的内容。

4.4.1.容器和虚拟机对比（重点★★★★★）

容器和虚拟机的对比如下表所示，其中标注灰色的运行原理，资源隔离方式，启动速度，资源占用，隔离性的不同之处都要给我背出来（不需要一字不差）。

对比维度	容器	虚拟机
运行原理	基于操作系统层面的虚拟化，共享宿主机内核，直接运行在操作系统之上	通过硬件虚拟化技术，模拟出完整的硬件环境，运行独立的操作系统
资源隔离方式	利用 cgroups 进行资源限制（如 CPU、内存、磁盘 I/O 等），通过 namespace 实现命名空间隔离（如进程、网络、文件系统等），不同容器共享内核但彼此隔离	依靠硬件虚拟化技术，在不同虚拟机实例间实现完全的资源隔离，每个虚拟机有独立的内核和硬件资源
启动速度	启动非常快，一般在秒级。因为无需启动完整操作系统，仅需加载应用及其依赖	启动相对较慢，通常需要数十秒到数分钟，要加载整个操作系统
资源占用	占用资源少，仅需运行应用所需的资源，因为共享宿主机内核	资源占用多，每个虚拟机都有独立的操作系统和完整硬件资源模拟，开销大
应用移植性	移植性好，将应用及其依赖打包在镜像中，环境一致性高，可在不同支持容器的环境中快速部署	移植有一定难度，受虚拟硬件环境和操作系统影响，迁移时需考虑兼容性

隔离性	隔离性相对较弱，虽能实现进程、网络等隔离，但共享内核，存在一定安全风险（如内核漏洞可能影响多个容器）	隔离性强，每个虚拟机相互独立，一个虚拟机出现问题基本不影响其他虚拟机
管理复杂度	相对简单，主要管理容器镜像、容器生命周期（创建、启动、停止、删除等），通过编排工具可实现集群管理	管理复杂，需管理多个虚拟机的操作系统安装、配置、补丁更新等，以及虚拟硬件资源的分配和管理

4.4.2.容器和容器编排技术（重点★★★★★）

容器和容器编排技术并非同一类事物的对比，容器是一种轻量级、可移植的应用打包和运行环境，而容器编排技术是用于管理和协调多个容器的技术，它们之间对比如下。它们的定义是最关键的部分，一定要记忆。

对比维度	容器	容器编排技术
定义	一种将应用及其依赖项打包成独立单元的技术，可在不同环境中运行，共享宿主主机内核	用于自动化部署、管理和扩展容器化应用程序的技术，能管理多个容器组成的集群
核心功能	隔离应用运行环境，实现应用的快速部署和迁移	容器集群管理、资源调度、服务发现、负载均衡、自动伸缩、故障恢复等
代表工具	Docker	Kubernetes（k8s）
部署难度	部署单个容器相对简单，通过拉取镜像并启动容器即可	部署和配置容器编排系统较为复杂，涉及多个组件的安装、配置和集群初始化
适用场景	适合部署单一、简单的应用，或作为微服务架构中单个服务的运行载体	适用于大规模、复杂的容器化应用集群管理，如大型互联网应用、企业级分布式系统，这些系统需要高效的资源调度、自动伸缩和服务治理等功能
管理粒度	主要管理单个容器的生命周期，如创建、启动、停止、删除，以及容器内应用的运行状态	从集群层面管理多个容器，包括容器的分组、调度到不同节点、监控容器的健康状态、动态调整容器数量等

4.4.3.容器运维指令（重点★★★★★）

有一年系分案例考过 docker 指令，所以还是要有所了解，至少有个印象。

Docker 常用指令	
dockerps	查看当前所有运行的容器
dockerps-a	查看所有容器（包括停止的）
dockerrun[选项][镜像名称][命令][参数]	启动一个容器
dockerstop[容器 ID 或名称]	停止一个运行中的容器
dockerkill[容器 ID 或名称]	强制停止一个运行中的容器
dockerrestart[容器 ID 或名称]	重启容器
dockerrm[容器 ID 或名称]	删除一个容器
dockerrm\$(dockerps-a-q)	删除所有容器
dockerlogs[容器 ID 或名称]	查看容器的日志
dockerexec-it[容器 ID 或名称]/bin/bash	进入容器内部
dockerbuild[选项]路径	构建镜像
dockerimages	查看镜像
dockerrmi[镜像 ID 或名称]	删除镜像
dockerpull[镜像名称]	拉取镜像
dockerpush[镜像名称]	推送镜像到仓库
dockerstats	查看 Docker 容器的资源使用情况
dockerinfo	查看 Docker 的信息
docker--help	查看 Docker 的帮助

Kubernetes（k8s）常用指令。

指令	说明
kubectlgetnodes	查看集群内节点信息
kubectlgetpods	查看当前命名空间下的 Pod 列表
kubectldescribepod<pod-name>	查看指定 Pod 的详细信息，用于故障排查

kubectl delete pod <pod-name>	删除指定的 Pod
kubectl get services	查看当前命名空间下的服务列表
kubectl get deployments	查看当前命名空间下的 Deployment 列表
kubectl set image deployment/<deployment-name><container-name>=<new-image>	更新 Deployment 中容器的镜像
kubectl logs <pod-name>	查看指定 Pod 的日志

4.4.4.典型例题（重点★★★★★）

（23 年 5 月系统分析师）随着嵌入式计算资源快速提升，容器技术（Docker）发挥重要作用，某公司对原有平台升级，公司将平台升级任务交给了张工，张工经过分析、调研，提出在嵌入式操作系统平台上采用容器技术的升级方案，但该方案引发了争议。

问：争论焦点是采用容器技术还是虚拟机（VM）技术。李工指出由于容器技术共享主机内核能像虚拟机一样完全隔离，系统存在安全问题；如果采用虚拟机技术除了满足需求外，还保证了系统的安全和稳定，会上领导根据系统升级的初衷选择了张工的升级方案，请用 300 字以内的文字说明容器技术和虚拟技术的含义，并简要论述公司领导采纳容器技术的原因。

这里实际就是讨论容器和虚拟机的区别，前面小节提到的表格这里可以排上用场。

答：容器技术是一种轻量级的隔离技术，多个容器共享主机内核，每个容器运行一个或一组应用，具备自己的文件系统、运行环境等，实现应用的快速部署和隔离。虚拟机技术通过虚拟化软件在物理主机上模拟出多个独立的虚拟计算机，每个虚拟机有自己的操作系统、硬件资源，实现完全隔离。

容器技术相比虚拟机更加轻量化，资源占用少，启动速度快，能更高效利用硬件资源，满足系统升级中对资源利用优化的需求。且在当前技术发展下，安全问题可通过安全策略、安全工具等手段进行防护，在满足业务需求的同时，能以更低成本、更高效率完成部署，契合系统

升级初衷。

问：下表给出了虚拟技术和容器技术的性能对比表，请根据下面的（a）~（h）的 8 个性能指标；判断这些指标属于哪类对比项，补充完善 3-1 的（1）-（8）的空白处。

（a）分钟级、（b）包含 GuestOS，G 量级以上、（c）跨操作系统平台迁移、（d）CPU 与内存按核、按 G 分配（e）毫秒级、（f）Cgroups，进程级别、（g）VM 伸缩，CPU/内存手动伸缩、（h）实例自动伸缩、CPU 内存自动在线伸缩。

对比项	虚拟机技术	容器技术
镜像大小	①	仅包含运行的Bin/Lib,M量级
资源要求	②	CPU与内存按单核、低于G量级分配
启动时间	③	④
可持续性	跨物理机迁移	⑤
弹性伸缩	⑥	⑦
隔离策略	操作系统、系统级别	⑧

此题考察虚拟机和容器技术的对比，本质上和第一问没有区别，见容器那一小节的讨论。

答：①b②d③a④e⑤c⑥g⑦h⑧f

5.上篇-面向服务架构（超级重点★★★★★）

面向服务架构（SOA），无论是架构还是系分都是超级重点的存在，属于不看必挂的内容。隔三差五出一道大题，有的是简答对比，有的是选词填空，出题形式特别多样。这一章最基本的还是要掌握 SOA 所谓的一个参考架构，它的各个组成部分的作用是什么，SOA 和其他技术，特别是微服务的对比等。

5.1.什么是 SOA（重点★★★★★）

有个印象即可，应付概念简答题。

从应用的角度定义，可以认为 SOA 是一种应用框架，它着眼于日常的业务应用，并将它们划分为单独的业务功能和流程，即所谓的**服务**。SOA 使用户可以构建、部署和整合这些服务，且无需依赖应用程序及其运行平台，从而提高业务流程的灵活性。这种业务灵活性可使企业加快发展速度，降低总体拥有成本，改善对及时、准确信息的访问。SOA 有助于实现更多的资产重用、更轻松的管理和更快地开发与部署。

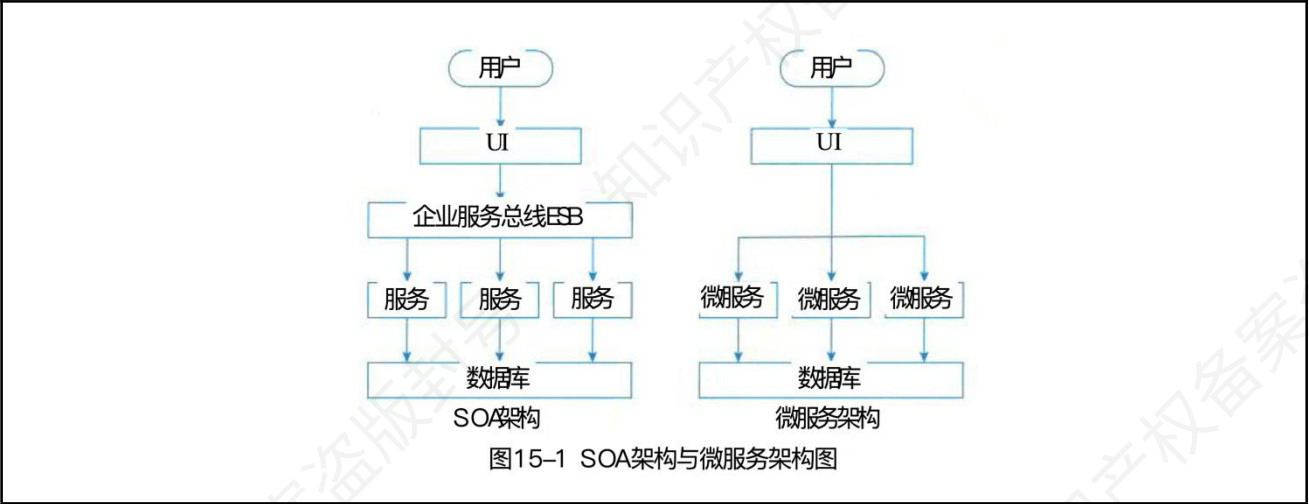
从软件的基本原理定义角度，认为 SOA 是一个组件模型，它将应用程序的不同功能单元（称为**服务**）通过这些服务之间定义良好的接口和契约联系起来。接口是采用中立的方式进行定义的，它应该独立于实现服务的硬件平台、操作系统和编程语言。这使得构建在各种各样的系统中的服务可以以一种统一和通用的方式进行交互。

5.2.SOA/微服务/单体架构对比（重点★★★★★）

这是书本上新增的内容，最近没有考过，值得重点关注。最近考过微服务和单体架构的区别，那么 SOA 和微服务，SOA 和单体就是大概率考的方向。下表是微服务和 SOA 的区别，

这里分为 5 个维度，应付考试没问题了。

特性	微服务	SOA（面向服务的架构）
服务粒度	服务粒度极为精细，每个微服务通常仅负责单一的业务功能，例如在电商系统中，商品展示、订单处理、支付等功能可分别由独立的微服务实现。这种细粒度的划分使得每个服务专注于特定任务，便于开发、维护和独立扩展。	服务粒度相对较大，一个服务可能涵盖多个业务功能。比如在企业资源规划（ERP）系统中，一个服务可能同时包含采购管理和库存管理的相关功能，其整合程度更高。
通信方式	主要使用 HTTPRESTfulAPI 进行通信，这种方式具有良好的语言和平台无关性，不同语言开发的微服务之间可以轻松交互，并且 RESTful 风格简单易用，便于与各种客户端进行集成。	通信方式更为多样化，除了 HTTP 外，还常使用 SOAP（简单对象访问协议）等协议。SOAP 具有严格的消息格式和标准，适用于对数据交互规范性和安全性要求较高的企业级应用场景。
数据管理	每个微服务通常拥有自己独立的数据库，形成数据源唯一的特性，确保数据的独立性和安全性。不同微服务的数据库可以根据自身业务需求选择合适的数据库类型，如关系型数据库、非关系型数据库等。	数据管理方式较为灵活，可能采用多个服务共享同一个数据库的方式，或者通过企业服务总线（ESB）进行数据交换和整合，实现不同服务间的数据交互和协调。
架构风格	遵循去中心化的架构风格，不存在中心化的服务总线。各个微服务独立部署、运行和管理，通过轻量级的通信机制相互协作，具有较高的灵活性和可扩展性。	可能包含中心化的企业服务总线（ESB），ESB 充当服务间通信和数据交换的枢纽，负责处理服务的注册、发现、路由以及消息转换等功能，在一定程度上增强了服务间的集成和管理能力。
复杂性	由于服务的独立性和去中心化特性，微服务架构的系统复杂性相对较低。每个微服务可以独立开发、测试和部署，降低了整体系统的耦合度。但同时，微服务数量众多也会带来服务治理、监控和运维等方面的挑战。	由于服务间可能存在较强的耦合关系，并且中心化的 ESB 增加了架构的复杂性，使得 SOA 架构的系统在设计、开发和维护上相对复杂，对团队的技术能力和管理水平要求较高。



SOA 和单体架构对比，如下所示。

特性	SOA	单体架构
服务粒度	将业务功能分解为多个相对独立、粒度较细的服务，每个服务专注于特定的业务功能，可被不同应用复用，提高了业务灵活性和可扩展性。	整个应用是一个紧密耦合的整体，所有功能模块都集成在一起，难以单独拆分和复用，新增或修改功能可能影响整体。
通信方式	采用多种通信协议，如 SOAP、HTTP 等，服务之间通过标准的接口进行通信，实现不同服务间的交互和数据共享，支持分布式部署。	内部通过方法调用的方式实现模块间交互，无需复杂的外部通信协议，主要在同一应用内部完成。
数据管理	数据管理较为灵活，可以是多个服务共享数据库，也可通过企业服务总线（ESB）进行数据交换和整合，实现不同服务间的数据交互和协调。	通常使用单一数据库，所有功能模块都直接访问和操作同一数据源，数据的一致性和管理相对集中。
架构风格	属于分布式架构，各个服务可以独立部署和运行，可能存在中心化的 ESB，用于协调服务间的通信和数据交换，实现服务的集成和管理。	是集中式架构，所有功能集中在一个应用程序中，运行在同一环境下，易于开发和部署，但后期维护和扩展较困难。
故障影响	某个服务出现故障时，一般不会对其他服务产生直接影响，仅会影响依赖该服务的特定功能，故障隔离性较好。	一旦应用中的某个部分出现故障，可能会导致整个应用无法正常运行，故障影响范围大。

5.3.服务注册模式（重点★★★★★）

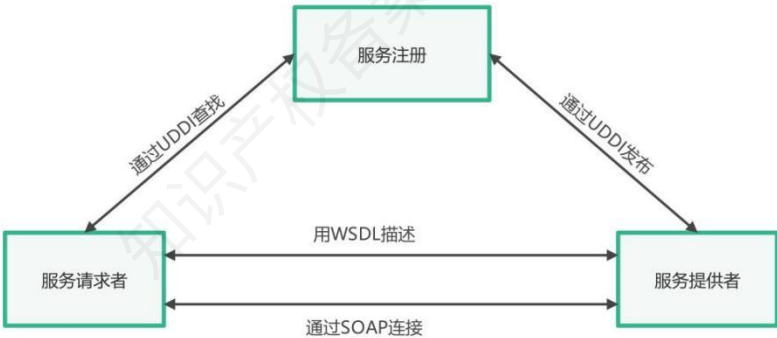
服务注册模式和 ESB 模式书本上是当作两种不同的模式分开介绍的，在真题中都考察过。

引入服务注册的根本目的，是为了解决服务提供者与服务请求者之间的寻址与发现问题。通过在服务注册中心统一发布和存储服务信息，调用方不必直接依赖具体地址，而是动态查询并获取调用方式，从而实现双方的解耦，支持服务的动态发现、负载均衡和版本演进，降低维护成本，提升系统的可扩展性与可维护性。

5.3.1.服务注册模式概念（重点★★★★★）

服务注册模式包含服务提供者、服务请求者和服务注册，它们的定义如下表所示。

角色	操作内容
服务提供者	使用 UDDI 将 Web 服务发布到服务注册中心，通过 WSDL 描述服务
服务注册中心	接收并存储服务提供者通过 UDDI 发布的服务信息，供服务请求者查找
服务请求者	借助 UDDI 在服务注册中心查找 Web 服务，依据 WSDL 了解调用方式，通过 SOAP 与服务提供者建立连接并使用服务



5.3.2.UDDI、WSDL 和 SOAP（重点★★★★★）

服务注册模式包含服务提供者、服务请求者和服务注册，它们之间通过 UDDI、WSDL 和 SOAP 三种协议协同工作。这些协议的概念、作用是必须要掌握的。如下表所示。

协议名称	概念	主要作用
------	----	------

UDDI (通用描述、发现和集成服务)	是一个用于发现和集成 Web 服务的协议标准,基于 XML 的跨平台的描述规范	提供统一的方式发布和搜索 Web 服务信息。服务提供者可在注册表注册服务,供使用者查找和连接
WSDL(Web 服务描述语言)	用于描述 Web 服务接口的 XML 格式语言	定义 Web 服务的功能、输入输出参数、绑定协议和访问端点等信息,让服务使用者了解如何调用和使用 Web 服务
SOAP (简单对象访问协议)	基于 XML 的消息传输协议,用于在 Web 服务之间进行通信	定义消息格式、编码规则和 RPC 约定,用于在应用程序之间交换结构化信息,可与多种因特网协议和格式结合使用,如 HTTP、SMTP 等

5.3.3.REST 规范/风格 (重点★★★★★)

这一块考的次数比较频繁,需要好好关注。书本上写的概念有点简略,不足以应付考试。凯恩在这里补充如下。

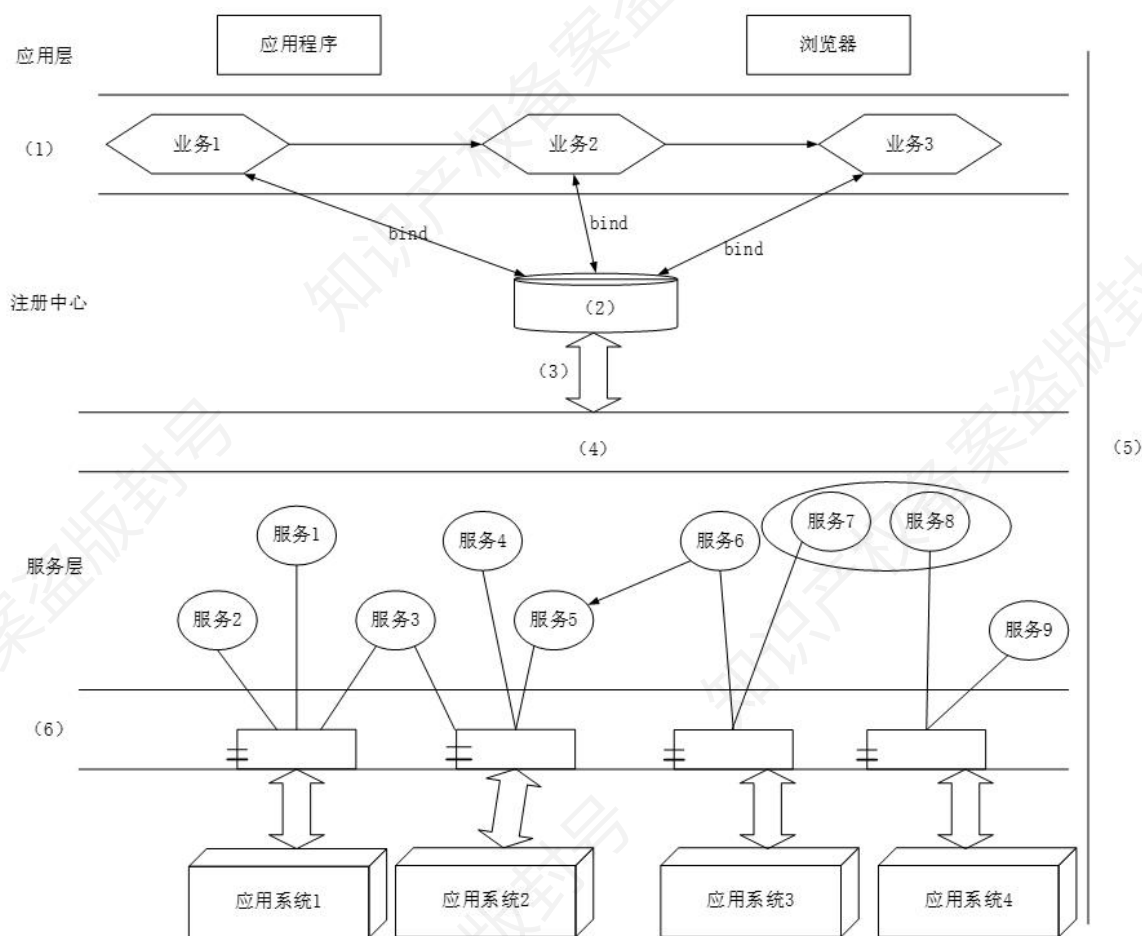
REST 全称是 RepresentationalStateTransfer, 中文意思是表述性状态转移。它首次出现在 2000 年 RoyFielding 的博士论文中。如果一个架构符合 REST 的约束条件和原则,我们就称它为 RESTful 架构。REST 本身并没有创造新的技术、组件或服务,而是更好地使用现有 Web 标准中的一些准则和约束,如下所示。

主题	内容要点
资源与 URI	资源是任何有被引用必要的事物,包括实体和抽象概念。URI 是资源唯一标识,可看作资源地址或名称。
统一资源接口	遵循统一接口原则,使用标准 HTTP 方法 (GET、PUT、POST、DELETE 等) 访问资源。
资源的表述	客户端获取的是资源的表述,资源表述形式多样,如文本可采用 html、xml、json 等格式。
资源的链接	REST 需利用超媒体概念,在表述格式中加入链接引导客户端,增强资源连通性。
状态的转移	服务端不保存客户端状态。客户端与服务端交互无状态,每次请求包含所需信息,从而可推进状态转移。

5.3.4.典型例题（重点★★★★★）

（18 年 11 月架构真题）某银行拟将以分行为主体的银行信息系统，全面整合为由总行统一管理维护的银行信息系统，实现统一的用户账户管理、转账汇款、自助缴费、理财投资、贷款管理、网上支付、财务报表分析等业务功能。但是，由于原有以分行为主体的银行信息系统中，多个业务系统采用异构平台、数据库和中间件，使用的报文交换标准和通信协议也不尽相同，使用传统的 EAI 解决方案根本无法实现新的业务模式下异构系统间灵活的交互和集成。因此，为了以最小的系统改进整合现有的基于不同技术实现的银行业务系统，该银行拟采用基于 ESB 的面向服务架构（SOA）集成方案实现业务整合。

问：基于该信息系统整合的实际需求，项目组完成了基于 SOA 的银行信息系统架构设计方案。该系统架构图如图所示：



请从 (a) ~ (j) 中选择相应内容填入上图 (1) ~ (6) 处，补充完善架构设计图。

(a) 数据层 (b) 界面层 (c) 业务层 (d) bind (e) 企业服务总线 ESB (f) XML (g)

安全验证和质量管理 (h) publish (i) UDDI (j) 组件层 (k) BPEL

(1) 填 (c) 业务层：图中该位置处于应用层下方，包含业务 1、业务 2、业务 3，主要负责处理具体业务逻辑，符合业务层的定义，所以是业务层。

(2) 填 (i) UDDI：这一行左侧写着注册中心，已经给出提示，注册中心这里的主要功能是服务的注册与发现，UDDI（统一描述、发现和集成协议）就是专门用于实现这一功能的，业务通过“bind”操作与它交互来查找服务，因此此处为 UDDI。

(3) 填 (h) publish：服务要能够被业务发现和调用，需要先在注册中心进行发布操作，publish 就是发布的意思，所以该位置应填 publish。

(4) 填 (e) 企业服务总线 ESB：从架构层次关系看，该位置处于注册中心和服务层之间，起到连接和通信的作用，企业服务总线 ESB 能实现不同服务之间的消息传递、协议转换等，符合此层的功能需求，故为 ESB。这个图也说明 ESB 和服务注册模式可以混用。

(5) 填 (g) 安全验证和质量管理：在系统架构的各个层次中，都需要对服务的安全性和质量进行管控，负责对服务调用进行安全验证以及质量管理，所以是安全验证和质量管理。

(6) 填 (j) 组件层：此层位于服务和应用系统之间，其作用是将应用系统的功能封装成可复用的服务组件，供外部系统调用，所以是组件层。

(1) (c) 业务层 (2) (i) UDDI (3) (h) publish (4) (e) 企业服务总线 ESB (5) (g) 安全验证和质量管理 (6) (j) 组件层（实际考试你写英文序号就行，不用写后面的代表的具体内容）

5.4.ESB 模式（重点★★★★★）

5.4.1.ESB 概念（重点★★★★★）

书本上把 ESB 当作是另外一种单独的，和之前提到的服务注册表模式有区别的设计模式。在引入 ESB 之前，书本花了不少篇幅来讲什么是 IBMWebsphere。但是凯恩调研了一下，这个技术底座虽然已经有二十多年的历史了，但是，相关资料奇少无比，应该不是重点。根据真题经验，这里只要掌握 ESB（书本定义为架构模式）的相关概念即可。

企业服务总线（ESB）模式由消息中间件发展而来，采用“总线”模式管理简化应用集成拓扑，基于开放标准支持应用在消息、事件和服务级别的互联互通。ESB 提供位置透明性的消息路由和寻址服务、服务注册和命名的管理功能，支持多种消息传递范型（如请求/响应、发布/订阅等）、多种可以广泛使用的传输协议、多种数据格式及其相互转换，还提供日志和监控功能。

5.4.2.典型例题（重点★★★★★）

（20 年 11 月架构真题）某银行拟将以分行为主体的银行信息系统，全面整合为由总行统一管理维护的银行信息系统，实现统一的用户账户管理、转账汇款、自助缴费、理财投资、贷款管理、网上支付、财务报表分析等业务功能。但是，由于原有以分行为主体的银行信息系统中，多个业务系统采用异构平台、数据库和中间件，使用的报文交换标准和通信协议也不尽相同，使用传统的 EAI 解决方案根本无法实现新的业务模式下异构系统间灵活的交互和集成。因此，为了以最小的系统改进整合现有的基于不同技术实现的银行业务系统，该银行拟采用基于 ESB 的面向服务架构（（SOA）集成方案实现业务整合。

问：请分别用 200 字以内的文字说明什么是面向服务架构（SOA）以及 ESB 在 SOA 中的作用与特点。

此题直接考察 SOA 的概念和 ESB 的概念，直接默写。2018 年架构也考过一模一样的问题。

答：SOA 是一个组件模型，它将应用程序的不同功能单元（称为服务）通过这些服务之间定义良好的接口和契约联系起来。接口是采用中立的方式进行定义的，它应该独立于实现服务的硬件平台、操作系统和编程语言。这使得构建在各种这样的系统中的服务可以一种统一和通用的方式进行交互。

企业服务总线（ESB）提供位置透明性的消息路由和寻址服务、服务注册和命名的管理功能，支持多种消息传递范型（如请求/响应、发布/订阅等）、多种可以广泛使用的传输协议、多种数据格式及其相互转换，还提供日志和监控功能。

5.5.SOA 设计原则（重点★★★★★）

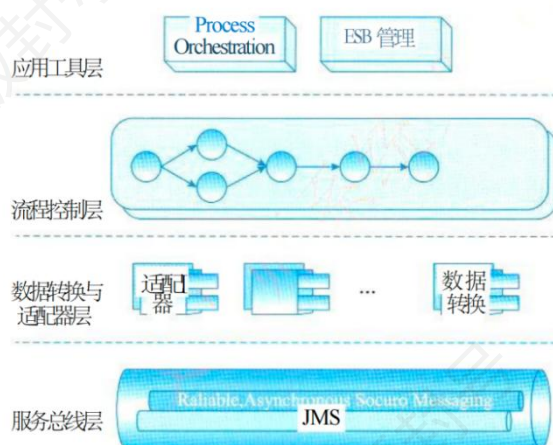
SOA 的设计需要遵循哪些原则，真题没有考过，下面的内容可以用口诀加以记忆。我们可以提取每个要点的首字，组成“无单明自粗松重互”——想象没有订单的人，自己的名字就会变得粗劣松散，失去重要客户。

特性	简要概括
无状态	避免服务请求者依赖服务提供者状态
单一实例	防止功能冗余
明确定义的接口	由 WSDL 定义，其中 WS-Policy 描述规约，XML 模式定义消息格式等
自包含和模块化	封装组件，实现功能实体独立部署、管理
粗粒度	服务数量适度，靠消息交互
松耦合性	使用者仅见接口，服务位置、实现等对其不可见
重用能力	服务具备可重用性
互操作性、兼容和策略声明	确保不同的系统、应用程序、设备或组件之间能够相互通信

5.6.典型架构（重点★★★★★）

书本上这里再介绍 SOA 的时候，把微服务也放进来作为典型架构进行讨论，这里的重点是清楚微服务的六个分类。

5.6.1.某 ESB 架构实现（重点★★★★★）



这是某 ESB 层次结构图，展示了该系统的分层架构，各层作用如下表所示。

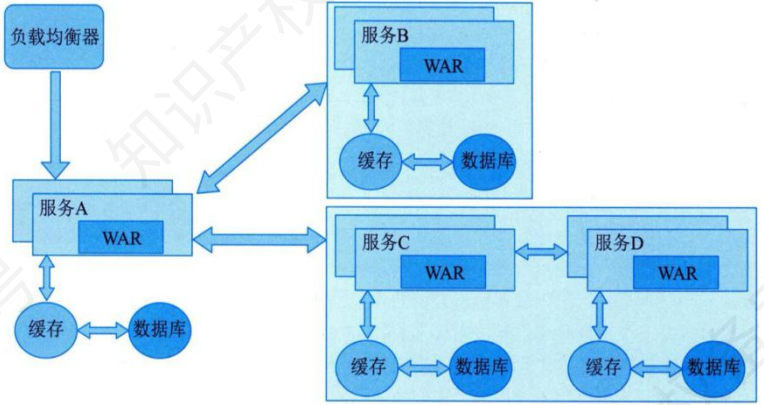
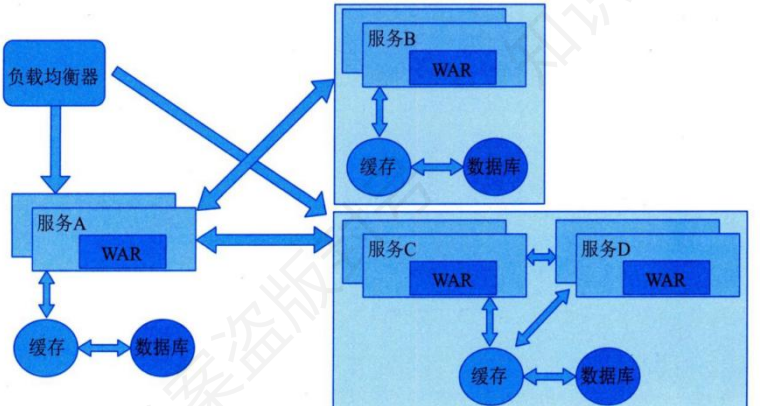
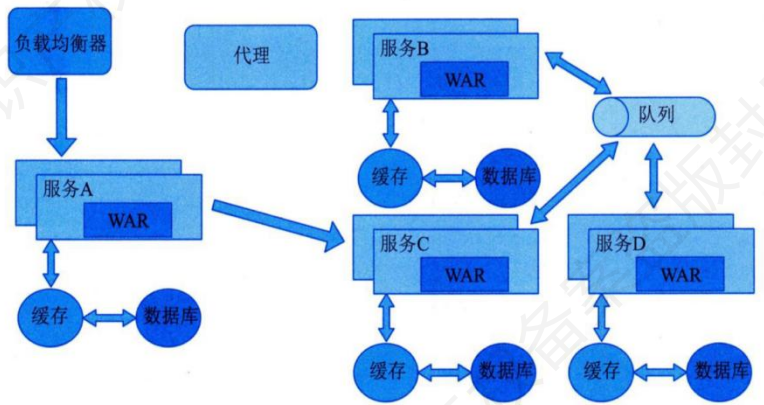
层次	作用
应用工具层	包含流程编排和 ESB 管理。流程编排负责对业务流程进行设计与协调
流程控制层	通过一系列相互连接的节点来控制业务流程的走向，对服务调用顺序、条件分支等进行管理，以确保业务流程按照预期执行。
数据转换与适配器层	有适配器和数据转换组件。适配器用于连接不同的系统或服务，实现系统间的通信交互；数据转换组件负责在不同格式的数据之间进行转换，保证数据在不同系统间能正确被识别和处理。
服务总线层	基于 JMS（Java 消息服务），提供可靠的、异步的、安全的消息传递机制，是整个 ESB 的基础通信层，为上层提供消息传输服务，使得不同的服务和系统能够通过消息进行交互。

5.6.2.微服务模式（重点★★★★★）

微服务架构将单一应用拆分成多个微服务，可单独地开发、管理和扩展每个服务。微服务架构优势包括：解耦复杂应用、独立部署、技术选型灵活、容错性好、易扩展。这一块架构展

开的不多，系分教材把微服务模式实际上分为六大类，这六大类非常有可能给你一个图例让你谈谈它的处理流程，如下表所示。

名称	定义	图示
聚合器微服务	将多个微服务的 API 组合成统一接口，整合响应后返回客户端，简化交互复杂度。	图 20-1 聚合器微服务模式示意图
代理微服务	在客户端与微服务间增加中间层，处理路由、负载均衡、安全验证等功能。	图 20-2 代理微服务模式示意图
链式微服务	微服务按顺序串联，每个服务处理部分功能并传递结果，形成完整流程。	图 20-3 链式微服务模式示意图

分支微服务	按业务子系统拆分独立微服务，通过 API 或消息队列通信，实现模块化开发。	 图 20-4 分支微服务模式示意图
数据共享微服务	提供统一数据访问接口或复制机制，支持多应用/服务共享数据。	 图 20-5 数据共享微服务模式示意图
异步消息微服务	通过消息队列异步通信，解耦服务依赖，提升系统吞吐量和响应能力。	 图 20-6 异步消息微服务模式示意图

6.上篇-系统架构设计（超级重点★★★★★）

质量属性与架构评估属于架构的超级重点章节，一旦出题就是 25 分，不容小觑，这里所有的概念背熟都不为过。25 年 5 月开始难度加大，不单单考概念还考图例，所以凯恩这里大力补充，并且给出解释。

6.1.软件架构风格（重点★★★★★）

6.1.1.概念辨析（重点★★★★★）

软件架构风格之前案例特别喜欢考，主要是概念的辨析，对比等，难度适中，只要你背了基本都能拿大分。案例简答意思对就行，所以不需要像选择那样记得那么准确，针对案例的特点，凯恩把红宝书对应的知识化解如下表所示。

一级风格	二级风格	核心概念	典型应用场景
数据流体系结构	批处理	分步执行，前一步完成后整体传递数据，无并行能力	传统数据处理、BAT 脚本
	管道-过滤器	前一步输出作为后一步输入，支持并行处理	UnixShell 程序、编译器阶段处理
调用/返回体系结构	主程序/子程序	单线程控制，模块化分层调用	早期结构化程序设计
	面向对象	数据与操作封装，通过消息传递交互	Java/C#应用程序
	客户端/服务器	资源分层（表示层/功能层/数据层），网络交互	Web 系统、数据库应用
以数据为中心体系结构	仓库	中央数据存储，独立构件操作	现代集成开发环境（IDE）、企业级数据管理系统
	黑板	分级解空间，多领域知识协同	语音识别、模式识别

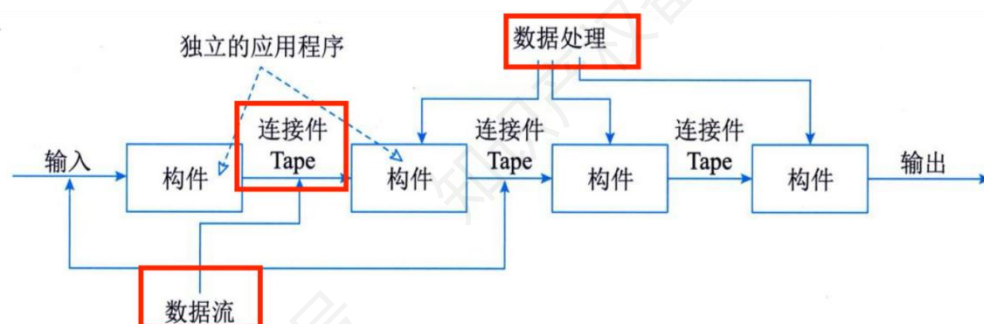
一级风格	二级风格	核心概念	典型应用场景
虚拟机体系结构	解释器	自定义语言解析执行, 支持动态扩展	游戏脚本引擎
	规则系统	基于规则的系统包括规则集、规则解释器、规则/数据选择器及工作内存	专家系统
独立构件体系结构	进程通信体系结构风格	在进程通信结构体系结构风格中, 构件是独立的过程, 连接件是消息传递	分布式系统、微服务架构
	事件驱动	隐式调用, 事件广播触发响应	UI 系统
特殊体系结构	C2	构件通过连接件分层连接, 强制顶部到底部通信	嵌入式系统、分布式应用
	过程控制	反馈循环调节物理环境状态	空调控温、汽车巡航系统

6.1.2.数据流风格（重点★★★★★）

简单理解, 数据流风格是一种软件架构风格, 它把系统抽象为一系列处理数据的独立构件, 数据以流的形式在构件间通过管道传递, 从而形成类似流水线的处理过程。

6.1.2.1.批处理体系结构风格

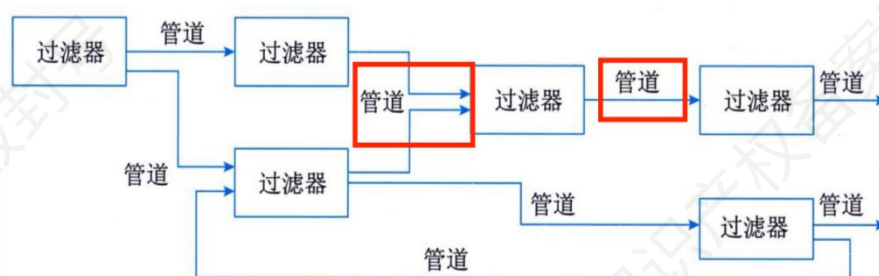
在批处理风格的软件体系结构中, 每个处理步骤是一个单独的程序, 每一步必须在前一步结束后才能开始, 并且数据必须是完整的, 以整体的方式传递。它的基本构件是独立的应用程序, 连接件是某种类型的媒介。连接件定义了相应的数据流图, 表达拓扑结构。



上图展示的是批处理体系结构风格，它的核心思想是：系统由一系列独立的处理构件（组件）组成，每个构件完成特定的数据处理任务，构件之间通过顺序连接件（Tape，通常对应磁带、文件或缓冲区）来传递数据。另外，书本上没提到，实际上批处理它适合定时离线的一些任务，不适合高性能要求的，需要流式处理的场景。

6.1.2.2.管道-过滤器体系结构风格

当数据源源不断地产生，系统就需要对这些数据进行若干处理（分析、计算、转换等）。现有的解决方案是把系统分解为几个序贯的处理步骤，这些步骤之间通过数据流连接，一个步骤的输出是另一个步骤的输入。每个处理步骤由一个过滤器（Filter）实现，处理步骤之间的数据传输由管道（Pipe）负责。每个处理步骤（过滤器）都有一组输入和输出，过滤器从管道中读取输入的数据流，经过内部处理，然后产生输出数据流并写入管道中。



上图可以看到有个细节，在管道过滤器风格里，数据像水流一样在过滤器之间连续流动，不需要“落盘”（批处理风格有个 Tape 落盘操作），因此比批处理更高效。从上图也可以看到，管道可以灵活组合，甚至可以并行（比如图中某个过滤器的输出同时传递给两个后续过滤

器)。系统整体上是流式处理的，所以它的性能，相比我们批处理要更高。。

6.1.3.调用/返回体系结构风格（重点★★★★★）

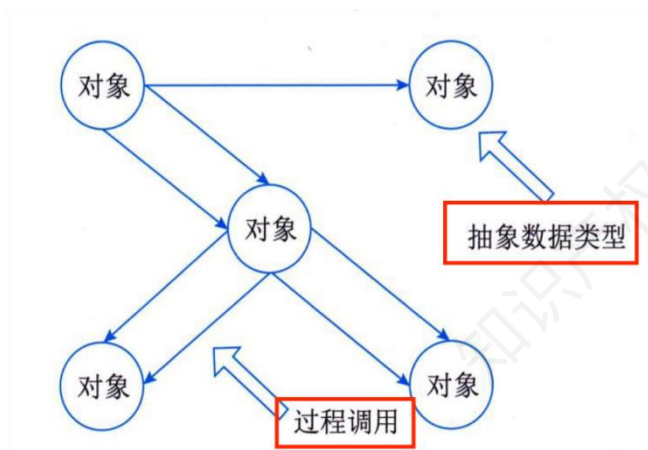
调用/返回体系结构风格以调用关系为核心，通过分解系统来降低复杂度，常见形式包括主程序/子程序、面向对象、层次型和客户端/服务器风格。

6.1.3.1.主程序/子程序风格

主程序/子程序风格一般采用单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。子程序通常可合成为模块。过程调用作为交互机制，即充当连接件。调用关系具有层次性，其语义逻辑表现为子程序的正确性取决于它调用的子程序的正确性。

6.1.3.2.面向对象体系结构风格

面向对象体系结构风格建立在数据抽象和面向对象的基础上，数据的表示方法和它们的相应操作封装在一个抽象数据类型或对象中。这种风格的构件是对象，或者说是抽象数据类型的实例。



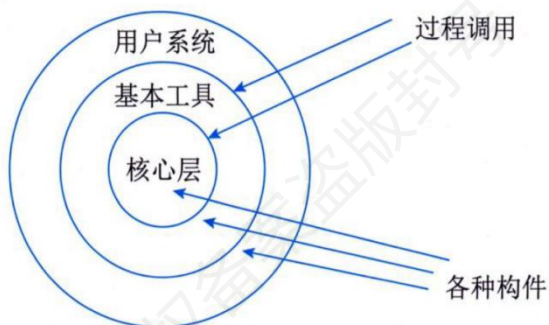
上图可以看到系统由对象组成，这里需要注意的是对象之间通过消息传递或方法调用进行交互（而不是直接访问对方的内部数据连接）形成协作网络，每个对象既封装数据也封装操作，

从而实现模块化、可复用和可扩展的软件体系结构。

6.1.3.3. 层次型体系结构风格

层次系统的组成为一个层次结构，每一层为上层提供服务，并作为下层的客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见。

在这样的系统中，构件在层上实现了虚拟机。连接件由决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。由于每一层最多只影响两层，同时只要给相邻层提供相同的接口，允许每层用不同的方法实现，这同样为软件重用提供了强大的支持。



以此图为例，层次型体系结构以同心圆方式展现，内核层提供最底层的核心服务，中间的基本工具层基于内核实现更高级的功能，最外层的用户系统直接面向用户。每一层由不同的构件组成，构件之间协同完成该层的功能。构件的调用受限于层次结构，只能访问相邻层。

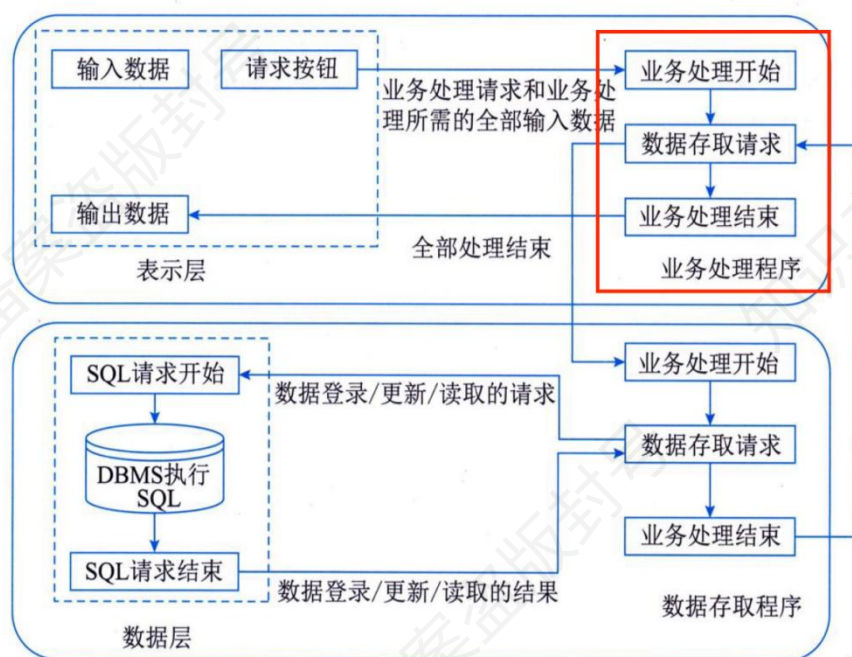
这里的箭头表示过程调用，也就是说表示上层调用下层提供的服务。层之间通过明确接口进行交互，而不是随意访问。

6.1.3.4. 客户端/服务器体系结构风格

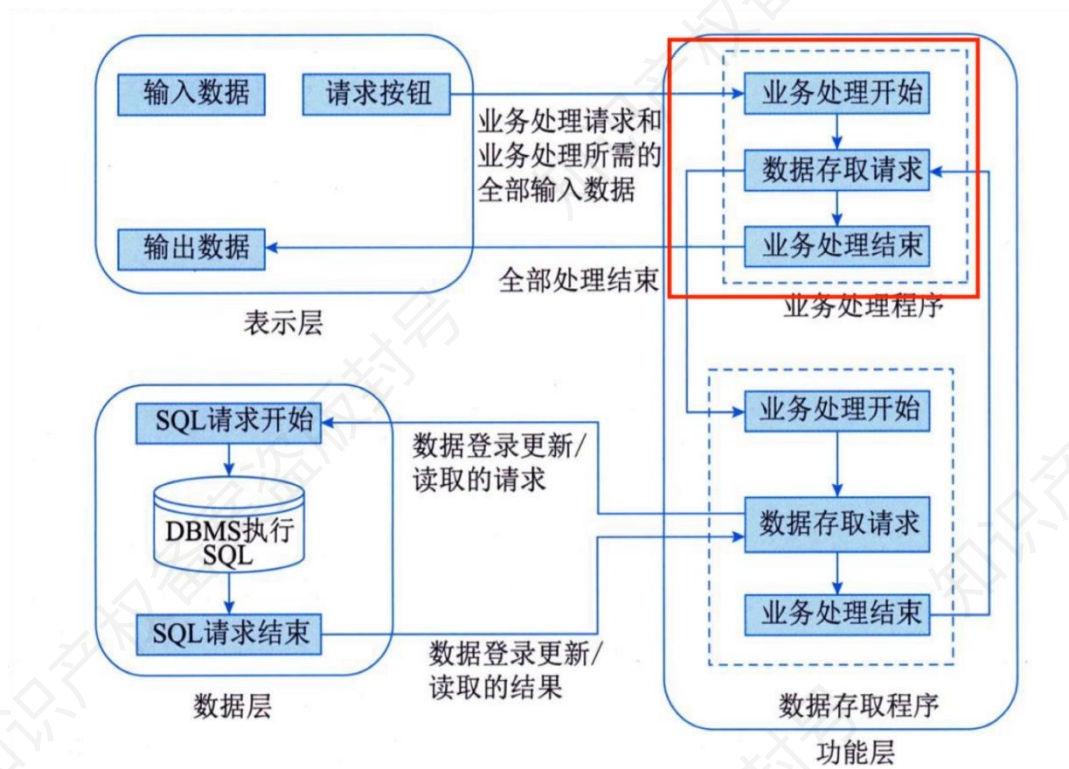
客户端/服务器体系结构风格主要包含三种形态，分别是二层 C/S 架构、三层 C/S 架构和 B/S 架构。

二层 C/S 架构由客户端和数据库服务器组成，客户端承担了表示层和业务逻辑，是典型的

“胖客户端”。它在小型系统中能够充分利用硬件能力，开发思路也较为直接，但随着应用复杂度增加，暴露出客户端程序庞大、升级维护困难、移植性差和安全性不足等问题，可扩展性也较差。



三层 C/S 架构在二层基础上增加了应用服务器，把业务逻辑从客户端剥离出来，使表示层、功能层和数据层分工明确。客户端变成“瘦客户机”，维护和升级更加方便，系统也具备更好的可扩展性和安全性，能支持更大规模的企业级应用。但同时，三层间通信成为关键的性能瓶颈，设计与实现复杂度相应提升。



B/S 架构是三层 C/S 的一种实现形式，采用“浏览器/Web 服务器/数据库服务器”结构，客户端零安装，维护升级集中在服务器端，跨平台能力和推广性极强。它大大降低了部署和运维成本，适合分布式和跨地域应用，但在动态交互、系统响应速度和安全性方面弱于传统 C/S，尤其不利于高并发的 OLTP 应用场景。

6.1.4.以数据为中心的体系结构风格（重点★★★★★）

以数据为中心的体系结构风格主要包括仓库风格和黑板风格。

6.1.4.1.仓库体系结构风格

仓库（repository）是存储和维护数据的中心场所。在仓库体系结构风格中，有两种不同的构件：中央数据结构（说明当前数据的状态），以及一组对中央数据进行操作的独立构件，仓库与独立构件间的相互作用在系统中会有大的变化。这种风格的连接件即为仓库与独立构件之间的交互。

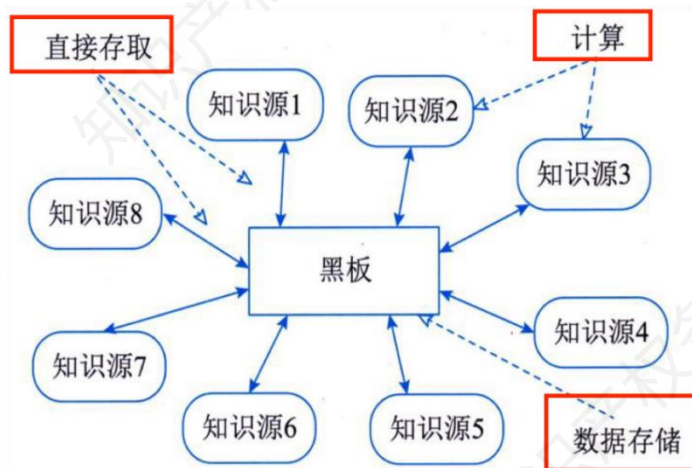


上图有若干细节容易忽略，中央数据库结构（数据仓库）：位于系统核心，所有数据集中存放在这里，相当于共享的数据仓库。独立构件：图中外围的椭圆形就是多个“独立构件”，它们之间没有直接联系，而是通过中央数据库来进行信息交换。一个构件可以把处理结果写入数据库；另一个构件可以从数据库中读取这些结果，作为输入继续处理。

箭头：表示数据的读写关系。所有交互都要经过“中央数据库”这一个枢纽。

6.1.4.2.黑板体系结构风格

黑板体系结构是一种将解空间组织为分级结构，并由多个独立知识源协同工作的框架，适合解决复杂非结构化问题，常用于语音/模式识别和分布式数据共享场景。



这里需要注意的是，图中间的矩形“黑板”是系统的共享数据结构，存放问题的中间状态和部分解。相当于一个全局的工作区，不同的知识源通过黑板间接通信，而不是直接互相调用。

围绕黑板的知识源 1~8 是独立的知识模块，每个模块掌握某种领域知识或处理方法。它们会监听黑板的变化，当发现自己能对现有信息做出推理或补充时，就会执行，把新的知识写回

黑板。例如：语音识别中，不同知识源可能负责声学特征提取、语音单元匹配、词汇映射、语义理解等。

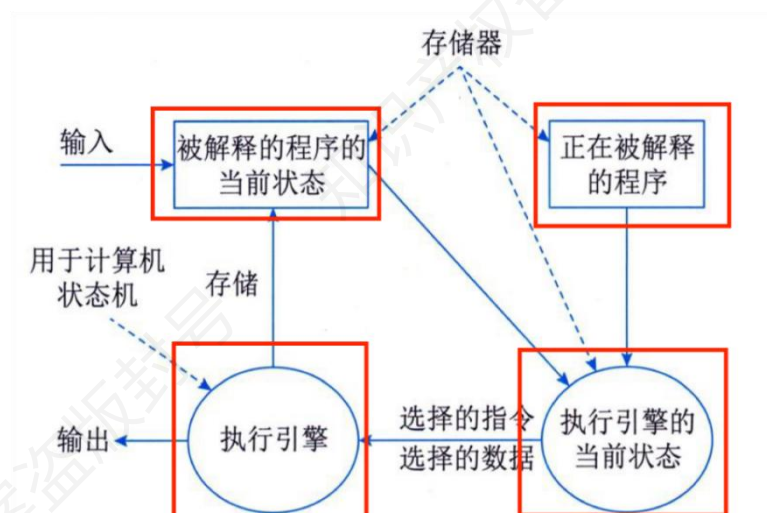
这里我们要注意的是实线箭头：表示知识源与黑板之间的读写交互（推理结果写入黑板，或从黑板读取信息）。虚线箭头：表示知识源还可能从外部获取信息或调用计算功能，例如：“直接存取”表示知识源可直接访问黑板外部数据；“计算”表示知识源可能依赖外部计算资源；“数据存储”表示黑板与外部数据库或持久化存储交互，保存解空间的中间结果。

6.1.5.虚拟机体系结构风格（重点★★★★★）

虚拟机体系结构风格的基本思想是人为构建一个运行环境，在这个环境之上，可以解析与运行自定义的一些语言，这样来增加架构的灵活性。虚拟机体系结构风格主要包括解释器风格和规则系统风格。

6.1.5.1.解释器体系结构风格

一个解释器通常包括完成解释工作的解释引擎，一个包含将被解释的代码的存储区，一个记录解释引擎当前工作状态的数据结构，以及一个记录源代码被解释执行进度的数据结构。具有解释器风格的软件中含有一个虚拟机，可以仿真硬件的执行过程和一些关键应用。



上图展示了解释器的核心工作机制：存储器保存程序与数据，执行引擎逐条取指令并解释执行，通过不断更新“被解释程序的当前状态”和自身状态，最终产生输出。重点关注上图这四个关键细节：

被解释的程序的当前状态：这是程序的运行上下文，包含输入数据、程序计数器、变量值等信息。它会随着解释器的执行不断变化，相当于被解释程序在某一时刻的“快照”。

正在被解释的程序：这是存储在存储器中的程序代码，解释器逐条读取指令。它与“执行引擎的当前状态”相连，执行引擎会根据程序指令来驱动解释。

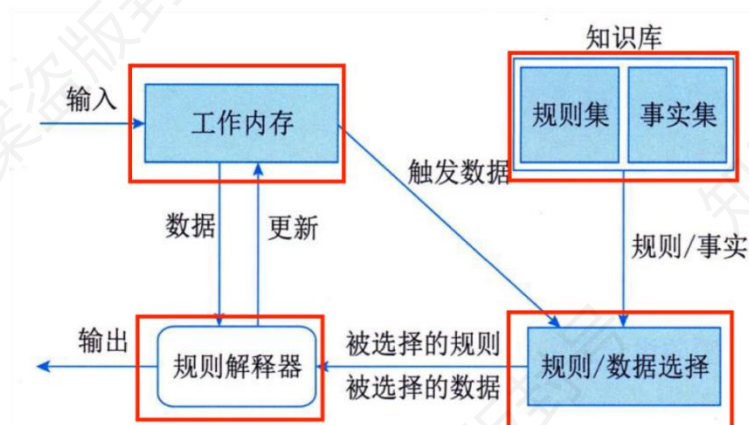
执行引擎的当前状态：表示解释器本身的内部运行状态，例如：当前指令、寄存器值、控制流位置。它决定解释器接下来执行什么动作。

执行引擎：核心部件，负责逐条读取被解释程序的指令，并根据当前状态和输入选择要执行的动作。产生执行结果（输出），并更新“被解释程序的当前状态”。相当于虚拟机或解释器的“大脑”。

存储器：存放程序代码（“正在被解释的程序”）和运行时数据（“被解释的程序的当前状态”）。执行引擎通过存储器读取指令和数据。

6.1.5.2.规则系统体系结构风格

基于规则的系统包括规则集、规则解释器、规则/数据选择器及工作内存。外部输入更新工作内存→选择器匹配知识库规则与数据→规则解释器执行→更新工作内存或输出→循环往复，直至问题解决。



其中基于规则的系统由四个核心部分组成：

工作内存：存放当前的输入数据、系统运行的中间状态，以及规则触发产生的新信息。是规则系统的动态数据区，会随着推理过程不断更新。

知识库：包含两个部分：规则集：一系列“条件→动作”的规则；事实集：领域相关的已知事实和知识。这部分是系统的静态知识来源。

规则/数据选择器：当工作内存发生变化时，选择器根据触发条件，从规则集中挑选出满足条件的规则，并确定相关数据。输出是“被选择的规则”和“被选择的数据”。

规则解释器：接收选择器挑选出的规则和数据，逐条执行规则中的动作。执行结果可能会：更新工作内存（产生新事实）；输出最终结果。

6.1.6.独立构件体系结构风格（重点★★★★★）

独立构件风格主要强调系统中的每个构件都是相对独立的个体，它们之间不直接通信，以

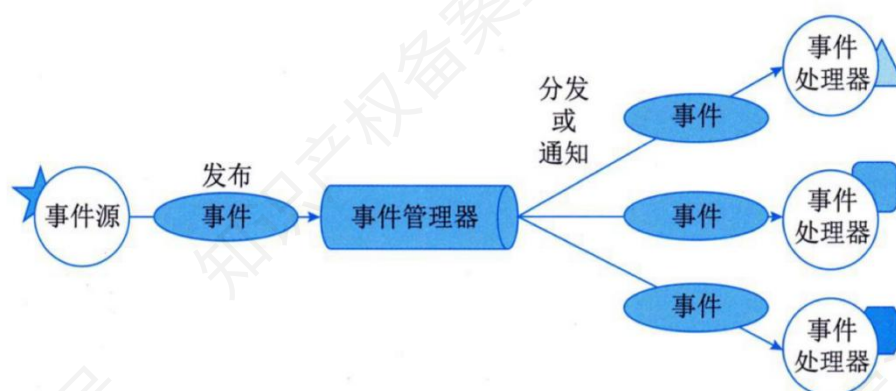
降低耦合度，提升灵活性。独立构件风格主要包括进程通信和事件系统风格。

6.1.6.1.进程通信体系结构风格

进程通信结构风格就是通过不同的消息传递方式让独立运行的进程协同工作，从而构建出一个松耦合的系统。消息传递的方式可以是点到点、异步或同步方式及远程过程调用等。

6.1.6.2.事件系统体系结构风格

事件系统风格基于事件的隐式调用风格的思想，构件不直接调用一个过程（这是和进程通信风格最大的区别），而是触发或广播一个或多个事件。系统中其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程，这样，一个事件的触发就导致了另一模块中的过程的调用。如下图所示。



6.1.7.典型例题（重点★★★★★）

（22 年 11 月架构真题）某电子商务公司拟升级其会员与促销管理系统，向用户提供个性化服务，提高用户的黏性。在项目立项之初，公司领导层一致认为本次升级的主要目标是提升会员管理方式的灵活性，由于当前用户规模不大，业务也相对简单，系统性能方面不做过多考虑。新系统除了保持现有的四级固定会员制度外，还需要根据用户的消费金额、偏好、重复性

等相关特征动态调整商品的折扣力度,并支持在特定的活动周期内主动筛选与活动主题高度相关的用户集合,提供个性化的打折促销活动。

问:针对该系统的功能,李工建议采用面向对象的架构风格,将折扣力度计算和用户筛选分别封装为独立对象,通过对象调用实现对应的功能;王工则建议采用解释器架构风格,将折扣力度计算和用户筛选条件封装为独立的规则,通过解释规则实现对应的功能。请针对系统的主要功能,从折扣规则的可修改性、个性化折扣定义灵活性和系统性能三个方面对这两种架构风格进行比较与分析,并指出该系统更适合采用哪种架构风格。

这里题干说明了要你从可修改性、灵活性和系统性能三个方面进行综合考虑。从可修改性来看,解释器风格折扣规则是独立的语法规则,由解释器可对变化的规则进行解析,修改更容易。而面向对象相对固定,有变化需要修改具体的类。从灵活性来看,解释器可以根据用户灵活解释执行规则,优于面向对象。从系统性能来看,解释器风格一般来说,理论上,软考里,因为多了一层,效率是低于面向对象,这个记住结论即可。综合三个方面来看,解释器的优势更大,所以该系统更适合采用解释器风格。

答:应该选择解释器架构风格。折扣规则的可修改性:解释器风格比面向对象方式实现强。因为解释器风格折扣规则是独立的语法规则,由解释器可对变化的规则进行解析,修改更容易。而面向对象相对固定,有变化需要修改具体的类。个性化折扣定义灵活性:解释器强于面向对象,解释器可以根据用户灵活解释执行规则,做到千人千面。系统性能:面向对象可能略优于解释器。综合来看还是解释器更好。

6.2.面向架构评估的质量属性（超级重点★★★★★）

在架构的新教材上先引入了质量属性的概念,然后又讲了面向架构评估的质量属性,但是奇怪的是后者又不是前者的子集,让人十分迷惑。但是总体只考下面的 9 个质量属性。案例主

要以选词填空的形式出现，主要目的是看你是否会辨别指定场景到底属于哪个质量属性。这里的对策需要熟悉一下。

6.2.1.九大必考质量属性（超级重点★★★★★）

属性	描述	对策
性能	指系统的响应能力，处理事务所需时间或单位时间内处理事务数量。	优先级队列、增加计算资源、减少计算开销、引入并发机制、采用资源调度
可靠性	指软件在应用错误、意外错误使用情况下维持功能特性的基本能力，通过平均失效时间来衡量。	冗余设计（硬件冗余、数据冗余）、错误检测与恢复机制（如奇偶校验、循环冗余校验）、异常处理（捕获并处理程序运行中的异常）、备份与恢复策略（定期备份数据，在故障时可恢复）、负载均衡（避免单个组件过载）
可用性	指系统正常运行的时间比例，表现为两次故障间时长或故障时恢复正常速度。	心跳机制（检测系统组件的运行状态）、冗余系统（如双机热备、集群）、快速恢复机制（使用备用设备或系统）、内置监控器监控与预警（实时监控系统状态，提前发现潜在问题）、故障转移（自动将工作负载转移到备用组件）。心跳是检测系统组件的在线状态的，内置监视器更多的是预警
安全性	指向合法用户提供服务同时阻止非授权访问或拒绝服务的能力，包括机密性、完整性、不可否认性、可控性等。	用户认证（如用户名密码认证、多因素认证）、授权管理（基于角色的访问控制）、数据加密（对敏感数据进行加密存储和传输）、防火墙（防止外部非法访问）、入侵检测与防范系统（检测并阻止网络攻击）、审计与日志记录（记录系统操作，便于事后审计）
可修改性	指快速、高性价比地变更系统的能力，包括可维护性、可扩展性、结构重组、可移植性。 <u>如果移植到新的系统需要做适当更改,则该可移植性就是一种特殊的可修改性。</u>	模块化设计（降低模块间耦合度）、接口标准化（便于替换和扩展组件）、抽象与封装（隐藏实现细节，提高可维护性）、配置管理（使用配置文件管理系统参数）、版本控制（管理代码和文档的变更）

互操作性	软件不是独立存在的，需要与其他系统或环境交互。为了支持互操作性，软件架构必须为外部功能和数据结构提供精心设计的入口。不同编程语言编写的软件系统之间的交互作用体现了互操作性问题。	使用标准协议和接口（如 RESTfulAPI、SOAP）、数据格式转换（将不同系统的数据格式进行转换）、中间件（作为不同系统间的桥梁）、服务注册与发现（便于系统间相互发现和调用服务）、跨语言开发工具（支持不同语言间的交互）
可移植性	软件或硬件适应不同运行环境的能力。通过抽象层（如虚拟机）、环境无关代码设计、减少平台依赖，使系统在迁移至新操作系统（如从 Windows 到 Linux）、硬件平台或云环境时，无需重大修改即可正常运行，如容器化应用跨云部署。	遵循相关标准协议并依赖标准 API，避免特定平台专有 API 以确保兼容性和减少平台依赖；抽象底层差异，通过封装底层代码和借助中间件或虚拟机隔离软件与底层的差异；进行兼容性测试，利用多种硬件平台、操作系统、浏览器等进行多平台测试。
可测试性	指软件能够被有效测试的难易程度，即通过设计和开发手段，使软件易于发现缺陷、验证功能正确性和性能指标达标性的特性。通常体现在测试用例的设计、执行效率，以及缺陷定位的便捷性上。	设计分层与模块化：将系统拆分为独立模块，减少模块间耦合度，便于针对单个模块编写测试用例，降低测试复杂度；提供测试接口：在代码中预留专用的测试接口，允许测试人员直接访问内部状态或调用特定功能，辅助进行白盒测试；
易用性	指系统让用户“好学、好找、好操作、少犯错”的程度，包括学习成本、操作效率、可理解性与可访问性。	统一交互与视觉规范；信息架构清晰（分组与层级、显式路径反馈）；可发现性与即时反馈；降低认知负荷；快捷操作（快捷键、批量操作、常用入口就近原则）；多语言与本地化；响应式与无障碍；新手引导与内嵌帮助；可配置主题/字号与个性化偏好保存等。

Q1:可靠性和可用性的区别是否需要辨析。实际上架构第二版上这两个不一样的概念，但是真题实际上也没出现过两者的辨析。同时根据国际标准 ISO/IEC25010 的解读，可用性实际上是可靠性的一个子集。其中可用性的定义包含了故障恢复速度的含义，可靠性的度量指标平均失效时间实际上也是这个意思（失效时间短也是恢复速度快）。近十年之内两者都没出现过辨析，以真题为例：

网络失效后，系统需要在 2 分钟内发现错误并启用备用系统；

主站点断电后，需要在 3 秒内将访问请求重定向到备用站点；

网络失效后，系统需要在 2 分钟内发现并启用备用网络系统；

系统主站点断电后，需要在 3 秒内将请求重定向到备用站点；

能够连续运行的时间不小于 240 小时，意外退出后能够在 10 秒之内自动重启。

网络失效后，系统需要在 10 秒内发现错误并启用备用系统；

系统主站点断电后，必须在 3 秒内将访问请求重定向到备用站点；

系统主站点断电后，应在 5 秒内将请求重定向到备用站点；

当系统发生网络失效后，需要在 15 秒内发现错误并启用备用网络；

系统主站点断电后，应在 3s 内将请求重定向到备用站点；

系统宕机后，需要在 15s 内发现错误并启用备用系统；

平台支持分布式部署，当主站点断电后，应在 20 秒内将请求重定向到备用站点；

平台主站点宕机后，需要在 15 秒内发现错误并启用备用系统；

这些都是归结到可用性上。所以这个不纠结。写可用性可靠性都没问题。默认可用性。

6.2.2.典型例题（超级重点★★★★★）

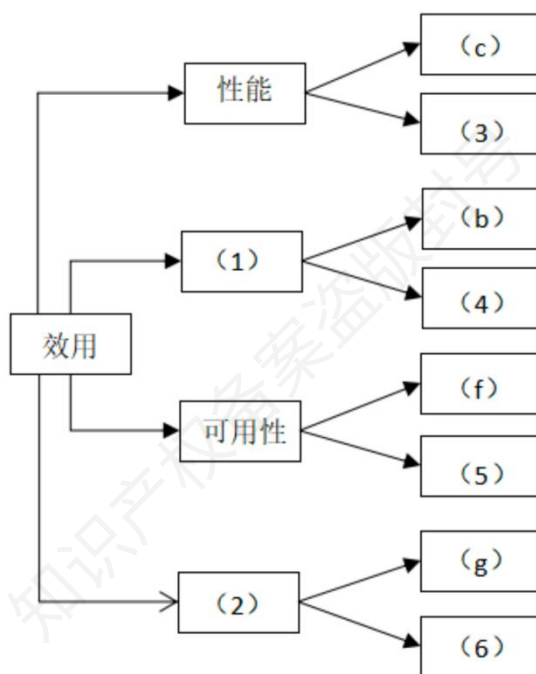
（22 年 11 月架构真题）某电子商务公司拟升级其会员与促销管理系统，向用户提供个性化服务，提高用户的粘性。在项目立项之初，公司领导层一致认为本次升级的主要目标是提升会员管理方式的灵活性，由于当前用户规模不大，业务也相对简单，系统性能方面不做过多考虑。新系统除了保持现有的四级固定会员制度外，还需要根据用户的消费金额、偏好、重复性等相关特征动态调整商品的折扣力度，并支持在特定的活动周期内主动筛选与活动主题高度相关的用户集合，提供个性化的打折促销活动。在需求分析与架构设计阶段，公司提出的需求和质量属性描述如下：

- （a）管理员能够在页面上灵活设置折扣力度规则和促销活动逻辑，设置后即可生效；
- （b）系统应该具备完整的安全防护措施，支持对恶意攻击行为进行检测与报警；
- （c）在正常负载情况下，系统应在 0.3 秒内对用户的界面操作请求进行响应；
- （d）用户名是系统唯一标识，要求以字母开头，由数字和字母组合而成，长度不少于 6 个字符；

- (e) 在正常负载情况下，用户支付商品费用后在 3 秒内确认订单支付信息；
- (f) 系统主站点电力中断后，应在 5 秒内将请求重定向到备用站点；
- (g) 系统支持横向存储扩展，要求在 2 人天内完成所有的扩展与测试工作；
- (h) 系统宕机后，需要在 10 秒内感知错误，并自动启动热备份系统；
- (i) 系统需要内置接口函数，支持开发团队进行功能调试与系统诊断；
- (j) 系统需要为所有的用户操作行为进行详细记录，便于后期查阅与审计；
- (k) 支持对系统的外观进行调整和配置，调整工作需要 4 人天内完成。

在对系统需求、质量属性描述和架构特性进行分析的基础上，系统架构师给出了两种候选的架构设计方案，公司目前正在组织相关专家对系统架构进行评估。

问：在架构评估过程中，质量属性效用树（utilitytree）是对系统质量属性进行识别和优先级排序的重要工具。请将合适的质量属性名称填入图 1-1 中 (1)、(2) 空白处，并选择题干描述的 (a) ~ (k) 填入 (3) ~ (6) 空白处，完成该系统的效用树。



(1) 安全性：与系统的安全防护、审计相关。题干中 (b) 描述的是对恶意攻击的检测，(j) 描述的是用户行为日志，两者均体现对系统运行过程中的可追踪、可检测能力，属于安全性范畴。

(2) 可修改性：指系统对变更（尤其是业务逻辑变更）的适应能力。(g) 描述的是系统支持扩展的能力、(k) 涉及界面调整均体现系统的可修改性。

而对应的需求项具体分析如下：

(e) 为性能属性的体现，用户支付后 3 秒内完成确认，对系统响应的时效性有明确要求

(j) 用户行为日志记录，归属安全性中的审计能力，保证系统运行的透明度与可追责。

(h) 系统宕机后的自动恢复体现了可用性属性中的故障恢复能力，与安全性密切相关。

(k) 界面调整配置支持，明确限定了人力工时范围，体现了系统在界面层的可配置性，属于可修改性。

答：(1) 安全性 (2) 可修改性 (3) (e) (4) (j) (5) (h) (6) (k)

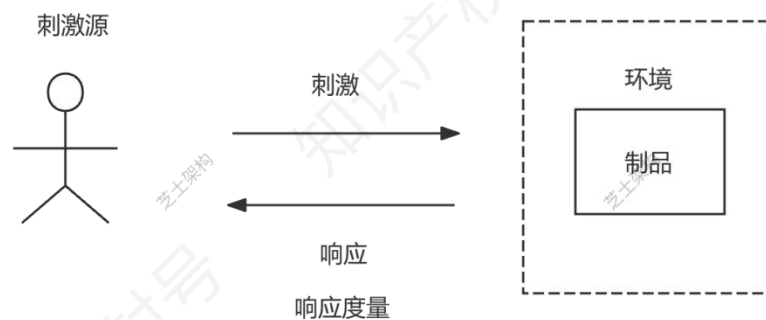
6.3.质量属性场景描述（超级重点★★★★★）

为了更精确描述软件系统的质量属性，引入场景是个不错的想法。它能让抽象的质量属性变得可感知、可衡量，从而为软件系统的设计、开发和评估提供清晰且实用的指引。书本上把这个说明的方法用一个公式表示，这是常考的重点和难点，往往放在案例第一题，不看必挂。

美国卡内基梅隆大学的软件工程研究所的资深研究员 LenBass、PaulClements 和 RickKazman 等人在著作《软件架构实践》(SoftwareArchitectureinPractice)中系统化了这一模型。在该书第 2 版中，作者明确提出了用于描述质量属性需求的“六部分场景”模板。这一模板包含六个要素：刺激源 (Sourceofstimulus)、刺激 (Stimulus)、环境 (Environment)、制品 (Artifact)、响应 (Response) 和 响应度量 (ResponseMeasure)。六要素场景模型提供了一种规范化方式来描述质量属性需求，使其可测试、可度量，从而成为软件架构领域广泛引用的“权威”模型。

6.3.1.质量属性场景描述的语言（重点★★★★★）

质量属性场景描述=刺激源+刺激+环境+制品+响应+响应度量，如下图所示



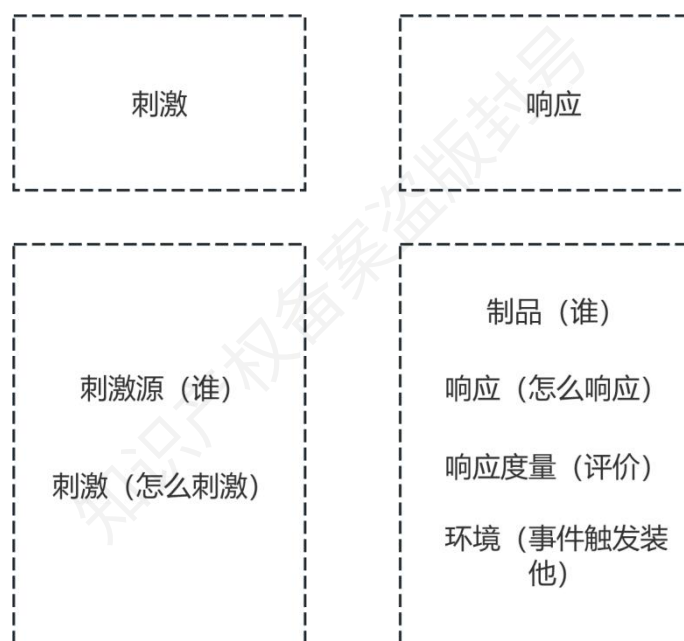
假如我们要描述一个电商平台秒杀场景的性能问题，我们应该怎么做。为了更好地理解这种描述语言（刺激源+刺激+环境+制品+响应+响应度量），凯恩把它们对应的语义组织成下表。

名词	理论	白话	例子
刺激源	人、计算机系统或者任何其他刺激器	质量属性测试参与者	大量用户
刺激	当刺激到达系统时需要考虑的条件	测试触发条件	用户操作的随机性、网络状况不同
环境	当刺激发生时，系统可能处于的状态	测试触发的时间/测试触发时测试对象所处状态（有的时候和刺激很像）	正常访问
制品	受刺激的模块或系统	测试对象	在线购物系统的商品展示模块、购物车模块、订单处理模块
响应	在激励到达后所采取的行动	测试对象的反应	快速处理请求，返回商品信息、更新购物车状态、处理订单
响应度量	当响应发生时，应当能够以某种方式对其进行度量以对需求进行测试	如何度量反应好坏与否	以系统响应时间、吞吐量衡量

串起来形成一个故事就是，大量用户（刺激源）在双十一 0 点同时点击秒杀（刺激），在系统正常运行状态下（环境），电商平台的商品展示/购物车/订单模块（制品）需要快速处理

请求并返回结果（响应），并以要求系统在 200ms（响应度量）内响应与吞吐率 500QPS（响应度量）为指标进行衡量（响应度量）。

那么我们记忆的时候，其实是可以把它分成两块。我们可以把它看成左侧是刺激，不管是刺激的对象还是刺激的方式，我们都把它归为刺激一类。右侧，我们把它当做“响应”。不管是响应的对象、响应的方式、响应的度量还是响应的环境，都把它当做响应。这样，你就把整个问题化解了。我们不是直接把它分解成很复杂的那几块，而是先把它分解到中间状态（左右两边），然后再去向下分解记忆，如下图所示。



6.3.2.质量属性场景描述案例（超级重点★★★★★）

可用性、可修改性、性能、可测试性、易用性和安全性的场景描述方法是书本重点，曾经考过一道大题，所以内容虽然多还是需要看和记。这里凯恩给你的内容是书本内容的简记，为了好记配合一个小故事把这些内容串起来，熟读一下会考的。这里的记忆方法参考 6.3.1 的化简的方法。

可用性质量属性场景描述	
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常模式到降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件，其记录事件，通知事件，隔离故障，等待恢复
响应度量	故障修复时间

串起来形成一个故事就是，硬件故障或程序错误（刺激源）导致数据库崩溃（刺激），在系统由正常模式切换到降级模式时（环境），订单数据库、通信链路等关键模块（制品）需要及时检测并记录事件、通知管理员并切换到备用节点（响应），并以故障修复时间 RTO 是否满足预定时限为指标进行衡量（响应度量）。

可修改性质量属性场景描述	
刺激源	最终用户、开发人员、系统管理员
刺激	改变功能、扩容等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	修改且不会影响其他功能
响应度量	成本、努力、资金；该修改对其他功能的影响

串起来形成一个故事就是，开发人员或系统管理员（刺激源）在运行时需要对系统进行功能修改或容量扩展（刺激），在系统处于正常运行环境下（环境），用户界面、平台或相关交互系统（制品）应当能够被修改且不影响其他功能（响应），并以修改所需的成本、时间和对其他功能的影响程度为指标进行衡量（响应度量）。

性能质量属性场景描述

刺激源	用户请求，其他系统触发等
刺激	事件到达
环境	正常模式到超载模式
制品	系统
响应	处理刺激、改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率

串起来形成一个故事就是，用户请求或其他系统触发事件（刺激源）在同一时间大量到达（刺激），在系统由正常模式进入高负载甚至超载模式时（环境），整个系统（制品）需要及时处理请求或动态调整服务级别（响应），并以响应等待时间、截止期限、吞吐量、抖动和缺失率等指标进行衡量（响应度量）。

可测试性质量属性场景	
刺激源	开发、测试、客户
刺激	分析、架构、设计、集成交付时（触发）（书上的写法会有点歧义，参看刺激词条，一般是指触发条件）
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、应用
响应	提供对状态值的访问，提供所计算的值和测试环境 （这里有点争议，有同学认为响应应该是系统的反应，按照其他的质量属性的说法，这里的响应都可以看成是对策，怎么保证质量属性得到满足。那这里是不是有问题？实际上这里的含义是，任何系统要测试可测试性，需要提供合适的测试环境以及提供对状态值的访问等）
响应度量	已执行的可执行语句的百分比、如果存在缺陷出现故障的概率、执行测试的时间、测试中最长依赖的长度、准备测试环境的时间

串起来形成一个故事就是，开发人员、测试人员或客户（刺激源）在系统分析、设计、集成或交付过程中触发测试活动（刺激），在设计、开发、编译或部署等阶段（环境），系统的设计、代码段或应用（制品）需要提供可访问的状态值、计算结果以及合适的测试环境（响应），并以语句覆盖率、缺陷触发概率、测试执行时间、最长依赖链长度和测试环境准备时间等指标

进行衡量（响应度量）。

易用性质量属性场景	
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统
环境	系统运行时或配置时
制品	系统
响应	用户友好界面及适应性设计，确保错误影响最小化
响应度量	用户满意度、成功操作在总操作中所占的比例等

串起来形成一个故事就是，最终用户（刺激源）在使用过程中希望学习系统特性并有效完成操作（刺激），在系统运行或配置时（环境），整个系统（制品）需要通过用户友好的界面和适应性设计来支持操作并尽量减少错误影响（响应），并以用户满意度、成功操作占比等指标进行衡量（响应度量）。

安全性质量属性场景	
刺激源	内部/外部的个人或系统
刺激	试图访问修改数据，访问系统服务
环境	系统所处网络环境
制品	系统服务、数据
响应	认证授权审计加密监控
响应度量	检测、溯源、恢复

串起来形成一个故事就是，内部或外部的个人或系统（刺激源）试图非法访问或修改数据、调用系统服务（刺激），在特定的网络环境下（环境），系统的服务与数据（制品）需要通过认证、授权、审计、加密和监控等机制进行防护（响应），并以能否及时检测、有效溯源以及完成恢复的能力作为衡量标准（响应度量）。

6.3.3.典型例题

(24 年 5 月架构)某互联网企业正在对其已有的在线服务平台进行架构升级,以提升系统的可维护性、可扩展性和服务稳定性。原系统采用传统的单体架构,随着业务模块的不断增加,系统部署越来越困难,修改一处代码往往需要整个系统重新打包上线,导致维护成本高、故障影响范围广。

为解决上述问题,架构团队计划引入微服务架构,将系统按业务边界划分为多个服务单元,分别部署和独立扩展,并配套引入容器化、服务注册与发现、统一配置中心等能力。同时,团队也在开展质量属性建模分析,明确如可用性、性能、可维护性等非功能需求的权衡与保障机制。请结合下列问题,完成系统架构设计分析任务。

问:用质量属性 6 要素描述可用性质量属性场景描述。

答:刺激源:系统内部、系统外部

刺激:疏忽、错误、崩溃、时间

环境:正常操作、降级模式

制品:系统处理器、通信信道、持久存储器、进程

系统响应:检测事件并进行以下一个或多个活动:将其记录下来。通知适当的各方,包括用户和其他系统。根据已定义的规则禁止导致错误或故障的事件源。在一段预先指定的时间间隔内不可用,其中,时间间隔取决于系统的关键程度。在正常或降级模式下运行。

响应度量:故障修复时间

6.4.系统架构评估 (重点★★★★★)

系统架构评估说人话就是看看它的结构设计是否合理,能否扛住高负荷、防得住攻击、方便未来升级改造,以及出问题时好不好修复。就是提前给系统做全面体检,避免后期出大毛病。

这里特别是敏感点和权衡点的辨析是系分架构选择最爱出的方向，看凯恩举的例子就能明白。案例中呢有时候会出现风险点和非风险点（基本不考），也需要注意。这些都是语文题，注意这里的句式。敏感点，风险点，权衡点，说多了可能觉得咬文嚼字，最简单的判定是根据句式，如下表所示。

概念	定义	举例
敏感点	一个或多个构件（和/或构件之间的关系）的特性	对查询请求处理时间的要求将影响系统的数据传输协议和处理过程的设计
权衡点	影响多个质量属性的特性，是多个敏感点的综合	还是上述在线文件存储系统，加密级别的选择就是权衡点。高加密级别（如 AES-256）能大幅提升安全性，但会增加数据加密和解密的计算量，降低系统性能；低加密级别（如 DES）安全性稍弱，但对性能影响较小。它影响了安全性和性能多个质量属性。
风险承担者 / 利益相关人	系统架构涉及到的利益相关者，会试图影响架构决策以实现自己目标	企业 ERP 系统中的财务部门、销售部门
场景	从风险承担者的角度对于系统的交互的简短描述，架构评估中一般采用刺激、环境和响应三方面来描述	客户在公共 Wi-Fi 环境下尝试登录银行网上银行系统，系统需 10 秒内完成身份验证并显示账户信息
风险点	可能导致系统架构出现问题或无法满足质量属性要求的特性、决策或情况。这些因素一旦发生，会给系统带来负面的影响，如降低性能、损害安全性等	如果“养护报告生成”业务逻辑的描述尚未达成共识，可能导致部分业务功能模块规则的矛盾，影响系统的可修改性（如果...，可能...，影响）
非风险点	不会对系统架构的质量属性造成负面影响，或者对系统架构的影响在可接受范围内的特性、决策或情况	选择一款成熟且持续更新的消息队列中间件（如 RabbitMQ），其性能、稳定性都经过市场验证，在当前系统架构下使用它不会给系统带来明显风险

6.4.1.系统架构评估方法（重点★★★★★）

书本上提到的系统架构评估方法有三种，除了场景评估，其他了解即可。

评估方法	描述	优点	缺点
基于调查问卷或检查表	通过设计问卷或检查表，利用相关人员的经验和知识进行评估。	能够充分利用评估人员的经验和知识。	在很大程度上依赖于评估人员的主观推断。
基于场景的评估	分析软件架构对场景的支持程度，判断架构对质量需求的满足程度。 SAAM、ATAM 都在其中。	能够系统地评估架构对特定场景的支持。	需要详细定义和分析场景，可能耗时。
基于度量的评估	建立质量属性和度量之间的映射原则，从架构文档中获取信息，分析推导出系统的质量属性。	客观性强，可以量化评估结果。	需要精确的度量方法和映射原则，可能需要专业知识。

6.4.2.基于场景的架构分析方法 SAAM（次重点★★★★☆）

这一部分了解即可，SAAM 最多选择题中出现。这里提到它，因为它是 ATAM 的基础。

基于场景的架构分析方法 SAAM，说人话就是用实际行动来对架构设计“挑刺”。通过模拟各种具体场景，检查架构是否扛得住，会不会有问题。它使用场景技术，主要分析可修改性这一质量属性，协调风险承担者达成共识。SAAM 主要输入是问题描述、需求声明和架构描述。SAAM 的评估过程包括：场景开发、架构描述、单场景评估、场景交互评估和总体评估。

6.4.3.架构权衡分析方法 ATAM（重点★★★★★）

ATAM 在 SAAM 的基础上发展起来的，主要针对性能、安全性、可修改性和可用性，在

系统开发之前，对这些质量属性进行评价和折中。它和 SAAM 最大的区别是，它用效用树等工具量化分析不同属性的优先级，基于优先级再去评估架构方案的设计好坏。ATAM 包括 4 个主要阶段：场景和需求收集、架构视图和场景实现、属性模型构造和分析、权衡。整个过程是迭代式的。

6.4.3.1.质量效用树（重点★★★★★）

效用树有两种考法，第一种是直接选择题的考法，下面划线的内容挖空让你填写，或者做简答。第二种就是给出质量效用树，结合具体场景让你判断应该归为哪类。

ATAM 方法采用效用树（Utilitytree）这一工具来对质量属性进行分类和优先级排序。效用树的结构包括：树根—质量属性—属性分类—质量属性场景（叶子节点）。需要注意的是，ATAM 主要关注 4 类质量属性：性能、安全性、可修改性和可用性，这是因为这 4 个质量属性是利益相关者最为关心的。得到初始的效用树后，需要修剪这棵树，保留重要场景（通常不超过 50 个），再对场景按重要性给定优先级（用 H/M/L 的形式），再按场景实现的难易度来确定优先级（用 H/M/L 的形式），这样对所选定的每个场景就有一个优先级对（重要度、难易度），如（H、L）表示该场景重要且易实现。

6.4.3.2.典型例题（重点★★★★★）

（21 年 11 月架构真题）某公司拟开发一套机器学习应用开发平台，支持用户使用浏览器在线进行基于机器学习的智能应用开发活动。该平台的核心应用场景是用户通过拖拽算法组件灵活定义机器学习流程，采用自助方式进行智能应用设计、实现与部署，并可以开发新算法组件加入平台中。在需求分析与架构设计阶段，公司提出的需求和质量属性描述如下：

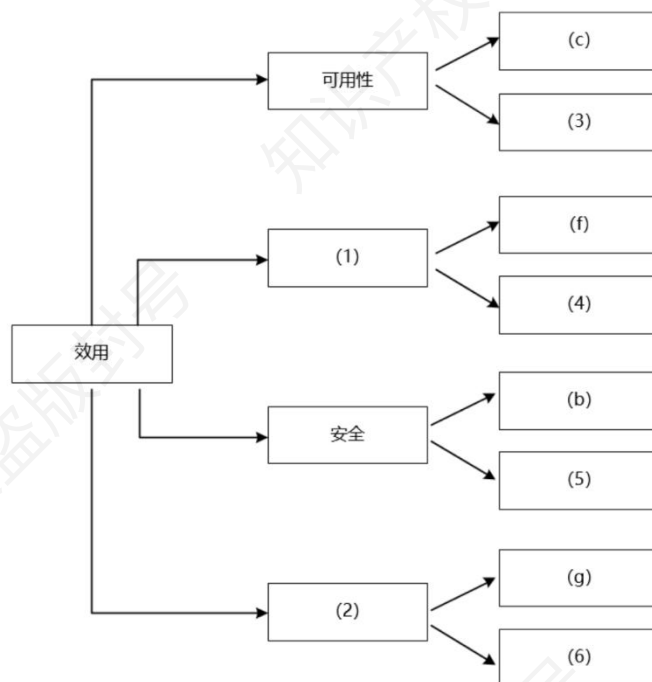
（a）平台用户分为算法工程师、软件工程师和管理员等三种角色，不同角色的功能界面有所

不同；

- (b) 平台应该具备数据库保护措施，能够预防核心数据库被非授权用户访问；
- (c) 平台支持分布式部署，当主站点断电后，应在 20 秒内将请求重定向到备用站点；
- (d) 平台支持初学者和高级用户两种界面操作模式，用户可以根据自己的情况灵活选择合适的模式；
- (e) 平台主站点宕机后，需要在 15 秒内发现错误并启用备用系统；
- (f) 在正常负载情况下，机器学习流程从提交到开始执行，时间间隔不大于 5 秒；
- (g) 平台支持硬件扩容与升级，能够在 3 人天内完成所有部署与测试工作；
- (h) 平台需要对用户的所有操作过程进行详细记录，便于审计工作；
- (i) 平台部署后，针对界面风格的修改需要在 3 人天内完成；
- (j) 在正常负载情况下，平台应在 0.5 秒内对用户的界面操作请求进行响应；
- (k) 平台应该与目前国内外主流的机器学习应用开发平台的界面风格保持一致；
- (l) 平台提供机器学习算法的远程调试功能，支持算法工程师进行远程调试。

在对平台需求、质量属性描述和架构特性进行分析的基础上，公司的架构师给出了三种候选的架构设计方案，公司目前正在组织相关专家对平台架构进行评估。架构进行评估。

问：在架构评估过程中，质量属性效用树 (utilitytree) 是对系统质量属性进行识别和优先级排序的重要工具。请将合适的质量属性名称填入图 1-1 中 (1)、(2) 空白处，并从题干中的 (a) - (l) 中选择合适的质量属性描述，填入 (3) - (6) 空白处，完成该平台的效用树。



此题是比较典型的质量效用树的考察，就是给你场景，让你做质量属性的分类。在架构评估中，质量效用树，默认有四大质量属性，分别为：性能、可用性、安全性和可修改性，这个条件题目一般不直接给出，需要掌握这个知识背景。(1)和(2)只能在性能和可修改性中选择。由于(f)是性能要求，所以(1)填性能，(2)为可修改性。(e)强调了系统出故障限定多长时间切换到备用系统，是典型的系统修复时间限定，属于可用性（准确地说是可靠，因为是关注故障修复时间。软考里，可用可靠同时出现的时候就要进行辨析，否则当做同义词）。(j)强调响应时间，应为性能。(h)强调记录操作并审计，属于安全性。(i)强调做系统修改时，时限要求，为可修改性。

答：(1) 性能 (2) 可修改性 (3) (e) 可用性 (4) (j) 性能 (5) (h) 安全性 (6)

(i) 可修改

6.5.ATAM 方法的架构评估实践（重点★★★★★）

这一章书上写了很多，但是从案例的出题角度来看，能考的东西不多，因为太过理论。但

是论文到有可能会考，假如出论文题，你就从下面四个阶段展开。

ATAM 方法的架构评估实践分为四个阶段：演示、调查和分析、测试和报告。



图 8-2 ATAM 分析评估过程

一个完整的 ATAM 实践活动是演示、调查和分析（场景和需求收集、架构视图和场景实现、属性模型构造和分析、权衡）、测试和报告。

阶段	主要活动
演示阶段	介绍 ATAM 评估流程、业务目标及待评估架构。
调查和分析	确定架构设计方法；生成质量属性效用树（并针对优先级排序）；分析架构与质量属性的匹配度。
测试阶段	通过头脑风暴提出候选场景；筛选优先级场景并验证架构对场景的支持能力。
报告阶段	汇总评估结果（风险、非风险、敏感点、权衡点）；提交效用树、场景列表及改进建议。

7.上篇-安全架构设计理论与实践（超级重点★★★☆☆）

这次凯恩把安全这一个章节的内容当作超级重点，因为 13 年之后就没当作大题考过了，再考的概率其实很大了。

7.1.安全架构特性（次重点★★★★☆☆）

安全架构应具备可用性、完整性和机密性等特性，它们的概念如下表所示。这里主要是会让你辨析完整性和机密性的区别。你把它们各自的概念写出来就行了。

特性	定义	典型例子
可用性	确保系统资源在需要时可正常访问，防止因故障或攻击导致服务中断	数据备份与恢复机制，负载均衡技术，双机热备系统
完整性	保护数据不被未经授权修改或破坏，确保数据一致性和准确性	文件哈希校验（如 MD5/SHA-1），数字签名技术，数据库事务回滚机制
机密性	防止敏感信息在未授权情况下被泄露或获取	数据加密存储，HTTPS 加密传输，访问控制列表（ACL）

7.2.数据加密技术（重点★★★★★）

数据加密技术 13 年考过一次真题，就是考你加密技术的概念，这里作为基本知识还是要了解。

7.2.1.对称加密概念（重点★★★★★）

对称加密是一种加密技术，它使用同一个密钥来加密和解密数据。也就是说，发送方使用

密钥将明文加密成密文，接收方使用相同的密钥将密文还原为明文。其工作原理是基于某种数学算法，对原始数据进行特定的变换，使得只有拥有正确密钥的人才能还原数据。对称加密的优点是加密和解密速度快，效率高，适合对大量数据进行加密。

7.2.2.非对称加密概念（重点★★★★★）

非对称加密则使用一对密钥，即公钥和私钥。公钥可以公开给任何人，用于加密数据；而私钥则由用户自己保密，用于解密数据。当发送方要向接收方发送数据时，使用接收方的公钥对数据进行加密，接收方收到密文后，使用自己的私钥进行解密。非对称加密的安全性基于数学难题，如 RSA 算法基于大整数分解难题，使得在不知道私钥的情况下，很难从密文还原出明文。它的主要优点是密钥管理方便，不需要像对称加密那样在通信双方之间安全地传递密钥。非对称加密常用于数字签名、密钥交换等场景。

7.2.3.非对称加密的应用场景（重点★★★★★）

非对称加密的基本核心是“公钥加密私钥解密，也可以私钥加密公钥解密”，这两种方法衍生出了两种不同的场景。

私钥加密公钥解密。在数字签名场景中，通常是使用私钥对消息进行签名，然后将消息以及签名一起发送给接收方，接收方使用对应的公钥来验证签名，以确保消息的来源和完整性。这个时候传递的内容是明文。

公钥加密私钥解密。在加密场景中，一般是使用接收方的公钥对消息进行加密，然后接收方使用自己的私钥进行解密。这样可以保证只有拥有相应私钥的接收方能够解密并读取消息内容。这个时候传递的内容是加密的。这个你结合数字签名那一章节来看会比较明白。

7.2.4.对称加密和非对称加密的区别（重点★★★★★）

案例前面提到过，最喜欢考场景的技术选型还有技术之间的对比，那么对称加密和非对称加密特别在加密的速度和适用的数据规模上各有不同，看凯恩下表的总结。

对比项	对称加密	非对称加密
密钥数量	单一密钥	密钥对（公钥+私钥）
加密速度	快（适合大数据）	慢（适合小数据）
密钥管理	需安全共享	公钥公开，私钥保密
主要用途	数据加密	密钥交换、数字签名、身份验证
数学基础	替代/置换操作	数论难题（分解质因数、离散对数等）
典型算法	AES、DES、3DES	RSA、ECC

7.3.认证技术（重点★★★★★）

认证技术是信息安全的核心技术之一，主要用于确认用户、设备或数据的真实性与合法性。

它保证了信息传输与访问过程中的身份确认、数据完整性、防伪造和防抵赖。总结如下表所示。

分类	具体内容
数字签名	附加在数据单元上的数据或密码变换，用于确认数据来源、完整性和防止伪造；基于非对称加密算法，有普通和特殊数字签名算法；主要功能是保证信息传输完整性、发送者身份认证和防止抵赖。
杂凑算法	利用散列函数加密，如 MD5 将任意长度字节串变成定长大数，产生 128 位消息摘要；SHA 家族算法对长度不超过 264 位消息产生 160 位消息摘要，SHA 有 5 个算法。
数字证书	由认证中心签发对用户公钥的认证，内容包括 CA 及用户信息、公钥、时间等；国际上遵从 X.509 体系标准；不同 CA 发放证书需证书链通信，CA 维护证书吊销列表。
身份认证	口令认证：用户名/密码简单常用但不安全，易泄漏和被截获。 动态口令认证：密码按时间或使用次数变化，一次一密较安全，但可能存在同步问题和输入不便。

生物特征识别：通过身体或行为等生物特征认证，分为身体和行为特征，又有高级、次级和深奥生物识别技术分类。

7.3.1.数字签名（重点★★★★★）

数字签名（DigitalSignature）是公钥基础设施（PKI）的核心组成部分。在 PKI 体系中，常见的要素包括数字证书、证书颁发机构（CA）、银行使用的安全 Key，以及 SSL/TLS 通信等，而数字签名正是这些应用的技术基础。

7.3.1.1.Hash 算法与数字签名（重点★★★★★）

在理解数字签名之前，必须先掌握 Hash（散列/杂凑）算法。Hash 算法的主要特性包括：易压缩性：可将任意长度的数据映射为固定长度的输出。单向性：从源数据得到哈希值容易，但无法由哈希值推回源数据。高灵敏性：输入数据发生微小变化，输出结果会有显著差异。抗碰撞性：不同输入得到相同哈希值的概率极低。

常用算法包括 SHA-1、SHA-256、SHA-384 以及我国的 SM3。由于计算能力的提升，SHA-1 已被证明存在安全风险，逐步退出历史舞台。由于具备这些特性，Hash 算法非常适合用于数据完整性校验，是数字签名的重要组成部分。

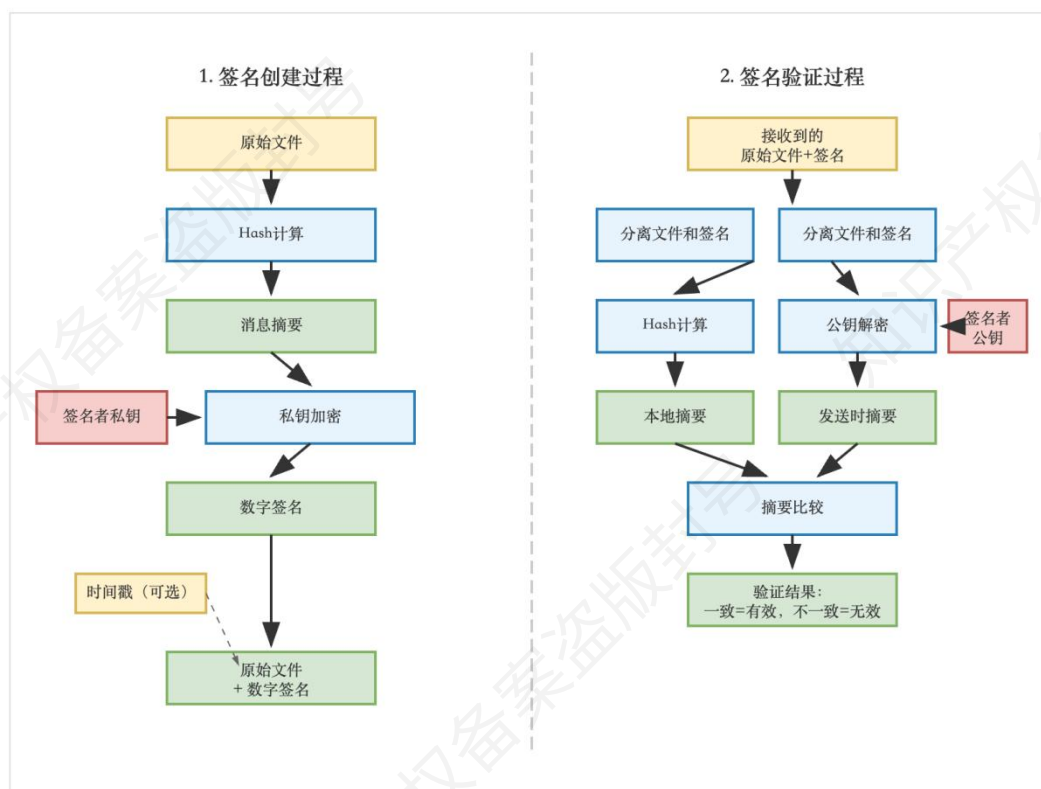
7.3.1.2.数字签名的工作原理（重点★★★★★）

1.签名的创建过程。对需要签名的文件进行 Hash 计算，得到消息摘要。使用签名者的私钥对消息摘要进行加密，生成数字签名。可以选择加入时间戳，标识签名的创建时间。最终发送的内容包括原始文件和数字签名。需要注意，数字签名只对文件的 Hash 值进行签名，而不是直接对整个文件签名，这样既节约了资源，又提高了效率。

2.签名的验证过程。接收方对收到的原始文件进行 Hash 计算，得到本地摘要。使用签名

者的公钥解密数字签名，得到发送时的摘要。比较两个摘要，如果一致，则文件完整且签名有效；若不一致，则说明文件被篡改或签名无效。

数字签名创建与验证流程示意图



7.3.2.数字证书（重点★★★★★）

在数字签名的过程中，私钥用于加密消息的摘要，而公钥则用于对端解密以进行认证。这个过程虽然能有效验证信息的完整性和来源，但也引出了一个关键问题：如何确保公钥确实属于消息的发送者，而不是被恶意篡改或伪造的？这个问题正是数字证书和证书认证体系的核心。

当接收方使用发送方的公钥来验证数字签名时，如果公钥在传输过程中被篡改或伪造，接收方将无法正确验证签名，从而导致安全漏洞。为了避免这种情况，就需要确保公钥的真实性——这正是数字证书的作用所在。

数字证书通过引入证书中心（CA）来解决这个问题。CA 充当了一个信任的第三方，它会验证公钥的归属关系，并通过自己的私钥对公钥及其相关信息进行加密，生成数字证书。接收

方可以通过 CA 的公钥解密证书，验证公钥的真实性，确保收到的信息来自真实的发送者，而非被恶意篡改。

下面是一个典型的过程。首先，客户端（如浏览器）向服务器发起加密通信请求。服务器响应时，会返回自己的数字证书，并使用私钥对信息进行加密。接着，客户端使用证书中的 CA 公钥解密证书，并验证其有效性，以确保证书由受信任的 CA 签发且服务器身份无误。如果验证通过，客户端就可以使用服务器的公钥加密敏感信息，并与服务器安全地交换数据。这一过程中，数字证书确保了双方身份的真实性和数据的加密保护。

7.3.3.非对称公钥的使用场景（重点★★★★★）

这里引申出非对称公钥的使用场景。这里我们看到的多是信源验证场景，验证信息来自于某个我们可靠的信息源。此时是通过私钥加密信息（摘要）形成数字签名，然后把公钥+信息+数字签名发送出来，在接收端公钥用于解密信息，并对传递过来的信息做一个摘要，然后进行比较，假如匹配说明信源可靠。

但是公钥私钥同样可以用于加密场景（比如交换对称加密的密钥，因为非对称加密效率低，对称加密效率高。但是对称加密双方要约定一个密钥用于通讯，所以此时出现了这个场景）。前面在签名信源验证场景，我们已经通过数字证书的形式把公钥带给客户端。此时客户端可以用此公钥加密双方通信所需密钥，然后再传给服务端。服务端收到加密信息后，就可以用对应的私钥解密，能拿到密钥。

7.3.4.典型例题（重点★★★★★）

（13 年 11 月架构真题）某软件公司拟开发一套信息安全支撑平台，为客户的局域网业务环境提供信息安全保护。该支撑平台的主要需求如下：（1）为局域网业务环境提供用户身份

鉴别与资源访问授权功能；（2）为局域网环境中交换的网络数据提供加密保护；（3）为服务器和终端机存储的敏感持久数据提供加密保护；（4）保护的主要实体对象包括局域网内交换的网络数据包、文件服务器中的敏感数据文件、数据库服务器中的敏感关系数据和终端机用户存储的敏感数据文件；（5）服务器中存储的敏感数据按安全管理员配置的权限访问；（6）业务系统生成的单个敏感数据文件可能会达到数百兆的规模；（7）终端机用户存储的敏感数据为用户私有；（8）局域网业务环境的总用户数在 100 人以内。

问：在确定该支撑平台所采用的用户身份鉴别机制时，王工提出采用基于口令的简单认证机制，而李工则提出采用基于公钥体系的认证机制。项目组经过讨论，确定采用基于公钥体系的机制，请结合上述需求具体分析采用李工方案的原因。

这里实际上考察的就是非对称加密的概念：首先，公钥体系相对于基于口令的简单认证机制更安全可靠，因为口令容易受到猜测、暴力破解等攻击，而公钥体系利用非对称加密技术，安全性更高。其次，公钥体系可以提供更强的身份验证和数据传输加密保护，确保用户身份的安全性和数据的机密性。这个在介绍非对称加密的时候都已经提到。

答：（1）基于口令的认证方式实现简单，但由于口令复杂度及管理方面的原因，易受到认证攻击；而在基于公钥体系的认证方式中，由于其密钥机制的复杂性，同时在认证过程中私钥不在网络上传输，因此可以有效防止认证攻击，与基于口令的认证方式相比更为安全。

（2）按照需求描述，在完成用户身份鉴别后，需依据用户身份进一步对业务数据进行安全保护，且受保护数据中包含用户私有的终端机数据文件，在基于口令的认证方式中，用户口令为用户和认证服务器共享，没有用户独有的直接秘密信息，而在基于公钥的认证方式中，可基于用户私钥对私有数据进行加密保护，实现更加简便。

（3）基于公钥体系的认证方式协议和计算更加复杂，因此其计算复杂度要高于基于口令的认证方式，但业务环境的总用户数在 100 人以内，用户规模不大，运行环境又为局域网环境，

因此基于公钥体系的认证方式可以满足平台效率要求。

7.4.网络安全体系架构设计（次重点★★★★☆☆）

7.4.1.安全服务和安全机制（次重点★★★★☆☆）

安全服务是网络安全体系中为满足特定防护需求而设计的功能性目标，回答“需要实现什么安全能力”。安全机制是支撑安全服务落地的具体技术手段或策略，回答“如何实现安全目标”。实际上这些安全服务作用在不同的业务场景中，有登录的，有访问控制的，有数据加密的，它们作用在不同场景，共同构成了系统的安全保障体系。凯恩总结如下表所示。

安全服务	对应安全机制	安全服务的作用
鉴别服务	认证机制、数字签名机制	验证通信实体（用户、设备等）的身份真实性，确保交互对象是声称的主体，防止身份伪造。
访问控制服务	访问控制机制、路由控制机制	管理和限制用户对资源的访问权限，确保只有授权主体能访问特定资源，防止未授权的资源访问或操作。
数据机密服务	加密机制、业务流填充机制	保护数据的保密性，防止数据在传输或存储过程中被未授权方获取，业务流填充还能避免因流量分析泄露信息。
数据完整性服务	数据完整性机制	确保数据在传输、存储过程中未被篡改、删除或插入非法内容，保证数据的准确性和一致性。
抗否认性服务	公证机制	通过记录和验证操作行为，防止实体对已执行操作（如发送消息、交易等）进行否认，确保行为可追溯。

7.4.2.典型软件架构的脆弱性分析（次重点★★★★☆☆）

这块内容是凯恩认为最有可能结合具体案例考察的，它可以在介绍完架构风格之后问你这些架构风格有什么样的脆弱性。那么此时下表梳理的内容就可以派上用场。

架构名称	脆弱性梳理
分层架构	层与层依赖强，某层出错易引发整体故障；层间通信可能出现性能瓶颈问题。
C/S 架构	客户端软件易藏安全漏洞；网络开放性高，存在被攻击风险；数据易遭破坏。
B/S 架构	易被病毒、恶意代码攻击，安全性较低。
事件驱动架构	组件控制能力不足；组件间数据交互逻辑复杂；易触发死循环、高并发处理难题；固定流程存在安全漏洞。
MVC 架构	架构设计复杂；视图与控制器耦合度高，影响模块复用；视图读取模型数据效率低。
微内核结构	整体优化难度大；进程间通信开销高，导致系统通信效率下降。
微服务架构	需应对复杂开发逻辑；需额外设计通信机制处理局部故障；管理维护复杂度高。

8.上篇-大数据架构（次重点★★★★☆）

系分新教材也新增了这一章的内容，并且要求更高了，凯恩把系分新增的部分也放进来作为补充。鉴于现在命题考察形式，凯恩这里重点梳理出来的对比、图表一定要好好背出来，拿下案例第一题就稳了。

8.1.系统架构原则（次重点★★★★☆）

这一小节的内容是系分新教材的补充内容，每个原则都有具体的展开，需要有个印象。这里的原則是指假如我要设计一个大数据系统，要做到哪些事情。一般不会直接考察这里的概念，而是给出指定场景让你归类属于什么设计界原则。

原则	要点	解释
可扩展	分布式环境	在分布式计算环境运行，分配任务、管理资源，实现负载均衡，支持横向扩展计算和存储资源
	模块化设计	将系统拆分为独立模块，便于设计、开发、维护，可添加或扩展模块
可管理	—	通过自动化管理、统一管理平台、可视化管理界面，实现高效管理监控，提升效率、可靠性等
数据安全	数据加密	应用于存储、传输、处理等，如用对称加密传输、非对称加密存储
	访问控制	通过用户名和密码等方式，为不同用户分配权限
	数据备份和恢复	定期备份数据，意外丢失时恢复
	审计日志	记录用户操作，用于审计跟踪、发现问题、满足监管
	持续监控	监控日志记录、访问模式等，及时发现解决安全问题
高性能	数据分区	将大数据集分块，分配给不同计算单元处理，提高并行度和扩展性
	分布式计算	将计算任务分布到多个节点，节点独立计算并合并结果，提高并行度和效率
	并行处理	将任务分成子任务，在多个计算单元同时执行

	内存计算	将数据存于内存计算，避免磁盘 I/O 和网络传输延迟，提高效率
高可用	数据冗余	将数据复制到多个节点或设备，故障时切换保证可用性
	自动故障转移	节点或设备故障时，自动切换到其他节点或设备，可通过 ZooKeeper 等工具实现
	负载均衡	将工作任务分配到多个节点或计算单元，平衡负载，可通过 Haproxy 等工具实现
稳定性	监控和预警	实时监控组件，异常时及时警报并处理
	性能调优	优化系统组件和配置参数，通过定期测试分析找到瓶颈优化
	故障排除	建立机制和团队，快速发现和解决故障
	升级和维护	及时更新组件和软件版本，定期维护修复，保证长期稳定可靠

8.2.批处理架构 VS 流处理架构（重点★★★★★）

根据业务场景不同，我们对数据处理时效要求高，有时候要求低。这就引申出两种不同的数据处理模式，分别是批处理还有流式处理。此章节属于系分新增章节，比架构概括的好，特别是批处理和流处理的基本概念，优势和局限需要背出来，案例会考考他们的概念和相互的对比，这一节的内容必须熟悉。

架构类型	概念	优势	局限	应用场景
批处理架构	处理大规模数据批量任务，通常离线执行，含数据采集、存储、处理、输出模块；大数据集分组，按预定时间间隔或触发条件处理	可处理大规模数据，吞吐量高、稳定性好，能充分利用计算资源提升效率	实时性差，无法满足实时分析处理需求	对实时性要求不高的大规模数据处理场景

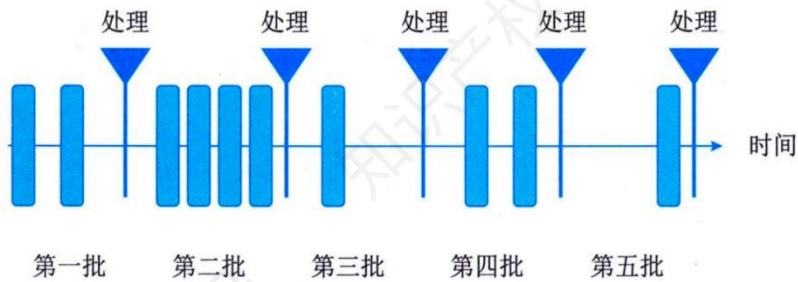


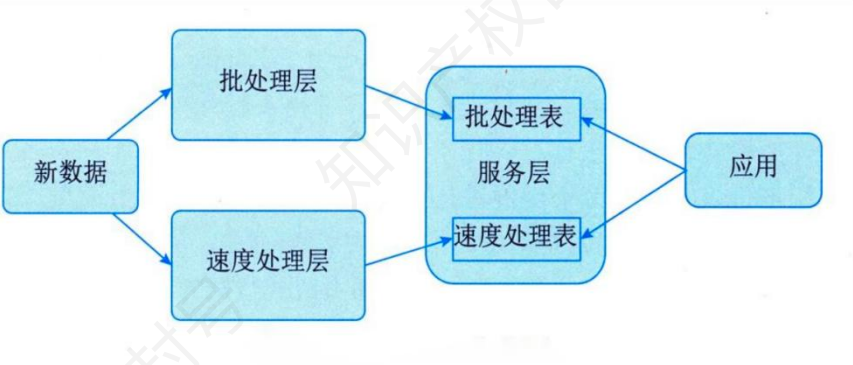
图 19-1 批处理过程

架构类型	概念	优势	局限	应用场景
流处理架构	数据连续、无限制流式处理，新数据到达即处理，由数据流、运算符组成，通过流处理引擎管理协调	实时处理数据，快速响应用户请求，适应数据快速变化	无法处理历史数据，可能漏数据，需考虑并发性和一致性问题	实时数据分析、监控、推荐等场景
混合架构	同时运用批处理和流处理，数据按需在两种处理模式间转换	兼具批处理和流处理优点，可处理历史和实时数据，快速响应用户请求	需在架构和数据处理上权衡，管理维护成本较高	既需处理历史数据又有实时数据处理需求的场景

8.3.Lambda 架构（重点★★★★★）

8.3.1.Lambda 分层介绍（重点★★★★★）

Lambda 架构的核心思想是：将批处理作业和实时流处理作业分离，各自独立运行，资源互相隔离。Lambda 架构可分解为三层，即批处理层、加速层（有的叫作速度层或者速度处理层）和服务层，如下图所示。



层次	概念	支撑技术	图例
批 处 理 层	负责所有批处理操作，存储整个数据集并计算批处理视图	Hive 、 Spark-SQL 、 MapReduce 等批处理技术	 图19-6 批处理层结构
加 速 层 / 速 度 层	使用流式计算技术实时处理当前数据，弥补批处理视图计算高延迟	Storm 、 SparkStreaming 、 Flink 等流处理技术	 图19-7 加速层结构
服 务 层	以批处理层和加速层结果数据为基础，对外提供低延时数据查询和即席查询服务	关系型数据库等传统技术，或 Kylin、Presto、Impala、Druid 等大数据 OLAP 产品	 图19-8 服务层结构

批处理层和加速层有羁绊的，常考它们的对比，分开设计的原因等，凯恩归纳为下表。

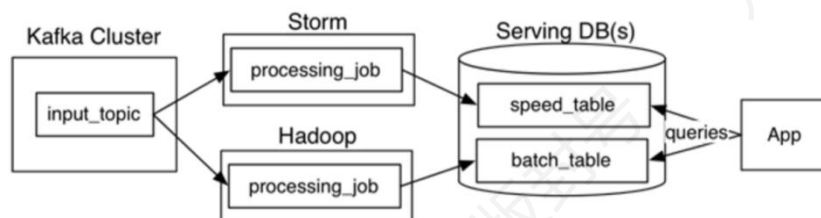
批处理层和加速层分开设计的原因	
容错性	加速层处理的数据持续写入批处理层。当批处理层重新计算的数据集涵盖了加速层处理的数据集后，可以修正加速层的一些错误
复杂性隔离	加速层运用增量算法处理实时数据，且其复杂性远高于批处理层，所以要进行隔离。
横向扩容	当系统面临的数据量或负载增大时，分层的设计可通过增加更多的机器资源来维持性能。

批处理层和加速层的对比		
对比内容	加速层	批处理层

处理目标	处理的数据是最近的增量数据流	处理的全体数据集
处理方式	接收到新数据时不断更新 RealtimeView	根据全体离线数据集直接得到 BatchView

8.3.2.Lambda 架构实现（重点★★★★★）

上一节的分层结构是逻辑上的，那么如何落地，就是实现要谈的事情。这一节最简单的就是考你概念每个技术是做什么的，稍微复杂一点是技术之间的对比，或者给定场景让你填写具体的技术方案。



这是 Lambda 架构实现概览图，Kafka 集群的 input_topic 接收原始数据，Storm 进行实时处理、Hadoop 进行批量处理，处理后数据分别存入服务数据库的 speed_table 和 batch_table，App 通过向服务数据库发送查询获取数据用于业务。各个层次使用技术总结如下表所示。

层次	名称	功能	详细解释
批处理层	Hadoop	处理大规模数据集	Hadoop 是一个开源的分布式计算平台，主要用于批量数据处理。图中的 processing_job 代表在 Hadoop 上执行的作业，它也从 Kafka 的 input_topic 读取数据，不过是进行批量处理。Hadoop 适合处理大规模数据集，对处理时间要求相对宽松的场景。
加速层	Spark 或 Storm	提供快速数据处理能力，适合迭代计算的数据挖掘和机器学习任务	Spark 和 Storm 均为分布式计算框架：Spark 基于批处理，通过微批实现准实时（秒级延迟），擅长处理大规模数据、复杂计算（如机器学习、图分析）及批流一体任务，适合离线分析、日志处理等场景；Storm 是纯流处理框架，逐条实时处理数据（亚秒级延迟），强调低延迟与消息可靠性，适用于实时监控、支付系统、事件驱动架构（如 IoT）等需立即响应的场景。Spark 侧重吞吐量与复杂逻辑，Storm 聚焦极致实时性。

服务层	HBase 或 Cassandra	提供实时的数据访问和更新	HBase 是 ApacheHadoop 生态中的分布式列存储 NoSQL 数据库，基于 HDFS 构建，支持海量数据的高并发随机读写，适合实时查询、日志存储和时序数据场景；Cassandra 是高性能分布式宽列数据库，支持高吞吐量读写和复杂数据模型，常用于社交平台动态流、实时推荐系统和金融交易记录存储。
服务层	Hive	创建可查询的视图，方便数据查询和分析	Hive 构建在 Hadoop 之上，提供了类似于 SQL 的查询语言 HiveQL，能将 SQL 语句转换为 Map-Reduce 任务在 Hadoop 上运行，使得熟悉 SQL 的用户可以方便地对存储在 Hadoop 中的大规模数据进行查询和分析

8.3.3.Lambda 架构优缺点（重点★★★★★）

Lambda 架构的优缺点也是案例喜欢考察的内容。

优劣	描述	例子
优点	容错性好	Lambda 架构为大数据系统提供了更友好的容错能力，一旦发生错误，我们可以修复算法或从头开始重新计算视图
	查询灵活度高	数据存储多样性：批、实时处理结果存于不同存储系统，满足不同查询需求；多种查询接口支持：服务层提供 SQL、RESTfulAPI 等接口，结合可视化工具展示结果
	易伸缩	分布式计算与存储：批、实时处理框架基于分布式架构，可按需添加节点；弹性资源调度：框架配备调度器，自动分配管理资源，依据任务需求动态调整
	易扩展	分层架构解耦性：分层设计，各层职责明确，扩展功能不影响其他层；插件化与接口化设计：各层组件采用插件、接口设计，便于集成新组件。书本上只从视图添加角度来讲，有点单薄。
缺点	全场景覆盖带来编码开销：技术栈复杂，开发维护成本高，代码重复；针对具体场景重新离线训练益处不大（针对推荐系统）；时间资源浪费，模型更新滞后（针对推荐系统）；重新部署和迁移成本高：环境配置复杂，数据迁移有一致性、速度等挑战	

8.4.Kappa 架构（重点★★★★★）

前面凯思提到了 Lambda 架构的缺点，Kappa 在 Lambda 的基础上进行了优化，删除了批

处理层，将数据通道以消息队列进行替代。因此对于 Kappa 架构来说，依旧以流处理为主，数据在数据湖层面进行了存储，当需要进行离线分析或者再次计算的时候，则将数据湖的数据再次经过消息队列重播一次则可。核心组件包括：

核心组件	内容
消息传输层	使用 Kafka 等消息队列实现高吞吐、低延迟的数据传输，支持数据持久化和重放。
流处理层	采用 Flink、Storm 等框架进行实时计算，生成低延迟结果并输出至服务层。
数据存储	处理后的数据存储于 HBase、Cassandra 等高性能存储系统，支持快速查询

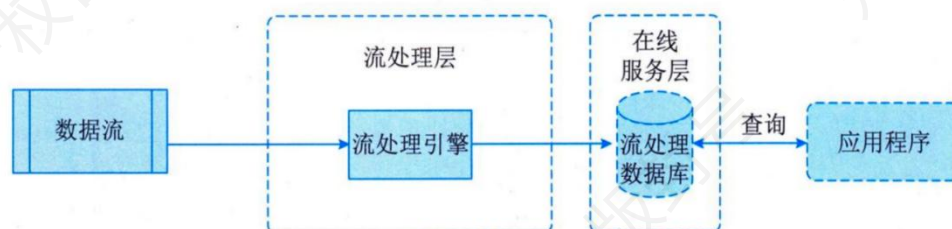


图 19-4 Kappa 架构图

从上图中可以看到，在 Kappa 架构中，数据从源头产生的那一刻起就被看作是一条持续不断的事件流。这些事件通常会首先进入一个支持高吞吐和持久化的消息中间件，如 Kafka 或 Pulsar，它既是数据的入口，也是整个系统的数据总线。每个事件都可以被多路消费、重复回放，从而为后续的计算和分析提供可靠的输入。接下来，流处理引擎（如 Apache Flink、Kafka Streams 或 Spark Structured Streaming）承担着核心角色，它持续地接收、计算和更新数据结果，通过窗口、聚合、过滤、关联等算子实现复杂的实时计算逻辑。与批处理不同的是，这种流计算具备状态管理能力，可以在数据不断到来的情况下保持结果的连贯与一致。

经过计算后的结果，会被写入不同的下游系统形成“服务层”。一些结果会进入 Redis、ClickHouse、Elasticsearch 等系统，用于实时监控、搜索和分析；另一些可能被同步到数据湖或数据仓库中（如 Hudi、Iceberg），以便后续离线分析和机器学习使用。整个系统中，Kafka

或消息总线起到“可回放日志”的作用——当算法逻辑需要调整或程序出现错误时，开发者只需重置消费位点并回放历史数据流，就能重新生成结果，而无需额外的离线批处理程序。

因此，Kappa 架构的最大优势在于**统一性与简洁性**。它用一套流处理框架，替代了原本分裂的批处理与流处理系统，使开发、测试、部署都更容易一致化。它更适合那些对实时性要求高、数据持续产生的场景，例如金融风控、日志分析、IoT 监控、用户行为追踪等。可以说，Kappa 架构代表了现代大数据系统从“数据最终一致”向“数据实时一致”演进的方向——让系统不仅能“知道过去发生了什么”，更能“当下立刻感知变化”，并实时驱动决策。

8.4.1.Kappa 架构的优缺点（重点★★★★★）

Kappa 架构的优缺点也是案例喜欢考察的内容，需要加以记忆。

优劣	描述	例子
优点	将实时和离线代码统一，方便维护，统一数据口径	使用 Kappa 架构处理订单数据，开发人员无需分别维护实时和离线两套代码，能更高效地进行功能迭代和问题排查，避免因数据不一致导致的分析偏差。
缺点	消息中间件缓存的数据量和回溯数据有性能瓶颈	当需要回溯过去几个月的消息数据进行分析时，由于消息中间件缓存数据量过大，可能导致回溯操作缓慢甚至超时，影响数据分析的效率。
	实时数据处理时，大量不同实时流关联易因数据顺序问题导致数据丢失	在交通流量监控系统中，有车辆位置流、交通信号灯状态流等多个实时流数据。当关联这些数据计算车辆在不同信号灯状态下的通行情况时，若数据到达顺序混乱，可能会遗漏某些车辆在特定信号灯状态下的记录。
	抛弃离线数据处理模块，同时抛弃离线计算更稳定可靠的特点	采用 Kappa 架构后，过度依赖实时计算，在网络波动或实时计算系统出现短暂故障时，可能无法及时、准确地识别风险。

8.4.2.Lambda VS Kappa（超级重点★★★★★）

架构爱考概念对比，下面的表格内容需要熟记，23 年架构已经考过挖空填空题。

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性（这个没错）	满足实时性
历史数据处理能力	批式全量处理，吞吐量巨大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱

8.5.IOTA 架构（次重点★★★★☆☆）

系分新增的内容，了解即可，IOTA 的主要特点是引入边缘计算和统一数据模型。

IOTA 架构是一种新兴的大数据处理系统架构，它强调数据流的连续性和一致性。IOTA 的整体思路是设定标准数据模型，通过边缘计算技术把所有的计算过程分散在数据产生、计算和查询过程当中，以统一的数据模型贯穿始终，从而提高整体的计算效率，同时满足计算的需要，可以使用各种即时查询来查询底层数据。下图是 IOTA 的基本架构。

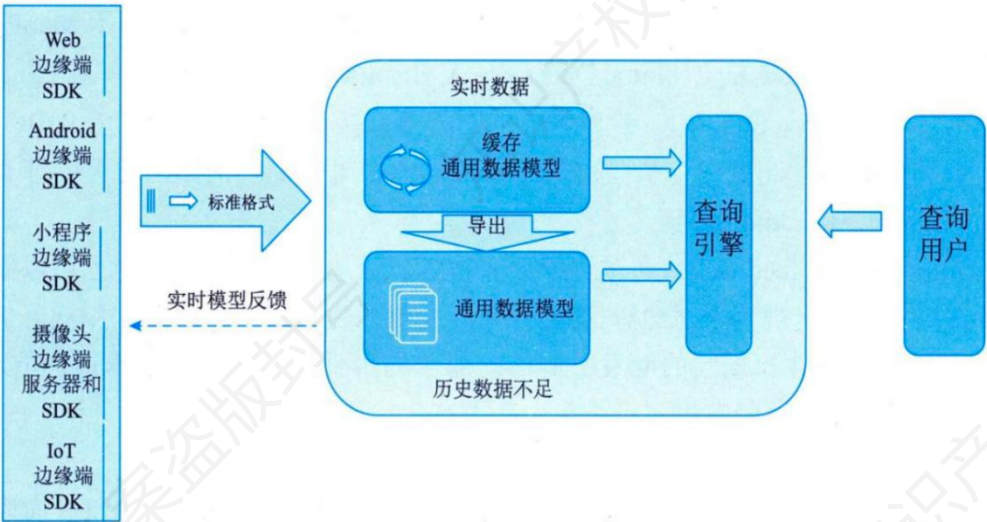


图 19-5 IOTA 架构图

书本上讲的很简单，看凯恩这里的补充。

统一数据模型（CommonDataModel）。IOTA 架构通过定义“主-谓-宾”或“对象-事件”等抽象数据模型，贯穿数据采集、处理、存储全流程，确保从 SDK 到查询引擎的数据一致性。例如，用户行为数据可统一表示为“X 用户 - 事件 1 - A 页面（时间戳）”。

边缘计算与实时处理。在设备端（EdgeSDKs）完成数据校验、格式转换和边缘聚合计算，减少中心化服务器的压力。例如，智能 Wi-Fi 采集的数据在设备端直接转换为“用户 MAC 地址 - 出现 - 楼层（时间）”的统一模型。

分层存储与实时缓存。实时数据缓存区（Real-TimeData）：使用 Kudu 或 HBase 暂存最近几分钟数据，支持低延迟查询。历史数据沉浸区（HistoricalData）：通过 Dumper 将实时数据合并至 HDFS 等存储系统，支持秒级复杂查询。

IOTA 的优缺点如下表所示。

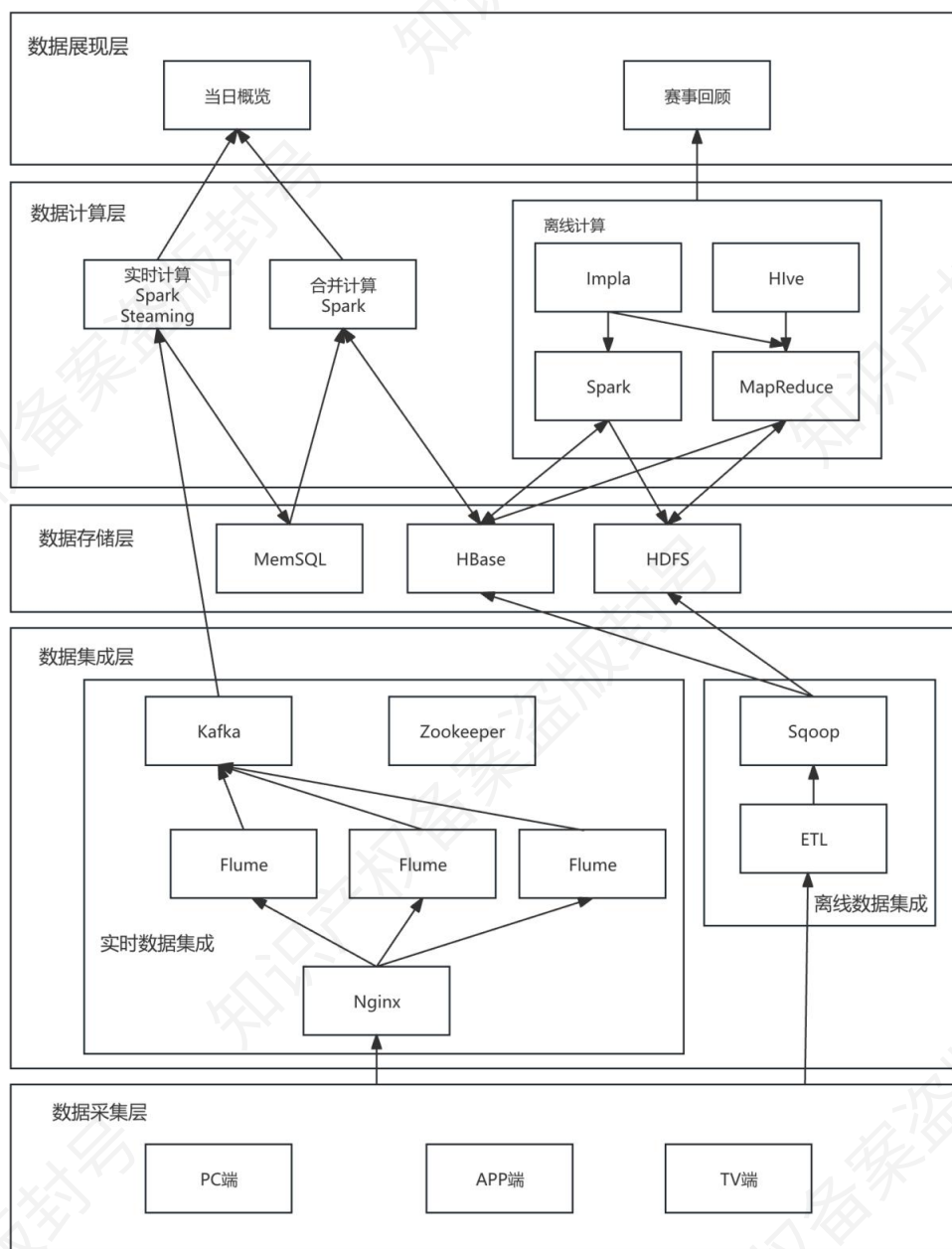
类别	要点	具体说明
优点	简化架构复杂度	删除 Lambda 架构的批处理层，仅需维护流式处理逻辑，降低开发和运维成本。

	低延迟处理	边缘计算（EdgeSDKs）减少中心化处理步骤，支持毫秒级实时响应。
	统一数据模型	通过“主-谓-宾”模型统一数据格式，避免批流计算结果不一致问题，提升数据一致性。
缺点	历史分析性能瓶颈	依赖流式重放全量数据日志，超大规模历史数据分析效率较低，难以支持复杂离线计算。
	开发与调试难度高	需处理数据乱序、延迟、幂等性等问题，业务逻辑复用性低，开发复杂度较高。
	存储资源消耗大	需长期保留全量数据日志以支持重放，存储成本显著高于仅保留结果的 Lambda 架构。

8.6.典型架构（重点★★★★★）

这里有三个典型架构，23 年出过挖空填空，目的是看你是否熟悉这些技术的具体应用。因此属于非常重点的精华章节，必须要看熟。

8.6.1.某网奥运（重点★★★★★）

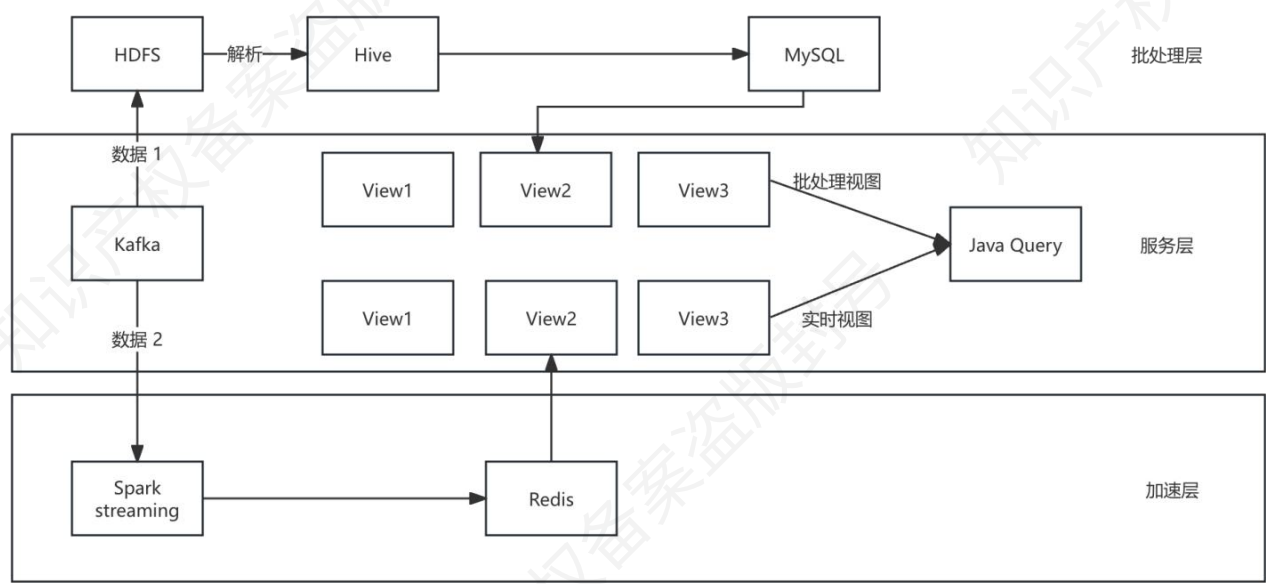


数据从 PC 端、APP 端、TV 端等来源，经数据采集层的 Flume、Kafka 等实时采集组件以及 Sqoop、ETL 等离线采集工具汇聚，存储于 MemSQL、HBase、HDFS 等存储层；计算层中，SparkStreaming 等负责实时计算，Impala、Hive 等进行离线计算；最终在数据展现层以日志展现、实时展现等形式呈现处理结果。此图中所涉及的技术可以归纳为下表，每个技术做什么的必须知道，选词填空中常见。

层级	涉及技术	用处说明
数据采集层	PC 端、APP 端、TV 端	作为数据来源，产生用户操作、业务等原始数据，是数据流程的起点。
数据集成层	Flume	实时收集、聚合、传输分散日志数据，高效传输至 Kafka。
	Nginx	负载均衡，优化数据采集节点流量分配，保障数据稳定收集。
	Kafka	分布式消息队列，缓存实时数据，解耦生产与消费，支持高吞吐量流式处理。
	Zookeeper	为 Kafka 等组件提供协调服务，管理集群节点状态与配置信息。
	ETL	对离线数据提取、转换、加载，清洗并转换为适合分析的格式。
	Sqoop	在关系型数据库与 Hadoop 生态间批量导入/导出数据，整合异构离线数据源。
数据存储层	MemSQL	内存数据库，高速读写，存储实时计算结果或高频访问热数据。
	HBase	分布式 NoSQL 数据库，存储海量半结构化/非结构化数据，支持高并发随机读写。
	HDFS	分布式文件系统，存储大规模离线数据，具备高容错性和扩展性。
数据计算层	SparkStreaming	流计算框架，处理实时数据流，实现实时统计、分析。
	Spark	通用分布式计算框架，处理批量数据合并计算，支持复杂数据处理逻辑。
	Impala	实时交互式查询引擎，配合 Hive 对 HDFS/HBase 数据快速 SQL 查询。
	Hive	将结构化数据映射为表，支持 HQL 查询，转换为 MapReduce 任务处理离线数据分析。

	MapReduce	离线批量计算模型，拆分任务为 Map 和 Reduce 阶段，处理大规模分布式计算。
数据展现层	当日概览、赛事回顾	将计算结果可视化呈现，供用户查看实时动态或历史数据，实现数据价值落地。

8.6.2.某网络广告平台（重点★★★★★）

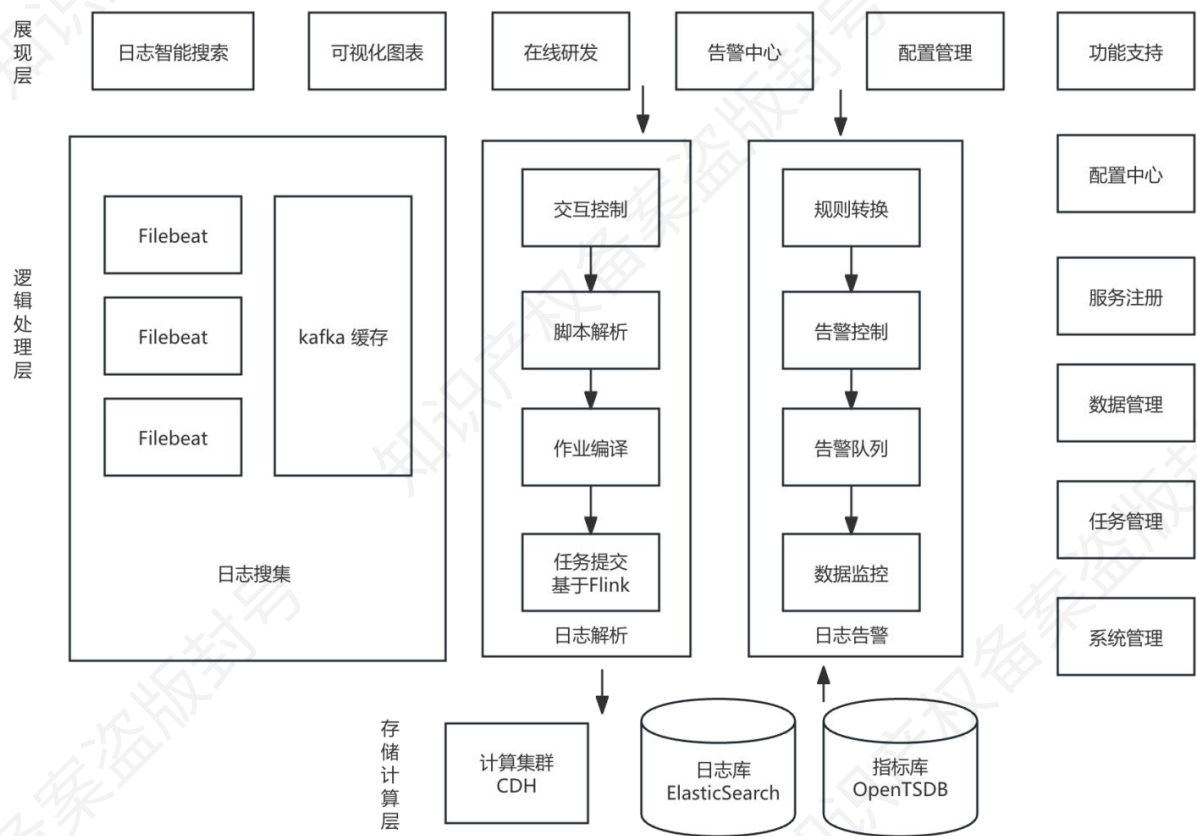


在批处理层，HDFS 存储原始数据，Hive 对其进行离线分析处理后存入 MySQL；服务层里，Kafka 作为消息队列传输数据，批处理视图和实时视图（View1-View3）基于不同处理结果构建，通过 JavaQuery 获取数据；加速层中，SparkStreaming 进行实时数据处理，Redis 缓存处理结果以提升性能，各层组件协同满足业务的数据处理与查询需求。

层级	涉及技术	用处说明
批处理层	HDFS	分布式文件系统，用于存储大规模原始数据，为后续批处理提供数据基础。
	Hive	基于 HDFS 的数据仓库工具，通过 HQL 解析处理 HDFS 中的数据，实现批量数据的分析、转换。
	MySQL	关系型数据库，存储 Hive 处理后的结构化结果数据，供上层服务层查询调用。

服务层	Kafka	分布式消息队列，接收并缓存数据（数据 1、数据 2），解耦数据生产与消费，支持实时数据流处理。
	View1-View3	分为批处理视图和实时视图，对批处理层（如 MySQL）和加速层处理后的数据进行逻辑封装，方便上层统一查询。
	JavaQuery	业务查询接口，通过调用批处理视图和实时视图，实现数据的业务逻辑查询与展示。
加速层	SparkStreaming	流计算框架，实时处理 Kafka 中的数据（数据 2），进行实时计算、清洗或聚合。

8.6.3.某证券公司日志大数据系统（重点★★★★★）

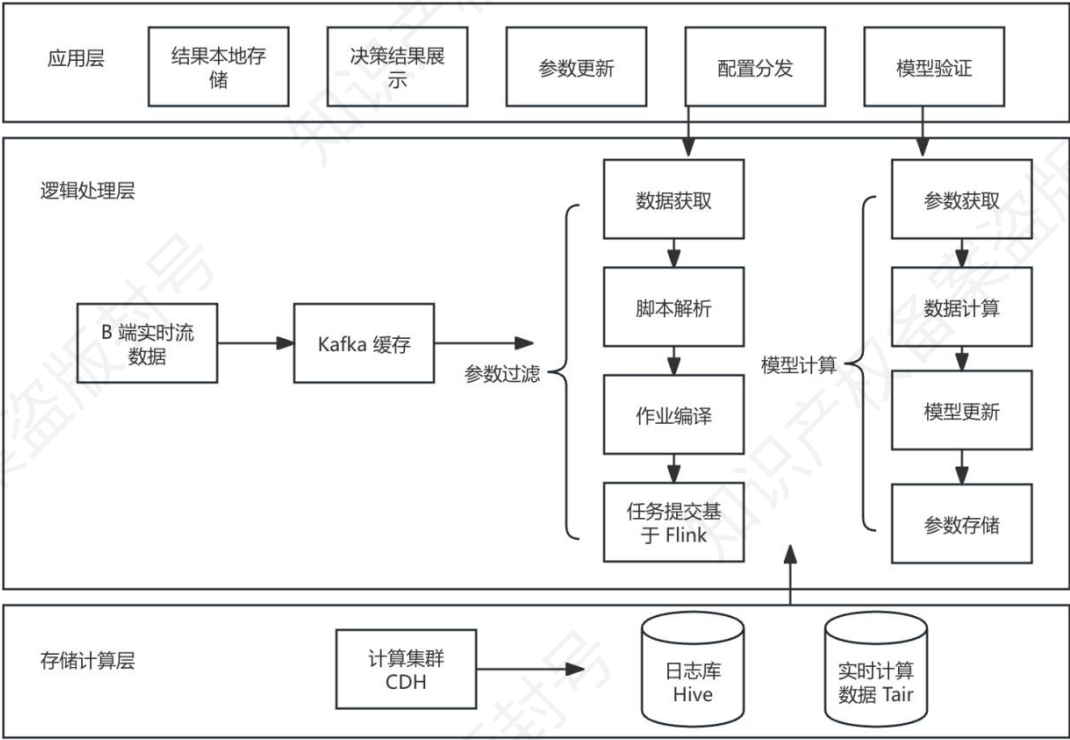


在逻辑处理层，Filebeat 负责日志搜集并暂存于 Kafka 缓存；搜集后的日志经交互控制、脚本解析、作业编译等流程基于 Flink 进行日志解析，任务提交至计算集群 CDH，同时规则转换、告警控制等实现日志告警，数据分别存入日志库 ElasticSearch 和指标库 OpenTSDB。在展

现层，提供日志智能搜索、可视化图表等功能，整个系统由配置中心、服务注册等进行管理支持。这里的关键是逻辑处理层和存储计算层，这里的技术梳理如下所示。

层级	涉及技术	用处说明
逻辑处理层	Filebeat	轻量级日志收集工具，收集不同数据源日志，传输至 Kafka。
	Kafka	缓存日志数据，解耦日志收集与处理，支持高吞吐量实时数据缓冲。
存储计算层	CDH	大数据计算集群，提供 Hadoop 生态组件，支持日志数据分布式存储与计算。
	ElasticSearch	作为日志库，存储处理后的日志，支持高效检索与分析。
	OpenTSDB	作为指标库，存储时间序列数据，适合高频更新的时序指标存储与查询。

8.6.4.某电商智能决策大数据系统（重点★★★★★）



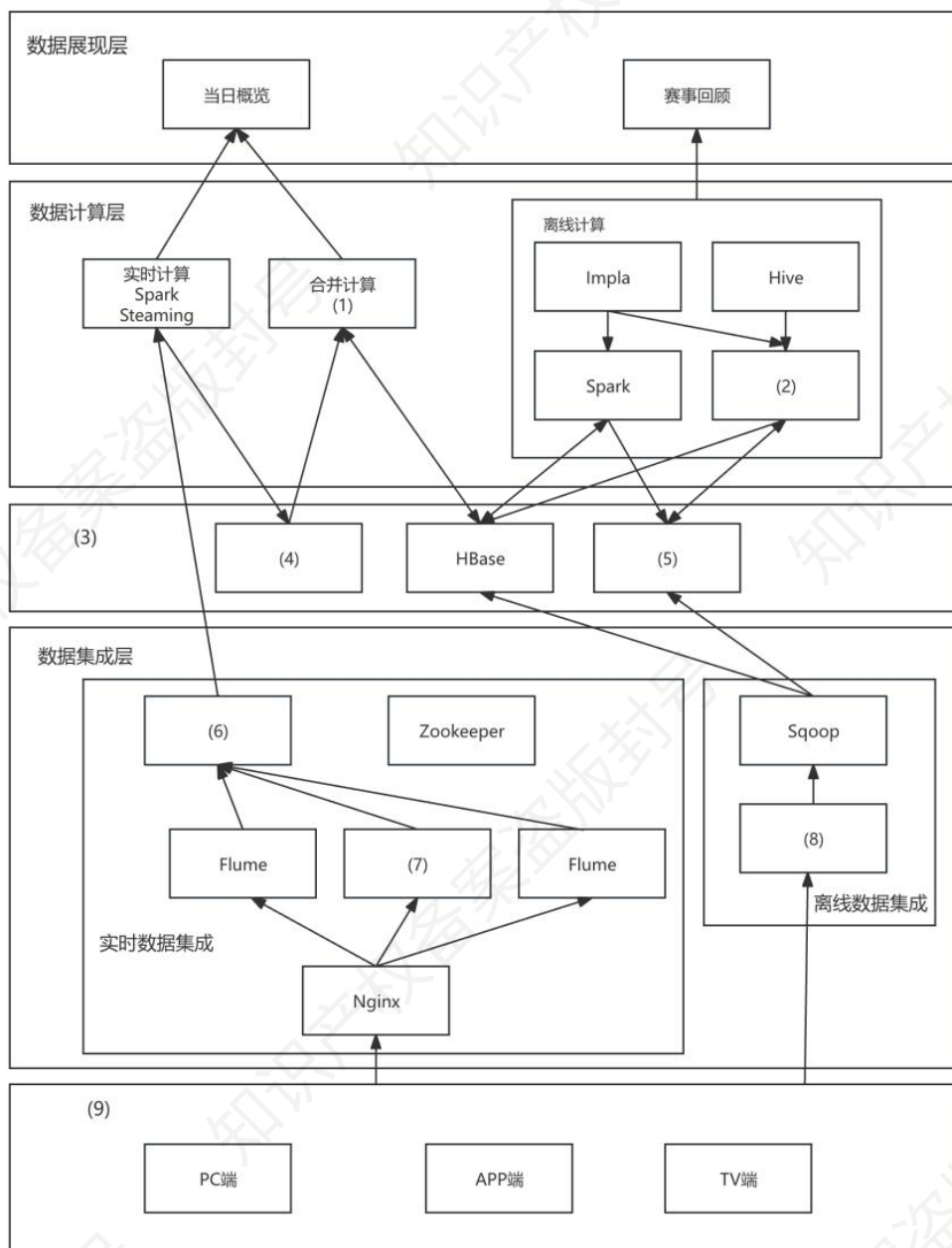
B 端实时流数据先经 **Kafka** 缓存，经参数过滤后，在逻辑处理层，一方面通过数据获取、脚本解析、作业编译，基于 **Flink** 提交任务；另一方面经参数获取、数据计算等实现模型计算及更新。存储计算层中，计算集群 **CDH** 配合日志库 **Hive**、实时计算数据 **Tair** 进行存储计算。应用层提供结果本地存储、决策结果展示等功能。这个架构图的逻辑和上一个很像，唯一区别是存储计算层的技术这里选的是 **Hive**。

层级	技术	用处说明
逻辑处理层	Kafka	作为消息队列，缓存 B 端实时流数据，解耦数据生产与消费，支持高吞吐量数据缓冲。
	Flink	基于 Flink 提交作业任务，处理实时流数据，实现数据的实时计算与作业执行。
存储计算层	CDH	提供大数据计算集群环境，支持分布式存储与计算，是底层数据处理的基础平台。
	Hive	作为日志库，存储处理后的日志数据，支持批量数据的存储、查询与分析。
	Tair	存储实时计算数据，利用其高效读写特性，支持实时计算结果的快速存储与获取。

8.7.典型例题（重点★★★★★）

（23 年 11 月架构真题）某网作为某电视台在互联网上的大型门户入口，某一年成为某奥运会中国大陆地区的特权转播商，独家全程直播了某奥运会全部的赛事，积累了庞大稳定的用户群，这些用户在使用各类服务过程中产生了大量数据，对这些海量数据进行分析与挖掘，将会对节目的传播及商业模式变现起到重要的作用。该奥运期间需要对增量数据在当日概览和赛事回顾两个层面上进行分析。

问：下图给出了某网奥运的大数据架构图，请根据下面的(a)~(n)的相关技术，判断这些技术属于架构图的哪个部分，补充完善下图 1 的(1)-(9)的空白处。

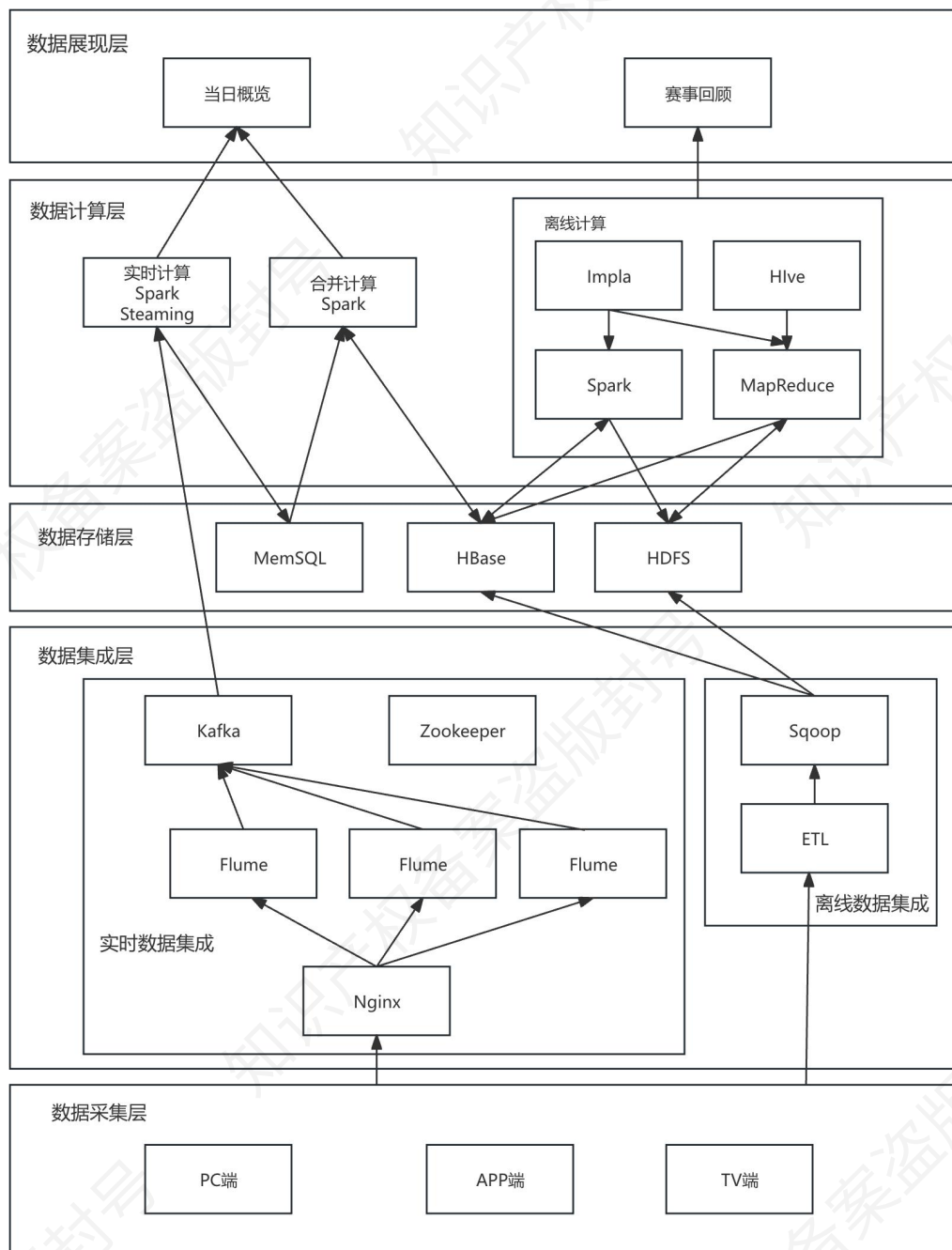


(a) Nginx; (b) Hbase; (c) SparkStreaming (d) Spark; (e) MapReduce; (f) ETL;

(g) MemSQL; (h) HDFS; (i) Sqoop; (j) Flume; (k) 数据存储层; (l) kafka; (m)

业务逻辑层; (n) 数据采集层

答: (1) d (2) e (3) k (4) g (5) h (6) l (7) j (8) f (9) n



9.下篇-微服务架构（次重点★★★★☆）

假设我们要开发一个类似京东、淘宝的商城系统，里面有商品、订单、用户、支付、物流、营销、搜索等功能。以 **Java** 语言为例，刚开始项目不大，我们下意识会把所有功能都写在一个 **Spring Boot** 项目里，打成一个 **jar** 包，部署到一台服务器上。这就是单体架构。

单体架构的好处是简单。开发简单，部署简单，排查问题也简单。用户下单时，订单代码可以直接调用商品代码，库存扣减也可以直接操作同一个数据库里的商品表。但随着业务变大，单体架构的问题就会越来越明显。

比如订单功能只是改了一个很小的逻辑，比如新增“预售订单”或者修改“订单超时取消规则”，按理说只影响订单模块。但在单体架构里，订单、商品、库存、支付都打在同一个包里，最后往往要把整个商城系统重新打包、重新部署。这样一来，改动虽然很小，但发布风险很大。比如商品模块出现了问题，原本只是商品详情页查得比较慢，但因为所有功能都运行在同一个应用里，它可能把数据库连接、线程池、内存资源都占满。结果用户不仅看商品变慢，连下单、支付、登录也可能受到影响。也就是说，一个局部模块的问题，可能拖垮整个系统。

在比如，平时商城访问量不大，一台服务器还能撑住。但到了双十一、秒杀活动或者直播带货的时候，压力主要集中在商品查询、库存扣减和下单这几个功能上。可单体架构没法只扩容订单模块或者库存模块，只能把整个商城系统复制多份一起扩容。这样虽然也能提升性能，但资源浪费比较大。

还有团队协作的问题。系统小的时候，几个人改一个项目还能接受。等团队变大以后，订单组、商品组、支付组、营销组都在同一个代码仓库里开发，代码依赖越来越复杂。一个人改了公共工具类，可能影响好几个模块；一个模块发布延期，可能拖住整个系统上线。

还有就是技术选型受限制。比如普通的商品、订单功能用 **Java** 开发没问题，但后面如果

要加入直播、音视频、实时推荐、AI 智能客服等功能，可能更适合用 Go、Python 或专门的音视频服务框架。如果系统仍然是一个大单体，这些新功能就很难独立开发、独立部署、独立扩展。

9.1.微服务架构下的特定问题（重点★★★★★）

微服务架构首先要面对的是服务拆分问题。一个商城系统不能随便拆。这里的拆分，是指要按照业务边界拆。商品服务只负责商品查询、商品上下架、库存维护；订单服务只负责创建订单、查询订单、取消订单；支付服务只负责支付和退款；用户服务只负责用户资料和登录信息。每个服务都应该能独立开发、独立部署、独立扩容。

第二个问题是服务之间通信。单体项目里，订单代码调用商品代码，只是一次本地方法调用。拆成微服务以后，订单服务和商品服务可能部署在不同机器上，订单服务要查商品信息，就要通过网络调用商品服务。问题是，商品服务到底在哪台机器上？如果商品服务扩容成三个实例，订单服务应该调哪一个？如果其中一台挂了，订单服务怎么避开它？这就需要服务注册和服务发现。

第三个问题是外部请求怎么进入系统。单体架构中，前端只访问一个商城应用即可。微服务架构中，后端变成了商品服务、订单服务、支付服务、用户服务、物流服务等多个服务。前端不可能记住每个服务的地址，也不应该把每个微服务都直接暴露出去。所以需要有一个统一入口，统一接收外部请求，再转发到对应的微服务。

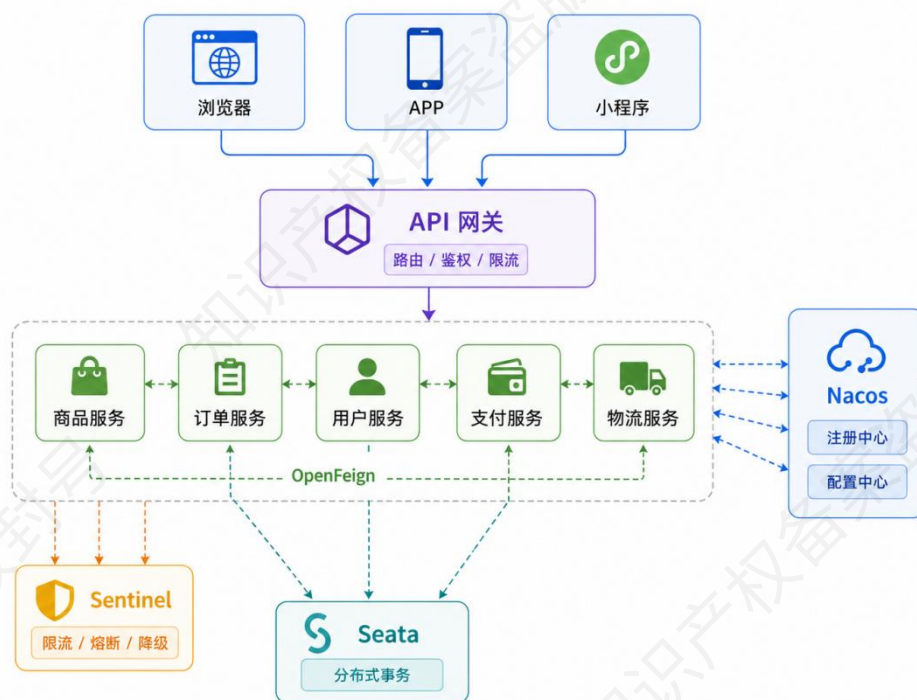
第四个痛点是微服务架构下配置难管理。单体项目只有一份配置文件，改起来还算容易。微服务多了以后，每个服务都有自己的配置，比如端口、数据库、缓存、超时时间、开关参数、限流规则等。如果几十个服务、几百个实例都靠本地配置文件维护，后期会非常麻烦。所以需要配置中心，把配置统一管理起来。

第五个痛点是跨服务、跨数据库的事务问题。单体项目里，下单、扣库存、扣余额可能都在一个数据库里，一个本地事务就能保证一致性。微服务架构里，订单服务操作订单库，商品服务操作商品库，账户服务操作账户库。创建订单成功了，但扣库存失败怎么办？库存扣了，但账户扣款失败怎么办？这就是分布式事务问题。这个我们单独开一章来讲。

这些问题，下面我们都会做出解答。

9.2. 基于 Spring Cloud 的微服务整体架构

虽然微服务架构大部分语言都有对应的框架实现，但是纯脱离语言来说又有点生硬，所以还是结合当下最流行的 Spring Cloud 来介绍一下微服务架构。一个典型的 Spring Cloud 微服务系统，可以按层次来理解。



最外层是访问层。用户通过浏览器、APP、小程序访问商城系统。前端不直接访问商品服务、订单服务、支付服务，而是统一访问网关。网关层负责统一入口、请求路由、鉴权、跨域、限流、安全防护等。比如用户访问 `/api/product/10001`，网关就把请求转发到商品服务；用户访

问 `/api/order/create`，网关就把请求转发到订单服务。服务层是一个个具体微服务，比如商品服务、订单服务、用户服务、支付服务、物流服务。每个服务都是独立的 **Spring Boot** 应用，可以独立部署，也可以横向扩容成多个实例。注册中心保存每个微服务的地址。商品服务启动以后，把自己的服务名、IP、端口注册到 **Nacos**；订单服务启动以后，也注册到 **Nacos**。后续订单服务想调用商品服务，不需要写死 IP，而是通过服务名去注册中心发现商品服务实例。远程调用组件负责服务之间的调用。**Spring Cloud** 里常用 **OpenFeign**。它可以让订单服务像调用本地接口一样调用商品服务，底层实际还是通过 **HTTP** 发起远程请求。配置中心负责统一管理配置。**Nacos** 不仅可以做注册中心，也可以做配置中心。比如订单超时时间、营销开关、库存预警阈值等，可以放在 **Nacos** 中统一管理，避免每次改配置都重新打包部署。服务保护层负责限流、熔断和降级。**Sentinel** 可以对秒杀接口、下单接口、远程调用进行保护。流量太大时限流，下游服务变慢时熔断，系统压力过高时降级。分布式事务组件负责跨服务一致性。**Seata** 可以用来处理订单服务、商品服务、账户服务之间的分布式事务问题。

9.3.服务拆分与集群部署（重点★★★★★）

前面提到，微服务架构的第一步，是把一个大应用按照业务边界拆开。

以电商系统为例，可以拆成商品服务、订单服务、用户服务、支付服务、物流服务、营销服务、搜索服务。每个服务只负责自己的业务。商品服务不要直接改订单数据，订单服务也不要直接查商品库。服务之间要通过接口协作，而不是互相访问数据库。这样做的好处是边界清楚。商品团队可以只维护商品服务，订单团队可以只维护订单服务。商品服务访问量大，就单独扩容商品服务；订单服务压力大，就单独扩容订单服务。某个服务升级，也不一定影响整个系统。拆分以后，每个微服务都可以有多个实例。比如商品服务可以部署在 9000、9001、9002 三个端口，形成商品服务集群。订单服务可以部署在 8000、8001 两个端口，形成订单服务集

群。这里会涉及几个基础概念。如下表所示。

概念	说明
节点	一台服务器，或者一个运行实例。
副本	同一个服务复制出来的多个实例。
集群	多个节点一起工作。它们可以提供同一种能力，也可以承担不同角色。
负载均衡	把请求尽量均匀地分配给多个节点，避免某个节点太忙、某个节点太闲。
扩容	增加节点。
缩容	减少节点。

关于集群这里多说一句，严格意义上来说，集群和分布式不是一回事。集群强调的是“多台机器一起工作”，哪怕每台机器部署的是同一个完整商城应用，也可以叫集群。分布式强调的是“一个大系统拆成多个小服务，分布在不同机器上协作”。微服务架构通常既有分布式，也有集群。

9.4.Nacos 服务注册与服务发现（重点★★★★★）

微服务架构下，服务之间如何通信也变成了一个问题。以前订单服务调用商品代码，只是一个本地方法调用。现在订单服务和商品服务部署在不同机器上，订单服务要获取商品信息，就要通过网络调用商品服务。最容易想到的方式，是在订单服务里写死商品服务地址：

```
String url = "http://127.0.0.1:9000/api/product/" + productId;
Product product = restTemplate.getForObject(url, Product.class);
```

这个写法的问题在于商品服务地址一旦变化，订单服务就要改代码。商品服务如果有多个实例，订单服务也不知道该调哪一个。另外，在这种硬编码的方式下，假如某个实例挂了，订单服务仍然会继续调用它。

所以需要注册中心。注册中心可以理解成微服务系统里的“服务通讯录”。每个服务启动以后，都把自己的服务名、IP、端口注册到注册中心。比如商品服务启动后告诉 Nacos：我是 service-product，我在 192.168.1.10:9000。订单服务启动后告诉 Nacos：我是 service-order，我

在 192.168.1.11:8000。

如果商品服务启动了三个实例，Nacos 里就会保存类似这样的列表：

```
service-product
  192.168.1.10:9000
  192.168.1.11:9001
  192.168.1.12:9002
```

订单服务要调用商品服务时，不再写死 IP，而是先问 Nacos: service-product 现在有哪些可用实例？Nacos 返回实例列表以后，订单服务再从中选择一个实例发起调用。这个过程就是服务发现。商品服务配置大概如下：

```
server:
  port: 9000

spring:
  application:
    name: service-product
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
```

订单服务配置大概如下：

```
server:
  port: 8000

spring:
  application:
    name: service-order
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
```

这里最关键的是 `spring.application.name`。它就是服务注册到 Nacos 里的名字。以后别的服务要找商品服务，不是找某个固定 IP，而是找 `service-product` 这个服务名。

9.5.OpenFeign：声明式远程调用（重点★★★★★）

有了注册中心以后，订单服务已经知道可以通过服务名找到商品服务。但如果每次调用都要自己获取实例、拼 URL、发 HTTP 请求，代码还是很麻烦。所以 Spring Cloud 提供了 OpenFeign。

OpenFeign 可以理解成声明式远程调用。所谓声明式，就是你不用自己拼 URL，不用自己写 RestTemplate，只要写一个 Java 接口，告诉 Spring Cloud：我要调用哪个服务的哪个接口。剩下的服务发现、负载均衡、HTTP 请求发送，都由框架帮你完成。比如订单服务要调用商品服务，可以先在启动类上开启 Feign：

```
@SpringBootApplication
@EnableFeignClients
public class OrderApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderApplication.class, args);
    }
}
```

然后定义一个商品服务客户端：

```
@FeignClient(value = "service-product")
public interface ProductClient {

    @GetMapping("/api/product/{id}")
    Product getProduct(@PathVariable("id") Long id);
}
```

这里的 service-product 就是 Nacos 中注册的商品服务名。/api/product/{id} 是商品服务暴露出来的接口路径。订单服务创建订单时，就可以像调用本地方法一样调用商品服务：

```
@Service
public class OrderService {

    @Autowired
    private ProductClient productClient;

    public Order createOrder(Long productId) {
        Product product = productClient.getProduct(productId);
    }
}
```

```
        Order order = new Order();
        order.setProductId(product.getId());
        order.setProductName(product.getName());
        order.setAmount(product.getPrice());
        return order;
    }
}
```

这段代码看起来像本地调用，实际上底层是远程 HTTP 调用。OpenFeign 会根据 service-product 去注册中心找商品服务实例，再通过负载均衡选择一个实例发送请求。

9.6.Gateway：微服务系统的统一入口（重点★★★★★）

第三个问题是外部请求怎么进入系统。在单体架构里，用户直接访问商城应用即可。可是在微服务架构里，后端变成了商品服务、订单服务、支付服务、用户服务、物流服务。前端不可能记住每个服务的地址，也不应该直接暴露所有微服务地址给外部用户。

所以需要网关，也就是 Gateway。

Gateway 可以理解成微服务系统的大门。所有外部请求先进入 Gateway，再由 Gateway 判断这个请求应该转发给哪个微服务。比如请求路径以 /api/product 开头，就转发给商品服务；请求路径以 /api/order 开头，就转发给订单服务；请求路径以 /api/pay 开头，就转发给支付服务。基本流程如下：前端请求→Gateway→根据路由规则判断→转发到具体微服务。

Gateway 的作用不只是转发请求。因为所有外部流量都会经过网关，所以它还可以做统一鉴权、跨域处理、限流、安全防护、日志记录、监控统计、灰度发布、协议转换等。

Spring Cloud Gateway 的配置可以这样写：

```
server:
  port: 80

spring:
  application:
    name: service-gateway
  cloud:
    nacos:
```

```
discovery:
  server-addr: 127.0.0.1:8848
gateway:
  routes:
    - id: product-route
      uri: lb://service-product
      predicates:
        - Path=/api/product/**

    - id: order-route
      uri: lb://service-order
      predicates:
        - Path=/api/order/**
```

这里的 `server.port: 80` 表示网关占用 80 端口。前端以后只需要访问网关，不需要记住商品服务 9000 端口、订单服务 8000 端口。

`id` 是路由规则的唯一标识。`uri` 是请求要转发到哪里。`lb://service-product` 中的 `lb` 表示 load balancer，也就是负载均衡。它不是写死某个 IP，而是根据服务名 `service-product` 去注册中心找可用实例。`predicates` 是断言规则。`Path=/api/product/**` 表示只要路径以 `/api/product/` 开头，就转发到商品服务。

有的同学会问，网关那么好，可以对外暴露统一入口，微服务之间的调用要不要经过网关？一般情况下，不经过。比如订单服务要调用商品服务，订单服务会通过 `OpenFeign` 根据 `service-product` 去 `Nacos` 中发现商品服务实例，然后直接调用商品服务。它不需要先绕到 `Gateway`，再由 `Gateway` 转发给商品服务。所以，`Gateway` 主要管外部流量入口，服务内部调用一般通过注册中心和 `OpenFeign` 直接完成。

9.7.Nacos 配置中心（重点★★★★☆）

前面提到微服务多了以后，配置管理会变成一个很现实的问题。在单体项目里，配置文件一般放在项目本地，比如 `application.yml`。如果配置变了，比如库存预警阈值、营销开关、接口超时时间发生变化，就修改配置文件，然后重新打包部署。

微服务多了以后，这种方式就不太适合了。几十个服务、几百个实例，如果都靠本地配置文件维护，会非常麻烦。而且有些配置只是临时调整，比如活动开关、限流阈值、超时时间，并不希望每次都重新发版。

配置中心就是把配置统一放到一个地方管理。服务启动时从配置中心读取配置，配置变化时还可以动态刷新。**Nacos** 除了可以做注册中心，也可以做配置中心。比如订单服务可以从 **Nacos** 配置中心读取配置：

```
spring:
  application:
    name: service-order
  config:
    import: optional:nacos:service-order.yml
  cloud:
    nacos:
      config:
        server-addr: 127.0.0.1:8848
```

然后在 **Nacos** 中创建一个 `service-order.yml` 配置文件：

```
order:
  timeout: 3000
  enable-coupon: true
```

代码中可以读取这些配置：

```
@Component
@ConfigurationProperties(prefix = "order")
public class OrderProperties {

    private Integer timeout;
    private Boolean enableCoupon;

    public Integer getTimeout() {
        return timeout;
    }

    public void setTimeout(Integer timeout) {
        this.timeout = timeout;
    }
}
```



```
}

public Boolean getEnableCoupon() {
    return enableCoupon;
}

public void setEnableCoupon(Boolean enableCoupon) {
    this.enableCoupon = enableCoupon;
}
}
```

这样配置就从本地文件转移到了 Nacos。后续要调整订单超时时间，不需要重新改代码，只要在配置中心修改配置即可。

9.8.Sentinel 服务保护与流量控制（重点★★★★★）

微服务系统里，最怕的不是某一个服务单独出问题，而是一个服务的问题沿着调用链不断放大，最后拖垮整个系统。比如用户下单时，请求链路可能是：

用户请求 → Gateway → 订单服务 → 商品服务 → 库存服务 → 支付服务

如果支付服务突然变慢，订单服务一直等支付服务返回。大量请求都卡在这里，订单服务线程被占满。用户继续请求订单服务，订单服务也开始卡。接着其他服务调用订单服务也卡住。最后，一个支付服务的问题，传导成整个系统的问题，这就是服务雪崩。

Sentinel 要解决的，就是微服务系统里的服务保护问题。它常用的保护手段主要包括限流、熔断和降级。

限流，是控制单位时间内通过的请求数量。比如秒杀接口每秒最多放行 100 个请求，多余请求直接拒绝，避免把后端服务打垮。

熔断，是当下游服务持续异常或响应太慢时，调用方暂时不再调用它，而是快速失败或返回兜底结果。比如支付服务连续超时，订单服务就先不再等待支付服务，而是返回“支付服务繁忙，请稍后重试”。

降级，是在系统压力大或依赖服务不可用时，返回一个简化结果。比如商品推荐服务挂了，首页可以先不展示个性化推荐，而不是整个首页打不开。

9.8.1.Sentinel 流量控制（重点★★★★★）

Sentinel 的流量控制效果常见有三种。

第一种是快速失败，也就是超过阈值以后立刻拒绝。比如秒杀接口 QPS 设置为 100，那么每秒超过 100 的请求会直接返回“当前人数过多，请稍后再试”。这种方式最简单，也最常用。

第二种是 Warm Up，也叫预热或冷启动。它不是一开始就把全部流量放进来，而是慢慢把流量提高。这个适合一些刚启动、缓存还没热起来的服务。比如商品推荐服务刚上线，如果一下子进来大量请求，缓存、连接池等都还没准备好，容易被打垮。Warm Up 的思想就是让系统先慢慢接流量，等状态稳定以后再逐步提高到目标阈值。

第三种是匀速排队。它不是简单地把多余请求拒绝掉，而是让请求排队，并且按照固定速度放行。比如 QPS 设置为 2，就表示每秒放行 2 个请求，平均下来大概每 500 毫秒放一个请求。如果同时来了 10 个请求，前面的请求可以排队等待，轮到它时再执行；但如果等待时间超过了最大等待时间，这个请求仍然会失败。匀速排队可以用“医院排队看病”来理解。医生每分钟只能看一个病人，后面的病人排队等。如果前面排了两三个人，可以等；如果前面排了五十个人，等到自己可能已经太久了，那就没有意义了。所以匀速排队通常会有一个最大等待时间，超过这个时间就直接拒绝。

服务限流主要包括 QPS 限流算法、令牌桶算法等。这里重点理解令牌桶算法。

令牌桶算法可以这样理解：系统会按照一个固定速度，往一个桶里放令牌。每来一个请求，都必须先从桶里拿到一个令牌，才能继续往后执行。如果桶里有令牌，请求就放行；如果桶里

没有令牌，请求就会被拒绝、排队或降级。

比如系统设置下单接口最多支持每秒 100 个请求，那么限流组件就可以理解为每秒往桶里放 100 个令牌。正常情况下，请求来了就拿一个令牌，通过一个请求。这样系统整体的处理速度就被控制在一个相对稳定的范围内。

令牌桶算法和简单的 QPS 限流不太一样。简单 QPS 限流是直接限制单位时间内最多通过多少请求，比如 1 秒最多通过 100 个。这种方式简单直观，但是有一个问题：如果请求集中在这一秒的前半段，可能瞬间就把 100 个名额用完；如果上一秒末尾和下一秒开头都来了大量请求，还可能在很短时间内放过超过预期的流量。

令牌桶算法的好处是，它不是简单地按时间窗口数请求，而是按照固定速度生成令牌。只要桶里有令牌，请求就能通过；没有令牌，就不能通过。这样放过的流量会相对均匀，更有利于保护后端服务。同时，令牌桶还有一个重要特点：它允许一定程度的突发流量。因为令牌可以在桶里暂存。比如系统一段时间内请求不多，桶里就可能攒下一些令牌。等到某一瞬间突然来了较多请求，这些请求可以先消耗桶里已有的令牌，从而被快速放行。等令牌消耗完以后，后面的请求又只能按照令牌生成速度慢慢通过。

在秒杀场景里，这个特点很有价值。比如秒杀开始时，下单接口可能瞬间涌入大量请求。如果完全没限制，订单服务、库存服务、支付服务、数据库都可能被打垮。使用令牌桶限流以后，系统可以先放过一部分突发请求，但不会无限制放行。令牌用完以后，后续请求就会被限流或降级，从而保护后端核心服务。

9.8.2.Sentinel 熔断降级（重点★★★★★）

除了限流，Sentinel 还有一个非常重要的能力，就是熔断降级。

熔断可以用“电路开关”来理解。正常情况下，订单服务调用商品服务，调用链路是通的，这时断路器处于关闭状态，也就是 **Closed**。请求可以正常通过。没有熔断时，即使商品服务已经明显不可用，订单服务每次还是会先发请求过去，等它超时或报错以后，再执行兜底。配置熔断以后，一旦断路器打开，订单服务在一段时间内就不再发请求给商品服务，而是直接兜底。这样系统更快，资源消耗更少，也更不容易雪崩。

如果商品服务开始变慢，或者大量请求返回异常，订单服务继续死等就没有意义了。**Sentinel** 可以把这条调用链路切断，让断路器进入打开状态，也就是 **Open**。断路器打开以后，订单服务再想调用商品服务，就不会真的发远程请求了，而是直接快速失败或返回兜底结果。这样订单服务不会继续被商品服务拖住。

但是断路器也不能永远打开。万一商品服务已经恢复了呢？所以过了一段熔断时间以后，断路器会进入半开状态，也就是 **Half Open**。半开状态会放一个探测请求过去试一试。如果这个请求成功了，说明商品服务恢复了，断路器就重新关闭，后面的请求可以正常调用。如果探测请求还是失败，断路器就继续打开，再熔断一段时间。

所以断路器有三个状态：**Closed** 关闭：正常调用；**Open** 打开：不再调用，快速失败；**Half Open** 半开：放一个请求试探，成功则恢复，失败则继续熔断

Sentinel 中常见的熔断策略有三种：慢调用比例、异常比例和异常数。

慢调用比例关注的是“慢”。比如订单服务调用商品服务，超过 1 秒还没返回，就认为是慢调用。如果 5 秒内至少有 5 个请求，并且 80% 的请求都超过 1 秒，那么就认为商品服务不稳定，断路器打开 30 秒。之后 30 秒内，订单服务不会再真正调用商品服务，而是直接走兜底逻辑。

异常比例关注的是“错误占比”。比如 5 秒内订单服务调用商品服务 100 次，其中 80 次返回 500 错误，那么异常比例就是 80%。达到阈值以后，断路器打开，后续请求直接兜底，

不再继续把请求打到商品服务。

异常数关注的是“错误数量”。比如 5 秒内只要出现 10 次异常，就触发熔断。它不管异常占比是多少，只看异常个数有没有达到阈值。

OpenFeign 远程调用一般要配合兜底返回。比如订单服务调用商品服务时，可以这样写：

```
@FeignClient(value = "service-product", fallback = ProductClientFallback.class)
public interface ProductClient {

    @GetMapping("/api/product/{id}")
    Product getProduct(@PathVariable("id") Long id);
}
```

兜底类可以这样写：

```
@Component
public class ProductClientFallback implements ProductClient {

    @Override
    public Product getProduct(Long id) {
        Product product = new Product();
        product.setId(id);
        product.setName("未知商品");
        product.setPrice(BigDecimal.ZERO);
        product.setStock(0);
        return product;
    }
}
```

如果要想 **OpenFeign** 和 **Sentinel** 配合，还需要开启 Feign 对 Sentinel 的支持：

```
feign:
  sentinel:
    enabled: true
```

这样远程调用失败、超时、被熔断时，Feign 才能走 fallback 兜底逻辑。

除了普通限流和熔断，**Sentinel** 还有一个很实用的规则，叫热点参数限流。

普通限流通常是对整个资源限流。比如 /api/order/seckill 这个秒杀接口每秒最多 100 个请求。但有时候我们希望限得更细。比如同一个用户不能疯狂刷秒杀接口，同一个热点商品不

能被无限访问。这时就可以用热点参数限流。热点参数限流的意思是：不只是看你访问了哪个接口，还要看你带了什么参数。比如秒杀下单接口有两个参数：

```
public Order seckill(Long userId, Long productId) {
    // 秒杀下单逻辑
}
```

我们希望每个用户每秒只能提交一次秒杀请求，就可以对第一个参数 `userId` 做热点限流。参数索引从 0 开始，所以 `userId` 是第 0 个参数，`productId` 是第 1 个参数。那么只要请求带了 `userId`，那么同一个用户每秒最多通过 1 次。如果 6 号用户是 VIP，希望他的限制宽松一些，也可以设置参数例外项。普通用户 QPS 是 1，但是 `userId=6` 的用户 QPS 可以设置成 1000。这样 VIP 用户就不会被普通规则卡住。如果 999 号商品已经下架，希望任何人都不能秒杀它，也可以对 `productId` 设置热点规则。`productId` 是第 1 个参数，如果参数值是 666，就把阈值设置成 0，相当于禁止访问。

在 Web 接口上使用热点参数限流时，通常要用 `@SentinelResource` 自定义资源名：

```
@RestController
@RequestMapping("/api/order")
public class OrderController {

    @GetMapping("/seckill")
    @SentinelResource(
        value = "seckillOrder",
        blockHandler = "seckillBlockHandler",
        fallback = "seckillFallback"
    )
    public Order seckill(Long userId, Long productId) {
        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(productId);
        order.setRemark("秒杀下单成功");
        return order;
    }

    public Order seckillBlockHandler(Long userId, Long productId, BlockException
ex) {

        Order order = new Order();
```

```
        order.setUserId(userId);
        order.setProductId(productId);
        order.setRemark("当前抢购人数过多，请稍后再试");
        return order;
    }

    public Order seckillFallback(Long userId, Long productId, Throwable ex) {
        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(productId);
        order.setRemark("下单失败，请稍后重试");
        return order;
    }
}
```


10.下篇-领域驱动设计（次重点★★★★☆）

领域驱动设计（Domain-Driven Design，简称 DDD）是一套面向复杂业务软件的分析与设计方法。它的核心主张是：在业务复杂度较高、规则变化频繁、多团队协作的系统中，软件结构应围绕“业务模型”来组织；团队应形成一套贯穿需求讨论、设计、编码与测试的共同语言；并用清晰的边界把不同业务语义隔离开，使系统在长期演进中仍可理解、可修改、可扩展。DDD 的代表性阐述来自 Eric Evans 的专著《领域驱动设计软件核心复杂性应对之道》，其出发点就是“在软件核心处应对复杂性”。

在架构的考试中，我们不需要关心如何使用这套工具去建模、去拆解业务。我们只需要了解领域驱动设计引申出来的若干概念即可。

为了便于说明，下面以“电商平台”为贯穿案例。平台典型能力包括：商品（上架与展示）、购物车、下单、支付、库存扣减、发货、售后退款、营销优惠（满减、券、会员价）、履约与物流查询等。并且为了避免同学觉得概念散，我用“大的边界→小的边界→代码边界”三层来串起 DDD 所有概念。

10.1.大的边界（业务版图边界）（次重点★★★★☆）

10.1.1.领域与子域（次重点★★★★☆）

领域是系统要解决的业务问题空间，例如“电商交易”。子域是领域中可相对独立的问题区域，例如“商品管理”“交易下单”“支付结算”“库存管理”“营销优惠”“售后服务”。DDD 鼓励先做子域划分，是为了把复杂度拆小，让团队把精力聚焦在最能体现业务价值的部分。

10.1.2.子域详解（重点★★★★★）

DDD 常把子域分为核心子域（Core）、支撑子域（Supporting）、通用子域（Generic）。核心子域是企业差异化竞争力所在，最值得做深模型与持续演进；通用子域尽量复用成熟方案，避免把研发资源消耗在可替代能力上；支撑子域则介于两者之间。该思想在业界实践与 DDD 介绍中被反复强调。

以电商平台为例子：对于大多数平台，“优惠计算引擎”“库存与履约协同”“售后规则”可能属于核心子域；“统一登录、短信邮件、通用支付渠道接入”更偏通用子域；“内容运营、活动页搭建”常是支撑子域（除非平台核心竞争力就在内容生态）。

10.2.小的边界（团队协作边界）（重点★★★★★）

10.2.1.统一语言（重点★★★★★）

统一语言是指业务人员与研发人员共同使用的一套业务术语与定义，并把它落实到代码命名、接口字段、测试用例、文档与监控指标中。它的目的不是写一本术语词典，而是确保“讨论的概念”和“实现的概念”一致，减少误解与口径漂移。DDD 的战略设计强调用语言来约束模型的一致性。

以电商平台为例子：业务说“取消订单”和“关闭订单”在口语上相近，但在规则上可能完全不同：取消可能发生在未支付状态；关闭可能由超时、风控、库存不足等触发。统一语言要求团队明确区分：取消（用户主动）、超时关闭（系统自动）、风控关闭（风险判定）等，并在代码里直接体现，如 `OrderStatus`、`CloseReason`、`CancelReason` 等，而不是一律用一个状态替代。

10.2.2.限界上下文（重点★★★★★）

限界上下文是 DDD 的中心概念之一。它规定在一个明确边界内，模型与语言保持一致；跨边界可以出现同名不同义，但必须通过契约协作。

以电商例子：至少可以划出这些上下文：

- 商品上下文：商品、SKU、类目、上下架、图文详情；
- 交易上下文：订单、订单项、价格快照、收货地址、订单状态机；
- 营销上下文：优惠券、活动、满减规则、凑单、会员价；
- 库存上下文：可售库存、预占、扣减、回补、库存流水；
- 支付上下文：支付单、渠道流水、退款单、对账；
- 履约物流上下文：发货单、包裹、物流轨迹、签收。

如果你把“优惠计算”直接写进交易上下文的订单类里，短期省事，长期会让订单模型被营销规则污染。这里的污染有些同学可能不太理解，这里做个说明。我们知道营销规则变化是“活动级”的：双十一、周年庆、清仓、拉新等，可能每天都在变。加入你把把“优惠计算”直接写进交易的逻辑里，那么只要营销在变，你就需要改订单里的算价逻辑：新增券类型、新增叠加规则等。这个就是被污染后的典型后果。

10.2.3.上下文映射与边界协作关系（次重点★★★☆☆）

当存在多个限界上下文时，需要描述它们的集成方式与依赖关系，这就是上下文映射。典型关系包括：客户/供应商（下游提需求、上游配合）、顺从者（下游被迫适配上游）、防腐层（用翻译层隔离外部模型）、共享内核（少量共享模型共同维护）等。其核心目的是让跨团队协作与系统演进的成本“可控”。

防腐层是指当系统要对接外部系统或遗留系统时，外部模型往往与本系统语言不一致。防腐层是一层翻译与适配，保证外部模型不会污染核心领域模型。它在多上下文协作中尤为关键，属于上下文映射的重要思想。

比如对接第三方支付渠道时，渠道返回的字段可能包含渠道特有的状态码、错误码、签名信息。支付上下文可以用 ACL 把它翻译成统一的“支付成功/支付失败/待确认”等领域概念，再向交易上下文发布稳定事件。这样未来更换支付渠道时，只需替换 ACL 适配，不会撕裂交易模型。

10.3.代码边界（重点★★★★★）

10.3.1.实体（Entity）（重点★★★★★）

实体以“身份标识”区分，同一实体在生命周期内属性可能变化，但身份保持不变。实体通常承载业务行为与状态演化。Fowler 在对 DDD 的介绍中也提到 Evans 对实体与值对象等分类的重要性。

比如订单（Order）是典型实体，它有 orderId；订单地址可能改、订单状态会变、订单备注会变，但它仍是同一张订单。商品（Product）也可视为实体（productId），但要注意：交易所需的商品信息通常应使用“价格快照/商品快照”，避免下单后商品改价影响历史订单。

10.3.2.聚合与聚合根（重点★★★★★）

聚合是“一致性边界”。聚合内的业务不变式（强约束）必须在一次事务或一次一致性操作中被维护；聚合外部不能随意修改聚合内部对象，而应通过聚合根统一入口来修改，从而保证规则不被绕开。

比如订单聚合（以 Order 为聚合根）通常包含订单项、价格明细、收货信息等。关键不

变式包括：订单已支付后，不允许再修改应付金额；订单关闭后，不允许发起发货；同一订单在同一时刻只能处于一个状态（状态机约束）。

这些规则应由订单聚合根的方法来保证，例如 `pay()`、`close(reason)`、`ship()`，而不是在控制器里直接改字段。

10.3.3.领域服务（重点★★★★★）

当一个业务操作天然涉及多个聚合，或难以归属于单一实体/值对象的行为时，可用领域服务承载。它仍属于领域层，表达业务规则而非技术流程。

比如优惠分摊是典型领域服务：一个订单有多个订单项，会员折扣后，需要把优惠合理分摊到各订单项，以便后续退款按项计算。分摊规则既不属于某个单独订单项，也不应落在应用层脚本里，更适合做成领域服务 `PricingService/PromotionApportionService`。

10.3.4.领域事件（重点★★★★★）

领域事件表示“领域里发生了一件重要的业务事实”，用于在边界内外传播变化、解耦模块协作。领域事件强调业务语义，而非技术日志。事件机制常与上下文边界协作结合使用：上游发布事件，下游订阅后执行自己的业务反应。

10.3.5.仓储与工厂（重点★★★★★）

仓储用于以“集合”的方式获取与保存聚合，隐藏持久化细节；工厂用于创建复杂聚合，隐藏构造过程与校验逻辑。它们的目的是让领域模型不被数据库结构牵引，使领域层保持业务表达的纯度。

比如 `OrderRepository` 提供 `findById`、`save` 等接口，领域层不直接写 SQL；`OrderFactory` 在创建订单时完成：生成订单号、固化价格快照、校验库存可售、写入初始状态等。这样“创

建订单”不会散落在控制器与多个服务类中。

10.3.6.应用编排（重点★★★★★）

DDD 常见实现会区分应用层与领域层。应用层负责“用例编排”（即把一次业务请求串起来：校验权限、加载聚合、调用领域方法、持久化、发布事件）；领域层负责“业务规则本身”。微软的微服务与 DDD 指南中也强调战略与战术两阶段，并在战术阶段用模型要素精确表达业务规则。

比如用户点击“支付订单”，应用层流程可能是：读取订单聚合→调用订单聚合的 `pay()`→生成支付单→调用支付网关→收到回调后发布 `PaymentSucceeded` 事件。这里“流程编排”属于应用层；而“订单能不能支付、支付后状态如何变化”属于领域层。

10.4.小结

DDD 可以概括为：先回答“大的边界，业务版图边界”（领域与子域、核心子域）；再回答“小的边界，团队协作边界”（统一语言、限界上下文、上下文映射与防腐层）；最后在每个边界内部回答“代码边界”（实体、值对象、聚合、领域服务、领域事件、仓储与工厂、应用层编排）。

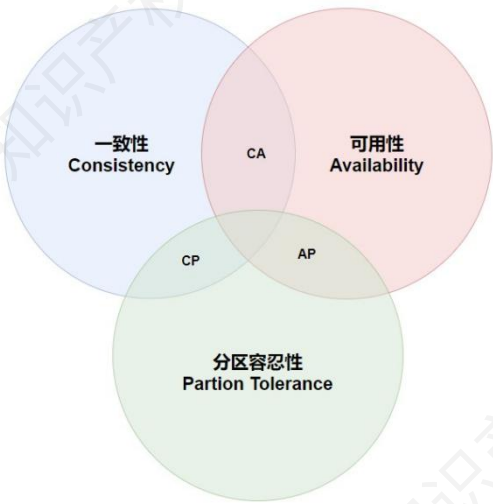
11.下篇-分布式系统（重点★★★★★）

分布式系统目前架构还没考过，这里主要是分为两块，一个是分布式系统的基本理论，另外一块分布式事务的理论解决方案。

11.1.CAP 理论（重点★★★★★）

CAP 理论是指计算机分布式系统的三个核心特性：一致性（Consistency）、可用性（Availability）和分区容错性（PartitionTolerance）。在 CAP 理论中，一致性指的是多个节点上的数据副本必须保持一致；可用性指的是系统必须在任何时候都能够响应客户端请求；而分区容错性指的是系统必须能够容忍分布式系统中的某些节点或网络分区出现故障或延迟。

CAP 理论认为，分布式系统最多只能同时满足其中的两个特性，无法同时满足全部三个特性。



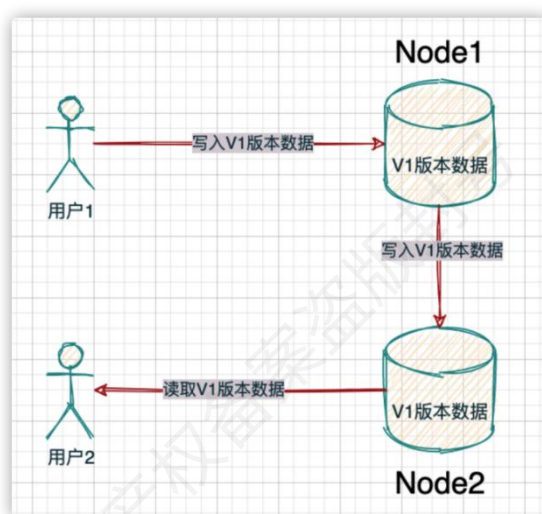
概念	定义
数据一致性	系统中写请求后各相关节点数据均变化，读请求能获取变化后数据
可用性	系统内节点接收读写请求后，都要处理并给出响应结果，即使数据可能有问题

分区容错性	系统中节点通信出现问题产生分区，系统仍需继续运行，各分区可独立对外服务
-------	-------------------------------------

11.1.1.数据一致性（重点★★★★★）

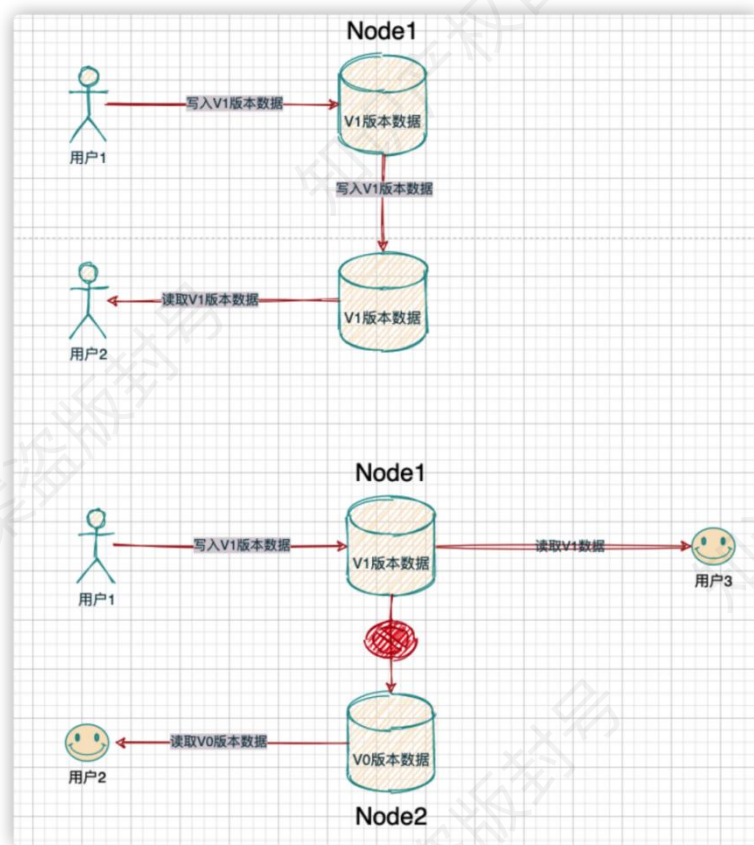
假设，我们的分布式存储系统有两个节点，每个节点都包含了一部分需要被变化的数据。

如果经过一次写请求后，两个节点都发生了数据变化。然后，读请求把这些变化后的数据都读取到了，我们就把这次数据修改称为数据发生了一致性变化。



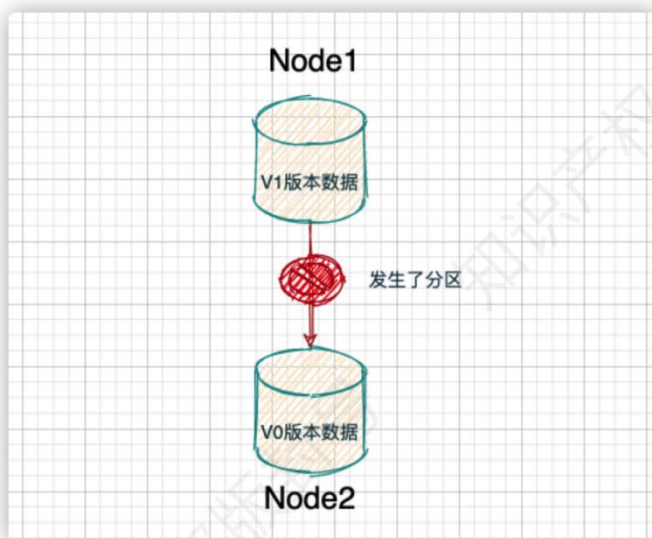
11.1.2.可用性（重点★★★★★）

如果节点能正常接收请求，但是发现节点内部数据有问题，那么也必须返回结果，哪怕返回的结果是有问题的。比如，系统有两个节点，其中有一个节点数据是三天前的，另一个节点是两分钟前的，如果，一个读请求跑到了包含了三天前数据的那个节点上，抱歉，这个节点不能拒绝，必须返回这个三天前的数据，即使它可能不太合理。

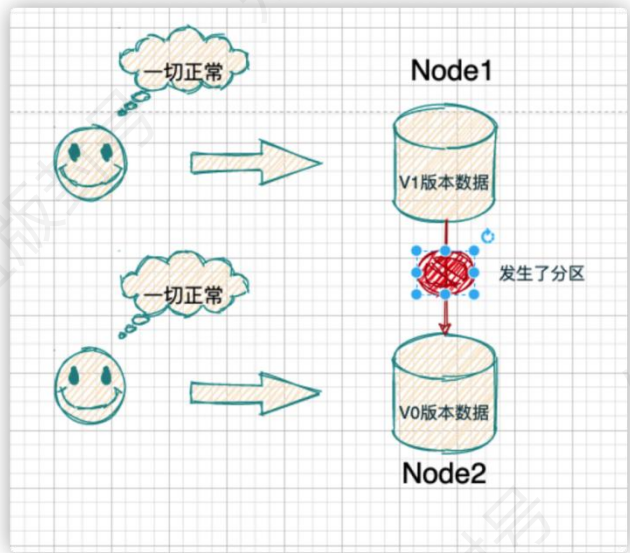


11.1.3.分区容错性（重点★★★★★）

分布式的存储系统会有很多的节点,这些节点都是通过网络进行通信。而网络是不可靠的,当节点和节点之间的通信出现了问题,此时,就称当前的分布式存储系统出现了分区。只要在分布式系统中,节点通信出现了问题,那么就出现了分区。



分区容忍性是指如果出现了分区问题，我们的分布式存储系统还需要继续运行。不能因为出现了分区问题，整个分布式节点全部就停止服务了。



分区容错性和可用性的区别在于，分区容错性本质上是由于通信故障导致分区产生，然后各个分区可以对外独立提供服务。而可用性是指非通信故障下，分布式系统整体对外提供服务的能力。

11.2.CAP 理论与系统实现（重点★★★★★）

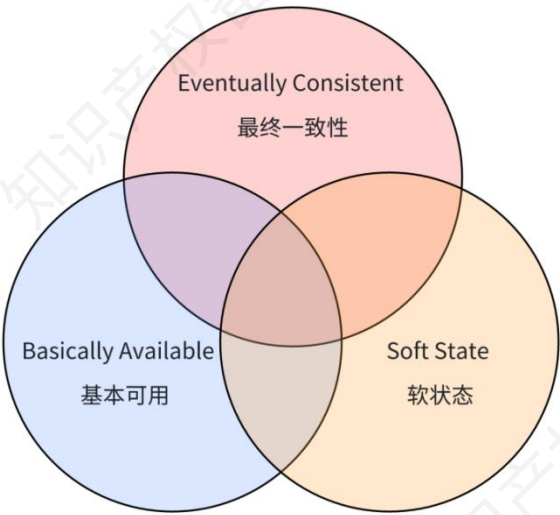
在分布式系统内，分区容错是必然会发生的。我们只能考虑当发生分区错误时，如何选择一致性和可用性。而根据一致性和可用性的选择不同，开源的分布式系统往往又被分为 CP 系统和 AP 系统。

组合方式	组合问题	例子
一致性和分区容忍性（CP）	在网络分区期间，某些操作可能不可用，牺牲了可用性。	Zookeeper 集群中存在一个 Leader 节点，其他是 Follower；所有写请求必须经过 Leader；写入要通过多数派（超过半数）确认后才算成功；如果 Leader 与其他节点失联，或集群中节点不足以形成多数派，Zookeeper 会暂停服务，直到重新选出 Leader。

可用性和分区容忍性 (AP)	在网络分区期间,不同节点可能有不同的数据副本,导致数据不一致。	Eureka, 当 Eureka 客户端心跳消失的时候, Eureka 服务端就会启动自我保护机制,不会剔除该服务,不会删除任何实例(即使心跳丢失),仍然允许其他客户端查询注册信息,保持整个系统继续可用。
一致性和可用性 (CA)	无法在分布式环境下实现。	

11.3.BASE 理论 (重点★★★★★)

BASE 理论起源于 2008 年,由 eBay 的架构师 DanPritchett 在 ACM 上发表。BASE 是对 CAP 中一致性和可用性权衡的结果,其来源于对大规模互联网系统分布式实践的结论,是基于 CAP 定理逐步演化而来的。其基本思路就是:通过业务,牺牲强一致性而获得可用性,并允许数据在一段时间内是不一致的,但是最终达到一致性状态。



在 BASE 理论涉及三个概念梳理如下表所示。

概念	定义
基本可用	分布式系统遇不可预知故障时,允许损失部分可用性

软状态	允许系统存在中间状态，且该状态不影响整体可用性，不同节点数据副本同步可存在延时
最终一致性	系统中所有数据副本经一段时间同步后，最终达到一致状态，无需实时强一致性

11.4.分布式事务理论解决方案

分布式事务是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。例如在大型电商系统中，下单接口通常会扣减库存、减去优惠、生成订单 id，而订单服务与库存、优惠、订单 id 都是不同的服务，下单接口的成功与否，不仅取决于本地的 db 操作，而且依赖第三方系统的结果，这时候分布式事务就保证这些操作要么全部成功，要么全部失败。本质上来说，分布式事务就是为了保证不同数据库的数据一致性。这一块架构几乎没考过，深入难度比较大。

凯恩这里已经尽力写的想让你能看懂，但对于没有做过开发的同学来说，估计还是很困难。对于这些同学，凯恩建议直接记 2PC，TCC 本地消息表的架构图，要能大致描述出方案。

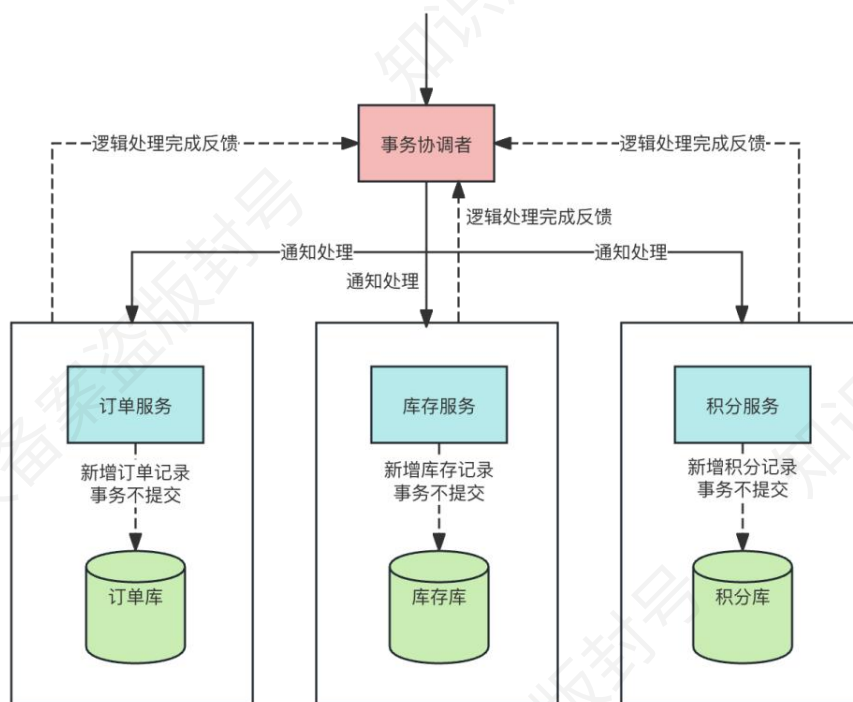
11.4.1.刚性事务 2PC（重点★★★★★）

2PC 即 Two-PhaseCommit，二阶段提交。广泛应用在数据库领域，为了使得基于分布式架构的所有节点可以在进行事务处理时能够保持原子性和一致性。绝大部分关系型数据库，都是基于 2PC 完成分布式的事务处理。顾名思义，2PC 分为两个阶段处理。

阶段一：事务预处理/投票阶段

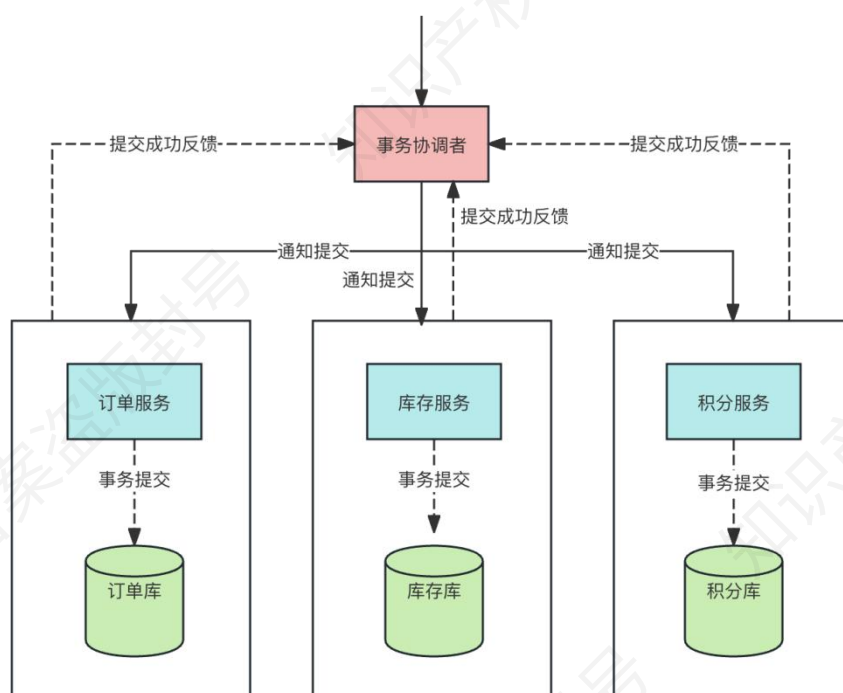
（1）事务询问。协调者向所有参与者发送事务内容，询问是否可以执行提交操作，并开始等待各参与者进行响应；（2）执行事务。各参与者节点，执行事务操作，并将 Undo 和 Redo 操作计入本机事务日志；（3）各参与者向协调者反馈事务询问的响应。成功执行返回 Yes，

否则返回 No。



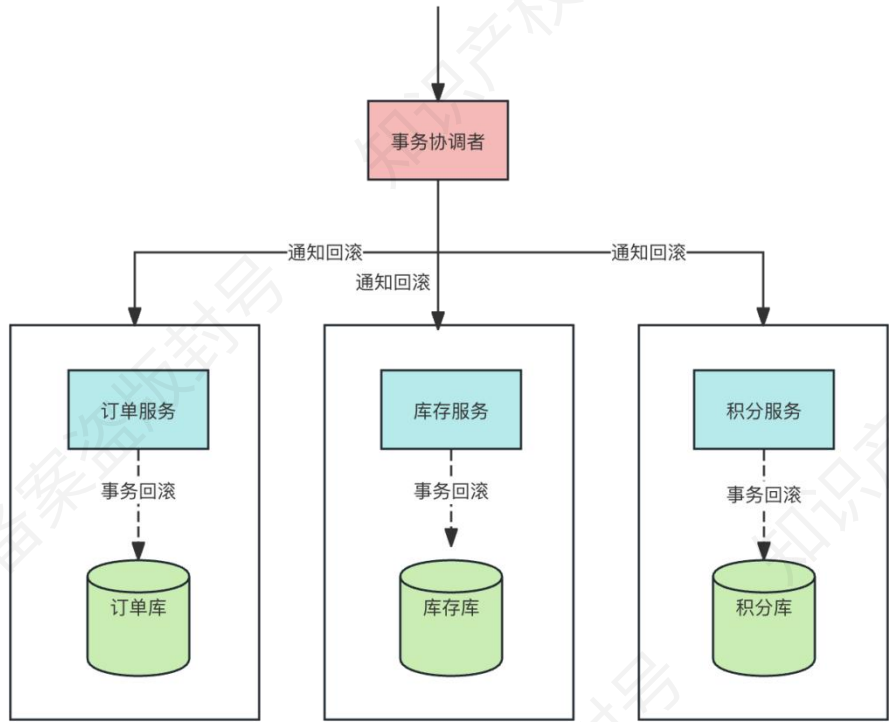
阶段二：提交阶段/执行阶段

协调者在阶段二决定是否最终执行事务提交操作。这一阶段包含两种情形执行事务提交和中断事务。执行事务提交。所有参与者 ReplyYes，那么执行事务提交。发送提交请求。协调者向所有参与者发送 **Commit** 请求；事务提交。参与者收到 **Commit** 请求后，会正式执行事务提交操作，并在完成提交操作之后，释放在整个事务执行期间占用的资源；反馈事务提交结果。参与者在完成事务提交后，向协调者发送 **ACK** 消息确认；完成事务。协调者在收到所有参与者的 **ACK** 后，完成事务。



中断事务。事情总会出现意外，当存在某一参与者向协调者发送 No 响应，或者等待超时。

协调者只要无法收到所有参与者的 Yes 响应，就会中断事务。发送回滚请求。协调者向所有参与者发送 Rollback 请求；回滚。参与者收到请求后，利用本机 undo 信息，执行 Rollback 操作。并在回滚结束后释放该事务所占用的系统资源；反馈回滚结果。参与者在完成回滚操作后，向协调者发送 ACK 消息；中断事务。协调者收到所有参与者的回滚 ACK 消息后，完成事务中断。



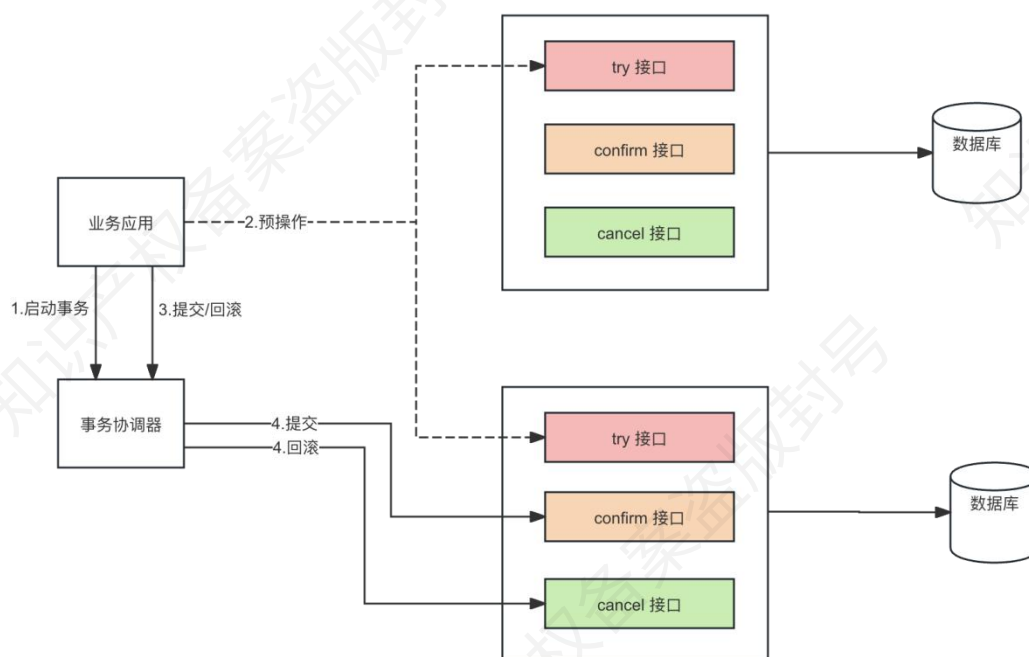
2PC 优点主要体现在实现原理简单，缺点如下

主要问题	解释
性能问题	在 2PC 提交过程中，所有参与事务操作的逻辑都处于阻塞状态，占用系统资源。
单点故障	2PC 协议的协调者是个单点，一旦协调者出现问题，其他参与者将无法释放事务资源，也无法完成事务操作。这是一个严重的可靠性隐患。
数据不一致	当执行事务提交过程中，如果协调者向所有参与者发送 Commit 请求后，发生局部网络异常或者协调者自身崩溃，可能只有部分参与者收到并执行了请求（实际操作只能超时部分要么做提交要么做回滚）。这会导致整个系统出现数据不一致的情况。
保守机制	2PC 没有完善的容错机制，当参与者出现故障时，协调者无法快速得知，只能严格依赖超时设置来决定是否继续提交或中断事务。这种机制比较保守，无法应对动态的网络环境。

11.4.2.柔性事务 TCC（重点★★★★★）

关于 TCC（Try-Confirm-Cancel）的概念，最早是由 PatHelland 于 2007 年发表的一篇名为《LifebeyondDistributedTransactions:anApostate’sOpinion》的论文提出。

TCC 事务机制相比于 2PC，（1）解决了协调者单点问题，由主业务方发起并完成这个业务活动。业务活动管理器也变成多点，引入集群。（2）同步阻塞：引入超时，超时后进行补偿，并且不会锁定整个资源，将资源转换为业务逻辑形式，粒度变小。（3）数据一致性，有了补偿机制之后，由业务活动管理器控制一致性。



Try 阶段：尝试执行，完成所有业务检查（一致性），预留必需业务资源（准隔离性）。

Confirm 阶段：确认执行真正执行业务，不做任何业务检查，只使用 Try 阶段预留的业务资源，Confirm 操作满足幂等性。要求具备幂等设计，Confirm 失败后需要进行重试。

Cancel 阶段：取消执行，释放 Try 阶段预留的业务资源。Cancel 操作满足幂等性 Cancel 阶段的异常和 Confirm 阶段异常处理方案基本上一致。

在 Try 阶段，是对业务系统进行检查及资源预览，比如订单和存储操作，需要检查库存剩余数量是否够用，并进行预留，预留操作的话就是新建一个可用库存数量字段，Try 阶段操作是对这个可用库存数量进行操作。

基于 TCC 实现分布式事务，会将原来只需要一个接口就可以实现的逻辑拆分为 Try、

Confirm、Cancel 三个接口，所以代码实现复杂度相对较高。

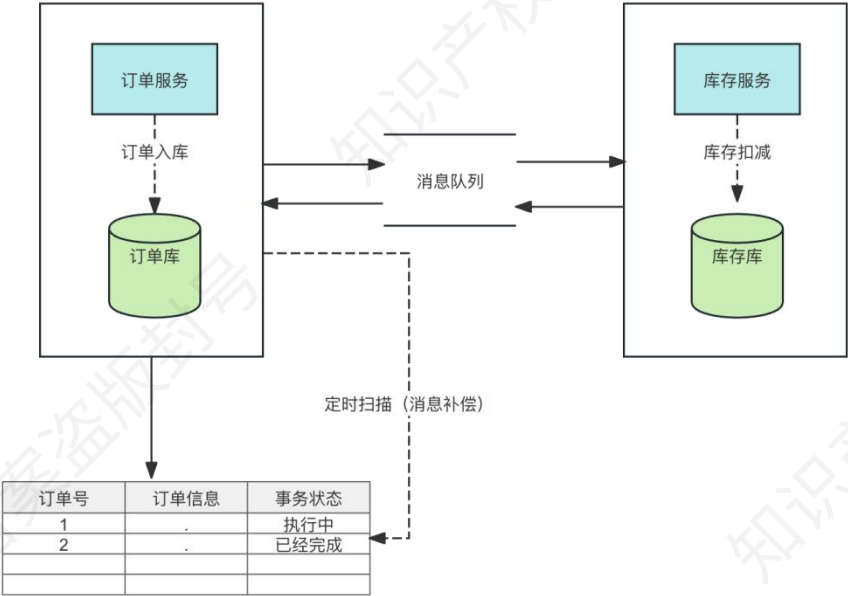
11.4.3.柔性事务本地消息表（重点★★★★★）

本地消息表这个方案最初是 ebay 架构师 DanPritchett 在 2008 年发表给 ACM 的文章。设计核心是将需要分布式处理的任务通过消息的方式来异步确保执行。它通过在本地数据库中记录消息的发送状态，结合异步消息队列，来实现事务的最终一致性。

发送消息方：需要有一个消息表，记录着消息状态相关信息。业务数据和消息表在同一个数据库，要保证它俩在同一个本地事务。直接利用本地事务，将业务数据和事务消息直接写入数据库。在本地事务中处理完业务数据和写消息表操作后，通过写消息到 MQ 消息队列。消息投递到 MQ。当消息成功投递到 MQ 后，如果收到消费方返回的 ACK 响应，说明消息被消费方成功处理，此时可以安全地删除本地事务消息表中的记录，以释放资源并避免重复处理。

如果发送失败，也就是没有收到 ACK 响应，确实需要进行重试。重试的目的是确保消息最终能够被消费方成功处理，从而保证分布式事务的一致性。可以通过设置重试次数和重试间隔等策略来控制重试过程，同时要注意处理可能出现的重复消息问题，比如消费方需要具备幂等性，以确保即使重复处理消息也不会产生错误的结果。

消息消费方：处理消息队列中的消息，完成自己的业务逻辑。如果本地事务处理成功，则表明已经处理成功了。如果本地事务处理失败，那么就会重试执行。如果是业务层面的失败，给消息生产方发送一个业务补偿消息，通知进行回滚等操作。



本地消息表实现了分布式事务的最终一致性，优缺点比较明显。优点：实现逻辑简单，开发成本比较低。缺点：与业务场景绑定，高耦合，占用业务系统资源，量大可能会影响数据库性能。由于消息是异步发送和处理的，可能会引入一定的延迟。

12.下篇-数据库系统（重点★★★★★）

数据库系统架构案例几乎不考范式，只考性能优化和高可用相关主题。

12.1.数据库性能优化（重点★★★★★）

数据库性能优化的主题软考主要涉及三个一个是索引，一个是数据分区、分片，还有一个就是读写分离，遇到题目你要有话可说。

12.1.1.索引建立注意事项（重点★★★★★）

索引是提高数据库查询速度的利器，而数据库查询往往又是数据库系统中最频繁的操作，因此，索引的建立与选择对数据库性能优化具有重大意义。索引相关知识也是案例常考内容，它的使用准则建议熟记。

索引的建立与选择可遵循以下准则：

（1）建立索引时，应选用经常作为查询，而不常更新的属性。避免对一个经常被更新的属性建立索引，因为这样会严重影响性能。

（2）一个关系上的索引过多会影响 UPDATE、INSERT 和 DELETE 的性能，因为关系一旦进行更新，所有的索引都必须跟着做相应的调整。

（3）尽量分析出每个重要查询的使用频度，这样，可以找出使用最多的索引，然后可以先对这些索引进行适当的优化。

（4）对于数据量非常小的关系不必建立索引，因为对于小关系而言，关系扫描往往更快，而且消耗的系统资源更少。

12.1.2.数据分片（重点★★★★★）

分片（Shard）是把数据库横向扩展（ScaleOut）到多个物理节点上的一种有效的方式，其主要目的是为突破单节点数据库服务器的 I/O 能力限制，解决数据库扩展性问题。分片（Shard）这个词的意思是“碎片”。如果将一个数据库当作一块大玻璃，将这块玻璃打碎，那么每一小块都称为数据库的碎片。将整个数据库打碎的过程就叫作分片，可以翻译为分片。

12.1.2.1.分片分类（重点★★★★★）

分片类型是架构设计的起点，决定数据拆分的维度（垂直/水平）。

数据分片	原理
水平分片	水平分片将一个全局关系中的元组分裂成多个子集，每个子集为一个片段。分片条件由关系中的属性值表示。对于水平分片，重构全局关系可通过关系的并操作实现。
垂直分片	垂直分片将一个全局关系按属性分裂成多个子集，满足不相交性。对于垂直分片，重构全局关系可通过连接运算实现。
导出分片	导出分片又称为导出水平分片，即水平分片的条件不是本关系属性的条件，而是其他关系属性的条件。像关系 SC，是一个学生选修课表（学号，课程号，成绩）。而是根据学号关联学生表（学号，性别），然后用学生的性别来分片。
混合分片	混合分片是在分片中采用水平分片和垂直分片两种形式的混合。

12.1.2.2.分片算法（重点★★★★★）

分片算法主要解决了数据被水平分片之后，数据究竟该存放在哪个表的问题。案例往往会结合具体的场景来考你，什么样的分片算法适合什么样的数据，看凯恩下面的梳理。

分片算法	原理	适用数据
哈希分片	对指定的键（如 id）进行哈希运算，再根据哈希值对存储节点数量取模，所得余数对应数据应存放的表或节点	数据分布较为均匀，无明显的顺序性或时间相关性；数据之间的关联性较弱，每个数据可独立读写
范围分片	按照特定的范围区间（如时间区间、ID 区间等）来划分数据，将满足某一范围条件的数据存放在一起	具有明显的顺序特征，如按时间先后产生、按编号顺序排列；数据在某个维度上呈现出可划分的区间特性
地理位置分片	依据地理位置信息（如城市、地域等）来分配数据，将同一地理位置相关的数据存储在同一个位置或相关联的存储单元中	数据与地理位置紧密相关，不同地理位置的数据使用场景、访问频率等有差异
融合算法	灵活组合多种分片算法，结合不同算法的优势来分配数据	数据特点多样，可能同时具有随机读写、范围查询需求，或兼具多种维度的关联特性

上面的是书本提到的分片算法，但是实际上你落实到具体的一个带分片功能的数据库中间件比如 MyCat，它支持的方式就更多了，这里简单介绍一下最常用的。

分片算法	原理	适用数据
取模算法	对数据的某个键值进行哈希运算，将哈希值对分片数量取模，所得结果作为数据存储的分片索引	适合随机访问的数据，数据分布期望均匀的场景，例如用户 ID、订单 ID 等主键数据
范围算法	按数据的某个字段（如时间、数值等）的范围划分分片	有范围查询需求的数据，例如按时间范围统计的日志数据、按数值区间划分的业务数据等
列表算法	列举数据某个字段的取值，指定每个取值对应的分片	业务场景中取值范围固定且明确的数据，例如地区、部门等有明确分类的数据
一致性哈希算法	将数据键值和节点通过哈希函数映射到固定哈希环上，数据从其哈希值位置顺时针找第一个节点存储；节点增减时，只影响相邻部分数据	分布式系统中节点需要动态调整的数据，例如缓存数据、分布式存储中的对象数据等

12.1.3.数据分区（重点★★★★★）

数据分区的底层逻辑是将一张表的数据按某种规则（如范围、哈希或复合条件）划分为多

个逻辑子集，称为分区。数据分区也在真题中出现过，这里的分区优势、算法特点都要重点记忆。

每个分区存储特定范围或条件的数据。当对分区表执行查询时，数据库查询优化器会根据查询条件和分区规则，确定需要访问哪些分区。数据分区的好处如下表所示。

角度	具体说明
查询性能角度	按特定规则分区，查询时仅访问相关分区数据，减少 I/O 操作，加快查询速度
管理角度	把大表拆分为小且易管理的块，方便数据的维护、备份、恢复等操作
数据隔离角度	可将不同类型或不同时间范围的数据分隔，简化数据管理流程，也利于权限控制等

12.1.3.1.分区算法（重点★★★★★）

这里的分区算法你会发现和前面分片算法几乎一样。

分区算法	原理	适用数据场景
范围分区	根据数据在某个维度上的连续范围，如日期、数字大小等划分，每个分区存储特定范围的数据	时间序列数据（如按日期存储的日志、交易记录），有明显数据范围区间（如按年龄区间存储的用户信息），方便进行范围查询的数据场景
列表分区	按照离散值列表进行分区，将具有特定离散值的数据划分到同一分区	数据值可枚举且有限的场景，如按地区（固定几个地区选项）、性别（男/女）等维度进行分区的数据
哈希分区	通过哈希函数计算数据的哈希值，依据哈希值将数据分配到不同分区，能让数据较为均匀分布	数据分布需要尽量均匀，对负载均衡要求较高的场景；不太需要范围查询，更注重随机读写性能的场景，如大规模用户数据的存储，可按用户 ID 哈希分区
复合分区	组合使用多种分区类型，先按一种规则进行粗粒度分区，再在每个分区内按另一种规则细分	数据量极大且具有多种特征维度的复杂场景，如既需要按时间范围（范围分区）又需要按业务类型（列表分区）进行管理的数据

12.1.3.2.分片与分区的差别（重点★★★★★）

前面谈到了分片和分区，那么它们到底有什么区别。一般来说数据库分片是将数据分散到

不同节点，分区是在单个节点内对数据进行分组。

特征	分片	分区
粒度	物理上的划分	物理上的划分
范围	可以跨多个数据库实例或服务器	通常在单个数据库实例内
目的	提高可扩展性和并发性	优化查询性能
实现方式	使用多个物理表	使用 PARTITIONBY 子句
管理难度	相对复杂	相对简单

12.1.4.分库分表（重点★★★★★）

随着用户量的激增，数据库里的数据越来越多，问题就随之出现了。最典型的三个现象：第一，资源报警。单机数据库的连接数、IO 和网络吞吐都是有限的，并发量上来之后，数据库就顶不住了。第二，查询变慢。一张表里的数据越来越多，B+树高度变大，访问一次就要更多的 I/O，性能就下降了。第三，热点集中。比如活动期间，某些表或者某些区间会被频繁访问，结果整个数据库都被拖慢。所以我们不得不做分库分表。分库分表的目的其实就是：分库，解决单机承载能力有限的问题，把请求分散到多台服务器上。分表，解决单表数据量过大的问题，把数据拆开，让查询在更小的范围内完成。

12.1.4.1.分库分表方案（重点★★★★★）

分库和分表都有两种解决方案，分别是水平和垂直，见下表凯恩的概括。

类型	定义	举例场景
水平分表	将同一张表的数据按行划分，分散到多个表中，降低单表数据量	电商订单表 orders 太大，按订单 ID 取模拆成 orders_0、orders_1、orders_2

垂直分表	将一张表的不同列拆分到多个表中，减少字段数量，提高查询效率	商品表 <code>product</code> ，基本信息放到 <code>product_base</code> ，大字段（描述、图片 URL）放到 <code>product_detail</code>
水平分库	将相同表结构的数据分布到多个数据库实例中，分摊读写压力	社交应用的消息表 <code>message</code> ，按用户 ID 拆到 <code>message_db1</code> 、 <code>message_db2</code> ，两个库里都有 <code>message</code> 表
垂直分库	将不同业务的数据分散到不同数据库实例中，按功能模块拆分	在线教育平台：用户数据放 <code>user_db</code> ，课程数据放 <code>course_db</code> ，支付数据放 <code>payment_db</code>

12.1.4.2.分库分表 VS 分片分区（重点★★★★★）

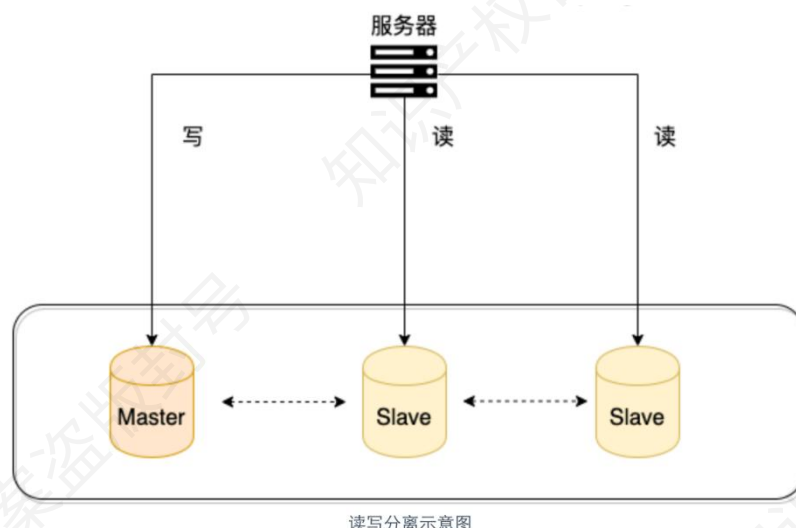
分库分表和分片的区别是什么？分库分表是一种物理实现方式，分片是抽象概念。你可以说“我的系统做了分片”，而实现手段就是“分库+分表”。

12.2.数据库高可用模式（重点★★★★★）

前面提到了，数据库的高可用模式是架构考试的常客，读写分离，主从复制都已经考过。

12.2.1.读写分离（重点★★★★★）

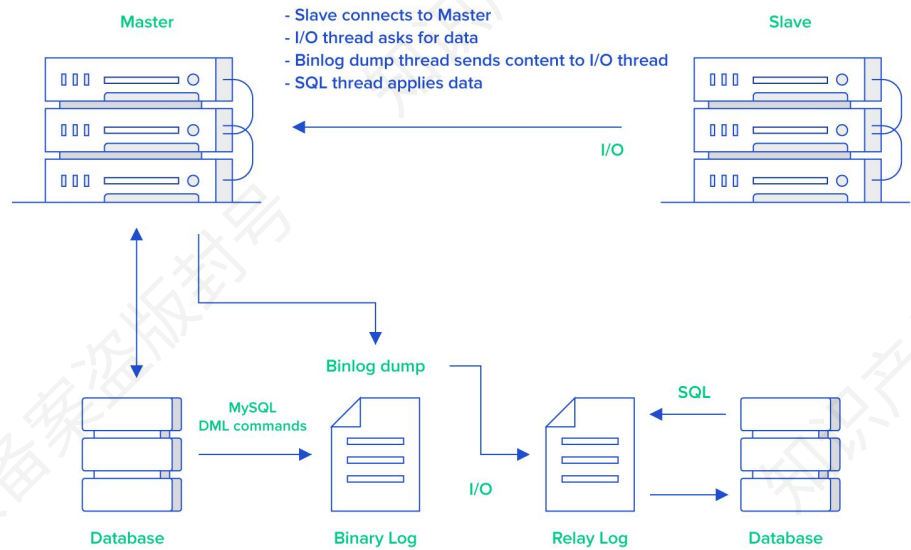
读写分离主要是为了将对数据库的读写操作分散到不同的数据库节点上。这样的话，就能够小幅提升写性能，大幅提升读性能。一般情况下，我们都会选择一主多从，也就是一台主数据库负责写，其他的从数据库负责读。主库和从库之间会进行数据同步，以保证从库中数据的准确性。这样的架构实现起来比较简单，并且也符合系统写少读多的特点。



抛开实践来谈架构没有意义，实际上，Java 生态下可以在应用层使用 ShardingSphere，也可以使用数据库中间件 MyCat 在数据库层面去做读写分离的工作。

12.2.2.主从复制（重点★★★★★）

上一节，凯恩一直在和大家讨论读写分离的场景和实现，但是忽略了一个问题，那就是如何保证主数据库和从数据库之间的数据同步的。其实主从复制利用了日志文件。以 MySQL 为例，MySQL 主要是通过二进制日志（BinaryLog，简称 binlog）实现数据的复制。主数据库在执行写操作时，会将这些操作记录到 binlog 中，然后推送给从数据库，从数据库重放对应的日志即可完成复制。



- (1) 主库将数据库中数据的变化写入到二进制日志。
- (2) 从库连接主库，并请求复制。
- (3) 从库会创建一个 I/O 线程向主库请求更新的二进制日志。
- (4) 主库会创建一个二进制日志 dump 线程来发送二进制日志，从库中的 I/O 线程负责接收。
- (5) 从库的 I/O 线程将接收的二进制日志写入到中继日志中。
- (6) 从库的 SQL 线程从中继日志中读取事件并应用到本地数据库中。

主从复制的优势如下表所示，这块也是需要记忆的。

优势	描述
数据冗余	提供数据副本，防止主库故障导致数据丢失
负载均衡	将读操作分发到从库，减轻主库负载
高可用性	主库故障时，能快速切换到从库，保障系统运行

12.2.3.一致性问题（重点★★★★★）

读写分离对于提升数据库的并发非常有效，但是，同时也会引来一个问题：主库和从库的数据同步往往是存在延迟，比如你写完主库之后，主库的数据同步到从库是需要时间的，这个时间差就导致了主库和从库的数据不一致性问题。这也就是我们经常说的主从同步延迟。一致性问题已经在 2023 年架构设计师考试中考到，需要引起注意，你要掌握解决延迟的三种方法以及优劣势。

同步方式	原理	优点	缺点
半同步复制	主库提交事务后，等至少一个从库接收并写入事务 binlog 到本地中继日志，收到从库 ACK 后主库才正式提交事务并返回客户端。这里涉及到 MySQL 的日志机制感兴趣自己查资料深入了解，软考到这里就已经完事了。	利用数据库原生功能，比较简单	主库写请求时延增长，吞吐量降低
数据库中间件	所有读写走中间件，写请求路由到主库，读请求路由到从库。记录写库 key，主从同步时间窗口内读请求路由到主库，时间过了再路由到从库，如 Canal、Otter。	能保证绝对一致	数据库中间件成本较高
缓存记录	写操作时记录 key 到 cache 并设超时时间（如 500ms）再修改主数据库；读时先查 cache，命中则路由到主库，未命中则路由到从库	相对数据库中间件成本较低	为保证一致性引入 cache 组件，读写数据库时多了缓存操作

拓展阅读

在半同步复制模式下，主库（master）在提交一个事务（即做数据改变操作）后，会等待至少一个从库（slave）接收并写入该事务的二进制日志（binlog）到其本地中继日志（relaylog）中。只有当从库发送确认信息（ACK）给主库后，主库才会正式提交该事务并返回给客户端。

这种机制确保了主从数据的一致性，但以牺牲主库的性能为代价。在 MySQL 中半同步有两种模式有主库先提交，也有主库后提交的，`AFTER_SYNC` 和 `AFTER_COMMIT`，看你半同步如何配置，这里大家注意一下就行。

MySQL 的 Replication 默认是一个异步复制的过程，从 MySQL5.5 开始，MySQL 以插件的形式支持半同步复制。半同步复制优点是利用数据库原生功能，比较简单，缺点是主库的写请求时延会增长，吞吐量会降低。

数据库中间件模式。在这种模式下，所有的读写都走数据库中间件，通常情况下，写请求路由到主库，读请求路由到从库。记录所有路由到写库的 key，在主从同步时间窗口内（假设是 500ms），如果有读请求访问中间件，此时有可能从库还是旧数据，就把这个 key 上的读请求路由到主库。在主从同步时间过完后，对应 key 的读请求继续路由到从库。相关的中间件有 Canal，Otter。优点是能保证绝对一致，缺点是数据库中间件的成本较高。

缓存记录（应用层）模式。写流程：如果 key 要发生写操作，记录在 cache 里，并设置“经验主从同步时间”的 cache 超时时间，例如 500ms，然后修改主数据库。相当于你自己做一个中间件。读流程：先到缓存里查看，对应 key 有没有相关数据，如果有相关数据，说明缓存命中，这个 key 刚发生过写操作，此时需要将请求路由到主库读最新的数据。如果缓存没有命中，说明这个 key 上近期没有发生过写操作，此时将请求路由到从库，继续读写分离。这种方式的优点是相对数据库中间件，成本较低，缺点是为了保证“一致性”，引入了一个 cache 组件，并且读写数据库时都多了缓存操作。

这里的 key 通常指的是一个能唯一标识本次写操作的数据主键（或业务主键），例如数据库表中的主键 ID、唯一约束字段（如用户 ID、订单号、题目 ID 等）等。它的作用是让中间件能在主从延迟期间识别哪些数据刚被写入主库，从而把后续对这些数据的读请求临时路由到主库。

12.3.数据库的选型（重点★★★★★）

文档型数据库、键值对数据库、列存储数据库、图数据库和时序数据库会放在一起考。一个比较常规的出题方向就是给定场景让你挑选合适的数据库，所以你主要掌握不同的数据库产品到底适合什么业务场景。

分类	场景	结构	产品
键值数据库	内容缓存，处理大量数据的高访问负载，日志系统等	Key 指向 Value 的键值对，通常用 hashtable 实现	Redis
列存储数据库	分布式文件系统，海量数据的高效存储与分析（如大数据场景）	以列簇式存储，将同一列数据存放在一起	Cassandra, HBase
文档型数据库	Web 应用、需要灵活数据结构存储（如电商商品信息管理）	Key-Value 对应的键值对，Value 为结构化数据	MongoDB
图形数据库	社交网络、推荐系统、知识图谱构建（如分析用户关系链）	图结构（节点、边、属性）	Neo4J
时序数据库	物联网设备监控数据存储、工业设备运行状态记录、金融交易数据时序分析等	以时间戳为核心，按时间序列组织数据记录	InfluxDB、TDengine
向量数据库	语义搜索、RAG 知识库、推荐系统、图像/文本相似度检索等 AI 应用场景	以高维向量为核心，基于向量索引组织数据，支持相似度检索	Milvus、Pinecone、Weaviate、Qdrant

13.下篇-Redis（重点★★★★★）

从 20 年开始 Redis 就几乎年年考，次次出，几乎把所有常用的主题都搞完了，假如之前是基础篇的话，现在 Redis 题目的难度高低也要算个进阶篇。所以 Redis 主题凯恩会写得略多一点。这一块没有开发基础的同学也别想着去本地部署测试了，赶紧背下来吧。八股不讲道理，背就是会，不背就是不会。

与传统数据库不同的是，Redis 的数据是保存在内存中的（内存数据库，支持持久化），因此读写速度非常快，被广泛应用于分布式缓存方向。并且，Redis 存储的是 KV 键值对数据。为了满足不同的业务场景，Redis 内置了多种数据类型实现（比如 String、Hash、SortedSet、Bitmap、HyperLogLog、GEO）。并且，Redis 还支持事务、持久化、Lua 脚本、发布订阅模型、多种开箱即用的集群方案（RedisSentinel、RedisCluster）。

13.1.数据类型（重点★★★★★）

Redis 支持的数据类型如下表所示，之前的考法是给你场景让你做数据类型的分类，所以下表所列的内容必须掌握。

属性名	含义	适用场景
String	字符串结构	存储配置数据、缓存对象、数据统计、限制时间内请求次数
Hash	键值对结构	存储用户信息、商品信息、文章信息、购物车信息
List	列表结构	记录最新文章、最新动态，实现简单消息队列
Set	集合数据结构，数据不重复，可进行交集、并集、差集运算，可随机获取元素	存放不重复数据（如网站 UV 统计、文章点赞等），获取多个数据源交集、并集、差集（如共同好友、好友推荐等），随机获取元素（如抽奖系统、随机点名）

Zset	有序集合数据结构，可根据权重排序	各种根据某个权重进行排序的场景（如直播间送礼物排行榜、微信步数排行榜、段位排行榜、话题热度排行榜）
Bitmap	二进制位，用 0/1 表示状态信息	保存状态信息（如用户签到情况、活跃用户情况、用户行为统计）
HyperLogLog	基数计数	数量巨大（百万、千万级别以上）的计数场景（如热门网站每日/每周/每月访问 ip 数统计、热门帖子 uv 统计）
Geospatial	地理空间索引	管理使用地理空间数据的场景（如附近的人）

这里最有可能做比较的是 String 和 Hash，它们都可以存储对象，到底用谁。

一般来说 String 存储的是序列化后的整个对象数据，相比 Hash 更节省内存，缓存相同量的对象数据内存消耗约为 Hash 的一半，存储具有多层嵌套的复杂对象更加方便。但是 Hash 可以单独存储和获取对象的部分字段信息，可以方便地修改或添加部分字段，节省网络流量，适合于对象中某些字段经常变动或需要单独查询的场景。

13.2.持久化机制（重点★★★★★）

Redis 作为内存数据库，为了避免数据丢失，提供了两种持久化机制，RDB(RedisDataBase) 快照和 AOF (Append-OnlyFile) 日志。以往的考试中直接考你 RDB 和 AOF 的对比。

RDB 持久化是默认的持久化方式，它会在指定的时间间隔内，将 Redis 中的数据以快照的形式写入磁盘。这种方式可以快速恢复数据，但如果宕机数据可能丢失。

AOF 持久化则是将每一个写命令都追加到文件中，可以做到秒级数据持久化，但恢复速度较慢。AOF 文件可以设置三种同步策略：always、everysec 和 no。AOF 重写机制可以压缩 AOF 文件，减小磁盘占用。从 Redis7.0 开始，AOF 重写机制得到了优化，引入了 MultiPartAOF 机制，解决了内存和 IO 资源浪费的问题。

13.3.Redis 集群（重点★★★★★）

Redis 集群是通过将多个 Redis 节点连接在一起实现高可用性、数据分片和负载均衡的技术。

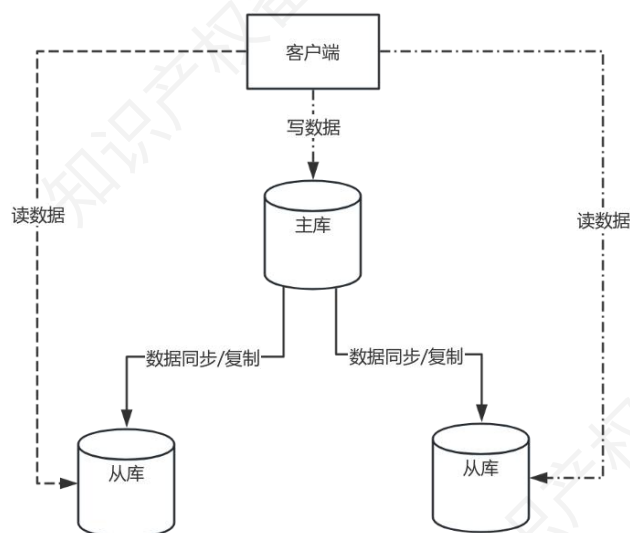
主要有三种集群模式：主从复制模式、哨兵模式和 Cluster 模式。Redis 集群的作用和优势包括高可用性、负载均衡、容灾恢复、数据分片和易于扩展。真题考过，特别关注三种模式的优缺点。

13.3.1.主从复制模式（重点★★★★★）

通过将一个 Redis 节点（主节点）的数据复制到其他节点（从节点）实现数据冗余和备份。

主节点负责写操作，从节点负责读操作，实现读写分离。

主从复制模式优点是配置简单、实现数据冗余和读写分离；缺点是无法自动故障转移，受单节点内存限制。适用于数据备份、读写分离和在线升级场景。



13.3.2.主从复制流程（重点★★★★★）

13.3.2.1.全量同步（重点★★★★★）

全量同步就是主节点把自己“全量的数据快照+期间新增的写操作”完整传递给从节点，从而让从节点的数据和主节点保持一致。它是主从复制中最重的一步，耗时耗资源，但能保证一致性。主要包括下面 5 个步骤，下图凯恩画的更加细致一点让你更好理解。

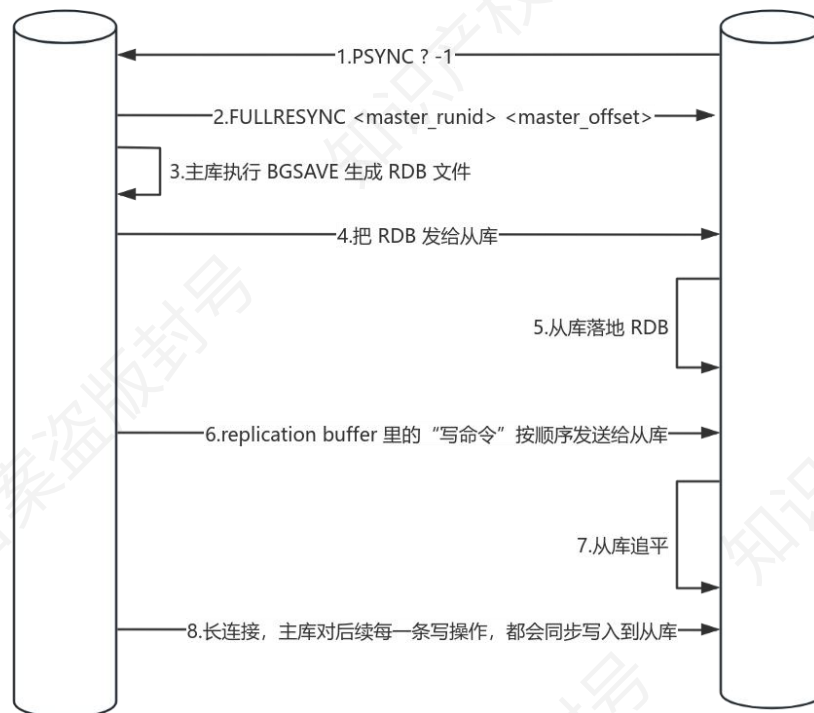
1.握手阶段。从库发送命令 `PSYNC?-1`，主库回应 `FULLRESYNC<runid><offset>`。这里，`runid` 是主节点的唯一身份标识。可以把它理解为主库的“身份证号”。`offset` 就是复制进度指针，是一个不断递增的数字。它记录了主从之间“写命令传输的字节数”位置。主库有一个全局的 `master_repl_offset`。每个从库也有自己当前已同步到的 `slave_repl_offset`。

2.主库生成快照。主库执行 `BGSAVE` 生成 `RDB` 文件。在生成期间，主库新的写命令先进入 `replicationbuffer`，避免丢失。

3.传输 `RDB`。主库将 `RDB` 文件发送给从库。从库清空旧数据，加载新的 `RDB`，得到主库当时的快照。

4.补齐追赶阶段。主库把生成 `RDB` 期间缓存的命令（在 `replicationbuffer` 中的）发给从库。从库执行完这批命令，就追平到主库最新状态。

5.进入常态复制。后续所有写命令实时通过长连接传递，从库保持持续同步。



13.3.2.2. 增量同步（重点★★★★★）

增量同步（部分重同步）是 Redis 主从复制中的一种优化方式。它的核心思想是：当主从复制因为网络抖动或短暂断开中断时，不用重新走一遍全量同步，而是只把断线期间缺失的那部分数据补给从库即可。主要包括下面 4 个步骤，不同的说法粒度不一样划分的步骤也不一样，这个了解即可。

1. 从库重连时带上凭据。从库拿出之前记住的主库 `runid` 和自己最新 `offset`，再次发送命令 `PSYNC<remembered_runid><my_offset>`

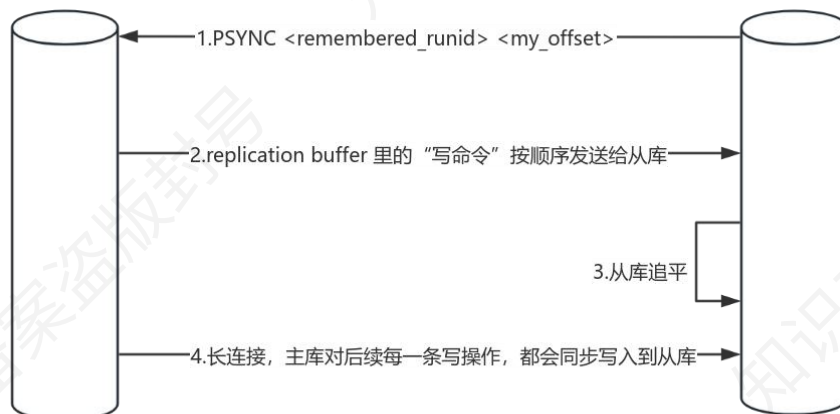
2. 主库收到命令后做两步校验（下图省略）：`runid` 一致？一致→说明还是原来的主库。不一致→主库换人了（例如主库重启/切换），必须全量同步。

需要同步的缺口数据还在 `backlog` 里吗？在：返回 `CONTINUE`，执行部分重同步。不在：只能全量同步。

3. 开启增量传输。主库从 `repl_backlog_buffer` 中找到“>offset”的那段日志，把它们放入

对该从库的 `replicationbuffer`，再发给从库执行。从库执行完就追平了。

4.进入常态复制。后续所有写命令实时通过长连接传递，从库保持持续同步。



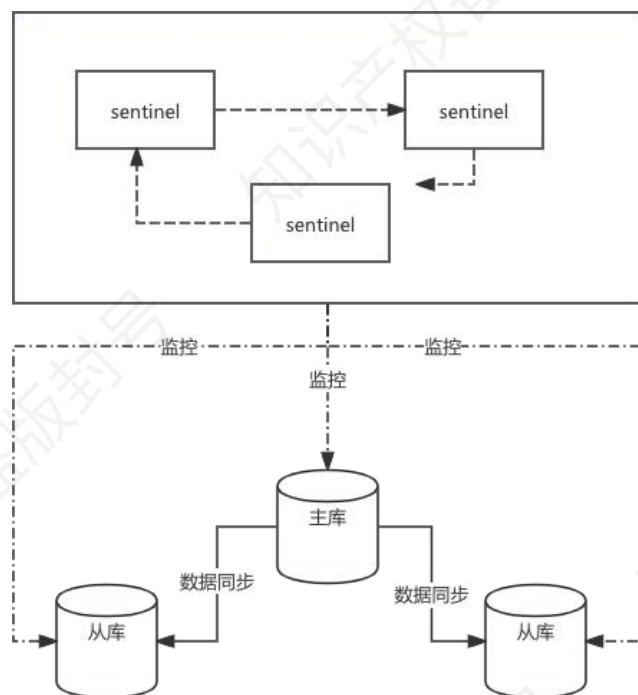
13.3.3.哨兵模式（重点★★★★★）

在主从架构中，假设主节点突然宕机，而没有新的主节点顶替上来，就会导致写请求在很长一段时间内无法响应，直接影响业务的正常运行。

针对这一问题，Redis 引入了哨兵机制。哨兵的核心作用就是持续监控主从节点的健康状态，一旦发现主节点下线，就会自动发起主从切换：将某个从节点提升为新的主节点，并通知其他从节点与之同步。这样能够最大限度地减少停机时间和数据丢失，保证系统的高可用性。在主从复制基础上添加哨兵节点，实现自动故障转移。

哨兵节点监控主从节点，当主节点故障时自动选举新的主节点。优点是提高高可用性，具有主从复制的其他优点；缺点是配置和管理相对复杂。适用于高可用性要求较高的场景。

从下图哨兵模式的架构图中可以看到有：哨兵节点（Sentinel）：负责监控、选举和故障转移。Redis 节点：包括 Master 与 Slave，是提供实际服务的数据库实例。通常至少三个哨兵节点组成哨兵集群，以保证投票的有效性和健壮性。



这里哨兵的主要功能有三个：

- 1. 监控（Monitoring）：**哨兵持续监控 Redis 主节点和从节点的运行状态。每隔一段时间向节点发送 PING 请求，判断是否正常。
- 2. 自动故障转移（Automatic Failover）：**当主节点发生故障时，哨兵会自动选举出一个新的主节点，并通知其他从节点与之建立复制关系。客户端也会被通知连接新的主节点，从而继续对外提供写服务。
- 3. 通知（Notification）：**哨兵可以通过消息、API 等方式，将节点状态变化及时通知系统管理员或其他服务，方便快速处理。

13.3.3.1.哨兵模式下故障转移流程（超级重点★★★★★）

1. 选举 SentinelLeader。当主节点被判定为客观下线后，首先需要在哨兵集群中选出一个 Leader 来负责主从切换。发现主节点异常的哨兵会作为候选者发起选举，每个哨兵节点只有一票，获得过半数选票的候选者才能成为 Leader。为避免平票，哨兵节点的数量一般会设置为奇数，如 3、5 或 7。

2.选举新的 Master 节点。Leader 哨兵选出新的主节点时，会从存活的从节点中进行筛选。

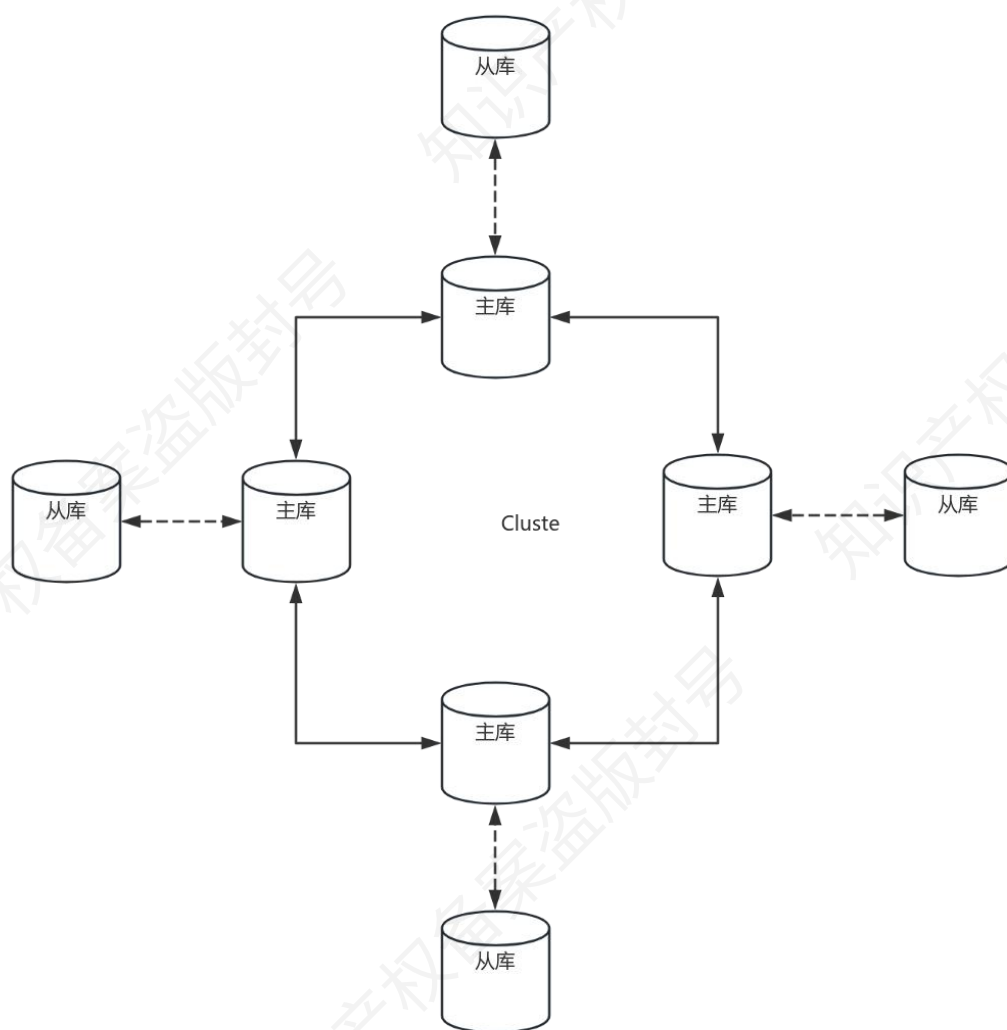
选择顺序通常为：优先级（slave-priority 越小越优先）、复制偏移量 offset（同步数据越多越优先）、以及实例 ID 大小（ID 小的优先）。通过这种方式，保证选出的新主节点数据尽量完整。

3.重建主从关系。新的主节点确定后，Leader 哨兵会向其他从节点下发命令，让它们与新主节点建立复制关系。同时，哨兵利用 Redis 的发布/订阅机制，将新主节点的地址信息推送给客户端，以便业务能够继续正常运行。

4.旧主节点恢复处理。如果旧的主节点后来恢复上线，哨兵会让它执行 `slaveof` 命令，自动降级为新主节点的从节点。这样可以避免出现两个主节点的冲突，并保证整个集群的主从关系始终一致。

13.3.4.Cluster 模式（重点★★★★★）

Redis 推出 Cluster 模式，主要是为了解决单机模式和主从+哨兵模式的局限性。单机受内存和性能限制，无法支撑超大规模的数据；而主从+哨兵虽然能实现高可用，但所有数据仍然集中在一个主库，无法水平扩展。Cluster 模式通过数据分片（slot）+自动故障转移，既能突破单机容量限制，又能保证集群的高可用性。

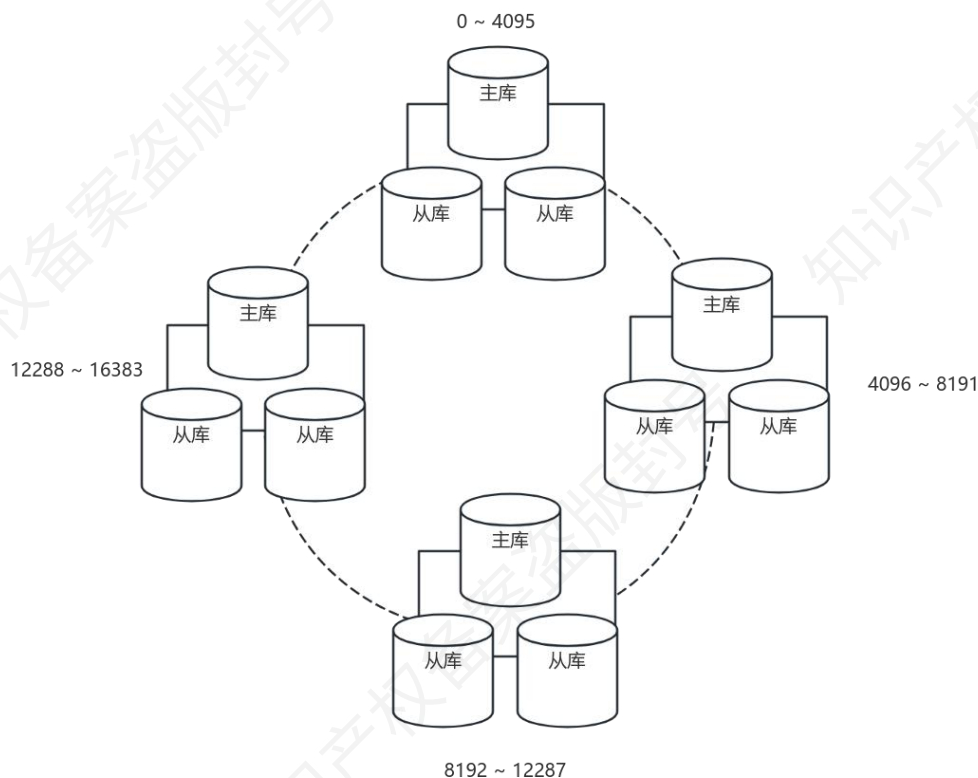


上图展示了 Redis 集群的基本结构。集群由多个主库（Master）组成，每个主库负责一部分数据分片；同时，每个主库还配有一个或多个从库（Slave），存储主库的数据副本，既能分担读请求，又能在主库故障时顶上来。

为了实现数据分布，Redis 集群采用哈希槽机制：整个键空间被划分为 16384 个槽，每个主库负责其中的一部分。客户端请求时，先根据键计算槽位，然后找到对应的主库节点；如果连接的节点不是目标节点，会通过重定向机制指引到正确的节点。

当某个主库发生故障时，从库会在投票机制下被提升为新的主库，集群继续对外提供服务。这样一来，Redis 集群既能通过多主分片实现水平扩展，又能依靠主从切换提供高可用，避免单点故障带来的系统不可用。

注意：Redis 哈希槽是 RedisCluster 的特定实现，把键空间固定分为 16384 个槽。每个主节点负责一部分槽位，数据通过 $\text{CRC16}(\text{key})\%16384$ 来定位。当集群扩容或缩容时，只需要迁移相应槽位的数据，迁移范围可控。



13.3.5.哨兵模式和 Cluster 模式区别（重点★★★★★）

RedisCluster 是 Redis 内置的集群方案，它把整个键空间切分成多个槽位，自动把数据分布到不同节点上。这样一来，存储和访问数据都能水平扩展，天然支持大规模场景。同时，它内置了类似 Sentinel 的故障检测和转移机制，当某个节点挂掉时，集群会自动把流量切到健康的节点，不需要额外组件干预。

而 Sentinel 本身并不负责数据分片，它的作用是监控多个 Redis 实例的运行状态，处理主从架构下的故障转移。当主节点宕机时，Sentinel 会选举一个从节点提升为新的主节点，保障系统的高可用性。

如果重点是横向扩展、分担超大数据量和高并发访问，应该选 RedisCluster；如果只是想提高单个 Redis 服务的可用性，做读写分离和主从切换，用主从复制配合 Sentinel 就足够。

13.4.Redis 常见生产问题（重点★★★★★）

Redis 缓存在高并发生产场景会出现三种问题如下表所示，这里要关注的是导致问题的原因和对应的解决方案。穿透：不存在的东西打进来→查/记/滤，击穿：单点崩溃被突破→固/锁/预，雪崩：群体崩溃压垮 DB→集/限/多/预。

问题	原因	解决方案
缓存穿透	大量请求的 key 不合理，在缓存和数据库中均不存在，导致请求都落到数据库上，对数据库造成压力	参数校验，合法性检查，对非法请求直接返回异常；缓存无效 key，即使查不到数据也将 key-value 写入缓存，设置短暂过期时间；使用布隆过滤器，先判断 key 是否存在，不存在则直接返回，存在再进行后续处理。布隆过滤器利用多个独立的哈希函数将元素映射到位数组中，可节省存储空间，但可能出现“假阳性”，错误率可通过调整位数组长度 m 和哈希函数个数 k 来控制。
缓存击穿	热点数据在缓存过期的瞬间，大量请求打到数据库，导致数据库压力陡增	设置热点数据永不过期或过期时间较长。请求数据写入缓存前，先获取互斥锁，保证只有一个请求会落到数据库。提前预热，将热点数据提前加载到缓存中。
缓存雪崩	大量缓存在同一时间失效，大量请求直接落到数据库，导致数据库压力巨大	采用 Redis 集群，避免单机故障。限流，避免同时处理大量请求。多级缓存，例如本地缓存+Redis 缓存。设置不同的失效时间，避免集中失效。缓存预热，主动加载热点数据到缓存

13.5.缓存和数据库一致性问题（重点★★★★★）

这里仅介绍常规做法——缓存旁路模式。24 年 11 月，架构考过。

读取数据时，先查询缓存，如果缓存中存在所需的数据，则直接返回。如果缓存中没有，则查询数据库，获取数据并将数据写入缓存。写入/更新数据时，先更新数据库，然后直接删除缓存中对应的数据。下次读取时会自动从数据库中加载最新数据到缓存。

这里会有一个同步问题。假设线程 1 做了一个写入数据库的操作，此时数据库的值是新的，缓存是旧的，我当然希望缓存也是立刻变成新的，这样肯定不会有业务问题。但是我们是一个高并发的系统，可能你在写完数据库之后，还没来得及改缓存，后面的一个线程就来读，此时它读的是缓存的就旧值，这样就可能发生业务问题。

所以这里的解决办法：一种是基于强一致性，也就是保证数据库中的值和缓存中的数据任何情况下都一致，假如不一致，缓存没改完前不给别人读，但是这种方法需要用分布式锁/事务/队列串行化，会显著影响性能，不太适合高并发场景。另外一种是最最终一致性，允许数据库和缓存短暂不一致，但是可以确保过了一段时间，数据库和缓存的值肯定是一样的，都是最新的。我们要做的就是缩短这个不一致的时间。

(1) 延迟双删（最终一致性）。在写入数据时，先删除缓存中的数据，然后更新数据源中的数据，最后在经过一段短暂的延迟后再次删除缓存（记住结论即可，推论比较复杂，这里就很玄学了，什么叫短暂的时间，多短多长，不同的时间设置，在不同的业务场景也有可能导致不一致，概率高低罢了）。

(2) 先写数据库再删除缓存（最终一致性）（会有失效的情况，但是概率很低，就是缓存失效和写数据库同时发生的时候，这时候会有旧数据填充到缓存里，记住结论即可，实践中常用这个方法，因为最简单）。

(3) 分布式锁（强一致性）。在写入数据时，先删除缓存中的数据，然后更新数据源中的数据。这一环节加锁解锁确保一致性（获取写锁、更新数据库、删除缓存、释放写锁）。读

数据的时候也要对访问数据库的操作获取读锁，缓存读取完毕之后，释放读锁，确保读写操作序列化。

(4) 消息队列进行序列化读写操作（最终一致性）。消息队列进行读写操作。把同一业务键的写请求路由到同一消息队列，天然顺序、无乱序覆盖。

(5) 订阅 binlog 日志（最终一致性）。在写入数据时，先修改数据源中的数据。同时可以使用阿里的 canal 将 binlog 日志采集发送到 MQ 队列里面，然后通过 ACK 机制确认处理这条更新消息，修改缓存，保证数据缓存一致性。因为队列消费是按顺序的，所以缓存最终会依据顺序修改达到最后的状态。

13.6.过期数据删除策略（重点★★★★★）

Redis 的过期数据删除策略是用于处理那些设置了过期时间的键（key）的机制，其目的是保证过期数据能及时从内存中移除，以此维持缓存的有效性和节省内存空间。常用的过期数据的删除策略：惰性删除和定期删除。

(1) 惰性删除。只会在取出 key 的时候才对数据进行过期检查。这样对 CPU 最友好，但是可能会造成太多过期 key 没有被删除。

(2) 定期删除。每隔一段时间抽取一批 key 执行删除过期 key 操作。并且，Redis 底层会通过限制删除操作执行的时长和频率来减少删除操作对 CPU 时间的影响。

13.7.内存淘汰策略（重点★★★★★）

除了这两个删除，还有一个机制也会淘汰 key，即当 Redis 内存使用达到设置的 maxmemory 限制时，会触发内存回收机制，它所用的策略如下表所示。

淘汰策略	策略描述	适用场景
------	------	------

noeviction（默认策略）	当内存使用达到上限时,拒绝所有写入操作	需要确保数据完整性,对可用性要求相对较低的场景
volatile-lru（最近最少使用）	从已设置过期时间的键中,淘汰最近最少使用的数据	缓存场景,特别是希望优先淘汰不常用缓存数据的情况
allkeys-lru（全局最近最少使用）	从所有键中,淘汰最近最少使用的数据	通用的缓存场景
volatile-random（随机淘汰）	从已设置过期时间的键中,随机淘汰数据	对数据淘汰顺序无严格要求的缓存场景
allkeys-random（全局随机淘汰）	从所有键中,随机淘汰数据	对数据淘汰顺序无严格要求,希望均匀淘汰数据的场景
volatile-ttl（最小 TTL 优先）	从已设置过期时间的键中,淘汰剩余 TTL（存活时间）最小的数据	缓存即将过期的数据,希望优先处理快过期数据的场景

13.8.热 Key 和大 Key 问题（重点★★★★★）

在使用 Redis 的过程中,如果未能及时发现并处理 Bigkeys(下文称为“大 Key”)与 Hotkeys(下文称为“热 Key”),可能会导致服务性能下降、用户体验变差,甚至引发大面积故障。

13.8.1.大 Key（重点★★★★★）

大 Key 一般的定义如下:Key 本身的数据量过大:一个 String 类型的 Key,它的值为 5MB; Key 中的成员数过多:一个 ZSET 类型的 Key,它的成员数量为 10,000 个; Key 中成员的数据量过大:一个 Hash 类型的 Key,它的成员数量虽然只有 2000 个但这些成员的 Value(值)总大小为 100MB。所以大 Key 的本质不是 Key 大,而是它对应的数据大,是 Value 大。

它会导致客户端执行命令的时长变慢; Redis 内存达到 MaxMemory 参数定义的上限引发操作阻塞或重要的 Key 被逐出(前面内存淘汰的时候提到过); 集群架构下,某个数据分片的内存使用率远超其他数据分片,无法使数据分片的内存资源达到均衡; 对大 Key 执行删除

操作，易造成主库较长时间的阻塞，进而可能引发同步中断或主从切换。如何解决大 Key 的问题，常规做法如下表所示，主要思路是清理+拆分。

处理方式	具体操作	作用或原因
拆分大 Key	将含有数万成员的一个 HASHKey 拆分为多个 HASHKey，确保每个 Key 的成员数量在合理范围	在 Redis 集群架构中，有助于实现数据分片间的内存平衡
清理大 Key	把不适用 Redis 能力的数据存到其他存储，并在 Redis 中删除此类数据	去除不适合 Redis 存储的数据，优化 Redis 使用
定期清理过期数据	通过定时任务清理失效数据，避免如在 HASH 数据类型中以增量形式写入大量数据却忽略时效性而产生大 Key 的情况	防止过期数据堆积导致大 Key 的产生

13.8.2.热 Key（重点★★★★★）

热 Key 的定义一般如下：QPS 集中在特定的 Key：Redis 实例的总 QPS（每秒查询率）为 10,000，而其中一个 Key 的每秒访问量达到了 7,000；带宽使用率集中在特定的 Key：对一个拥有上千个成员且总大小为 1MB 的 HASHKey 每秒发送大量的 HGETALL 操作请求；CPU 使用时间占比集中在特定的 Key：对一个拥有数万个成员的 Key（ZSET 类型）每秒发送大量的 ZRANGE 操作请求。

它会导致占用大量的 CPU 资源，影响其他请求并导致整体性能降低；集群架构下，产生访问倾斜，即某个数据分片被大量访问，而其他数据分片处于空闲状态，可能引起该数据分片的连接数被耗尽，新的连接建立请求被拒绝等问题；热 Key 的请求压力数量超出 Redis 的承受能力易造成缓存击穿，即大量请求将被直接指向后端的存储层，导致存储访问量激增甚至宕机，从而影响其他业务。

解决思路	具体操作	优点	缺点
------	------	----	----

多级缓存	使用本地缓存（如 GuavaCache、Caffeine、HashMap 等），发现热 key 后将其加载到系统 JVM 中，热 key 请求直接从本地缓存获取，不直接请求 Redis	将同一 key 的大量请求根据网络层负载均衡分散到不同机器节点，避免全部打到单个 Redis 节点，减少一次网络交互	对于要求缓存强一致性的业务，需花费更多精力保证分布式缓存一致性，增加系统复杂度
热 key 备份	在多个 Redis 节点上备份热 key，使备份 key 分散在各个节点，有热 key 请求时随机选取一个备份 key 所在节点访问取值	缓解 Redis 单点热 key 查询压力，使读写操作不集中于单个节点，提升系统应对热 key 问题的能力	需占用更多 Redis 节点存储空间
热 key 拆分	将热 key 拆分成多个带后缀名的 key，分散存储到多个实例中，客户端请求时按规则算出固定 key，使多次请求分散到不同节点	缓解热 key 集中访问压力，提升系统性能和用户体验	用户可能只能获取部分数据，需在热点降温后汇总数据重新推送
业务隔离	提前做好核心与非核心业务的 Redis 隔离，将存在热点 key 的 Redis 集群与核心业务隔离开	保障核心业务不受热点 key 引发的问题影响，确保核心业务的稳定性和可用性，提升系统整体的可靠性和容错能力	增加系统架构复杂度和成本

13.9.分布式锁（重点★★★★★）

24 年考察过分布式锁的用法，它是从数据库表实现出发让你提出改进方案的。分布式锁在分布式系统环境下，用来协调多个节点对共享资源进行互斥访问的一种机制。它的目标和单机锁一样：保证在同一时刻，某个临界资源只能被一个线程（或进程）访问。但由于分布式环境下有多个进程、多个服务器，普通的线程锁无法跨进程、跨节点起作用，就需要分布式锁。这里你要掌握三种方案。

基于数据库表。要实现分布式锁，最简单的方式就是直接创建一张锁表，然后通过操作该表中的数据来实现锁。当要锁住某个方法或资源时，就在该表中增加一条记录，想要释放锁的时候就删除这条记录。这种实现方式存在的问题：（1）锁强依赖数据库的可用性，数据库是单点，一旦数据库挂掉，会导致业务系统不可用。（2）锁没有失效时间，一旦解锁操作失败，就会导致锁记录一直在数据库中，其他线程无法再获得锁。（3）锁只能是非阻塞的，因为数

据的 `insert` 操作，一旦插入失败就会直接报错。没有获得锁的线程并不会进入排队队列，要想再次获得锁就要再次触发获得锁操作。（4）这把锁是非重入的，也就是说同一个线程在没有释放锁之前无法再次获得该锁。因为数据中数据已经存在。

使用 Redisson 的分布式锁，支持 Redis 单实例、Redis 主从、RedisSentinel、RedisCluster 等多种部署架构。Redisson 框架会开启一个定时器的守护线程，每 `expireTime/3` 执行一次，去检查该线程的锁是否存在，如果存在则对锁的过期时间重新设置为 `expireTime`，即利用守护线程对锁进行“续命”，防止锁由于过期提前释放。Redisson 的分布式锁也是非阻塞的，同样需要 `while` 重复执行。Redisson 的分布式锁是可重入的。因为 Redisson 的锁是 `hset` 结构，`key` 值就是客户端的身份标识，`value` 是加锁次数，从而实现了可重入加锁。

使用 ZooKeeper 实现分布式锁的过程：客户端连接 ZooKeeper，并在 `/lock` 下创建临时且有序的子节点，第一个客户端对应的子节点为 `lock-0000`，第二个为 `lock-0001`，以此类推。客户端获取 `/lock` 下的子节点列表，判断创建的节点是否为当前子节点列表中序号最小的节点，如果是则认为获得锁，否则监听前一个子节点的删除消息。获取锁后，执行业务代码流程，删除当前客户端对应的子节点，锁释放。使用 Zookeeper 实现分布式锁的优点：有效地解决单点问题、不可重入问题、非阻塞问题以及锁无法释放的问题。实现起来较为简单。

使用 Zookeeper 实现分布式锁的缺点：因为 Zookeeper 集群采用 `zab` 一致性协议，所以高并发场景，性能上不如使用 Redis 实现分布式锁。

13.10.布隆过滤器（重点★★★★★）

前面讲到过布隆过滤器，它是一种高效的概率数据结构，常用于检测一个元素是否在一个集合中，可以有效减少数据库的查询次数，解决缓存穿透等问题。它的底层结构是由一个很大

的位数组（bit array）和 k 个独立的哈希函数组成（这里 k 个哈希很重要）。我们可以这么来做标记和判断

1.添加元素：当我们把某个元素加入集合时，会通过这 k 个哈希函数计算出几个下标，把位数组对应的位置都标记为 1。

2.判断存在性：当要判断某个元素是否在集合中时，同样用这 k 个哈希函数算出下标。如果发现这些位置有任意一个是 0，那就能完全确定这个元素一定不在集合里；如果都为 1，只能说它可能存在，但不能保证一定存在。

这种“可能存在”的情况，是因为不同元素经过哈希函数可能会映射到同样的位置，从而造成误判。换句话说，它有“假阳性”（false positive）的概率，但绝不会出现“假阴性”。

实际上 Redis 本身支持布隆过滤器的实现，一种是通过位图，一种是通过自带的模块，如下表所示，了解即可。

实现方式	描述	优缺点
位图实现	利用 Redis 的位图结构 SETBIT/GETBIT 操作，自定义哈希函数映射元素到位图位置	优点：轻量级，无需额外模块；缺点：需手动管理哈希函数和位图大小，易出错
RedisBloom	通过官方模块控制哈希函数、位图大小	优点：开箱即用，自动优化哈希函数和位图大小，支持动态调整

13.11.Redis 事务（超级重点★★★★★）

Redis 的事务机制其实非常简洁，它的核心思想是把一组命令打包在一起顺序执行，从而避免并发干扰，有点类似我们的批处理，重新定义事务（狗头）。

使用时，客户端先通过 MULTI 命令开启事务模式，此时后续输入的命令不会立即执行，而是被放入一个队列中；等到执行 EXEC 时，Redis 会将事务中排队的所有命令一次性、按顺

序依次执行。这样可以保证执行过程中不会被其他客户端的命令插入，提供了串行化的效果。如果在事务中途想要放弃，可以使用 **DISCARD** 来清空队列并结束事务。

Redis 事务并不完全符合传统数据库的 **ACID** 特性，它主要不满足原子性和持久性的严格定义，更适合作为轻量级的批量执行机制。

比如 **Redis** 事务并不严格满足原子性。**EXEC** 提交后，命令会按顺序依次执行，中途如果某条命令执行失败（比如语法错误），之前的命令不会回滚，之后的命令仍然会继续执行。因此 **Redis** 事务更像是“部分原子”，保证要么所有命令都被发送到执行队列，要么全部放弃，但不保证执行过程中回滚。

Redis 的持久性取决于配置。默认情况下如果只在内存中运行，则事务结果可能在宕机后丢失；如果开启了 **RDB** 或 **AOF** 持久化，数据才会写入磁盘，才具备一定的持久性保障。因此持久性并不是事务本身保证的，而依赖持久化策略。

13.12.Redis 指令集（重点★★★★★）

有一年考过 **Zset** 的相关操作指令，所以这里凯恩把常用的指令列出来方便考前回顾。

类型	指令
字符串 (Strings)	SETkeyvalue : 设置字符串的值。 GETkey : 获取字符串的值。 INCRkey : 将字符串（整数值）增加 1。 DECRkey : 将字符串（整数值）减少 1。 APPENDkeyvalue : 向字符串追加值。
列表 (Lists)	LPUSHkeyvalue : 在列表头部插入元素。 RPUSHkeyvalue : 在列表尾部插入元素。 LPOPkey : 从列表头部弹出元素。 RPOPkey : 从列表尾部弹出元素。 LRANGEkeystartstop : 获取列表中指定范围的元素。
集合 (Sets)	SADDkeymember[member..] : 向集合添加一个或多个成员。 SREMkeymember[member..] : 从集合中删除一个或多个成员。 SPOPkey : 从集合中随机移除并返回一个成员。

	SCARDkey: 获取集合的成员数。
有序集合 (SortedSets):	ZADDkeyscoremember[scoremember..]: 向有序集合添加一个或多个成员, 每个成员关联一个分数。 ZREMkeymember[member..]: 从有序集合中删除一个或多个成员。 ZRANGEkeystartstop[WITHSCORES]: 获取有序集合中指定分数范围的成员。
散列 (Hashes)	HSETkeyfieldvalue: 在散列中设置字段的值。 HGETkeyfield: 从散列中获取字段的值。 HGETALLkey: 获取散列中的所有字段和值。
位图 (Bitmaps)	SETBITkeyoffsetvalue: 在字符串对象中设置位的值。 GETBITkeyoffset: 获取字符串对象中位的值。
超日志 (HyperLogs)	PFADDkeyelement[element..]: 向 HyperLogLog 中添加元素。 PFCOUNTkey: 返回 HyperLogLog 的近似基数。
地理空间 (Geospatial)	GEOADDkeylongitudelatitude[member][longitudelatitude[member..]]: 将地理空间位置的元素添加到键中。 GEODISTkeymember1member2: 返回两个地理空间位置之间的距离。 GEORADIUSkeylongitudelatitude[m km ft mi][WITHCOORD][WITHDIST][WITHHASH]: 返回位于给定坐标周围指定半径内的所有元素。
键管理	KEYSPattern: 查找所有匹配给定模式的键。 DELkey[key..]: 删除一个或多个键。 EXPIREkeyseconds: 为键设置生存时间。
事务	MULTI: 开始一个事务块。 EXEC: 执行事务块中的所有命令。 WATCHkey[key..]: 监视一个或多个键, 如果在执行事务期间这些键被修改, 则事务将失败。
持久化	SAVE: 将数据同步到磁盘。 BGSAVE: 在后台异步保存数据到磁盘。

14.下篇-ElasticSearch（重点★★★★★）

Elasticsearch（简称 ES）是一个开源的分布式、高度可扩展的全文搜索和分析引擎。它能够快速、近乎实时地存储、搜索和分析大量数据。适用于包括文本、数字、地理空间、结构化和非结构化数据等在内的所有类型数据。可以用来实现分布式数据存储、日志统计、分析、系统监控、地理空间查询等功能。Elasticsearch 最底层的搜索引擎技术是 Apache 基金会开源的搜索引擎类库 Lucene，Lucene 提供了搜索引擎核心 API。ES 在 Lucene 的基础上提供了分布式支持，可以水平扩展，提供了 Restful 这种简洁的访问接口，能被任何语言调用。24 年 11 月年软考开始出现 Elasticsearch。Redis 已经很难出题了，ES 要来凑了。

14.1.ES 概述（重点★★★★★）

Elasticsearch 以全文搜索为基础，具有出色的文本搜索功能。它使用分析器和标记化技术来处理文本数据，支持词干分析、停用词过滤、同义词处理等，从而能够实现高效的文本搜索和相关性排名。24 年 11 月架构考过分词器原理，这里注意一下。

Elasticsearch 是一款基于全文检索技术的分布式搜索与分析引擎。它最大的特点是能高效处理大规模文本数据，并支持自然语言查询。与传统数据库的精确匹配不同，Elasticsearch 在建立索引时会对文档进行分词，把文本拆解为一个个词项，然后存入倒排索引。这样一来，用户在输入搜索条件时，系统会用同样的方式对查询语句进行分词，并快速在倒排索引中找到相关文档。最终结果会结合相关性评分算法（如 BM25、TF-IDF）进行排序，让更符合语义的内容排在前面。所以这里凯恩就从分词、倒序索引还有相关性算法三个角度展开说明。

14.1.1.分词原理（重点★★★★★）

在 Elasticsearch 中，分词是通过分析器（Analyzer）来实现的。分析器可以看作是一条流

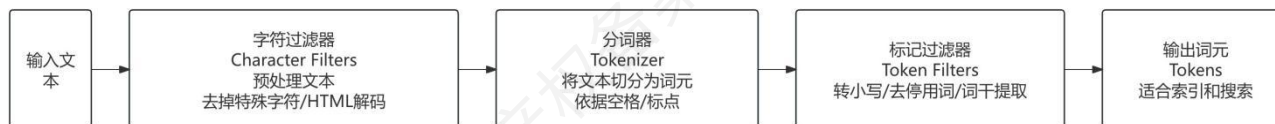
水线，主要由三个环节组成：字符过滤器、分词器和标记过滤器，它们依次对输入文本进行处理。

首先是字符过滤器（CharacterFilters）。这一环节会对原始文本做预处理，比如去掉或替换特殊字符，或者把 HTML 实体转换成可读文字。这一步是可选的，但能让输入更干净。

接着进入分词器（Tokenizer）。这是分词的核心步骤，它会按照特定规则把文本切割成一个个词元（Token）。例如，标准分词器会依据空格、标点来分割，同时保留这些切分出来的词。

最后是标记过滤器（TokenFilters）。在这里，分词结果会进一步被加工，比如全部转成小写，去掉停用词（如“the”、“and”），或做词干提取等，让结果更适合搜索。

分词过程就是：清理文本→切分词元→优化结果。三部分配合起来，把一段原始文本转化成能被高效索引和搜索的词汇单元。如下图所示。



14.1.1.1.Tokenizer（分词器）（重点★★★★★）

在 Elasticsearch 的全文检索体系里，Tokenizer（分词器）它的职责，就是把原始文本切割成一个个最小的词元（term）。词元才是被 Elasticsearch 真正拿来索引、存储和搜索的基本单位。

你可以把 Tokenizer 想象成“碎纸机”：输入是一段长文本，输出是一袋袋小碎片，每片都能单独拿出来做检索。后续的过滤器（Filter）和标准化操作（如去除停用词、转小写、同义词替换）都是在这碎片的基础上完成的。Elasticsearch 支持的分词器如下所示。

Tokenizer 类型	输入文本	输出结果	特点
--------------	------	------	----

standard(标准分词器)	Hello,thisisElasticsearch!	["hello","this","is","elasticsearch"]	去掉标点,统一转小写,适合自然语言处理
whitespace (空白分词器)	HelloWorld,你好Elasticsearch	["Hello","World","你好","Elasticsearch"]	仅按空格切分,标点保留
keyword (关键词分词器)	OrderID-12345	["OrderID-12345"]	不切分,整个输入作为一个词元,常用于ID、手机号等精确匹配
uax_url_email (URL/Email 分词器)	请联系support@example.com 或访问 https://cheko.cc	["请","联系","support@example.com","或","访问","https://cheko.cc"]	能正确识别邮箱和URL,把它们当作整体

14.1.2.倒排索引 (重点★★★★★)

倒排索引也叫反向索引,首先对文档数据按照 id 进行索引存储,然后对文档中的数据分词,记录对词条进行索引,并记录词条在文档中出现的位置。这样查找时只要找到了词条,就找到了对应的文档。概括来讲是先找到词条,然后看看哪些文档包含这些词条。通俗地来讲,正向索引是通过 key 找 value,倒排索引则是通过 value 找 key。倒排索引的优势在于它允许快速的全文搜索。当用户查询一个词时,搜索引擎可以直接查找词典中的词,并获取到所有包含该词的文档的引用,而不需要扫描整个文档集合。这大大提高了搜索效率。

拓展阅读:简单来说,当你向 Elasticsearch 提交一个文档时,系统会将文档拆分成单词(词项),并在倒排索引中记录每个词项所属的文档 ID。检索时,系统通过查询词典找到相关的词项,再通过倒排表确定包含这些词项的文档,最后汇总返回结果。假设我们有以下三个文档:

苹果是一种水果。

香蕉是一种热带水果。

苹果和香蕉都是水果。

我们为这些文档创建倒排索引:

可以得到如下索引：

苹果：[1, 3]

香蕉：[2, 3]

在这个倒排索引中，如果我们搜索“苹果”，索引列表就会告诉我们“苹果”包含在文档 1 和文档 3 内。

14.1.3.相关性算法（重点★★★★★）

在 Elasticsearch 中，相关性得分决定搜索结果排序的关键指标，它衡量文档与查询条件的匹配程度。简单来说，得分越高，说明文档越符合用户的查询需求。

相关性计算由 Lucene 内核完成，常用的算法有 TF-IDF 和 BM25(了解即可)。其中 TF-IDF 基于词在文档中的出现频率(TF)和在所有文档中的稀有程度(IDF)来计算。BM25 是 TF-IDF 的改进版，尤其在长文档处理和参数可调性上更优秀，因此是 Elasticsearch 默认的相关性算法。

14.1.3.1.影响相关性的主要因素（重点★★★★★）

判断是否相关和你具体的检索方式、关键词和文档的关系密切相关。具体来说如下表所示。

因素	含义	影响方式	举例说明
查询类型	不同的查询方式决定相关性计算规则	match 会计算相关性；term 更偏向精确匹配，不参与复杂评分	搜索“手机”：match 会考虑词频和相关性，term 只找完全一致的“手机”
词频（TF）	一个词在文档中出现的次数	出现越频繁，得分越高	文档 A 出现 10 次“手机”，文档 B 出现 1 次 → A 得分更高
逆文档频率（IDF）	一个词在所有文档中出现的稀有程度	越稀有的词贡献得分越高	“iPhone17”比“手机”更稀有，因此区分度更强

文档长度归一化 (Norms)	考虑关键词在短文档 vs 长文档中的代表性	短文档中出现一次的关键词更有代表性	标题 “iPhone17 发布” 中出现一次 “iPhone”，比长评测文档更突出
字段权重 (Boost)	手动调整字段的重要性	被 Boost 的字段匹配会得到更高分	给 title 设置 boost=2，比 description 更优先

14.2.ES 核心概念（重点★★★★★）

前面已经讲了分词、倒排索引和相关性算法。接下来还要把 ES 里的几个核心概念讲清楚，不然后面一说索引库、文档、mapping、DSL、分片、副本，很多同学就容易混在一起。我们可以先把 ES 和传统关系型数据库做一个不完全类比。

ES 概念	含义	可以类比关系型数据库中的什么
Index（索引库）	一类文档的集合，比如商品索引、酒店索引、题目索引	表
Document（文档）	ES 中的一条 JSON 数据	一行记录
Field（字段）	文档中的一个属性	列
Mapping	对字段类型、是否分词、是否索引等的约束	表结构
DSL	ES 使用的 JSON 风格查询语法	SQL
Analyzer（分析器）	负责把文本拆成词条	无
Shard（分片）	一个索引被拆开的若干份数据	分库分表里的分片
Replica（副本）	Replica（副本）	Replica（副本）

这里要注意，ES 里的 Index 容易和数据库索引混淆。数据库里的索引通常是某个表上的辅助结构，比如 B+ 树索引，方便快速定位到行；而 ES 里的 Index 更多是“索引库”的意思，可以粗略理解成一类文档的集合。

14.2.1.Index 索引库（次重点★★★★☆）

Index，叫索引库，可以理解成 ES 中存放同一类数据的地方。比如电商系统中，可以建

立一个 **product** 商品索引库，用来存放所有参与搜索的商品数据。它和关系型数据库中的“表”有点像，但不能完全等同。**MySQL** 的表更多用于事务型数据管理，而 **ES** 的索引库更多用于搜索和分析，比如商品关键词搜索、筛选、排序和聚合统计。

创建一个商品索引库时，可以通过 **DSL** 写成下面这样：

```
PUT /product
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "ik_smart"
      },
      "brand": {
        "type": "keyword"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "double"
      },
      "sales": {
        "type": "integer"
      },
      "status": {
        "type": "keyword"
      }
    }
  }
}
```

这里的 **PUT /product** 表示创建一个名为 **product** 的索引库。**mappings** 表示定义字段结构。

这个例子里，商品标题 **title** 要支持全文搜索，所以使用 **text** 类型，并指定中文分词器 **ik_smart**；品牌 **brand**、类目 **category**、状态 **status** 通常用于精确过滤，所以使用 **keyword**；价格 **price** 和销量 **sales** 要做范围查询或排序，所以使用数值类型。

14.2.2.Document 文档（次重点★★★★☆）

Document，叫文档，是 ES 中真正存储的一条 JSON 数据。在电商商品搜索中，一个商品就可以看成一个文档。比如“华为 Mate 70 Pro 旗舰手机”就是一条商品文档，里面会包含商品 ID、标题、品牌、类目、价格、销量、库存、状态等信息。它可以粗略类比为关系型数据库表中的一行记录。

比如向 product 索引库中写入一条商品文档，可以这样写：

```
POST /product/_doc/10001
```

```
{
  "title": "华为 Mate 70 Pro 旗舰手机 512GB",
  "brand": "华为",
  "category": "手机",
  "price": 4999,
  "sales": 12000,
  "status": "上架"
}
```

这里的 10001 可以理解成这条商品文档的 ID。真实项目中，一般会把 MySQL 中的商品 ID 作为 ES 文档 ID，这样后面修改、删除、同步商品数据时更方便。

14.2.3.Field 字段（次重点★★★★☆）

Field，叫字段，是文档中的一个属性。比如上面这条商品文档中，title、brand、category、price、sales、status 都是字段。它可以类比关系型数据库中的列。用户搜索和筛选商品时，本质上都是围绕这些字段展开的。比如搜索框输入的关键词主要查 title 字段；筛选品牌时查 brand 字段；价格区间筛选查 price 字段；按销量排序查 sales 字段。

14.2.4.Mapping 映射（次重点★★★★☆）

Mapping，叫映射，可以理解成 ES 的字段结构设计。它决定每个字段是什么类型，是否分词，是否建立索引，是否能参与搜索。这里最容易混的是 text 和 keyword。text 会分词，适合商品标题、商品描述、商品卖点这类需要全文检索的字段；keyword 不分词，适合品牌、类目、状态、商品 ID 这类整体匹配的字段。

比如“华为 Mate 70 Pro 旗舰手机 512GB”这个标题，如果设置为 text，ES 可以把它拆成“华为”“Mate”“70”“Pro”“旗舰”“手机”“512GB”等词条，用户搜“华为手机”也能命中。如果把它设置成 keyword，那 ES 会把整句话当成一个整体，用户必须非常精确地匹配整个字符串，搜索体验就会很差。反过来，如果品牌“华为”设置成 text，它会参与分词，反而不适合做精确过滤。所以字段类型不是随便选的，要根据搜索场景来判断。

14.2.5.DSL（重点★★★★★）

DSL，是 ES 使用的 JSON 风格查询语法。它可以粗略类比为关系型数据库中的 SQL，但写法完全不同。SQL 通常写成 select ... from ... where ...，而 ES 的 DSL 一般由三部分组成：HTTP 方法、请求路径和 JSON 请求体。比如 GET /product/_search 表示查询商品索引库，后面的 JSON 请求体描述具体怎么查。

比如用户搜索“华为手机”，同时要求品牌是华为、价格不超过 5000 元，可以这样写：

```
GET /product/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "title": "华为手机"
          }
        }
      ]
    }
  }
}
```

```

    }
  ],
  "filter": [
    {
      "term": {
        "brand": "华为"
      }
    },
    {
      "range": {
        "price": {
          "lte": 5000
        }
      }
    }
  ],
  {
    "term": {
      "status": "上架"
    }
  }
]
}
}
}
}

```

这段 DSL 的意思是：**match** 用来做全文检索，会对“华为手机”进行分词；**term** 用来做精确匹配，比如品牌必须是“华为”；**range** 用来做范围查询，比如价格小于等于 5000；**bool** 用来把多个条件组合起来。这里 **must** 中的条件会参与相关性评分，**filter** 中的条件只做过滤，不参与评分。

14.2.6.Analyzer 分析器（重点★★★★★）

Analyzer，叫分析器，也就是通常说的分词器。严格来说，**Analyzer** 不只是一个“切词工具”，它更像是一条文本处理流水线。它会先对文本做预处理，然后通过 **tokenizer** 把文本切成词元，再通过 **token filter** 对词元做加工，比如转小写、去停用词、拼音转换、同义词处理

等。关系型数据库中没有完全对应的概念，因为传统数据库主要做精确匹配，而 ES 的全文检索非常依赖分词效果。

我们可以直接用 ES 的 `_analyze` API 测试分词效果。例如使用默认的 `standard` 分析器测试商品标题：

```
POST /_analyze
{
  "analyzer": "standard",
  "text": "华为 Mate 70 Pro 旗舰手机 512GB"
}
```

默认分词器对英文、数字还可以，但对中文经常不够理想，可能会把中文拆得比较碎。比如“旗舰手机”可能不能很好地按业务语义拆成“旗舰”“手机”或者“旗舰手机”。如果分词结果不好，后面的倒排索引也会跟着不准确，搜索结果就会变差。

中文搜索一般会使用 `IK` 分词器。比如使用 `ik_smart` 测试：

```
POST /_analyze
{
  "analyzer": "ik_smart",
  "text": "华为 Mate 70 Pro 旗舰手机 512GB"
}
```

`ik_smart` 更偏粗粒度切分，分出来的词更少，结果更干净，适合大多数普通搜索场景。比如它可能把“华为”“Mate”“70”“Pro”“旗舰手机”“512GB”作为词条。用户搜索“华为手机”时，就能通过这些词条找到相关商品。

如果使用 `ik_max_word`，则会尽可能切出更多词：

```
POST /_analyze
{
  "analyzer": "ik_max_word",
  "text": "华为 Mate 70 Pro 旗舰手机 512GB"
}
```

`ik_max_word` 更偏细粒度切分，可能会切出“华为”“Mate”“70”“Pro”“旗舰”“手机”“旗舰手

机”“512GB”等更多词条。这样召回能力更强，用户搜短词时更容易命中，但词条更多，索引体积也会变大，还可能带来一些噪声。工程上经常会采用“建索引用 `ik_max_word`，搜索用 `ik_smart`”的方式，既保证召回，又避免搜索时过度扩展。

例如商品标题字段可以这样配置：

```
PUT /product
{
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "ik_max_word",
        "search_analyzer": "ik_smart"
      },
      "brand": {
        "type": "keyword"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "double"
      }
    }
  }
}
```

这里的 `analyzer` 表示写入文档、建立倒排索引时使用的分析器；`search_analyzer` 表示用户搜索时使用的分析器。也就是说，商品标题入库时尽量多切一些词，方便后面被搜到；用户搜索时则切得相对干净一些，避免召回太多不相关商品。

如果要支持拼音搜索，还可以自定义 `analyzer`。比如用户输入“`huawei`”时，也希望能搜到“华为”。这时可以把 `IK 中文分词` 和 `pinyin token filter` 组合起来。思路是：先用 `IK` 把中文按词切开，再把中文词条转换成拼音词条，同时保留原始中文词条。

示例 DSL 可以写成：

```
PUT /product
```

```

{
  "settings": {
    "analysis": {
      "analyzer": {
        "my_pinyin_analyzer": {
          "tokenizer": "ik_max_word",
          "filter": "py"
        }
      },
      "filter": {
        "py": {
          "type": "pinyin",
          "keep_full_pinyin": false,
          "keep_joined_full_pinyin": true,
          "keep_original": true,
          "limit_first_letter_length": 16,
          "remove_duplicated_term": true,
          "none_chinese_pinyin_tokenize": false
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "my_pinyin_analyzer",
        "search_analyzer": "ik_smart"
      }
    }
  }
}

```

这段配置里，`tokenizer` 使用 `ik_max_word`，表示先按中文语义切词；`filter` 使用自定义的 `py`，表示把切好的中文词条再转成拼音。`keep_original: true` 表示保留原始中文词条，比如“华为”；`keep_joined_full_pinyin: true` 表示保留完整拼音，比如“`huawei`”。这样用户既可以用中文搜“华为”，也可以用拼音搜“`huawei`”。

但是这里要注意，拼音分词不能乱用。创建倒排索引时可以保留拼音词条，但用户搜索中文时，不应该把中文查询也全部转成拼音。否则一些同音词可能互相误召回，比如“苹果”和其

他拼音接近的词。比较合理的做法是：索引阶段多存一些入口，搜索阶段尽量保持查询词本身的语义干净。

14.2.7.Shard 分片（重点★★★★★）

Shard，叫分片，是 ES 实现水平扩展的重要方式。一个商品索引库中的数据可能非常多，比如有几千万甚至上亿条商品数据。如果所有数据都放在一台机器上，磁盘、内存、CPU 和查询并发都会成为瓶颈。ES 可以把一个索引库拆成多个 shard，每个 shard 存放一部分商品文档，并分散到不同节点上。这样既能提升存储能力，也能让多个节点一起参与查询，提高搜索并发能力。创建索引时可以指定分片数量，例如：

```
PUT /product
{
  "settings": {
    "number_of_shards": 3,
    "number_of_replicas": 1
  },
  "mappings": {
    "properties": {
      "title": {
        "type": "text",
        "analyzer": "ik_smart"
      },
      "brand": {
        "type": "keyword"
      },
      "price": {
        "type": "double"
      }
    }
  }
}
```

这里的 `number_of_shards: 3` 表示商品索引库被拆成 3 个主分片。用户查询商品时，ES 会把查询请求分发到相关分片上，各分片分别查询自己的数据，再由协调节点汇总结果并返回给用户。这个过程不是简单的一台机器查到底，而是多个分片共同参与。

Replica，叫副本，是主分片的一份拷贝。它的作用主要有两个：第一，提高高可用；第二，提高查询能力。比如商品索引有 3 个主分片，每个主分片配置 1 个副本分片，那么最终会有 3 个主分片和 3 个副本分片，一共 6 个分片。主分片和副本分片一般会分布在不同节点上。这样某个节点宕机时，只要副本还在其他节点上，商品搜索就不会完全不可用。

副本也可以参与查询。比如用户搜索“华为手机”时，ES 不一定只查主分片，也可以查副本分片，从而分摊查询压力。所以 **replica** 既是“备份”，也是“读扩展”的一部分。

这里要特别注意，主分片数量通常在索引创建后不建议随便修改，因为文档路由和主分片数量有关；副本数量则可以根据集群压力和高可用要求进行调整。比如大促期间搜索压力变大，可以适当增加副本数量，让更多节点参与查询；如果只是单机开发环境，没有其他节点，那么副本可能无法分配，集群状态会变成 **yellow**，但主分片正常时数据仍然可以查询。

14.3.ES 与传统关系型数据库的区别（重点★★★★★）

ES 和传统关系型数据库的主要区别在于：关系型数据库更擅长事务型数据管理，ES 更擅长搜索型数据管理。**MySQL** 这类关系型数据库适合做主库，负责订单、支付、库存、用户资料等需要强一致、强事务的数据。ES 更适合做搜索库，负责商品搜索、日志检索、全文检索、地理位置搜索、聚合分析等场景。在真实项目里，业务数据往往先写入 **MySQL**，再通过消息同步机制（消息队列）写入 ES。用户搜索场景查 ES，用户下单、支付、修改资料时仍然查 **MySQL**。

所以在案例题里，如果题目强调“海量日志检索”“全文搜索”“关键词搜索”“地理位置附近查询”“搜索结果按相关性排序”，就可以考虑 ES。如果题目强调“资金扣减”“订单支付”“库存扣减”“事务回滚”“强一致”，就要考虑传统关系型数据库。其他区别归纳总结如下。案例概念题会考。

对比维度	Elasticsearch (ES)	传统关系型数据库 (以 MySQL 为例)
数据库类型	分布式搜索引擎，基于文档型存储	关系型数据库，基于表结构存储
核心功能	全文搜索、复杂查询、日志分析、实时数据检索	结构化数据存储、事务处理、数据一致性保障
应用场景	全文检索（如电商商品搜索）、日志分析、监控数据查询	订单管理、财务系统、用户信息管理（需强事务场景）
优点	搜索性能高、支持实时搜索、分布式扩容便捷、灵活的全文检索功能	支持事务回滚、数据一致性强、生态成熟、查询语法标准化
缺点	部署运维复杂、查询学习成本高、数据一致性保障弱	实时搜索能力差、大数据量扩展难度大、全文检索功能有限
数据结构	文档（JSON 格式），灵活的半结构化数据存储	二维表结构，严格的结构化数据定义
分布式支持	原生支持分布式集群，横向扩展能力强	需通过分库分表实现分布式，架构设计复杂
事务支持	仅支持简单文档级别的操作，事务能力较弱	支持完整的 ACID 事务，保障数据原子性、一致性

14.4.MySQL 与 ES 数据同步（重点★★★★★）

前面提到，真实项目中，ES 通常不是主库。主库一般还是 MySQL、PostgreSQL 这类关系型数据库，ES 作为搜索库使用。这样就会带来一个问题：MySQL 中的商品数据变了，ES 里的商品索引怎么同步？

比如商品后台修改了价格、库存、标题、上下架状态，MySQL 中的数据已经变化了，但 ES 中的数据如果没有同步，用户搜索时看到的仍然可能是旧价格、旧库存或已经下架的商品。这就是 MySQL 与 ES 的数据一致性问题。常见方案有三种。

同步方案	做法	优点
同步调用	写完 MySQL 后，直接调用搜索服务更新 ES	实现简单

MQ 异步通知	写完 MySQL 后发送消息, 搜索服务监听消息更新 ES	解耦, 主业务性能较好
监听 binlog / Canal	监听 MySQL binlog, 根据数据库变更同步 ES	业务侵入小, 解耦更彻底

比较常见的微服务做法是 MQ 异步通知: 商品服务先写 MySQL, 写成功后发送一条消息, 消息里通常只放商品 ID。搜索服务监听消息后, 根据商品 ID 查询 MySQL 最新数据, 再写入 ES。如果是商品下架或删除操作, 就根据商品 ID 删除 ES 文档, 或者把 ES 文档中的 status 改成下架。如下所示:

商品数据变更 → 写入 MySQL → 发送 MQ 消息 → 搜索服务消费消息 → 查询商品最新数据 → 更新 ES 商品索引

这里要特别强调一致性问题。MySQL 和 ES 同步通常是最终一致, 不是强一致。也就是说, 运营刚修改完商品价格的一瞬间, ES 中可能还没来得及更新, 搜索结果可能短暂显示旧价格。

15. 下篇-MongoDB (重点★★★★★)

MongoDB 中的记录是一个文档, 它是由字段和值对组成的数据结构。MongoDB 文档类似于 JSON 对象。字段的值可以包括其他文档, 数组和文档数组。它的优势是文档 (即对象) 对应于许多编程语言中的内置数据类型。嵌入式文档和数组减少了对连接的需求。近几年开始, 架构特别喜欢考 MongoDB 相关内容。

15.1. 数据类型 (重点★★★★★)

15.1.1. BSON (重点★★★★★)

BSON 是一种类 JSON 的一种二进制形式的存储格式, 简称 BinaryJSON, 它和 JSON 一样,

支持内嵌的文档对象和数组对象,但是 BSON 有 JSON 没有的一些数据类型,如 Date 和 BinData 类型。BSON 和 JSON 非常有可能拿来对比,如下表所示。

对比项	BSON (BinaryJSON)	JSON (JavaScriptObjectNotation)
数据表示	二进制格式	纯文本格式
数据类型支持	支持更多原生数据类型,如日期、整型、浮点型、二进制数据等	支持的数据类型相对有限
存储效率	更紧凑,速度更快,文件体积稍大	相对不紧凑,速度较慢
易读性	需专门浏览器或工具查看	可直接以人类可读格式打开查看
适用场景	对读写速度要求高、数据结构复杂且需要额外数据类型支持的数据库操作	互联网数据传输、开发调试(文本格式便于调试和日志记录)
传输效率	存储紧凑,但压缩优化不如文本数据明显	压缩算法对其优化效果更明显

15.1.2.GeoJSON (重点★★★★★)

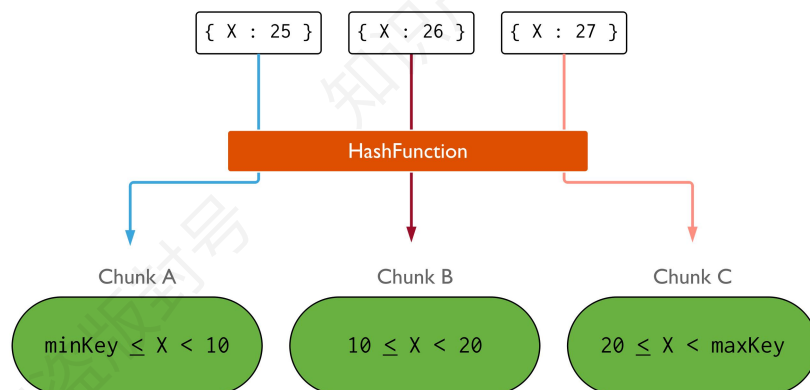
GeoJSON 是一种用于描述地理数据的开放标准格式,它不是 MongoDB 独有的。MongoDB 提供了对 GeoJSON 格式的原生支持,允许您在 MongoDB 数据库中存储和查询地理空间数据。地理空间索引和查询可用于实现诸如位置服务、位置分析等应用场景。

15.2.分片集群 (重点★★★★★)

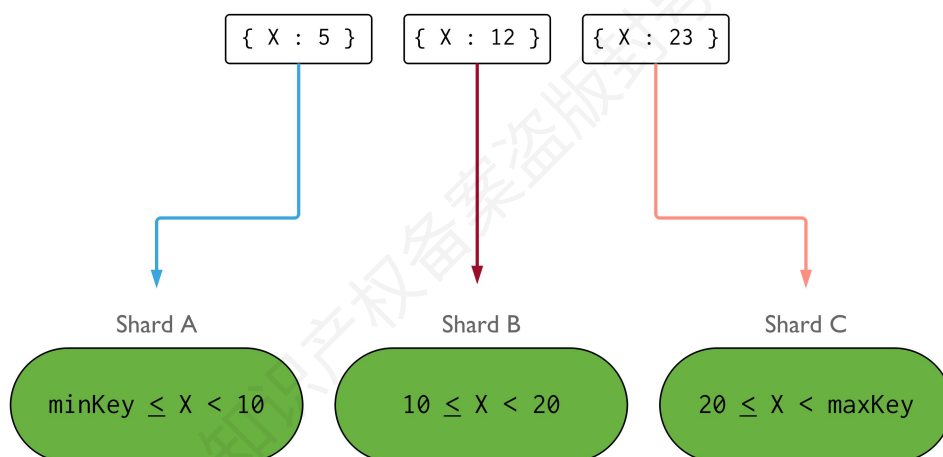
MongoDB 分片集群技术用于解决海量数据的存储问题。比如存储容量受单机限制,即磁盘资源遭遇瓶颈。读写能力受单机限制,可能是 CPU、内存或者网卡等资源遭遇瓶颈,导致读写能力无法扩展。MongoDB 分片集群支持的分片策略有哈希分片和范围分片。具体什么是分片凯恩在前面介绍数据库分片的时候也讨论过。

哈希分片使用哈希索引来在分片集群中对数据进行划分。哈希索引计算某一个字段的哈希

值作为索引值，这个值被用作片键。



基于范围的分片会将数据划分为由片键值确定的连续范围。这允许在连续范围内读取目标文档的高效查询。但是，如果片键选择不佳，则读取和写入性能均可能降低。



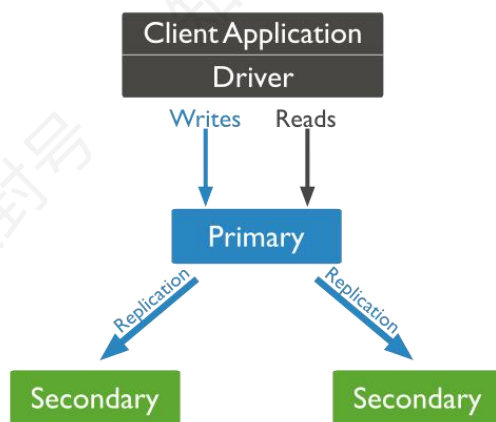
凯恩总结如下表所示。

分片类型	适用场景
哈希分片	随机的数据访问，无法确定特定的数据范围
范围分片	数据按某个属性连续增长的场景，比如按时间先后顺序存储的数据

15.3.高可用（副本复制）（重点★★★★★）

MongoDB 复制集由一组 MongoDB 实例（进程）组成，包含一个 Primary 节点和多个 Secondary 节点，MongoDBDriver（客户端）的所有数据都写入 Primary，Secondary 从 Primary

同步写入的数据，以保持复制集内所有成员存储相同的数据集，提供数据的高可用。下图是一个典型的 MongoDB 复制集，包含一个 Primary 节点和两个 Secondary 节点。



Primary 与 Secondary 之间通过 oplog 来同步数据，Primary 上的写操作完成后，会向特殊的 local.oplog.rs 集合写入一条 oplog，Secondary 不断地从 Primary 获取新的 oplog 并应用。这个有点类似 MySQL 的 binlog 机制。

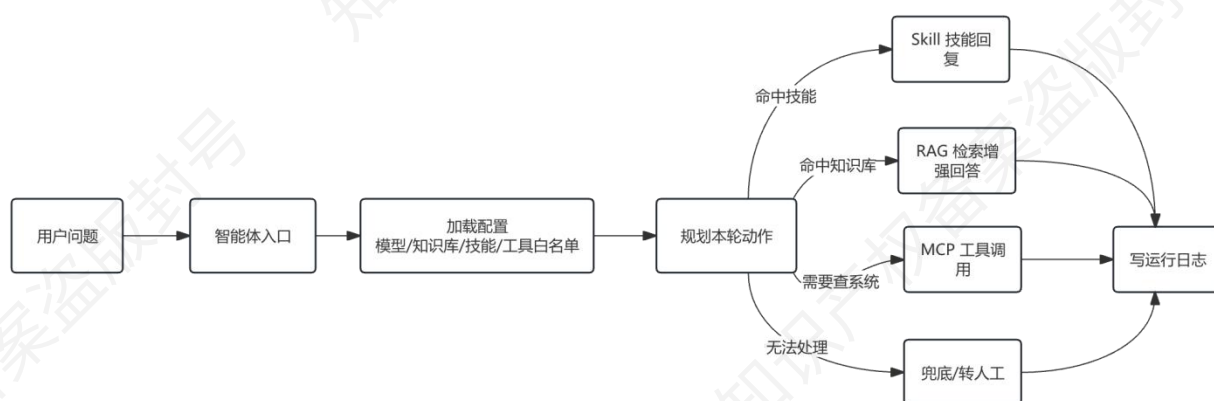
16.下篇-大模型应用（重点★★★★★）

25 年 5 月架构案例真题中已经小小涉及到大模型应用，因此这里凯恩把 RAG、MCP、Agent 作为补充知识点讲一下。

之前我们提到大模型，更多可能说是大模型的问答。现在，我们希望大模型去做一些事情了，但在做事的时候，我们会发现它有一些问题。比如用户问公司制度，模型可能凭常识乱答；用户问“我要退一下”，系统不知道他是要退货、退款，还是退订会员；用户让系统查订单，模型自己不能直接访问订单系统。再比如我们的软考问答场景，学生问“什么叫做 ABSD”，模型如果只靠通用知识回答是完全答不好的。

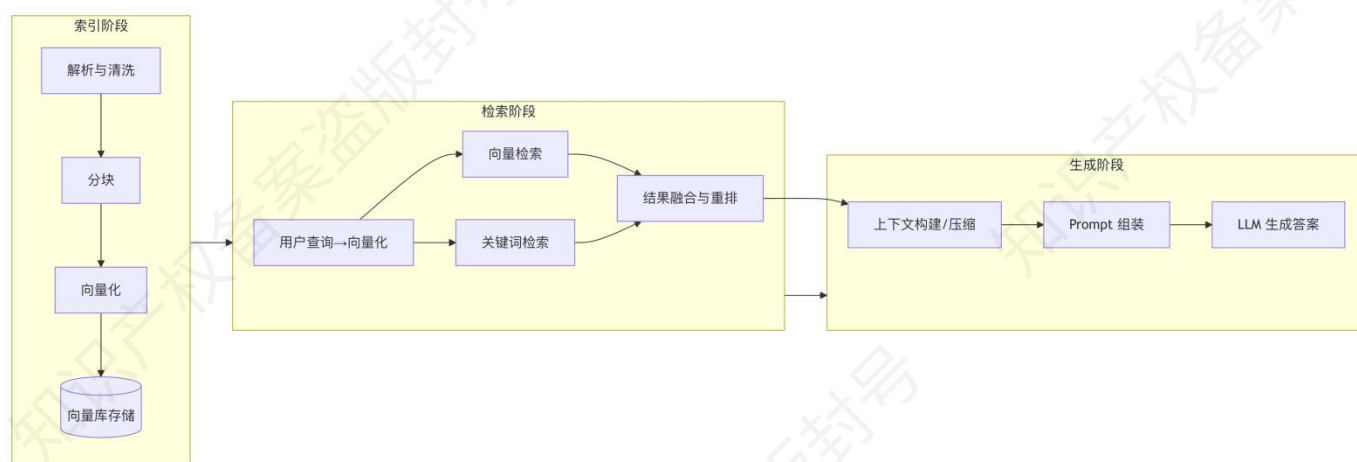
所以大模型应用开发真正要解决的，是下面这些问题：模型怎么拿到企业自己的知识？模型怎么判断用户到底想干什么？模型怎么安全地调用外部系统？模型回答不了怎么办？模型答错了以后，研发怎么复盘问题出在哪里，类似可观测性？

假如不出意外，当你看到这份资料的时候，我们芝士架构的智能客服已经上线了。如果用这个客服智能体举例，整体链路大概是这样：



16.1.检索增强生成（RAG）（重点★★★★★）

在大模型应用开发中，检索增强生成（RAG）RAG 几乎是最常见的知识增强方案。它的核心思想是“检索+生成”，既发挥大语言模型的语言生成能力，又引入外部知识来弥补模型本身的记忆局限。一个标准的 RAG 流程，可以分为索引阶段、检索阶段、生成阶段三大部分，也可以细分为五个核心环节：数据准备→向量化→索引存储→检索增强→生成答案。



RAG 的效果，很大程度上取决于文档怎么清洗、怎么切块、用什么 Embedding、怎么检索、要不要重排、Prompt 怎么约束，以及没有命中资料时怎么兜底。

16.1.1.数据准备（重点★★★★★）

数据准备阶段的质量，直接决定后续检索与回答的准确度。一般来说需要将原始文档转化为结构化的、可处理的文本。常见步骤包括：

- 1.清洗文档：**去掉多余的格式信息，如 HTML 标签、PDF 页眉页脚等。
- 2.分块处理：**长文档不能直接输入模型，所以要切成较小的文本块，也叫 **chunk**。每个 **chunk** 最好语义相对完整，不能切得太碎，也不能太大。
- 3.记录来源：**每个 **chunk** 要能回到原始文档、原始章节或原始 FAQ。这样用户问“依据是什么”时，系统能说清楚答案来自哪里。其实这个主要还是为了调试方便，便于观测不同的分片模式下它的效果。

16.1.2.分块策略（重点★★★★★）

分块是 RAG 中非常容易被低估的一步。块切得不好，后面 Embedding 再好、模型再强，也会出问题。常见分块策略有三种。

第一种是固定长度切分。比如每 500 个字或每 800 个 token 切一块，并设置一定重叠。它的好处是简单稳定，适合普通文本。但缺点是可能把一个完整语义切断。假如你做开发，你就知道很多参数里面有一个叫 `overlap`。如果你做过 RAG 开发，会经常看到一个参数叫 `overlap`，也就是“重叠长度”。它的作用，就是让相邻两个切片之间保留一部分重复内容。

比如原来按 500 个字切，如果没有 `overlap`，切法大概是：

第 1 块：1~500 字，第 2 块：501~1000 字，第 3 块：1001~1500 字，如果设置 `overlap = 100`，切法就会变成：，第 1 块：1~500 字，第 2 块：401~900 字，第 3 块：801~1300 字

你会发现，第 1 块和第 2 块之间，有 100 个字是重复的。这样做可以防止关键信息刚好卡在切片边界上。

比如一段话前面说“退款需要满足三个条件”，后面才列出具体条件。如果刚好从中间切开，前一个切片只有引子，后一个切片只有条件，模型检索时可能拿到的信息都不完整。设置 `overlap` 后，边界附近的内容会被带到下一个切片里，语义就不容易断。

当然，`overlap` 也不是越大越好。重叠越多，切片数量越多，向量库里的数据也越多，检索成本和存储成本都会上升，还可能导致检索结果里出现很多重复内容。一般可以把 `overlap` 设置为切片长度的 10% 到 20%。比如 500 字切一块，可以重叠 50 到 100 字；800 token 切一块，可以重叠 80 到 160 token。

第二种是结构化切分。按照标题层级切分，比如一级标题、二级标题、三级标题。它适合

Markdown、HTML、教材章节、制度文档等结构比较清晰的内容。比如“16.1 RAG”“16.1.1 数据准备”“16.1.2 向量化”，天然就可以作为切分依据。

第三种是 FAQ 切分。一条问答就是一个知识单元。比如“如何申请退费？”和对应答案作为一个整体入库。这种方式特别适合客服知识库。再往前说，其实以前有人用语义槽的方法来做。我们在之前做真题的时候也会看到，有一年呢，考察了“医药知识问答系统”，里面用到了所谓的模板匹配。实际上，现在已经不用这种技术了，都是用大模型去做意图匹配，或者用 embedding 去做语义关联，找到对应的这个问答对。它的泛化性能会更好。

16.1.3.向量化 Embedding（重点★★★★★）

完成分块后，每个文本块需要通过嵌入模型（Embedding Model）转换为向量。常见的模型有 BERT、Sentence-BERT、BGE 等。向量本质上是一个高维坐标点，用来表示文本的语义信息。比如“苹果手机”和“iPhone”会被映射到相近的向量空间位置，从而实现语义层面的检索。

完成分块后，每个文本块需要通过嵌入模型，也就是 Embedding Model，转换成向量。向量可以理解成文本在高维空间中的一个坐标点。文本语义越接近，它们在向量空间中的距离通常也越近。

比如下面几句话：“怎么申请退款？”“退费流程是什么？”“我想把钱退回来。”这几句话字面不完全一样，但语义很接近。传统关键词搜索可能要依赖“退款”“退费”这些关键词是否命中，而向量检索可以从语义层面发现它们接近。

这就是 Embedding 的价值：它把文本变成可以计算相似度的向量。常见 Embedding 模型包括 BERT、Sentence-BERT、BGE 等。在具体项目里，系统会把每个 chunk 的 content 输入 Embedding 模型，得到一个向量，然后和 chunkId、模型名称、向量维度一起保存。这里有两个问题要注意。

第一个是维度必须一致。比如某个 Embedding 模型输出 1024 维向量，那么向量数据库字段也要按 1024 维设计。如果后面换了一个 768 维或 1536 维模型，旧索引就不能直接混用。

第二个是文档更新后要重建索引。用户看到的文档是新的，但向量库里还是旧内容，这种情况在生产中很常见。所以知识库更新后，要有重新切块、重新向量化、重新写入索引的机制。

可以用下面这个简单过程理解：

原始文本：“用户申请退费时，需要提交订单号，并由客服核验支付状态。”

Embedding 后：[0.013, -0.245, 0.781, ..., 0.092]

写入向量库：

chunkId = 10001

content = “用户申请退费时，需要提交订单号……”

embedding = [0.013, -0.245, 0.781, ..., 0.092]

modelName = "bge-large"

dimension = 1024

嵌入模型的训练，可以理解成让模型反复学习一个目标：意思相近的内容，转成向量以后要更近；意思不相关的内容，转成向量以后要更远。比如在电商场景里，用户问“这件衣服尺码不合适，怎么退货？”，相关文档可能是“商品签收后 7 天内，在不影响二次销售的情况下，可以申请退货退款”。虽然这两句话字面不一样，但都在表达“退换货规则”，所以训练时会让它们的向量距离变近。相反，如果另一段文档是“本款羽绒服采用 90% 白鸭绒填充”，它讲的是商品材质，不是退货规则，模型就要把它和用户问题的向量距离拉远。

训练数据通常会整理成“问题、相关内容、不相关内容”这样的形式。比如“尺码买小了能不能退”是问题，“七天无理由退货规则”是相关内容，“商品面料说明”是不相关内容。模型每训练一轮，都会计算这些文本向量之间的相似度，然后根据结果调整自己的参数。

经过大量类似样本训练后，模型就逐渐学会了：用户问“衣服不合身怎么办”“尺码买错了

能退吗”“穿了一次还能退吗”，虽然说法不同，但都应该优先匹配到“退换货规则”，而不是匹配到商品详情、物流说明或促销活动。

16.1.4.索引存储（重点★★★★★）

向量化后的文本会被存入 向量数据库，如 Milvus、Faiss、Elasticsearch、Pinecone 等。这类数据库的优势是能够在海量向量中快速计算相似度，从而找到与查询语义最接近的内容。通常，数据库还会保存原始文本，以便在检索到相关向量时能够回溯真实内容。

在芝士架构 AI 客服里，我把使用 `pgvector`。它的好处是基于 PostgreSQL，我可以把知识表、`chunk` 表、向量表、日志表放在同一个数据库体系里，调试和关联查询比较方便。一个简化版表结构可以这样理解：

```
CREATE TABLE knowledge_chunks (  
    id BIGINT PRIMARY KEY,  
    knowledge_id BIGINT NOT NULL,  
    document_id BIGINT,  
    chunk_no INT,  
    section_path TEXT,  
    content TEXT,  
    metadata JSONB  
);  
  
CREATE TABLE knowledge_chunk_embeddings (  
    id BIGINT PRIMARY KEY,  
    chunk_id BIGINT NOT NULL,  
    model_name VARCHAR(100),  
    dimension INT,  
    embedding VECTOR(1024)  
);
```

这里的 `VECTOR(1024)` 只是举例，真实项目要和 `Embedding` 模型输出维度保持一致。

索引存储阶段，至少要保存三类数据。

第一类是原始知识内容。包括文档标题、FAQ 问题、答案、章节、状态等。

第二类是分块数据。包括 `chunkNo`、`content`、`tokenCount`、`sectionPath`、`metadata` 等。

第三类是向量数据。包括 `chunkId`、`embedding`、`modelName`、`dimension` 等。

这样检索时，系统可以先通过向量相似度找到 `chunk`，再回溯到原始文档、FAQ 和章节来源。

16.1.5.检索增强（重点★★★★★）

当用户提出问题后，系统会把用户问题也转成向量，然后在向量库里查找相似文本块。基本流程如下：

1. 加载本次要检索的知识库。
2. 用 `Embedding` 模型把用户问题转成向量。
3. 在向量库中查找最相似的 `chunk`。
4. 按 `topK` 取候选结果。
5. 按 `scoreThreshold` 过滤低质量结果。
6. 必要时调用 `Reranker` 模型重排序。
7. 把最终结果送入 `Prompt`。

```
SELECT
  c.id AS chunk_id,
  c.content,
  1 - (e.embedding <=> :query_embedding) AS score
FROM knowledge_chunk_embeddings e
JOIN knowledge_chunks c ON c.id = e.chunk_id
WHERE c.knowledge_id = :knowledge_id
ORDER BY e.embedding <=> :query_embedding
LIMIT 20;
```

这里 `<=>` 可以理解成 PG 里的向量距离计算。距离越小，说明越相似。为了更直观，也可以转成 `score`，`score` 越高表示越相关。但是只做向量检索还不够。实践中经常会配合关键词检索、混合检索和 `Rerank`。

Rerank 是指，一般做法是向量检索先粗筛出 20 条候选资料，**Rerank** 再判断这 20 条里哪几条最能回答当前问题。这样可以减少“看起来相似但其实没用”的资料进入 **Prompt**。

有的同学不理解为什么我已经用 **Embedding** 找去相似语义了，还要再去 **rerank**。**Rerank** 的作用可以理解成“二次筛选”。可以这样理解：**Embedding** 是粗筛，**Rerank** 是精排。**Embedding** 检索的优势是快。它会先把用户问题和知识库里的每个文本块都转换成向量，然后通过向量相似度，从海量文档里快速找出一批“看起来相关”的内容。比如知识库里有 10 万个 **chunk**，系统不可能让大模型逐条阅读判断，所以先用 **Embedding** 快速召回 **Top20** 或 **Top50**。

但问题是，语义相似不等于一定能回答问题。比如用户问“iPhone 退货需要满足什么条件”，**Embedding** 可能召回“iPhone 商品介绍”“手机退货流程”“AppleCare 服务说明”“订单售后规则”等内容。这些内容看起来都和 iPhone、退货、售后有关，但真正最能回答问题的，可能只有“手机退货流程”和“订单售后规则”。如果直接把前几条塞给大模型，可能会带入无关信息，影响回答质量。**Rerank** 的作用就是在 **Embedding** 召回结果的基础上做二次筛选。它不再只是简单比较两个向量距离，而是把“用户问题”和“候选文档片段”放在一起重新判断。所以 **Rerank** 通常比单纯向量相似度更精细，能把真正有用的内容排到前面。

但是 **Rerank** 通常更慢、成本更高，所以不能直接拿它去扫完整个知识库。所以比较合理的做法是：先用 **Embedding** 从全库快速召回一批候选内容，再用 **Rerank** 对这批候选内容重新排序，最后只把最相关的几段资料放进 **Prompt** 里让大模型回答。

混合检索是指，比如用户问：“MCP 是不是 API 网关？”关键词检索可能会找到“API 网关”相关资料，向量检索可能会找到“MCP 工具调用”“外部系统接入”相关资料。混合检索可以把两类结果合并，再通过 **Rerank** 判断哪些资料最适合回答这个问题。

16.1.6.生成答案（重点★★★★★）

最后，系统会将 用户问题 + 检索到的相关文档 一并输入大语言模型。这里的关键是 Prompt 设计。下面就是一个例子。

你是严谨的助手。请仅依据“相关文档”作答，不知道就说不知道，不要编造。

【问题】{query}

【相关文档（带编号）】

{contexts} # 例如：[1] 段落 A ... [2] 段落 B ...

【要求】

1) 先给结论，再给依据；2) 在依据处用[编号]标注来源；3) 若文档冲突，指出冲突并分别给出可能结论；4) 如问题与文档无关，明确说明无法回答并建议更精确的提问。

这里可以设置两种模式。

第一种是 **strict** 模式。只能基于参考资料回答，资料不足就说无法确认。企业客服、制度问答、售后政策、考试教材问答，都适合 **strict** 模式。

第二种是 **assist** 模式。优先基于参考资料回答，资料不足时可以给辅助建议，但要说明哪些内容不是知识库确认信息。比如学习助手、写作助手、头脑风暴类场景，可以适当使用 **assist** 模式。

16.2.模型上下文协议协议（MCP）（重点★★★★★）

RAG 解决的是“模型怎么读知识”，MCP 解决的是“模型怎么用工具”。

比如一个客服智能体，只靠知识库可以回答“退费流程是什么”。但如果用户问：“帮我查一下订单 12345 的退款进度”，模型就必须访问订单系统。再比如用户问：“现在服务器时间是多少？”“帮我创建一个工单”“读取一下本地文件里的配置”，这些都需要模型调用外部工具。

过去每接一个工具，都要单独适配。MCP 的出现，就像是为 AI 世界引入了一个类似 USB-C 的标准化接口，让模型能够即插即用访问各种数据源和工具。

MCP（模型上下文协议，Model Context Protocol）是 Anthropic 于 2024 年 11 月提出的一种新标准，旨在为大语言模型（LLM）与外部世界之间提供一个统一的“通用接口”。在过去，AI 模型往往只能依赖于预训练数据或用户手动上传的资料，这使得它们对新数据的处理效率低，扩展性差。

从机制上看，MCP 为 LLM 提供了一种“即插即用”的方式：任何符合 MCP 标准的外部资源——无论是数据库、API、文档库还是本地文件系统——都可以直接接入，模型无需知道底层的实现细节，只需通过 MCP 就能理解并利用这些资源。这极大地降低了开发难度和维护成本，也让模型具备了实时获取数据、调用工具和执行任务的能力。换句话说，MCP 打破了 AI 对静态知识的依赖，让模型能够像人类一样，在需要时去查询最新信息、调用外部服务、执行操作。

16.2.1. 解决什么问题？（重点★★★★★）

MCP 主要解决三类问题。

第一，工具接入标准化。不同系统、不同 API、不同数据源，都可以包装成 MCP 工具。模型看到的是统一的工具名称、描述、参数 schema 和返回结构。

第二，降低重复开发成本。如果没有标准协议，每个模型应用都要重新适配一遍工具。有了 MCP，一个工具服务可以被多个模型应用复用。

第三，方便治理和审计。工具调用不是普通问答，它可能真的修改数据、发送消息、查询隐私信息。所以 MCP 工具必须配合权限、白名单、参数校验、超时和日志。

这里要特别注意：MCP 只是标准化工具接入方式，不代表模型可以随便调用工具。生产

环境里一定要做权限控制。

16.2.2.MCP 工具的组成（重点★★★★★）

一个 MCP 工具通常至少包括下面几个部分

组成	含义	举例
toolName	工具名称	server_time
description	工具描述	获取服务器当前时间
inputSchema	参数结构	timezone 可选
handler	工具执行逻辑	调用系统时间函数
output	返回结果	content 和 structuredContent

比如一个获取服务器时间的工具，可以理解成这样：

```
{
  "name": "server_time",
  "description": "获取服务端当前时间",
  "inputSchema": {
    "type": "object",
    "properties": {
      "timezone": {
        "type": "string",
        "description": "时区, 例如 Asia/Shanghai"
      }
    }
  }
}
```

这里，

name 就像函数名，模型调用时要用它。

description 是给模型看的说明，模型靠它判断“什么时候该用这个工具”。

inputSchema 是参数约束，告诉模型参数有哪些、是什么类型、是否必填。

handler 是后端真正执行的代码，比如调用系统时间函数、调用数据库、调用第三方接口。

这个一般不直接暴露给模型，而是写在 **MCP Server** 里面。

执行后可以返回：

```
{
  "content": "当前服务端时间是 2026-06-20 15:30:00",
  "structuredContent": {
    "timestamp": 1781940600000,
    "timezone": "Asia/Shanghai",
    "datetime": "2026-06-20 15:30:00"
  }
}
```

这里 `content` 适合给模型或用户阅读，`structuredContent` 适合系统进一步处理。比如调试台展示、日志记录、后续工具继续使用，都更适合用结构化结果。

16.3.智能体（AGENT）（次重点★★★★☆☆）

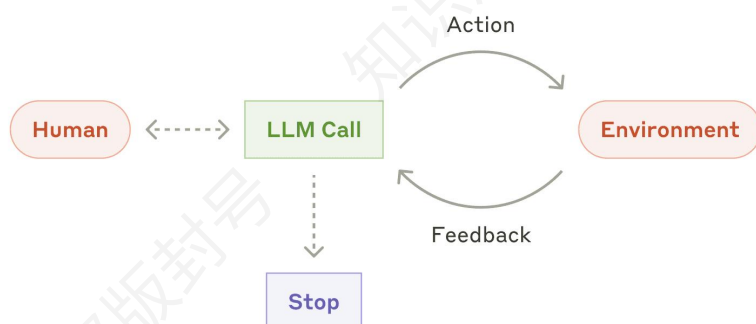
16.3.1.智能体简介（次重点★★★★☆☆）

智能体可以被视为一种以目标为导向的智能系统：它能够感知环境，基于观测进行推理与规划，并通过行动（调用工具或接口）对环境施加影响，从而让任务朝目标收敛。智能体的本质是一套“决策—行动—反馈”的闭环系统。

实际上为了说清楚智能体，还需要明确聊天机器人（Chatbot）、工作流（Workflow）的概念。聊天机器人的中心能力是对话生成与交互体验，它可以很聪明、很拟人，但不一定要去调用外部系统、更不需要对结果负责。

工作流强调步骤可复现、路径可控，适合稳定流程与合规场景，但它的适应性弱：当输入、约束或目标发生变化时，工作流往往只能通过“改流程”来应对。

智能体介于两者之上：它以目标为牵引，在循环决策中选择工具、执行动作、读取反馈并修正计划，因此具备自适应与迭代推进的能力。如下图所示。



这里不要把 Agent 神化。工程上更准确的理解是：**Agent = 大模型 + 目标 + 规划 + 工具 + 记忆/状态 + 反馈 + 兜底 + 日志**。为了说清楚 Agent，我们还要和 Chatbot、Workflow 做一下区分。

类型	核心能力	特点	适合场景
Chatbot 聊天机器人	对话生成	重点是会说话	闲聊、简单问答、内容生成
Workflow 工作流	固定流程编排	路径稳定，可控性强	审批、报销、固定业务流程
Agent 智能体	目标驱动决策	会判断、会调用工具、会根据反馈调整	智能客服、自动排障、研究助手

16.3.2.智能体架构（次重点★★★☆☆）

智能体架构（Agent Architecture）指的是：为了让一个 LLM 系统能够像“会做事的人”一样完成目标任务，我们在模型外部搭建的一整套结构与组件组织方式。它回答的不是“模型会不会说”，而是“系统如何感知信息、如何决策、如何调用工具、如何记忆与自我修正、如何停机交付”。换句话说，架构决定了智能体的“骨架”和“工作方式”，模型只是其中的“大脑”。下面介绍的 ReAct 和规划-执行模式就是两种常用的智能体架构。

16.3.3.ReAct 模式（次重点★★★☆☆）

ReAct 模式（Reason + Act）是一种典型的智能体（Agent）推理与执行框架，其核心思想是将“推理（Reasoning）”与“行动（Acting）”交替编排，使系统在完成任务的过程中不断从外部环境获取反馈，并基于反馈修正后续决策。与一次性生成答案的对话式模型不同，ReAct 强调“闭环”：模型不是凭空把结论写完，而是通过工具调用、检索、计算或系统操作获得“地面事实”，再据此逐步推进任务直至完成。

ReAct 的劣势主要来自“回合数与成本控制”。由于每做一步动作通常都要回到模型进行下一步决策，它容易产生较高的调用次数与时延，并可能出现循环失控——不停检索、不停尝试，迟迟不给最终交付。此外，ReAct 对工具质量与观测可靠性依赖较强：工具返回不稳定、观测噪声大或工具选择不当时，会导致智能体在错误反馈上反复迭代，甚至形成“越走越偏”的路径。因此工程上往往需要预算约束（最大回合/最大工具调用/超时降级）来配套治理。

ReAct 适用的典型场景是“信息不完整且需要边做边看”的任务：例如从多个来源检索资料并综合、根据日志逐步定位故障原因、对代码或配置进行试探性修改并观察运行结果、需要计算与验证的问答（先算再答、先验再结论）。不太适合的场景则是流程高度固定、要求强一致性与低成本的任务；这类任务用预编排工作流往往更稳更省。

16.3.4.规划-执行模式（次重点★★★☆☆）

规划-执行模式（Plan-and-Execute）是一种面向复杂任务的智能体组织方式，它把任务处理过程明确分为“规划阶段”和“执行阶段”两个部分。在规划阶段，系统先基于目标生成一份可落地的计划，计划通常包含步骤顺序、依赖关系、所需资源与工具、以及每一步的验收标准。随后进入执行阶段，智能体不再一口气完成全部任务，而是严格按照计划逐步推进：每完成一步就产出阶段性结果，并将结果作为下一步的输入，从而把复杂任务拆成一串可管理、可检

查的子任务链条。

其劣势集中在“计划与现实的偏差风险”。当问题空间不确定、信息缺失或环境变化快时，规划阶段容易生成理想化步骤，执行中频繁遇到依赖不可用、资料不足或约束改变，导致偏航与返工。如果系统过于“死守计划”，反而会拖慢任务。因此规划-执行模式通常必须配套“动态重规划”机制：当观测结果不满足计划假设时，允许局部调整后续步骤，必要时回到规划阶段重做计划，并保留已验证有效的阶段成果，避免从头推倒重来。

规划-执行模式更适用于“可拆解、可验收、目标明确”的多步骤任务，例如：撰写一份研究报告——先列提纲与资料清单，再逐段填充并校验引用。不太适合的场景是强探索式问题，比如目标模糊、路径不确定、需要频繁试错的场景，因为此时计划的稳定性很差，往往会退化成“计划—推翻—重来”，更适合直接采用 ReAct 或在执行阶段嵌入局部 ReAct 来应对不确定性。

16.3.5. 客服智能体架构探索（次重点★★★★☆☆）

前面提到，智能体架构指的是：为了让一个 LLM 系统像“会做事的人”一样完成任务，我们在模型外部搭建的一整套结构与组件组织方式。它回答的是：系统如何加载配置？如何判断用户意图？如何选择知识库、技能或工具？如何执行动作？如何失败兜底？如何记录过程？如何复盘等等。

从代码分层角度来看，芝士架构客服智能体的 Agent 架构，可以分成四层。

第一层是配置层。配置层定义一个 Agent 在运行时能使用的全部边界条件，包括 Agent 名称、描述、状态、模型配置、系统提示词、欢迎语、知识库、技能、MCP 工具白名单、Graph 工具白名单、转人工策略、兜底模式、兜底话术和排序等。

第二层是规划层。规划层负责在每一轮用户输入后决定“走哪条路径”。当前我设计的优

优先级是：显式 Skill 错误处理、Graph 工具、Direct/MCP 工具、Skill、知识库 RAG、兜底、普通模型回复。这里 Skill 路由不是直接把所有 Skill 都塞给主模型，而是先加载 Agent 绑定的候选 Skill，我是先用 code/name 做轻量关键词匹配，不命中再调用 LLM 路由器，让模型从候选 Skill 中输出一个 skillCode 或 NONE。

第三层是执行层。执行层根据规划结果执行具体动作。当前已经真正接入的是 RAG 问答和模型回复。Skill 命中后，并不是执行一个独立 Skill handler，而是把命中的 Skill 信息注入最终 system prompt，包括 Skill 名称、code、说明和指令，再由模型生成回复。

第四层是观测层。记录运行日志、技能路由结果、RAG 检索结果、工具调用结果、耗时、错误信息等，方便后续调试。

16.3.6.Skill（重点★★★★★）

16.3.6.1.为什么要有 SKILL（重点★★★★★）

前面我们讲 MCP 工具的时候，说工具更像是 Agent 可以调用的一个“函数”。比如查订单、查退款状态、获取服务器时间、搜索网页。但是在真实业务里，很多任务不是调用一个工具就结束了。用户说“我要退款”，系统要先判断用户是不是确实要退款，再收集订单号、退款原因，再查询订单状态，再判断是否满足退费条件，最后还要组织一段合适的话回复用户。

在智能客服里，用户的问题大致可以分成两类。

第一类是普通问答，比如“会员有什么权益”“软考什么时候出成绩”。这种问题通常只需要从知识库里检索答案，然后组织语言回复。

第二类是业务办理，比如“我要退款”“帮我查一下订单”“怎么改手机号”“我要转人工”“帮我开发票”。这类问题背后是一个流程。

如果系统只有几个业务，我们可以把所有规则都写进一个大 Prompt，但业务一多就会出

问题。

比如系统里有退费、订单查询、账号修改、发票申请、物流查询、投诉建议、转人工等几十个业务，如果全部塞进一个超级 **Prompt**，模型会越来越容易混淆。举个例子，用户明明只是问“退费多久到账”，模型可能直接进入退款申请流程。所以工程上一般的做法是：不要让一个超级 **Prompt** 管所有事，而是把不同业务拆成一个个 **Skill**。

用户问题

↓

Skill 路由器判断属于哪个业务技能

↓

进入对应 **Skill**

↓

按照该 **Skill** 调用工具

↓

生成业务回复

这样以后，每个 **Skill** 的边界是清楚的。退费 **Skill** 只管退费，订单查询 **Skill** 只管订单查询，账号资料 **Skill** 只管资料修改，转人工 **Skill** 只管转人工。**Skill** 可以理解成 **Agent** 中的一个可配置业务能力。一个 **Skill** 通常包括：

字段	含义	举例
code	技能编码	refund_apply
name	技能名称	退费申请
description	技能职责边界	处理用户退费咨询和申请
instruction	命中后给模型的指令	按退费流程引导用户
examples	典型问法	我要退款、怎么退费
slots	需要抽取的信息	订单号、退款原因
toolWhitelist	允许调用的工具	order_query、refund_status

一个 **Skill** 可以简单配置成这样：

```
{
  "code": "refund_apply",
  "name": "退费申请",
  "description": "处理用户关于退费条件、退费流程、退费进度的咨询",
  "examples": [
    "我要退款",
    "怎么申请退费",
    "我的订单能不能退",
    "退费多久到账"
  ],
  "requiredSlots": [
    "orderId",
    "refundReason"
  ],
  "toolWhitelist": [
    "order_query",
    "refund_status_query"
  ]
}
```

这里的 **code** 是技能编码，相当于系统内部识别这个 **Skill** 的唯一标识。

description 用来描述这个 **Skill** 的职责边界。模型可以根据它判断用户的问题是否应该进入这个 **Skill**。

instruction 是命中 **Skill** 后的业务指令。也就是说，一旦用户进入退费 **Skill**，模型就不能再像闲聊一样随意回答，而要按照退费流程引导用户。

examples 是典型问法，用来帮助路由模型判断用户的问题和这个 **Skill** 是否匹配。

requiredSlots 是这个业务必须收集的信息。比如退费申请通常至少需要订单号和退款原因。

toolWhitelist 是工具白名单。它限制这个 **Skill** 只能调用哪些工具。比如退费 **Skill** 可以调用订单查询工具和退款状态查询工具，但不能调用修改手机号、删除账号之类的工具。

这个白名单非常重要，因为 **Agent** 不是只要“聪明”就行，还要可控。不同 **Skill** 能调用的工具应该是不一样的，查询类 **Skill** 不应该随便调用写入类工具，普通咨询 **Skill** 也不应该越权调用敏感操作工具。

16.3.6.2.渐进式披露（重点★★★★★）

Skill 机制里有一个很重要的概念，叫渐进式披露，也可以叫渐进式加载。其实很好理解：不要一开始就把所有 Skill 的完整说明都塞给模型，而是先让模型看到 Skill 目录；等模型判断某个 Skill 真的有用时，再加载这个 Skill 的详细说明；如果还需要模板、脚本、参考资料，再继续读取对应资源。

比如一个客服系统里有退费申请、订单查询、发票申请、修改手机号、转人工、投诉建议等很多 Skill。如果每个 Skill 都有几千字说明，包括适用场景、不适用场景、处理流程、槽位要求、工具白名单和回复模板，那么系统不可能每轮对话都把这些内容全部塞给大模型。这样不仅浪费上下文、增加 Token 成本，还会让模型受到干扰。用户只是说“我要退款”，模型没有必要同时阅读发票、物流、手机号修改、投诉建议的完整流程。规则越多，模型越容易混淆，最后反而不知道该按哪个流程走。

渐进式披露的做法，就是把 Skill 信息分阶段给模型。第一阶段，只给模型看 Skill 的名称和简介，相当于一本书的目录。比如系统提示里只写：refund_apply 用来处理退款、退费流程、退款进度；order_query 用来处理订单状态查询；invoice_apply 用来处理发票申请；human_transfer 用来处理转人工。模型先知道“系统里有哪些技能，每个技能大概能干什么”。当用户说“我要退款，订单号是 20260620001”时，模型看到目录后，就能判断当前更像是 refund_apply。第二阶段，系统才去读取 skills/refund_apply/SKILL.md，也就是退费 Skill 的完整说明。这里面才会写清楚：什么情况适用，什么情况不适用，处理流程是什么，需要哪些槽位，可以调用哪些工具，回复时有哪些注意事项。第三阶段，如果这个 Skill 还带有模板、脚本、政策文件等资源，也不是一开始全部加载，而是任务推进到需要时再读取。比如 SKILL.md 里写着：退款成功时参考 templates/refund_success_reply.md，退款失败时参考 templates/refund_reject_reply.md，需要判断规则时读取 docs/refund_policy.md。这时候模型才

继续打开这些附件。

17.下篇-锁机制（重点★★★★★）

在高并发系统中，锁是保证共享资源安全访问的重要手段。只要多个线程、多个进程，甚至多个服务实例同时修改同一份数据，就可能出现数据不一致问题。常见场景包括库存扣减、余额扣减、优惠券核销、订单状态变更、秒杀抢购等。锁要解决的问题就是：同一时间，只允许一个执行单元修改关键资源，避免并发修改造成错误结果。这一章作为重点已经多次在案例论文中考察到，大家要重视起来。

从 Java 应用来看，在单体应用中，可以使用 Java 自带的同步机制，例如 `synchronized`、`ReentrantLock`、CAS（Compare And Swap）等方式控制并发。但当系统从单体架构演进到分布式微服务架构后，应用会部署在多个 JVM、多个进程、多个服务实例中，单机锁就无法控制跨进程、跨服务的并发访问。这时就需要数据库锁、Redis 分布式锁、Zookeeper 分布式锁等机制来实现跨节点的互斥控制。

17.1.为什么要使用锁（重点★★★★★）

使用锁的根本原因，是为了防止并发操作导致共享资源出现错误结果。

例如库存扣减场景中，某商品库存只有 100 件。此时有两个请求同时进来，每个请求都要扣减 100 件。如果两个请求都先读到库存为 100，然后分别执行扣减，就可能出现库存被扣成负数的问题，这就是典型的超卖。

余额扣减场景也是一样。账户余额只有 100 元，如果两个扣款请求同时读取余额并判断“余额充足”，就可能都扣款成功，最终造成余额异常。所以，高并发下使用锁，本质上是为了保证关键操作的原子性和互斥性。尤其是“先判断、再修改”这一类操作，最容易出问题。因为判断和修改之间存在时间窗口，其他线程可能已经把数据改掉了。

17.2.常见锁类型（重点★★★★★）

高并发系统中常见的锁（以 Java 应用为例），可以大致分为 Java 本地锁、数据库锁、乐观锁和分布式锁几类。

锁类型	典型实现	适用范围	特点
Java 重量级锁	synchronized	单 JVM 内部	使用简单，但无法解决分布式部署问题
Java 轻量级锁	CAS、Atomic 类	单 JVM 内部	无阻塞，性能较好，但适合较轻量的并发控制
数据库行锁	select ... for update	跨进程、跨服务	依赖数据库事务，能解决分布式并发，但容易阻塞
数据库乐观锁	version 版本号、条件更新	跨进程、跨服务	不显式加锁，通过更新条件判断是否冲突
Redis 分布式锁	SET key value NX EX seconds	分布式系统	性能高，但要处理超时、误删、续期等问题
Zookeeper 分布式锁	临时节点、顺序节点	分布式系统	一致性较强，但实现和运维成本较高
分布式队列	MQ、FIFO 队列	串行化处理任务	通过排队降低并发冲突，适合异步削峰

锁不是越多越好，也不是越重越安全。锁的范围越大，性能通常越差。所以在实际系统中，设计锁机制时要尽量缩小锁的粒度。

17.3.乐观锁与悲观锁（重点★★★★★）

锁机制中最常见的分类是乐观锁和悲观锁。

17.3.1.悲观锁（重点★★★★★）

悲观锁的思想是：总是假设数据会被别人修改，所以在读取或修改共享数据之前，先把它

锁住。常见的悲观锁包括：Java 中的 `synchronized`；Java 中的 `ReentrantLock`；数据库行锁，如 `select ... for update`；一些分布式锁实现，如 Redis 锁、Zookeeper 锁。

悲观锁的优点是理解简单，互斥效果明确。缺点是容易导致线程阻塞、性能下降。如果锁持有时间过长，还可能造成连接池耗尽、请求堆积甚至死锁。

17.3.2.乐观锁（重点★★★★★）

乐观锁的思想是：先假设别人不会修改数据，不提前加锁，只在更新时判断数据是否已经被别人改过。常见的乐观锁包括：条件更新；数据库 `version` 版本号机制；基于条件的 SQL 更新；Java 中的 `CAS`；`AtomicInteger`、`AtomicLong` 等原子类。

例如库存扣减时，可以不先加锁，我们用条件更新机制，直接执行：

```
update product
set stock = stock - 1
where id = 1 and stock > 0;
```

它在数据库内部是一条完整的更新语句。数据库执行时，会对命中的行加锁，然后判断 `where stock > 0`，再执行 `stock = stock - 1`。这个过程对同一行来说，不会被另一个事务插进来改一半。类似于

线程 A 先拿到这行的锁，看到 `stock = 1`，满足 `stock > 0`

执行 `stock = stock - 1`

库存变成 0

提交/语句结束

线程 B 再拿到这行的锁

重新检查 `stock > 0`

发现 `stock = 0`，不满足条件

影响行数 = 0

扣减失败

所以不会出现两个线程都基于 `stock = 1` 成功扣减，然后变成 `-1`。关键点在这里：第二个 `UPDATE` 等到拿到行锁后，会基于最新的数据重新判断 `where` 条件。如果已经被第一个事务改成 `0`，那么第二个 `UPDATE` 的 `stock > 0` 就不成立了。所以这条 SQL 能防止超卖。

乐观锁的优点是锁范围小、性能较好，适合读多写少、冲突不严重的场景。缺点是高并发冲突严重时，可能需要不断重试，重试成本也会变高。

17.4.实际场景中的锁方案（重点★★★★★）

为了更好地理解这些锁，可以用库存扣减这个典型场景来分析。假设商品库存为 1 万条，现在有多个请求同时扣减库存。如果控制不好，就可能出现超卖。围绕这个问题，常见方案有四类：Java 本地锁、数据库行锁、Redis 分布式锁、数据库乐观锁。

17.4.1.使用 `synchronized` 本地锁（重点★★★★☆）

最直接的方案是在扣减库存的方法上加 `synchronized`，让同一时间只有一个线程进入扣减逻辑。

```
public synchronized void deductStock(Long productId) {  
    // 查询库存  
  
    // 判断库存是否充足  
  
    // 扣减库存  
}
```

这种方式在单体应用中可以起到互斥作用。它的优点是实现简单，适合初学者理解并发控制。

但是缺点也很明显。

第一，性能较差。`synchronized` 会让并发请求串行执行，高并发场景下吞吐量会下降。

第二，它只能解决单 JVM 内部的并发问题。如果系统部署了多个实例，每个实例都有自己的 JVM 锁，那么实例 A 的锁管不到实例 B，仍然可能发生并发扣减问题。

所以，`synchronized` 适合单体应用内部的简单互斥，不适合分布式微服务场景下的共享资源扣减。

17.4.2.使用数据库行锁 `select for update`（重点★★★★★）

数据库行锁是一种典型的悲观锁。它可以通过数据库事务锁住某一行数据，避免其他事务同时修改。

常见写法如下：

```
begin;  
select stock  
from product  
where id = 1  
for update;  
update product  
set stock = stock - 1  
where id = 1;  
commit;
```

`select ... for update` 会对查询到的记录加排他锁。在当前事务提交之前，其他事务如果也想修改这条记录，就需要等待。

这种方案相比 `synchronized` 的优势是：它可以解决跨 JVM、跨服务实例的并发问题。因为锁加在数据库行上，不管请求来自哪个服务实例，只要访问的是同一条数据库记录，就会受到数据库锁的约束。

但是它也有明显缺点。

第一，性能问题。数据库行锁本质上仍然是串行执行。高并发请求会在数据库层排队等待，

容易造成响应变慢。

第二，事务持有时间不能太长。如果在事务方法中除了数据库操作，还调用了第三方接口、远程服务或复杂计算，就可能导致数据库连接长时间被占用。

第三，容易打满数据库连接池。如果大量请求都阻塞在 `select for update` 上，数据库连接会被长时间占住，最终可能导致系统整体不可用。

第四，可能产生死锁。比如事务 1 先锁住记录 A，再尝试锁记录 B；事务 2 先锁住记录 B，再尝试锁记录 A。此时双方互相等待，就会形成死锁。

所以数据库行锁可以解决分布式并发问题，但一定要注意：事务要短，锁范围要小，锁顺序要一致，避免在事务中调用慢接口。

17.4.3.使用 Redis 分布式锁（重点★★★★★）

Redis 分布式锁是分布式系统中常见的互斥方案。它的基本思想是：在执行业务逻辑之前，先到 Redis 中抢一把锁。抢到锁的服务实例可以执行扣减逻辑，抢不到锁的实例要等待、重试或直接失败。这个在 Redis 这张也讲过。

常见加锁命令如下：

```
SET lock:product:1 requestId NX EX 30
```

其中：

NX 表示 key 不存在时才设置成功；

EX 30 表示设置 30 秒过期时间；

requestId 用来标识当前锁的持有者。

Redis 分布式锁的优点是性能高，避免大量请求直接竞争数据库行锁，可以减轻数据库压力。

17.4.4.使用数据库乐观锁（重点★★★★★）

数据库乐观锁是库存扣减场景中非常常见、也非常推荐理解的一种方案。它的核心思想是：不提前加锁，而是在更新时通过条件判断保证不会扣成负数。

最简单的写法是：

```
update product
set stock = stock - #{num}
where id = #{productId}
and stock >= #{num};
```

如果影响行数为 1，说明扣减成功；如果影响行数为 0，说明库存不足，或者并发情况下已经被其他请求扣减了。

这种方案的好处是锁范围非常小。它不需要先查询再加锁，而是把判断和扣减合并在一 SQL 中完成。数据库会保证单条更新语句的原子性。

如果使用 version 版本号，也可以写成：

```
update product
set stock = stock - #{num},
    version = version + 1
where id = #{productId}
and version = #{oldVersion}
and stock >= #{num};
```

这类方式适合高并发扣减场景，尤其是库存、余额、名额、次数这类资源扣减。

17.4.5.几种方案对比（重点★★★★★）

方案	是否支持分布式	性能	实现复杂度	主要问题	适用场景
synchronized	不支持	一般	低	只能控制单 JVM	单体应用内部同步
数据库行锁	支持	较低	中	阻塞、连接占用、死锁	强一致、并发不太高的场景
Redis 分布式锁	支持	较高	较高	超时、误删、续期、可靠性设计	分布式互斥、短临界区
数据库乐观锁	支持	较高	中	高冲突下可能失败或重试	库存扣减、余额扣减、次数扣减

分布式队列	支持	取决于 队列消 费能力	中	实时性较弱	削峰、异步串行 处理
-------	----	-------------------	---	-------	---------------

这里凯恩建议大家记住一句话：能用条件更新解决的扣减问题，优先考虑数据库乐观锁；需要跨服务互斥的复杂业务，再考虑 Redis 分布式锁；事务强一致但并发不高的场景，可以使用数据库行锁。

18.下篇-消息队列与异步架构（重点★★★★★）

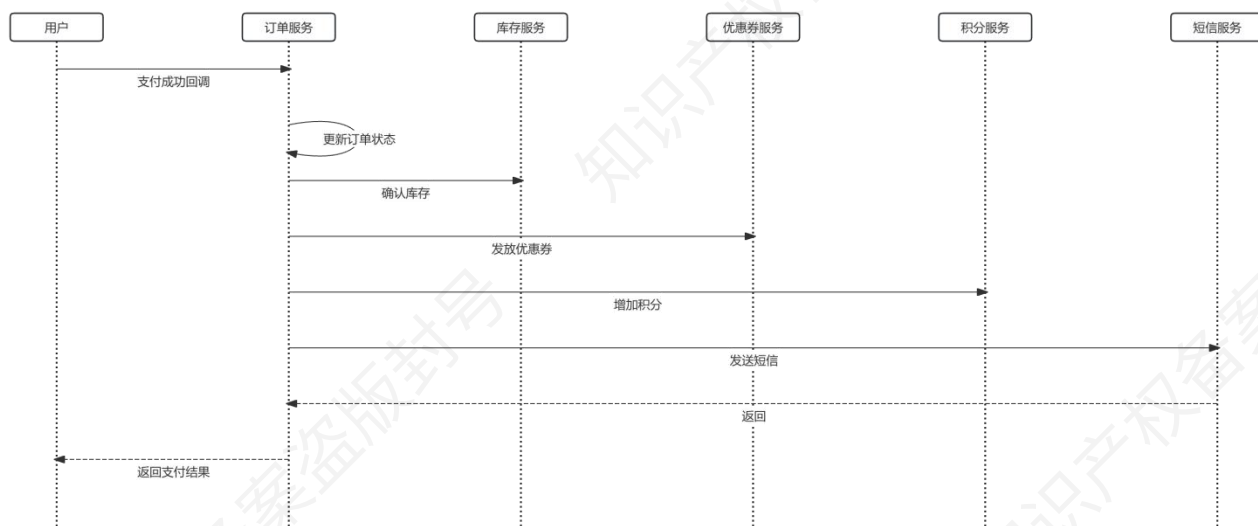
消息队列软考里的比重近几年不断增强，从概念考察、作用考察，到架构图填空都有涉及，是相对比较重要的章节。消息队列往往不是单独考察，而是和具体的解决方案放在一起考察。

18.1.为什么需要消息队列（重点★★★★★）

消息队列 MQ，本质上是系统之间传递消息的中间件。为什么要引入 MQ？这个问题实际上不好回答。因为角度太多了，比如哪些业务适合丢到 MQ 里？MQ 能不能保证主流程一定成功？引入 MQ 以后，消息丢失、重复消费、积压这些问题怎么办？所以这一节不要急着背结论。我们先从一个电商支付场景看起。

18.1.1.先看一个同步调用的问题

假设用户在商城里完成支付。站在业务视角看，要做下列这些操作：更新订单状态、确认库存、发优惠券、加积分、发短信等。如果所有动作都放在一个同步调用链里，大概会变成这样：



这个方案有下面这些问题：

第一，链路太长。每个下游接口哪怕只慢几十毫秒，加起来用户就要多等一段时间。

第二，耦合太重。订单服务必须知道后面有哪些系统、每个系统怎么调用、失败了怎么处理。以后如果新增“会员成长值”“运营消息推送”，订单服务还得跟着改。

第三，容错性差。短信服务慢了，积分服务报错了，都可能把订单主流程拖慢，甚至失败。

也就是说，同步调用最大的问题是它会把响应时间、系统依赖和故障影响都绑在一条链路上。

引入 MQ 后，订单服务不再挨个调用所有下游系统，而是先完成核心动作，然后发出一条事件消息，后续谁关心这件事，谁就订阅这类消息，也就是所谓的异步处理。这样一改，系统逻辑就清楚很多：比如订单服务只关心“支付成功”这个事件已经产生。优惠券、积分、短信、数据分析各自消费消息。新增一个下游系统，只需要新增消费者，不需要改订单主流程。

不过这里要特别注意：不是所有动作都应该异步，比如“订单是否支付成功”“核心库存是否确认”这类会影响主业务结果的动作，通常仍然要放在核心流程里。发券、加积分、发短信、写分析日志这类可以延后处理的动作，才更适合交给 MQ。

18.2.消息队列的作用（重点★★★★★）

18.2.1.异步解耦（重点★★★★★）

异步解耦是 MQ 最核心的价值。比如订单支付成功后，订单系统不一定要同步调用积分系统、优惠券系统、短信系统、推荐系统。订单系统只需要发送一条“订单已支付”的消息，后面的系统各自订阅并处理即可。这样做有三个好处：

第一，订单系统不需要知道后面有哪些系统要处理这件事，系统之间耦合降低。

第二，某个下游系统暂时故障，不会直接拖垮订单主流程。

第三，后续如果新增“发会员成长值”“推送运营消息”等功能，只要新增消费者即可，不需要大改订单系统。

18.2.2.削峰填谷（重点★★★★★）

削峰填谷是高并发场景中 MQ 最常见的用途。比如秒杀系统中，用户瞬间涌入。如果所有请求都直接打到订单服务和库存服务，数据库很容易被压垮。引入 MQ 后，可以先把请求写入队列，然后由消费者按照系统设定的速度慢慢处理。

18.2.3.引入 MQ 的代价（重点★★★★★）

讲 MQ 不能只讲优点。它不是银弹，引入以后系统会明显变复杂。那么考题中可能会问消息队列适用的场景。

适合引入 MQ 的场景，通常有几个明显特征：上游系统不想直接依赖太多下游系统，希望通过消息把双方解耦；业务中存在一些非核心、可以延后处理的动作，比如通知、积分、日志、数据同步等；系统会遇到突发流量，需要通过队列削峰填谷；同时业务能够接受最终一致性，并且团队已经具备补偿、重试、监控、告警等配套能力。

不适合引入 MQ 的场景往往是：如果调用方必须立即拿到完整结果，不能异步等待，就不适合用 MQ；如果业务本身很简单，引入 MQ 反而会增加系统复杂度，也没有必要；如果团队还没有监控、告警、重试、死信队列、补偿机制这些治理能力，贸然上 MQ 容易把问题藏起来；如果业务完全不能接受最终一致性，只能接受同步强一致，也不适合通过 MQ 来拆开处理。

18.3.消息队列核心概念（重点★★★★★）

18.3.1.消息流转过程（重点★★★★★）

还是用订单支付成功这个例子。订单服务发现用户支付成功后，会构造一条消息，比如：

消息 ID: msg_10001

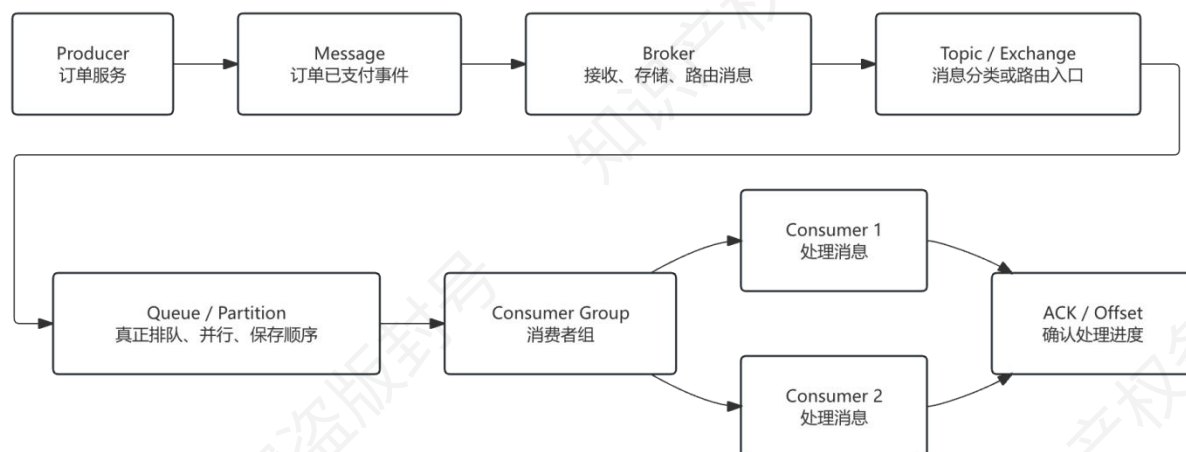
业务 ID: order_1001

事件类型: ORDER_PAID

业务数据: 订单金额、用户 ID、支付时间

Trace ID: 方便后续排查链路

然后订单服务把这条消息发给 MQ。MQ 收到消息以后，会根据主题、路由规则、队列或分区，把消息存起来。后面的积分服务、优惠券服务、短信服务再从 MQ 中拿消息处理。



这张图里出现的概念，就是一个涉及到 MQ 应用的流程全貌。

简单说：

Producer（生产者）负责发送消息。

Message（消息）是生产者和消费者之间传递的数据载体。

Broker（消息代理 / 消息服务器）负责接收、存储、路由和投递消息。

Topic（主题）或 Exchange（交换机）负责表达“这类消息应该怎么分类，或者应该怎么路由”。

Queue（队列）或 Partition（分区）负责真正承载消息排队、并行消费和局部顺序。

Consumer（消费者）负责消费消息。

Consumer Group（消费者组）负责把多个消费者组织起来，提高整体消费能力。

ACK（确认机制）或 offset（消费位点 / 偏移量）负责告诉 MQ：这条消息是否处理完成，或者这个消费组已经消费到哪里了。

18.3.2.Producer、Consumer、Message、Broker（重点★★★★★）

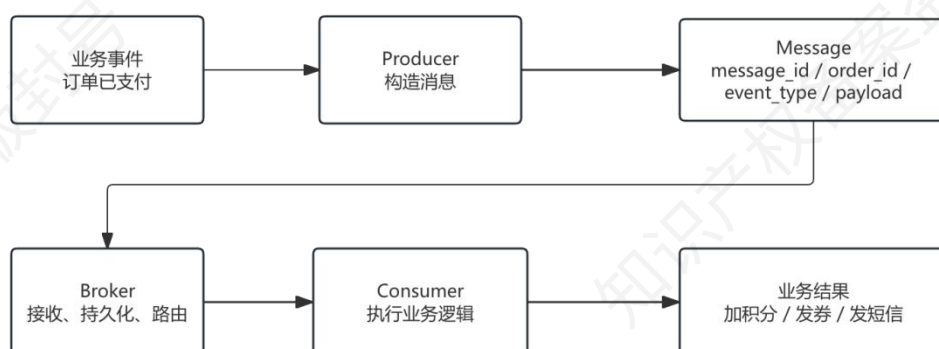
先看四个最基础的角色：Producer（生产者）、Consumer（消费者）、Message（消息）和 Broker（消息代理 / 消息服务器）。

Producer（生产者），就是发送消息的一方。比如订单服务发送“订单已支付”事件，支付服务发送“扣款成功”事件，日志采集程序发送“用户点击行为”事件，这些发送方都可以叫 **Producer**。

Consumer（消费者），就是处理消息的一方。比如积分系统消费“订单已支付”事件后增加积分，短信系统消费同一条事件后发送短信，风控系统消费交易事件后做风险分析，这些处理方都可以叫 **Consumer**。

Message（消息），就是 MQ 中传递的数据本身。生产系统里的消息通常至少要包含这些信息：消息 ID：用于追踪和幂等。业务 ID：比如 `order_id`、`user_id`、`payment_id`。事件类型：比如 `ORDER_PAID`、`ORDER_CANCELLED`。业务数据：消费者真正需要处理的数据。时间戳：用于排查延迟、排序和审计。重试次数：用于判断是否进入死信队列。**Trace ID**：链路追踪 ID，用于串联生产、存储、消费链路。

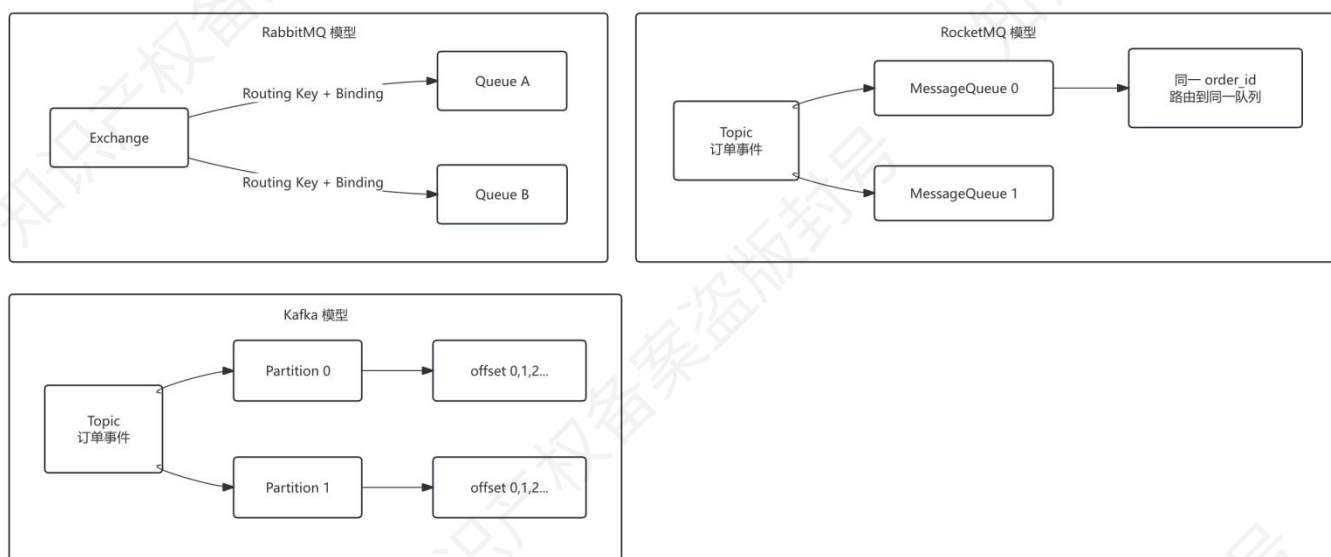
Broker（消息代理 / 消息服务器），就是 MQ 的服务端。它的工作是接收消息、存储消息、根据规则路由消息，再把消息投递给消费者，或者等待消费者主动拉取。可以把 **Broker** 理解为 MQ 的核心中转站。没有 **Broker**，生产者和消费者就又变成直接调用了，也就谈不上解耦、缓冲和削峰。我们常说的 Redis，



18.3.3.Topic、Queue、Partition（次重点★★★★☆）

Topic（主题）是消息分类，Queue（队列）或 Partition（分区）才是消息实际排队、存储、并行消费和局部有序的承载单位。

比如订单服务发出“订单已支付”消息，这类消息可以放到 `order_paid_topic`。这个 Topic 表示：这里面都是订单支付成功事件。但是消息真正怎么存、怎么并行消费、怎么保证同一个订单内有序，通常要落到 Queue 或 Partition 上。不同的消息队列内部实现不一样。



在 Kafka 中，一个 Topic（主题）会拆成多个 Partition（分区）。Partition 是 Kafka 并的基本单位。同一个 Partition 内，消息按照 offset（消费位点 / 偏移量）有序；不同 Partition 之间，不保证全局有序。

在 RocketMQ 中，一个 Topic（主题）下面有多个 MessageQueue（消息队列）。顺序消息通常要把同一个业务 Key，比如同一个订单 ID，路由到同一个 MessageQueue，这样才能保证同一个订单相关消息的顺序。

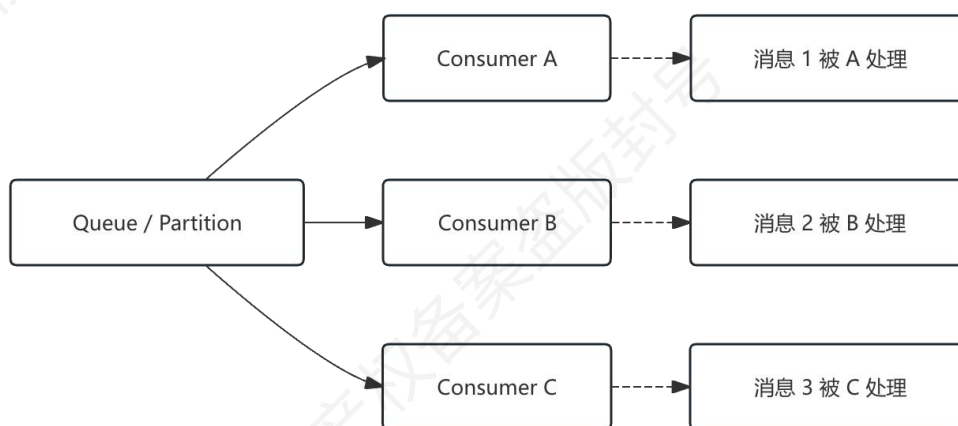
在 RabbitMQ 中，模型又不一样。生产者一般不是直接把消息发到 Queue（队列），而是先发到 Exchange（交换机）。Exchange 会根据 Routing Key（路由键）和 Binding（绑定

关系)，把消息路由到一个或多个 Queue。RabbitMQ 里，Queue 才是消息真正等待消费者取走的地方。

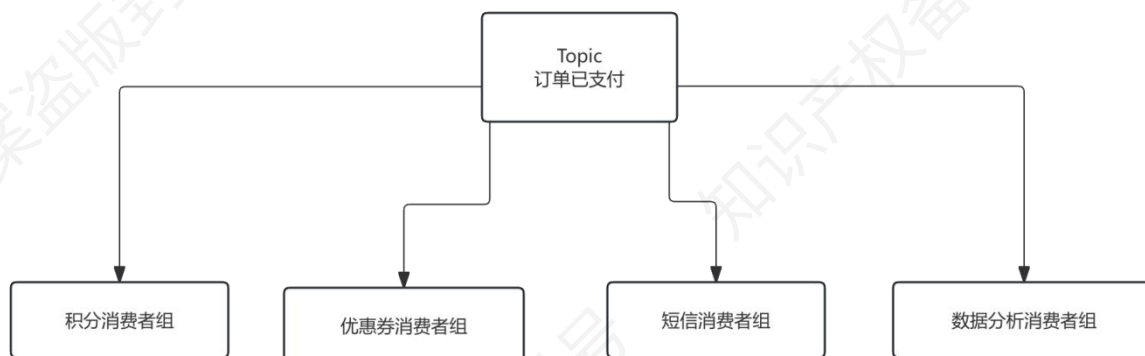
18.3.4.点对点模式与发布订阅模式（重点★★★★★）

消息消费的方式，主要有两种，分别是点对点和发布订阅。

点对点强调的是：从同一个消费关系看，一条消息通常只交给一个消费者处理。比如图片处理任务、订单创建任务、异步导出任务，都属于“谁拿到谁处理”的模式。



发布订阅模式：一条消息可以被多个订阅方处理。适合事件通知，例如订单支付成功后，积分系统、优惠券系统、通知系统都要感知。

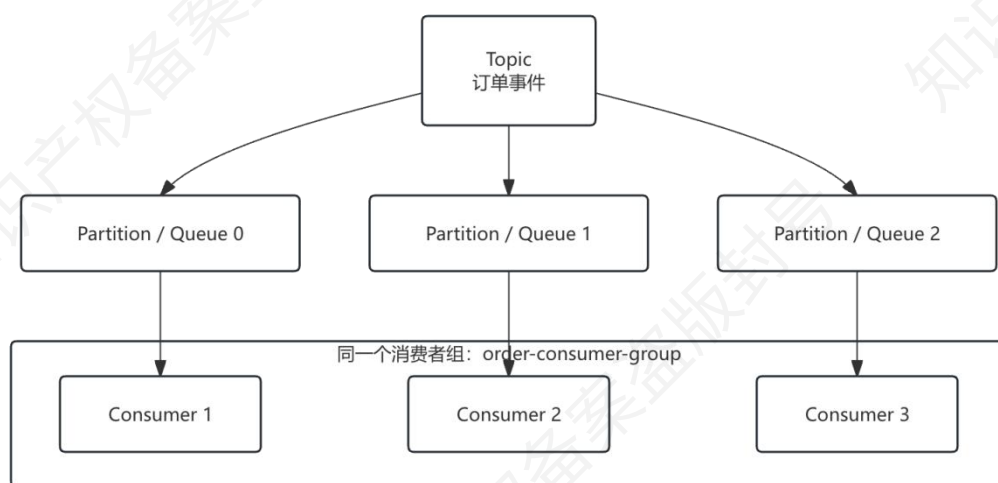


举个例子，订单系统发送一条“订单已支付”消息：积分消费者组收到一份，用来加积分。

优惠券消费者组收到一份，用来发券。短信消费者组收到一份，用来发短信。

18.3.5.Consumer Group（重点★★★★★）

消费者组的核心作用，是提高消费能力和隔离消费进度。如果一个消费者处理不过来，就可以启动多个消费者实例，让它们组成同一个消费者组，一起消费同一个 Topic 或队列里的消息。



这样做有两个好处。

第一，可以水平扩展消费能力。一个消费者处理不过来，就增加消费者实例。

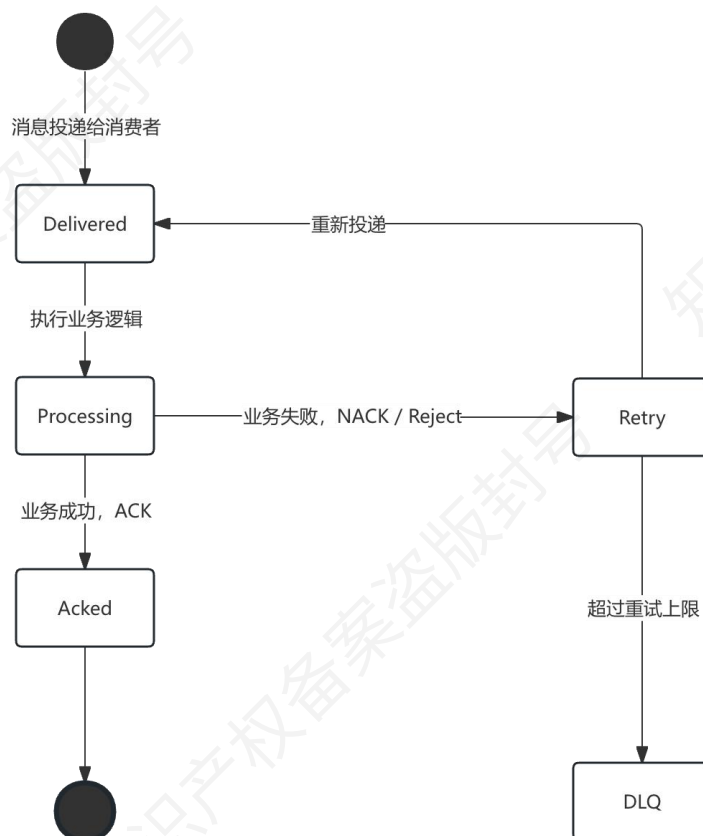
第二，不同业务可以维护各自的消费进度。积分系统、短信系统、数据分析系统都订阅订单事件，但它们属于不同消费者组，所以互不影响。

18.3.6.消息确认机制 ACK、NACK 和 offset（重点★★★★★）

消息被消费者拿到，不代表业务已经成功。比如消费者刚拿到“订单已支付”消息，还没来得及加积分，服务就宕机了。如果它一拿到消息就告诉 MQ“我处理完了”，那这条消息就可能丢了。所以很多 MQ 都有确认机制。

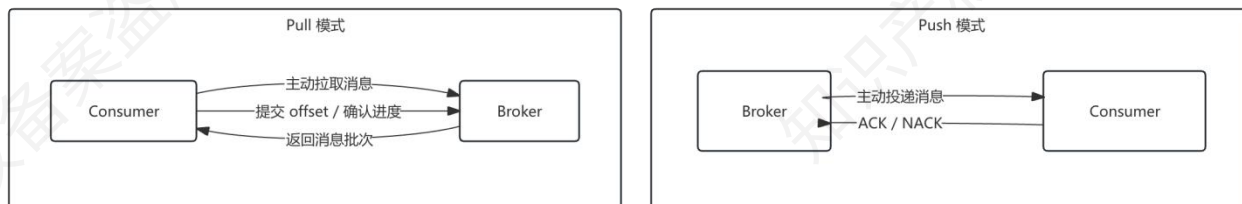
ACK 的意思是：消费者处理成功后，告诉 MQ 这条消息可以认为完成了。

NACK 或 Reject 的意思是：这条消息处理失败了，需要重试、重新入队，或者最终进入死信队列（后面会解释）。



18.3.7.Push 和 Pull（重点★★★★☆）

消费者拿消息，大体可以分成 Push 和 Pull 两种思路。



Pull 是消费者自己去 Broker 拉消息。消费者可以根据自己的处理能力，决定什么时候拉、一次拉多少。它的好处是节奏可控，但需要处理轮询、拉取间隔和空拉取等问题。

Push 是服务端主动把消息推给消费者。消息一来，消费者就被通知去处理。它的好处是实时性好，但消费者压力控制要做好，否则服务端推得太快，消费者可能处理不过来。

Kafka 典型是 **Pull** 模式。消费者主动拉取消息，自己控制消费节奏。

RabbitMQ 常见表现更接近服务端投递给消费者，但可以通过 **prefetch** 控制消费者一次最多拿多少未确认消息，避免消费者被压垮。

RocketMQ 里既有 **PushConsumer**，也有 **PullConsumer** 和 **SimpleConsumer**。很多所谓 **PushConsumer** 底层也可能通过客户端拉取来实现，只是对业务代码表现成“监听器回调”的形式。

18.3.8.JMS、AMQP 的区别

JMS、AMQP 在我们的选择题中也出现过。

JMS 可以理解为 **Java** 体系里的消息服务 **API** 规范。它定义了 **Java** 应用如何创建、发送、接收和读取消息，也定义了点对点、发布订阅这类模型。早期的 **ActiveMQ** 就是基于 **JMS** 规范实现的。

AMQP 则更像一个真正的消息协议。它不是某个语言里的 **API**，而是面向消息中间件的应用层协议，天然支持跨语言、跨平台。**RabbitMQ** 就是 **AMQP** 的典型实现。**RabbitMQ** 里 **Exchange**、**Routing Key**、**Binding**、**Queue** 这些概念，很大程度上就是 **AMQP** 模型带来的。

概念	更像什么	典型代表
JMS	Java API 规范	ActiveMQ
AMQP	跨语言消息协议	RabbitMQ

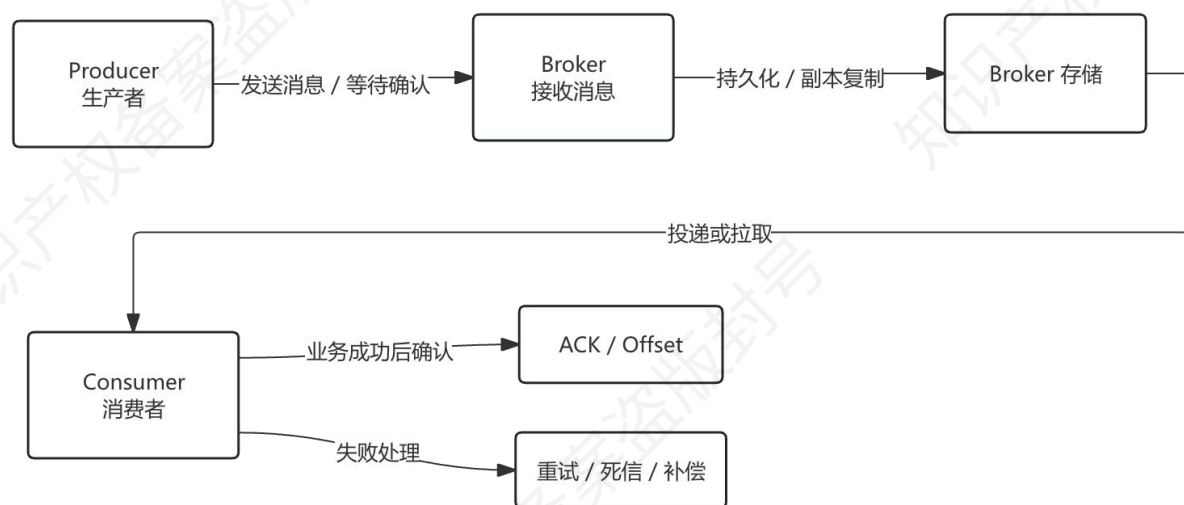
18.4.MQ 常见生产问题（重点★★★★★）

MQ 学到这里，才算真正进入生产问题。前面讲“解耦、异步、削峰”时，MQ 看起来很美

好：但只要把它放到真实业务里，问题马上就来了：

消息发出去以后，Broker 真的收到了吗？Broker 收到了以后，宕机了会不会丢？消费者拿到消息以后，业务还没处理完就挂了怎么办？消息被重复投递，积分会不会加两次？同一个订单的状态消息乱序了怎么办？

在分析具体业务场景的时候，我们可以参照下面这张图。



18.4.1.消息丢失（重点★★★★★）

如果题目问“如何保证消息不丢失”，完整的回答可以从生产者、Broker、消费者。

第一，生产者可能没发成功。比如订单服务以为自己已经把“订单已支付”消息发出去了，但网络抖动导致 Broker 根本没收到。或者 Broker 已经收到了，但返回给生产者的确认响应丢了，生产者不知道到底成功还是失败。因此，我们可以开启生产者发送确认。发送失败后重试。重要业务记录发送日志或本地消息表。对一致性要求更高的链路，使用事务消息或 Outbox 模式。

第二，Broker 丢失。Broker 收到消息不等于消息一定安全。如果消息还没刷盘，Broker 就宕机了；或者主节点确认了消息，但副本还没同步完成，主节点突然故障，也可能出现消息不

可用。因此，我们可以做消息持久化。合理设置刷盘策略。主从复制或多副本机制。

第三，消费者可能处理失败。这是很多人容易忽略的地方。消费者拿到消息，不代表业务已经完成。如果消费者一拿到消息就 ACK，结果业务还没处理完就宕机，MQ 会认为消息已经处理成功，后面就不会再投递了。

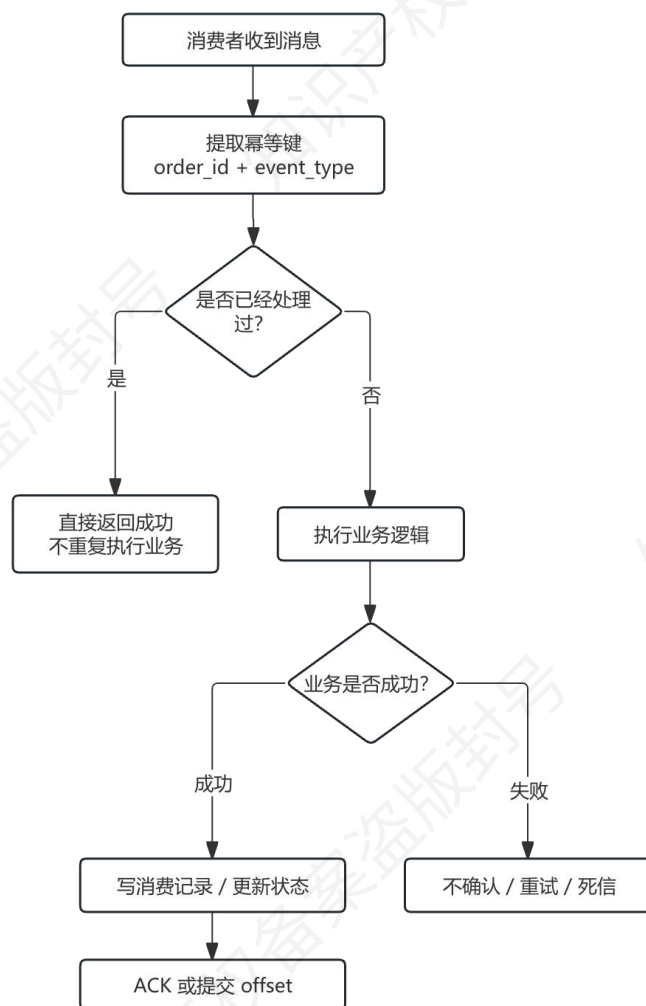
18.4.2.消息重复消费（重点★★★★★）

生产系统里，不要假设消息只会被消费一次。很多 MQ 更容易做到“至少一次投递”，也就是消息尽量不丢，但它可能重复。重复消费常见原因有如下几类类：

1. 生产者没收到发送确认，于是重试发送，导致消息重复进入 MQ。
2. 消费者业务已经处理成功，但 ACK 失败，Broker 又把消息投递了一次。
3. 网络抖动、超时重试，导致同一条消息再次被处理。

可以看到，有的时候重复消费是无法避免的，比如说网络抖动和超时重试。那么这个问题的核心解法不是“让 MQ 绝对不重复”，而是做业务幂等。

幂等的意思是：同一个操作执行一次和执行多次，最终结果应该一致。比如“给订单 1001 加积分”这件事，不能每消费一次消息就加一次。更合理的做法是用 `order\_id + event\_type` 做唯一键，先判断这类事件是否已经处理过。



常见幂等方案：

1. 使用业务唯一键去重，比如 `order\_id + event\_type`。
2. 数据库唯一索引，防止重复插入。
3. 消费记录表，记录 `message\_id`、处理状态、处理时间。
4. 状态机判断，比如订单已经支付，就不能再次执行支付。

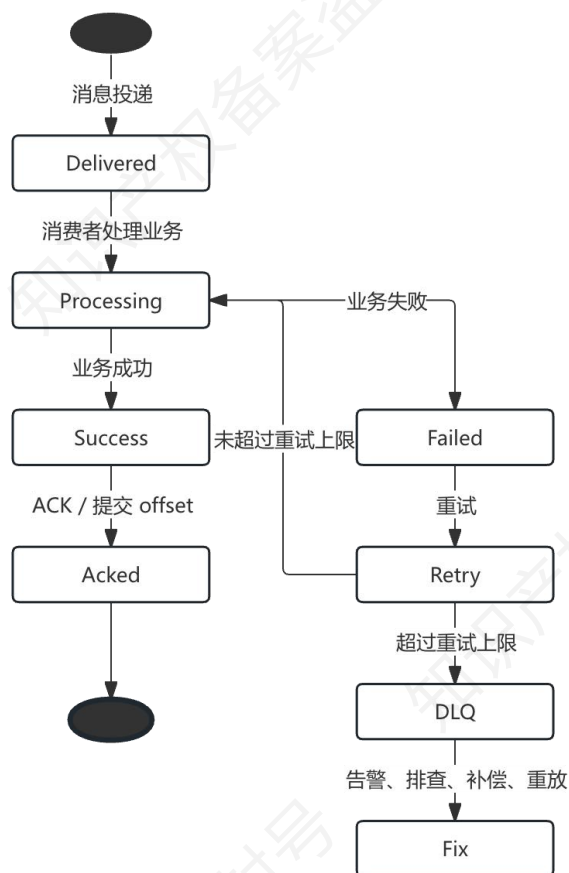
18.4.3.死信队列（DLQ）（重点★★★★★）

前面提到 ACK 解决的是“消息什么时候算处理成功”的问题。消费者不是拿到消息就算成功，而是业务逻辑真正执行成功以后，才应该确认。那如果业务处理失败怎么办？这时才会引

出 NACK、重试和死信队列。

一般先判断这个失败是不是“可恢复”的：如果只是网络抖动、数据库短暂超时，可以不 ACK，或者 NACK / Reject 后重新入队，让消息稍后再试。如果重试几次以后还是失败，就不能让它无限重试。如果消息本身就是坏数据，比如订单不存在、参数格式不对、业务状态永远不允许处理，也不应该继续堵着正常消息。这时候就需要一个专门的失败出口，也就是死信队列。

死信队列可以理解成“正常消费流程处理不了的消息，先隔离到另一个地方”。它解决的是两个矛盾：不能一直重试。否则一条坏消息可能反复失败，占用消费者线程，甚至在顺序队列里堵住后面的正常消息。也不能直接丢。因为这条消息可能关系到订单、支付、库存、积分，后面还要排查、修复、补偿或者重放。



常见进入死信队列的原因：（1）消息被拒绝，并且不再重新入队。（2）消息过期。（3）

队列长度超过限制。（4）重试次数超过上限。

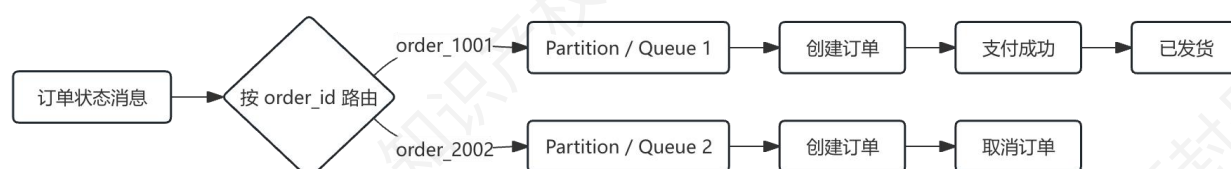
18.4.4.顺序性（重点★★★★★）

这里涉及到两个概念，分别是全局有序和局部有序。

全局有序的意思是：所有消息严格按一个顺序处理。这个代价很高，往往意味着单队列、单消费者，吞吐量很低，实际使用场景不多。

生产中更常见的是局部有序。比如同一个订单的状态变化：订单创建。支付成功。已发货。已完成。这几个事件对同一个订单来说必须有序。但不同订单之间，一般不需要严格有序。订单 1001 的支付事件和订单 2002 的取消事件，谁先谁后通常不影响业务。

实现局部有序的核心思路是：按业务 ID 路由，比如按 `order_id` 路由。同一个业务 ID 的消息进入同一个 Queue 或 Partition。同一个 Queue 或 Partition 内顺序追加。消费端对同一业务 ID 串行处理，避免并发打乱顺序。



18.4.5.消息积压（重点★★★★★）

消息积压说明生产速度大于消费速度。常见处理方式：

1. 看消费者日志，确认是否报错、阻塞、死循环。
2. 看下游数据库、缓存、RPC 是否变慢。
3. 看队列深度、消费延迟、重试次数、死信数量。
4. 如果队列或分区支持并行，再临时扩容消费者。

5. 优化消费逻辑，减少同步调用，做批量消费、批量写库。
6. 拆分 Topic、Queue、Partition，提高并行度。
7. 毒消息进入死信，避免反复重试阻塞正常消息。
8. 必要时限流生产者，或者降级非核心业务。

18.5.MQ 与分布式事务（重点★★★★★）

18.5.1.本地消息表（重点★★★★★）

本地消息表是一种经典的最终一致性方案。前面分布式事务那一章节也做过介绍。这里从 MQ 角度来看，可能感觉又不太一样。它的思路不复杂：不要在本地事务里直接发 MQ，而是在本地事务里同时写业务数据和消息记录。

这里所谓“不要在本地事务里直接发 MQ”，不是说代码里绝对不能调用发送接口，而是说不要把 MQ 发送当成本地数据库事务的一部分来依赖。因为本地事务只能管本地数据库。订单表能不能提交、消息表能不能提交，数据库可以保证原子性；但 MQ 是另一个系统，数据库事务管不了它。如果在一个 `@Transactional`（Java 事务注解）方法里这样写：

更新订单状态

发送 MQ 消息

提交数据库事务

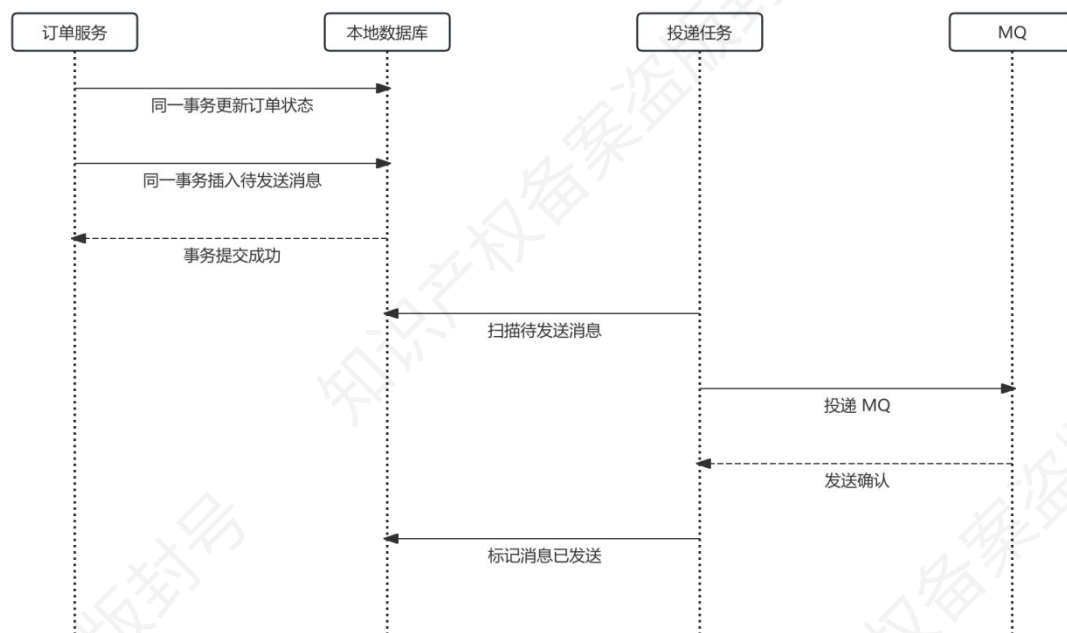
可能会出现：MQ 发送成功了，但是数据库事务最后回滚：那怎么去回滚下游已经做的事情？

本地消息表的思路，就是把“我要发一条消息”这件事先变成一条数据库记录。这样业务数据和消息记录就能放进同一个本地事务里，由数据库保证它们要么一起成功，要么一起失败。

比如订单支付成功时，同一个数据库事务里做两件事：（1）更新订单状态为已支付。（2）插入一条待发送消息到本地消息表。

只要这个事务提交成功，就说明业务数据和消息记录一定同时存在。后面由一个后台任务扫描消息表，把待发送消息投递到 MQ。这样做的好处是：不会出现“订单已支付，但系统里连一条待发送消息都没有”的情况。即使 MQ 暂时不可用，也只是消息表里有一条待发送记录，后台任务后续可以继续重试。

有同学可能会担心：消息记录刚插入本地消息表，还没提交事务，会不会被后台任务扫描到并发送出去？不会。因为后台任务是另一个数据库事务，只能看到已经提交的数据。业务事务未提交时，插入的消息记录对外不可见；如果业务事务回滚，消息记录也会一起回滚。只有当业务事务提交成功后，后台任务才能扫描到这条消息，并进行 MQ 投递。因此，本地消息表是依赖数据库本地事务和事务隔离机制来保证“业务成功后才发送消息”的。



当然，本地消息表也有代价：（1）要多一张消息表。要有扫描任务或投递组件。（2）要处理发送状态、重试次数、失败原因。（3）要避免重复投递，所以消费者仍然要幂等。（4）历史消息要归档或清理，避免消息表无限膨胀。

18.6.常见 MQ 对比（次重点★★★★☆）

一般软考不会考你选哪个消息队列，只要知道哪些名词是消息队列即可。这里的内容只是做个简单扩充。

从选型角度看，消息队列没有绝对的好坏，关键是看业务场景。简单来说，如果系统主要处理日志、埋点、监控数据、CDC 数据同步这类高吞吐事件流，通常优先考虑 **Kafka**。它的优势在于分区并行、吞吐能力强、消息可回放，适合和大数据、实时计算、流式处理系统结合使用。但如果业务特别强调复杂事务、延时消息、顺序消息和交易链路的可靠投递，**Kafka** 就不一定是最自然的选择。

如果是订单、支付、交易、库存、营销活动削峰这类典型业务消息场景，**RocketMQ** 通常更贴近业务系统的使用习惯。它对事务消息、延时消息、顺序消息等能力支持比较完整，更适合解决“业务动作发生以后，后续系统如何可靠感知并处理”的问题。比如订单支付成功后通知积分、优惠券、短信、物流等系统，这类场景用 **RocketMQ** 会比较顺手。不过它的概念模型和运维体系需要专门掌握，不能只把它当成一个简单队列来用。

RabbitMQ 更适合复杂路由和传统任务队列场景。它的 **Exchange**、**Queue**、**Routing Key** 模型比较灵活，可以支持直连、广播、主题匹配等多种路由方式，适合“不同类型消息分发到不同队列”的业务。比如根据不同告警类型、不同业务事件、不同任务优先级进行分发，**RabbitMQ** 就比较合适。但如果面对的是海量日志流、埋点流、监控流，**RabbitMQ** 一般不是首选。

Redis Stream 和 **Redis Pub/Sub** 更适合轻量级场景。**Redis Pub/Sub** 更像一个简单广播通知机制，消息发出去以后，如果消费者不在线，通常就收不到，因此不适合可靠消息投递。**Redis Stream** 相比 **Pub/Sub** 更完整，支持消息持久化、消费组、**ACK**、**PEL** 等机制，可以承担一些轻量级 **MQ** 场景。但它毕竟不是专业消息队列，一旦涉及核心交易链路、大规模堆积、复杂消费治理，还是建议使用 **Kafka**、**RocketMQ** 或 **RabbitMQ** 这类专业 **MQ**。

19.下篇-秒杀场景设计（重点★★★★★）

秒杀系统是高并发场景里非常经典的一类题，最近也时常在软考里出现。它看起来是在问“抢购功能怎么做”，本质上是在考查系统在高并发下的整体设计能力，包括流量控制、缓存、库存扣减、消息削峰、异步下单、数据库保护和系统监控等。

下面我们先来分析秒杀系统的核心痛点，再按架构层次展开：访问层、中间转发层、服务层、消息队列层和数据层。

19.1.秒杀系统的核心痛点（重点★★★★★）

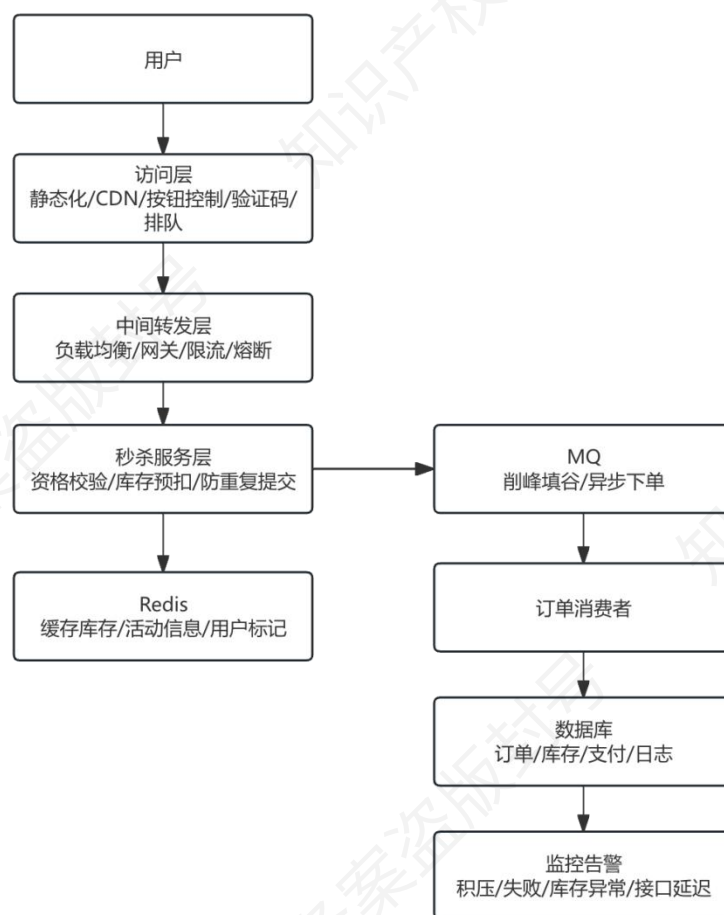
秒杀系统首先要面对的是瞬时并发量大。秒杀开始的一瞬间，大量用户会同时访问商品页、点击秒杀按钮、提交下单请求。平时几百 QPS 的系统，在秒杀开始时可能瞬间涨到几万甚至几十万 QPS。如果没有提前设计好限流、缓存、削峰和降级机制，服务端、数据库和下游系统都可能被打垮，最终造成系统雪崩。

第二个痛点是库存不能超卖。秒杀商品的库存通常很少，而参与抢购的用户很多。系统必须保证库存扣减是安全的。比如库存只有 100 件，最后就不能生成 120 个有效订单。也就是说，库存数量、成功请求数量和最终订单数量之间要保持一致。

第三个痛点是无效请求多。秒杀场景里，不是所有请求都是真实用户的正常请求。里面可能会混入爬虫、脚本、重复提交、黄牛请求等。如果这些请求全部进入服务端，会白白消耗系统资源。所以秒杀系统要尽量在前面挡掉无效流量，不要让所有压力都落到后端。

19.2.秒杀系统的整体架构（重点★★★★★）

秒杀系统一个典型的层次架构。可以分层来理解，如下图所示。



访问层主要负责减少无效访问和提升用户体验，比如页面静态化、CDN、按钮控制、验证码、排队提示等。

中间转发层主要负责承接和分发流量，比如负载均衡、网关、限流、熔断、动态扩容等。

服务层主要负责秒杀核心逻辑，比如校验活动、判断用户资格、扣减库存、防重复提交等。

消息队列层主要负责削峰填谷和异步下单。秒杀成功后，不一定马上同步写订单库，而是先写入 MQ，再由消费者按系统承受能力慢慢处理。

数据层主要负责最终数据落库和一致性保障，比如订单表、库存表、分库分表、读写分离、幂等处理等。

19.3.访问层设计（重点★★★★★）

访问层的核心目标是：能在前面挡住的流量，就不要放到后端。这个其实不管是秒杀系统还是正常的网站设计，都是要遵循这个逻辑。

首先是商品页静态化。秒杀商品页一般包含商品图片、描述、活动规则、价格、库存展示等内容。如果这些内容都由服务端动态生成，那么大量用户同时访问页面时，服务端压力会非常大。更好的做法是提前生成静态页面，把 HTML、CSS、JavaScript、图片等静态资源放到 CDN 上。用户访问商品页时，优先从 CDN 获取资源，服务端只处理真正的秒杀请求。

这样做有两个好处。

第一，用户访问速度更快。静态资源可以就近从 CDN 节点返回。

第二，服务端压力更小。服务端不需要处理大量商品页静态资源请求，而是集中处理真正的秒杀请求。

第三是验证码、滑块或者答题。验证码的作用不仅是防脚本，还可以把原本集中在同一毫秒的请求打散到几秒内。对于秒杀系统来说，只要能把瞬时流量稍微打散，就能明显减轻后端压力。

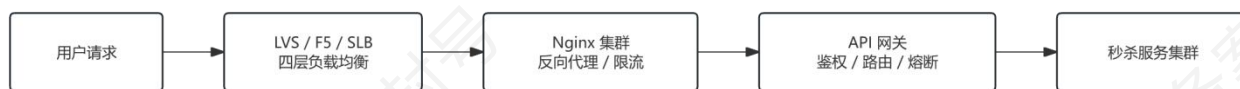
第四是排队体验。用户点击秒杀按钮后，如果页面没有任何反馈，很容易反复刷新或者重复点击。更好的做法是给用户明确的状态提示，比如“正在排队”“正在生成订单”“请稍后查看结果”。这既能提升用户体验，也能减少重复请求。

这一层的设计是尽量减少进入后端的请求数量。通过前端和 CDN 挡住一批请求，后端服务和数据库就会少一点压力。

19.4.中间转发层设计（重点★★★★☆）

访问层之后，请求会进入中间转发层。这个层次主要解决两个问题：第一，如何把流量分发到后端多个节点；第二，如何在流量过大时及时限流、熔断和扩容。

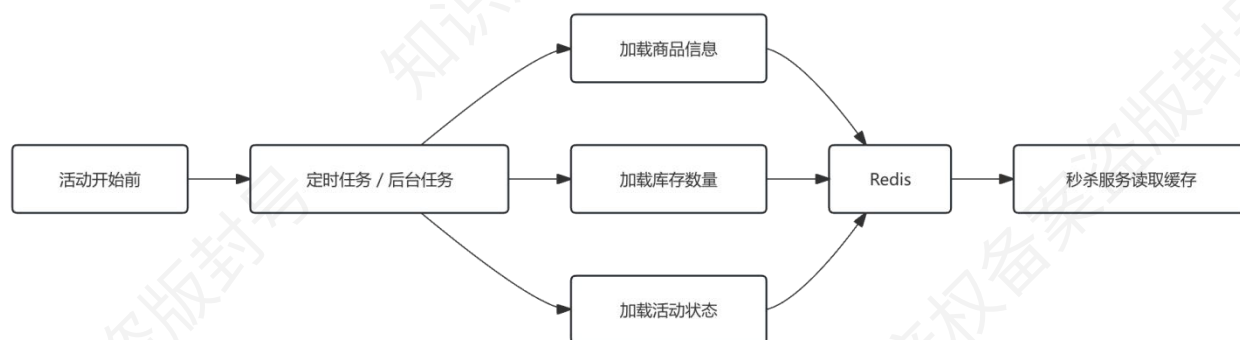
常见的做法是使用四层负载均衡配合 Nginx 集群,如下图所示。比如在最前面使用 LVS、F5 或云厂商 SLB 做四层负载均衡,再把流量分发到多台 Nginx。Nginx 继续做反向代理、限流和转发,后面再进入 API 网关和秒杀服务集群。



另外,由于秒杀请求中会混入大量无效请求和恶意请求,因此限流不应该只放在一个位置,而应该分层处理。在中间层,Nginx 可以先按 IP 或接口维度拦截明显异常的请求。

19.5.服务层设计（重点★★★★☆）

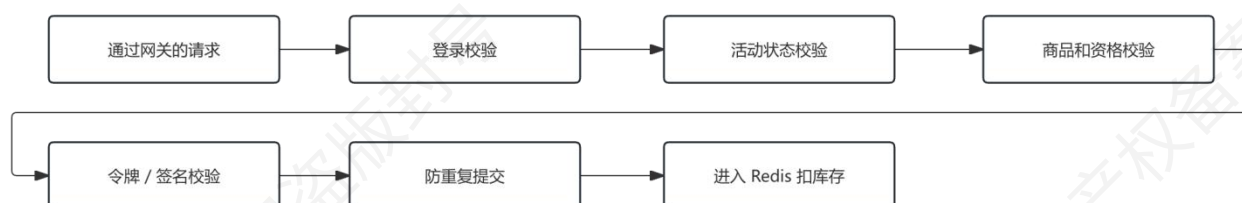
请求进入秒杀服务后,为了减少数据库磁盘读写压力,一般不会直接操作数据库。在这个层次上首先要做的是基础校验和缓存读取。一般在秒杀开始前,需要把秒杀商品信息、活动信息、库存信息提前预热到 Redis。这样用户请求进入服务层后,优先访问 Redis,而不是每次都查询 MySQL。



缓存预热不是把数据随便放进 Redis 就结束了。还要检查 key 是否存在、库存值是否正确、过期时间是否合理、热点 key 是否会被集中访问。如果活动开始时 Redis 中没有库存数据,大量请求可能直接穿透到数据库,数据库就会提前暴露在流量洪峰面前。

除了缓存预热,服务层还要做用户和活动校验。常见校验包括:用户是否登录、活动是否

开始或结束、商品是否参与本次秒杀、用户是否有资格、请求签名是否合法、同一用户是否已经抢过等。这些校验的目的，是在扣库存之前继续过滤请求。能在服务层挡掉的请求，就不要让它进入库存扣减逻辑。



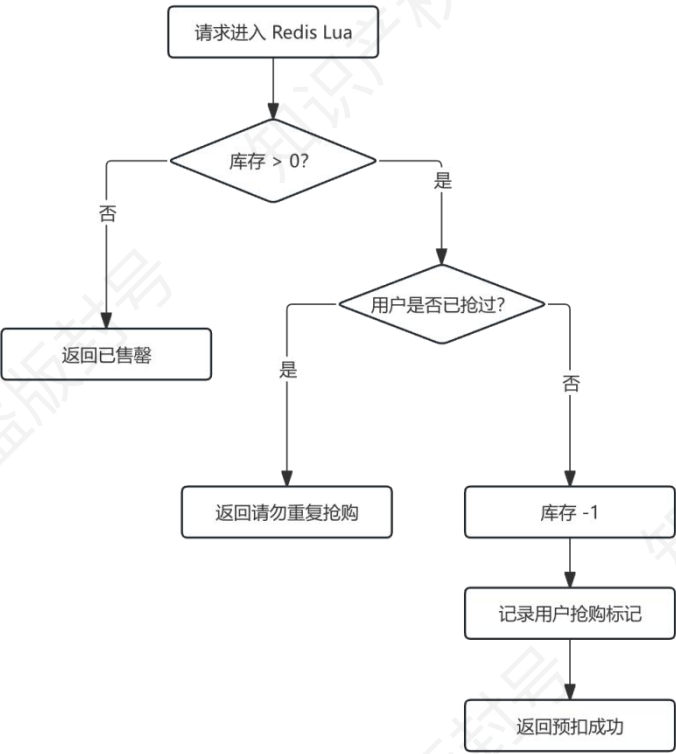
有的同学可能认为，这里的防重复提交是不是冗余？我们是不是可以直接把这个唯一性判断交给数据库呢？不的。虽然数据库唯一索引确实是最后一道兜底，但如果大量重复请求都打到数据库，再由唯一索引拒绝，数据库压力仍然很大。更好的做法是服务层和 Redis 层先挡一遍，数据库层再兜底一遍。

19.6.Redis 扣库存（重点★★★★★）

秒杀系统中最关键的动作是库存扣减。最容易想到的写法是：先查询 Redis 中的库存，如果库存大于 0，就把库存减 1，再记录用户已经抢到。

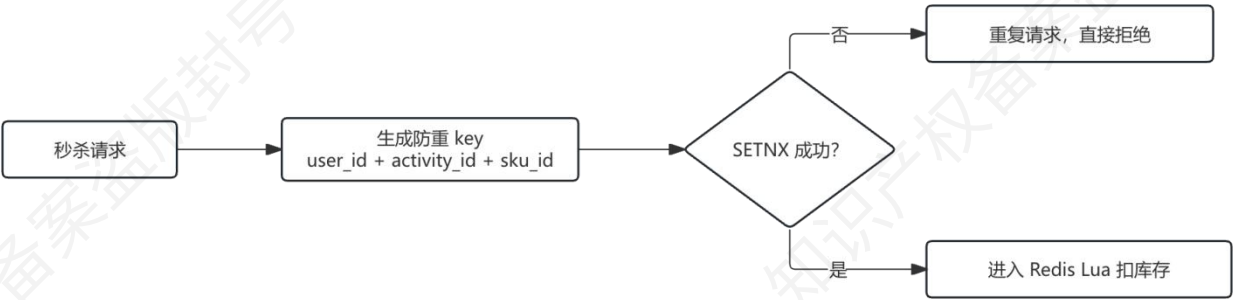
这个写法看起来简单，但在高并发下有明显问题。因为“查库存”和“扣库存”不是一个原子动作。假设库存只剩 1 件，两个请求同时查到库存大于 0，然后都去扣减，就可能出现超卖。

更稳妥的做法是使用 Redis Lua 脚本，把判断库存、判断用户是否已抢、扣减库存、记录用户抢购标记这几个动作放在 Redis 内部一次性执行。



Lua 脚本的价值在于原子性。它可以把多个操作合并成一个整体执行，中间不会被其他请求插入。这样既可以保证库存扣减安全，也可以减少应用层和 Redis 之间的多次网络交互。

除了 Lua 脚本，服务层还可以使用 Redis `SETNX` 做请求防重。比如用 `user\_id + activity\_id + sku\_id` 作为 key，保证同一个用户对同一个活动商品在同一时间只有一个有效请求。



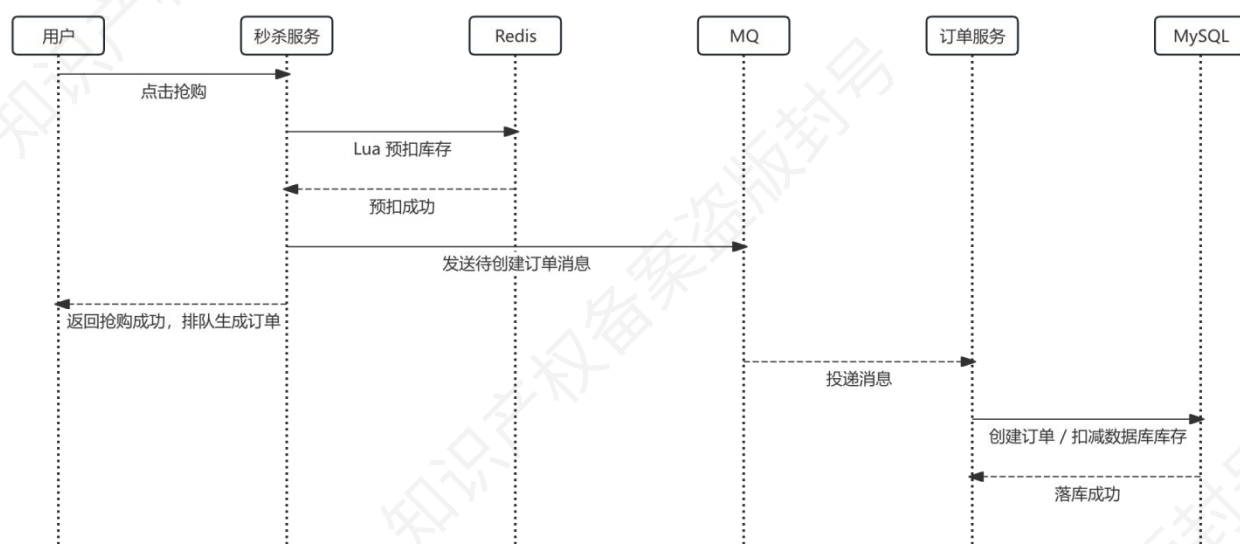
这里要注意，`SETNX` 更适合做防重标记，不能直接替代库存扣减逻辑。库存扣减仍然要依赖 Redis 原子操作、Lua 脚本或其他可靠的并发控制机制。分布式锁也可以实现互斥，但秒杀库存扣减属于高频操作，通常更推荐 Lua 脚本。它链路更短，性能更适合秒杀场景。

19.7.消息队列层设计（重点★★★★★）

Redis 预扣库存成功后，并不建议马上同步创建订单、扣数据库库存、调支付、发通知。

原因很简单：同步链路太长，任何一个下游变慢，都会拖慢用户请求；同时，预扣成功的请求虽然已经少很多，但仍然可能在短时间内集中打到数据库。

更合理的做法是：Redis 预扣成功后，秒杀服务发送一条“秒杀成功，待创建订单”的消息到 MQ，然后先返回给前端“抢购成功，正在生成订单中”。订单服务作为消费者，从 MQ 中按数据库能够承受的速度慢慢消费消息、创建订单、扣减数据库库存。



这里凯恩提醒大家要特别区分两个概念：这里，MQ 负责削峰填谷，不负责防超卖。防超卖主要靠 Redis 原子预扣和数据库层兜底。MQ 的作用是把瞬时成功请求变成平稳的后台处理流量，让数据库按照自己的承载能力处理订单。举个例子，如果数据库稳定处理能力是每秒 100 单，那么订单消费者就可以按照这个速度消费 MQ。外部流量再大，也不会直接冲击数据库。

19.8.数据层设计（重点★★★★★）

经过重重关卡，我们终于来到了数据层设计。这是因为秒杀系统里，数据库不应该直接承接秒杀峰值流量，但最后必须由数据库来负责落盘。也就是说，Redis 可以做预扣，MQ 可以做削峰，但最终订单是否创建成功、库存是否真正扣减成功，仍然要以数据库落库结果为准。

数据库层通常要做下面几件事。

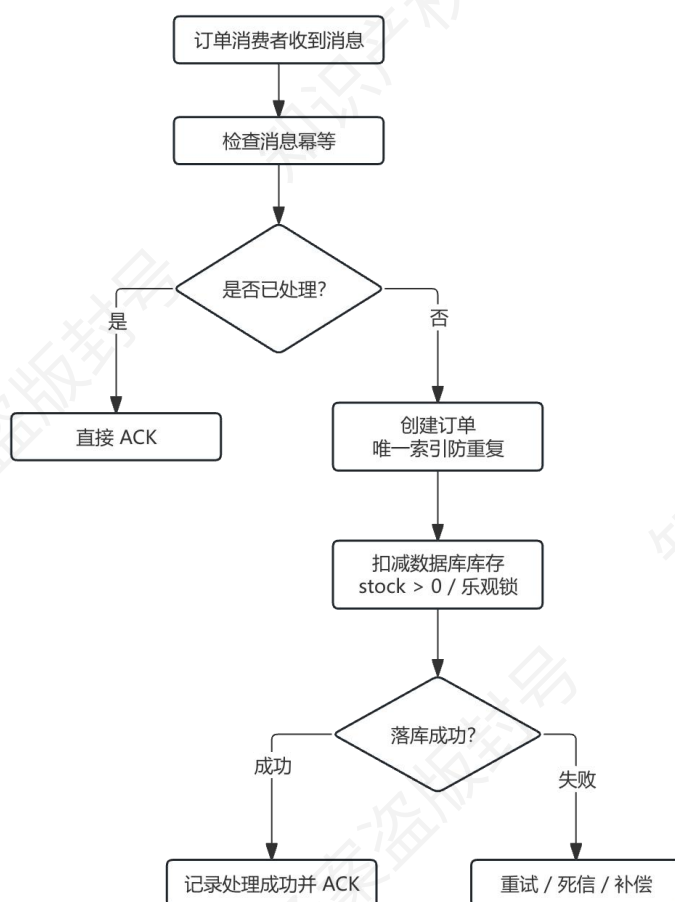
第一，订单表加唯一约束。比如使用 `user\_id + activity\_id + sku\_id` 作为唯一键，防止同一个用户对同一个秒杀商品重复下单。

第二，库存扣减要加锁防止超卖。比如更新库存时带上 `stock > 0` 条件，或者使用版本号做乐观锁，防止数据库层出现超卖。

第三，消息消费要做幂等。消费者可以记录 `message\_id` 或业务唯一键，避免同一条消息重复投递时重复创建订单。

第四，订单状态机要设计好。订单状态一般会经历待支付、已支付、已取消等状态，状态流转应该是单向且可控的，不能随意回退。

第五，数据量很大时可以考虑分库分表。比如订单表按用户、活动或时间维度拆分。查询压力较大时，也可以使用读写分离缓解查询压力。



无论是论文还有案例答题一提到高并发场景，有的同学都喜欢提到读写分离。这里需要注意的是，读写分离并不是核心环节。读写分离主要缓解查询压力，比如订单查询、活动信息查询、后台统计查询。核心环节仍然是前面提到的上游消息队列削峰、数据库约束、分库分表和异步消费来处理。

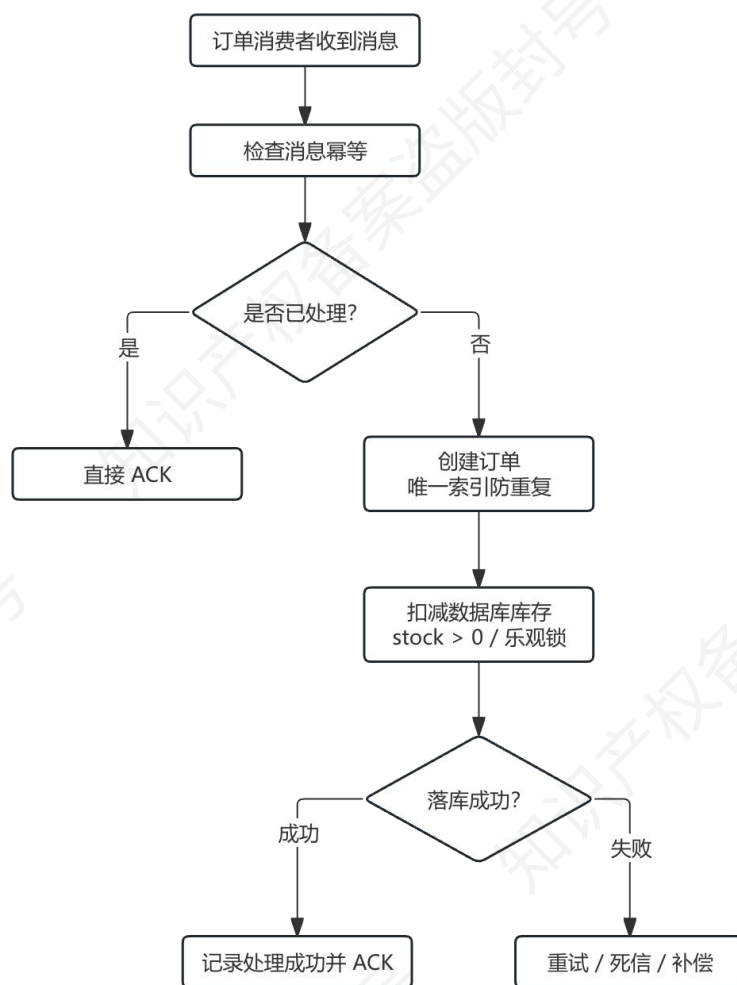
19.9.异常补偿与最终一致性（次重点★★★★☆）

秒杀系统不能只设计正常流程，还必须考虑异常场景，这个场景解决方案是偏复杂的，软考一般不做考察。常见异常包括：Redis 预扣成功但 MQ 发送失败，MQ 发送成功但消费者创建订单失败，订单创建成功但 ACK 失败导致消息重复投递，用户抢到资格但一直未支付，Redis 库存和数据库库存不一致等。这些问题通常要通过可靠消息、重试、死信队列、补偿任

务、库存回补和定时对账来处理。

前面我们也提到用本地消息表去解决分布式事务的问题，里面也提到了消息补偿。这是因为，本地消息表或事务消息主要解决“本地动作成功了，但消息没有可靠发出去”的问题。比如 Redis 已经预扣成功，或者业务状态已经变化，但 MQ 没有收到消息，这时系统不能假装没有发生，必须有重试和补偿入口。

在 MQ 的那一章节，我们提到了死信队列。消息进入死信，说明正常消费流程已经处理不了它，后面需要人工排查或补偿任务处理，比如回补库存、取消异常订单、补发消息、修正用户状态等。



19.10.用户体验设计（次重点★★★★☆）

为了减少后端压力，在抢购过程中更常见的做法是后端先返回给前端一个中间状态，比如“抢购成功，正在排队生成订单”。前端可以通过轮询、WebSocket、站内信等方式查询最终结果。常见状态包括：未开始、排队中、抢购成功待支付、已售罄、重复抢购、系统繁忙、超时未支付已取消等。排队体验本身也是系统保护的一部分。用户看到明确状态后，就不容易反复刷新和重复点击。用户请求减少了，后端压力自然也会下降。

20.下篇-知识图谱（次重点★★★★☆☆）

在前面的章节中，我们更多讨论的是数据库、大语言模型等技术。但如果从“知识组织”的角度来看，还有一个非常重要的概念，就是知识图谱。知识图谱并不是一个特别新的概念，它本质上是用图结构来表达现实世界中的实体、关系和属性。简单说，普通数据库更擅长存储一行一行的数据，而知识图谱更擅长表达“谁和谁有什么关系”。

比如在医疗领域，我们可以把“流行性感冒”“发热”“奥司他韦”“儿童”都看成实体，再用关系把它们连接起来：流行性感冒有症状发热，流行性感冒可以使用奥司他韦治疗，奥司他韦适用于某些病毒感染疾病。这样一来，系统就不只是保存了一堆文本，而是把医学知识整理成了可以查询、可以推理、可以解释的结构化网络。

在电商领域也是一样。商品、品牌、类目、属性、用户、订单、评价、使用场景都可以成为图谱中的实体。比如“iPhone 15”属于“手机”类目，品牌是“Apple”，颜色有“黑色”，适合“拍照”“商务”“学生”等场景，和“手机壳”“充电器”之间存在搭配关系。这样一张图谱建立起来以后，就可以支持商品搜索、智能推荐、客服问答、相似商品发现、搭配推荐和用户画像分析。知识图谱的核心组成如下表所示。

组成要素	具体说明	举例
实体	现实世界中的对象，可以是人、商品、疾病、药品、组织、地点、知识点等	流行性感冒、奥司他韦、iPhone 15、Apple、用户 A
关系	实体之间的连接，用来表达两个实体之间是什么关系	流感-有症状-发热，iPhone 15-属于-手机，用户 A-购买过-AirPods
属性	实体或关系上的补充信息，用来描述对象的具体特征	商品价格、颜色、药品剂量、疾病描述、用户年龄段
标签	对实体进行分类，方便后续查询和管理	疾病、药品、商品、品牌、用户、知识点
路径	多个实体和关系串联形成的推理链路	流感 → 可用药 → 奥司他韦 → 适用于 → 病毒感染

20.1.知识图谱的由来（次重点★★★★☆☆）

知识图谱并不是一个凭空出现的新概念。它的思想来源比较早，和语义网、本体、RDF、Linked Data 等技术都有关系。早期的语义网希望让互联网上的信息不只是给人看的网页，而是让机器也能理解其中的含义。比如“姚明”不是两个普通汉字，而是一个真实人物；“Apple”在不同语境下可能是水果，也可能是苹果公司。为了让计算机理解这些对象之间的语义关系，学术界提出了 RDF、本体、三元组等表达方式。

主体	关系	客体
流行性感冒	常见症状	发热
奥司他韦	可治疗	流感
iPhone 15	属于	手机类目
Apple	生产	MacBook

这种“主体—关系—客体”的结构，就是知识图谱最基本的表达方式。后来，DBpedia 等项目开始从 Wikipedia 这类百科数据中抽取结构化知识，把人物、地点、组织、作品、事件等信息整理成可查询、可链接的数据网络。

真正让“Knowledge Graph”这个词出圈的是 Google。2012 年，Google 正式推出 Knowledge Graph，也就是谷歌知识图谱。它当时要解决的核心问题是：搜索引擎不能只做关键词匹配，而要理解用户搜索词背后的真实事物。

举个例子，用户搜索“姚明”，传统搜索引擎会去网页中匹配“姚明”这个字符串，然后返回包含这个词的网页。而知识图谱要做的是进一步理解：姚明是一个人，是篮球运动员，出生于上海，曾效力于 NBA 火箭队，身高 2.26 米，还和中国篮球、NBA、休斯顿火箭等实体存在关系。所以，Google 当时提出的一个重要思想是：从 strings 到 things。也就是说，搜索引擎要从“理解字符串”走向“理解真实世界中的事物”。

20.2.知识图谱相比关系型数据库的优势（重点★★★★★）

关系型数据库非常重要，有它自己的使用场景，目前仍然不会被知识图谱替代。像用户表、订单表、支付表、库存表这些结构清晰、事务要求高的数据，仍然非常适合使用 MySQL、

PostgreSQL、Oracle 这类关系型数据库。

但是，当问题的重点不是“这一行数据是什么”，而是“这些数据之间有什么关系”时，知识图谱就更有优势。知识图谱和关系型数据库的区别如下表所示。

对比角度	关系型数据库	知识图谱
数据组织方式	以二维表为核心，数据按行和列存储	以节点和边为核心，数据按实体和关系组织
关系表达方式	主要通过外键、关联表和 JOIN 表达关系	关系直接作为图中的边存在
适合场景	订单、支付、库存、用户资料等结构化业务数据	搜索、推荐、问答、风控、知识推理、关系分析
查询特点	适合精确查询、事务处理、聚合统计	适合多跳查询、路径分析、关系发现
扩展方式	新增字段或表结构通常需要调整 Schema	可以逐步增加实体、关系和属性
可解释性	能查到数据，但关系链路不一定直观	可以返回实体之间的路径，解释答案来源
典型产品	MySQL、PostgreSQL、Oracle、SQL Server	Neo4j、JanusGraph、NebulaGraph、TigerGraph

第一，知识图谱更擅长表达复杂关系。关系型数据库当然也能表达关系，但通常需要通过外键和多表 JOIN 来实现。比如电商系统中，商品表要关联品牌表、类目表、属性表、评价表、用户表、订单表、优惠券表。如果关系越来越多，表结构会越来越复杂，查询 SQL 也会越来越长。知识图谱中，关系本身就是一等公民。系统看到的不再是一张张孤立的表，而是一张“商品—品牌—类目—用户—行为—场景”的关系网络。

第二，知识图谱更适合多跳查询。在传统 SQL 中，如果我们要查询“买过 iPhone 15 的用户还买过哪些配件”，可能需要订单表、订单明细表、商品表、用户表、类目表之间多次 JOIN。路径越长，SQL 越复杂。而在知识图谱中，这类问题可以理解为：从“iPhone 15”这个节点出发，沿着“被购买”“购买过”“属于”“可搭配”等关系往外走几步，看看能找到哪些相关节点。例如在 Neo4j 中，可以写成类似下面的 Cypher 查询（类似关系数据库的 SQL）：

```
MATCH (p:Product {name:'iPhone 15'})-[:可搭配|属于|品牌是|适合场景*1..2]-(x)
```

RETURN x

这段语句的意思是：从 iPhone 15 出发，沿着指定关系查找 1 到 2 跳范围内的相关节点。

第三，知识图谱更适合融合异构数据。现实系统里的数据来源往往很杂。医疗知识可能来自医学指南、论文、药品说明书、医院问诊记录、结构化数据库；电商知识可能来自商品库、评论、搜索日志、用户行为、客服问答和运营标签。关系型数据库通常要求先设计好表结构，字段也要相对稳定。知识图谱则可以用“实体 + 关系 + 属性”的方式逐步扩展。

第四，知识图谱更容易解释答案来源。现在大语言模型很流行，可以直接用它生成答案，但问题在于，它的答案有时不容易解释来源。知识图谱的优势是可以返回一条明确的关系路径。比如用户问：流感为什么可以使用奥司他韦？知识图谱可以给出类似路径：

流感 → 属于 → 病毒感染性疾病

奥司他韦 → 作用于 → 流感病毒

奥司他韦 → 可治疗 → 流感

这样答案就不只是“可以用”，而是可以说明它是根据哪些实体和关系推导出来的。

第五，知识图谱更适合做知识问答和推荐。传统问答系统往往依赖关键词匹配，比如用户问“感冒发烧吃什么药”，系统可能只是查找包含“感冒”“发烧”“药”的文本。知识图谱问答则可以先识别出问题中的实体和意图，再把问题转成图查询。所以，知识图谱特别适合处理关系密集型场景，比如智能客服、医疗问答、电商推荐、金融风控、用户画像、政务知识检索、教育知识点关联等。知识图谱的优势可以总结如下表所示。

优势	具体说明
关系表达能力强	实体之间的关系直接存储为边，适合表达复杂关系网络
多跳查询方便	可以从一个节点出发，沿着关系查找多层关联数据
适合知识推理	可以基于路径、规则、上下位关系进行一定程度的推理
扩展性较好	可以逐步新增实体、关系和属性，适合知识持续演化的场景
可解释性较强	可以返回答案背后的关系路径，方便解释“为什么得到这个答案”

适合异构数据融合	能把文本、结构化数据、日志、标签、百科知识等统一组织到一张图中
适合搜索推荐问答	天然适合实体检索、相似发现、推荐路径、智能问答等场景

20.3.知识图谱的基本属性与 Neo4j（重点★★★★★）

20.3.1.实体、关系、属性、标签和路径（重点★★★★★）

理解知识图谱，先要抓住几个基本概念：实体、关系、属性、标签和路径。

实体就是图中的节点。现实世界里可以被识别出来的对象，都可以成为实体。比如在医疗场景中，“糖尿病”“发热”“阿司匹林”“华法林”都是实体；在电商场景中，“手机”“iPhone 15”“Apple”“256GB”“快充”也都是实体。

关系就是实体之间的连接。比如“阿司匹林——治疗——血栓性疾病”，“阿司匹林——副作用——胃肠道出血”，“流行性感冒——症状——发热”，“iPhone 15——属于——手机类目”，“手机壳——适配——iPhone 15”。

知识图谱中常见的表达方式是三元组：

实体 A — 关系 — 实体 B

例如：

阿司匹林 — 治疗 — 血栓性疾病

阿司匹林 — 副作用 — 胃肠道出血

流行性感冒 — 典型症状 — 发热

iPhone 15 — 品牌 — Apple

手机壳 — 适配 — iPhone 15

属性是实体或关系上的补充信息。比如“药品”节点可以有名称、规格、生产厂家、适应症、禁忌症等属性；“商品”节点可以有价格、品牌、型号、颜色、材质、库存等属性。属性解决的

是“节点自身有什么特征”的问题，关系解决的是“节点之间有什么联系”的问题。

标签是对节点类型的分类。比如在 Neo4j 里，可以给节点打上 Disease、Symptom、Drug、Check、Food、Product、Brand、Category 等标签。标签的作用类似于关系数据库里的表名，可以帮助我们快速区分不同类型的实体。

路径则是图谱里非常重要的概念。关系数据库中我们经常通过 join 把多张表连起来，而图数据库可以天然地沿着关系边进行多跳查询。比如从“华法林”出发，可以查询它与哪些药物存在相互作用；从“糖尿病”出发，可以查到症状、检查项目、治疗药物、饮食禁忌、并发症等多类知识。路径查询是知识图谱区别于普通表结构的重要能力。

20.3.2.Neo4j（重点★★★★★）

在工程上，Neo4j 是最常见的图数据库之一。它使用节点、关系和属性来组织数据，并使用 Cypher 语言进行查询。Cypher 可以理解为图数据库里的 SQL，只不过 SQL 更像是“查表”，Cypher 更像是“找点、找边、找路径”。比如创建一个疾病和症状之间的关系，可以写成：

```
MERGE (d:Disease {name: '流行性感'})
```

```
MERGE (s:Symptom {name: '发热'})
```

```
MERGE (d)-[:HAS_SYMPTOM]->(s);
```

这里的 Disease 和 Symptom 是节点标签，name 是节点属性，HAS_SYMPTOM 是关系类型。MERGE 的含义是：如果节点或关系已经存在，就复用；如果不存在，就创建。相比 CREATE，MERGE 更适合知识图谱导入，因为真实数据中经常会重复出现同一个疾病、药品或症状。你可以看到，它这里的一些语法比 SQL 可能更好读，它中间的方括号表达的是一种关系，两边就是实体。

如果要查询“流行性感有哪些症状”，可以写成：

```
MATCH (d:Disease {name: '流行性感'})-[:HAS_SYMPTOM]->(s:Symptom)
```



```
RETURN s.name;
```

如果要查询“阿司匹林有哪些副作用”，可以写成：

```
MATCH (d:Drug {name: '阿司匹林'})-[:HAS_SIDE_EFFECT]->(s:SideEffect)
```

```
RETURN s.name;
```

如果要查询“某个商品属于什么类目、是什么品牌”，可以写成：

```
MATCH (p:Product {name: 'iPhone 15'})
```

```
OPTIONAL MATCH (p)-[:BELONGS_TO]->(c:Category)
```

```
OPTIONAL MATCH (p)-[:HAS_BRAND]->(b:Brand)
```

```
RETURN p.name, c.name, b.name;
```

如果要查多跳路径，比如“某个疾病和某种药品之间是否通过症状、治疗、禁忌等关系发生关联”，可以写成：

```
MATCH p = (a {name: '流行性感冒'})-[*1..3]-(b {name: '奥司他韦'})
```

```
RETURN p
```

```
LIMIT 10;
```

这里的 `[*1..3]` 表示查询 1 到 3 跳之间的关系路径。对于问答系统来说，多跳查询非常重要。因为用户问的问题往往不是简单的“查一个属性”，而是要在多个实体之间寻找关系链。

Neo4j 还可以建立索引、全文索引和向量索引。普通索引用于精确匹配，全文索引用于处理名称、别名、关键词等文本匹配，向量索引可以配合 `embedding` 模型进行语义相似度检索。比如用户输入“感冒发烧怎么办”，系统不一定能直接匹配到“流行性感冒”这个标准实体，这时候就需要实体对齐。实体对齐的思路是：先识别出用户问题中的候选实体，再通过别名词典、全文检索、向量相似度，把口语化表达映射到图谱中的标准实体。

比如“感冒”“流感”“小儿流感”“儿童流行性感冒”可能都要对齐到某个标准疾病节点。否则，图谱里虽然有知识，但用户一问，系统却查不到，这就是知识图谱问答中经常遇到的问题。

20.4.医药知识图谱问答实例（重点★★★★★）

在这个章节里，我主要是探讨一下知识图谱的在问答系统中的应用。虽然这个问答系统还

是比较粗浅的，但我们可以通过这个例子学习到知识图谱的构造过程，这也和我们芝士架构知识图谱构造的有一些相似之处。医药知识图谱问答系统的基本流程可以分成两大部分：一部分是知识图谱构建，另一部分是问答系统构建。

知识图谱构建阶段，系统需要从医学网站、医学百科、药品说明书、临床指南、医学教材、问答社区或结构化数据库中采集数据。数据来源有两类：一类是结构化或半结构化数据，比如疾病页面中已经有“症状、检查、药品、并发症、饮食禁忌”等字段；另一类是非结构化文本，比如一段医学描述、指南段落、药品说明书正文。结构化数据可以通过规则直接转成三元组，非结构化文本则需要通过自然语言处理模型做实体抽取和关系抽取。

问答系统构建阶段，系统要理解用户的问题，识别问题意图，抽取问题中的医学实体，再把问题转成对应的 **Cypher** 查询语句，到 **Neo4j** 中查询答案，最后把查询结果组织成自然语言返回给用户。这个和 **RAG** 是不一样的思路。

一个典型流程可以写成：

用户问题

↓

问题分类：判断用户问的是症状、药品、检查、禁忌、病因还是治疗方案

↓

实体识别：识别问题中的疾病、症状、药品等实体

↓

实体对齐：把口语化名称映射到图谱标准实体

↓

模板匹配：根据问题类型选择 **Cypher** 查询模板

↓

Neo4j 查询：查询相关节点、关系或路径

↓

答案生成：把查询结果组织成自然语言答案

这里面最关键的是实体抽取、关系抽取和问题模板。

20.4.1.使用 BERT 识别实体（次重点★★★★☆）

在传统医药知识图谱问答系统中，常见做法是使用 BERT 类模型进行命名实体识别。

BERT 的作用是把文本中的每个字或词转换成上下文相关的语义向量。比如“发热”在普通语境里可能只是一个词，但在医学语境中，它很可能是症状实体。具体模型训练过程这里省略，我们要知道的就是模型训练完成后，

输入一句话：小儿流行性感冒的病症有哪些？

模型输出：

疾病实体：小儿流行性感冒

问题意图：查询疾病症状

有的同学提到很多论文或系统里说“使用 BERT 进行训练”，准确理解应该是：通常不是从零开始预训练一个 BERT，而是在已有中文 BERT 或医学 BERT 的基础上做继续预训练或下游任务微调。真正从零预训练 BERT 需要海量语料和较大算力，普通项目一般不会这样做。

更常见的做法是：先准备大量医学文本，比如医学百科、药品说明书、临床指南、问答语料；然后使用已有的中文 BERT 作为基础模型，在医学语料上继续进行任务训练，让模型更熟悉医学表达；最后再用人工标注好的实体识别数据进行微调。实体识别训练数据通常长这样：

```
{
  "text": "小儿流行性感冒常见症状包括发热、咳嗽、头痛。",
  "labels": [
    {"start": 0, "end": 7, "type": "Disease", "text": "小儿流行性感冒"},
    {"start": 15, "end": 17, "type": "Symptom", "text": "发热"},
  ]
}
```

```
{ "start": 18, "end": 20, "type": "Symptom", "text": "咳嗽" },  
{ "start": 21, "end": 23, "type": "Symptom", "text": "头痛" }  
]  
}
```

训练过程可以简单理解为：

准备医学文本

↓

人工或半自动标注疾病、症状、药品、检查等实体

↓

转换成 BIO 标签

↓

输入 BERT

↓

BERT 输出上下文向量

↓

BiGRU 捕捉前后文

↓

CRF 输出最合理的标签序列

↓

得到实体识别模型

如果引入大模型，也可以让大模型先做预标注。比如把一段医学文本交给大模型，让它抽取疾病、症状、药品和检查项目，再由人工审核、修正，最后把修正后的结果作为训练数据，去训练一个更轻量、更稳定的 BERT 实体抽取模型。这样做的好处是减少人工从零标注的成本，但不能直接把大模型输出当成最终标准答案，因为大模型可能漏抽、误抽，所以要用到一些可视化的标注工具进行数据清洗。

20.4.2.使用 BERT 识别关系（次重点★★★★☆）

实体抽取解决的是“文本里有哪些实体”，关系抽取解决的是“这些实体之间是什么关系”。

比如句子：小儿流行性感冒常见症状包括发热、咳嗽、头痛，可使用奥司他韦进行治疗。

实体抽取可以识别出：

小儿流行性感冒：疾病

发热：症状

咳嗽：症状

头痛：症状

奥司他韦：药品

关系抽取要进一步得到：

小儿流行性感冒 — HAS_SYMPTOM — 发热

小儿流行性感冒 — HAS_SYMPTOM — 咳嗽

小儿流行性感冒 — HAS_SYMPTOM — 头痛

小儿流行性感冒 — RECOMMEND_DRUG — 奥司他韦

如果数据原本是半结构化的，比如页面字段已经写明：

```
{
  "疾病": "小儿流行性感冒",
  "症状": ["发热", "咳嗽", "头痛"],
  "推荐药品": ["奥司他韦"],
  "检查项目": ["血常规", "病毒抗原检测"]
}
```

那么关系抽取可以直接通过规则完成：

疾病字段 + 症状字段 → HAS_SYMPTOM

疾病字段 + 推荐药品字段 → RECOMMEND_DRUG

疾病字段 + 检查项目字段 → NEED_CHECK

但如果数据是非结构化文本，就需要训练关系分类模型。关系分类模型的输入通常是：原文、头实体、尾实体，输出是关系类型。例如：

```
{
  "text": "小儿流行性感冒常见症状包括发热、咳嗽、头痛。",
  "head": "小儿流行性感冒",
  "tail": "发热",
  "relation": "HAS_SYMPTOM"
}
```

再比如：

```
{
  "text": "华法林与阿司匹林合用可能增加出血风险。",
  "head": "华法林",
  "tail": "阿司匹林",
  "relation": "DRUG_INTERACTION"
}
```

关系抽取模型可以使用 BERT 做句子编码，然后把头实体和尾实体的位置标记出来，让模型判断它们之间的关系类型。训练数据中还要包含负样本，也就是两个实体虽然出现在同一句话中，但并不存在目标关系。否则模型很容易把所有共现实体都误判成有关联。

关系抽取的训练流程可以简化为：

输入医学句子

↓

识别句子中的实体

↓

枚举实体对

↓
构造训练样本：文本 + 头实体 + 尾实体
↓
BERT 编码
↓
分类器输出关系类型
↓
形成三元组
↓
导入 Neo4j

导入 Neo4j 时，可以使用 MERGE 语句，避免重复创建节点：

```
MERGE (d:Disease {name: $disease})  
MERGE (s:Symptom {name: $symptom})  
MERGE (d)-[:HAS_SYMPTOM]->(s);
```

或者：

```
MERGE (d:Drug {name: $drug1})  
MERGE (x:Drug {name: $drug2})  
MERGE (d)-[:DRUG_INTERACTION]->(x);
```

这样，一个面向医药问答的基础知识图谱就逐渐构建起来了。

20.4.3.医药问答例子：查询疾病症状（重点★★★★★）

假设用户问：小儿流行性感冒的病症有哪些？

系统首先进行问题分类，判断这是一个“疾病—症状”类问题。然后通过实体识别模型识别出疾病实体：疾病实体：小儿流行性感冒

接着进行实体对齐。如果图谱中的标准名称正好是“小儿流行性感冒”，就可以直接查询；如果图谱中叫“儿童流行性感冒”，系统就要通过别名表或相似度检索（记得吗前面说过 Neo4j 也是支持向量检索的）把它们对齐。

然后系统选择对应的 Cypher 模板：

```
MATCH (d:Disease {name: $disease})-[:HAS_SYMPTOM]->(s:Symptom)
RETURN s.name;
```

传入参数：

```
{
  "disease": "小儿流行性感冒"
}
```

Neo4j 返回：

发热、咳嗽、头痛、乏力、流涕

最后答案生成模块把查询结果组织成自然语言：

小儿流行性感冒常见的病症包括发热、咳嗽、头痛、乏力、流涕等。实际症状会因个体差异有所不同，如果症状持续或加重，应及时就医。

这里的关键点是：答案不是大模型凭空编出来的，而是由图谱查询结果支撑的。模板负责把自然语言问题转成 Cypher，Neo4j 负责查出事实，答案生成模块负责把事实组织成人能看懂的表达。

20.4.4.医药问答例子：查询药物相互作用（重点★★★★★）

再看一个药物关系问题。用户问：

哪些药物不能与华法林同时使用？

系统先做问题分类，判断这是一个“药物—相互作用”类问题。然后实体识别模型抽取：

药品实体：华法林

系统选择药物相互作用模板：

```
MATCH (d:Drug {name: $drug})-[:DRUG_INTERACTION]->(x:Drug)
RETURN x.name;
```

也可以考虑关系方向不固定，写成：

```
MATCH (d:Drug {name: $drug})-[:DRUG_INTERACTION]-(x:Drug)
RETURN DISTINCT x.name;
```

传入参数：

```
{  
  "drug": "华法林"  
}
```

Neo4j 返回若干药品名称后，系统生成回答：

图谱中与华法林存在相互作用风险的药物包括阿司匹林、氯吡格雷、部分非甾体抗炎药等。

这类药物可能增加出血风险，具体用药需要遵医嘱，不建议自行合用。

20.4.5.现实应用差距与落地问题（重点★★★★★）

知识图谱看起来很美，但真正落地时会遇到很多问题。

第一个问题是数据来源。论文或教学案例中，往往默认已经有大量干净的医学数据、商品数据或问答数据。但现实中，数据经常分散在不同系统里，字段不统一，格式不一致，还有大量缺失、重复、错误和过期内容。知识图谱不是把数据导入 Neo4j 就结束了，前面的数据治理才是最费力的部分。

第二个问题是实体标准化。比如“流感”“流行性感冒”“病毒性感冒”“小儿流感”可能指向相近或不同概念；电商中“苹果”“Apple”“苹果手机”“iPhone”也可能指向品牌、类目或商品。实体不对齐，图谱就会碎片化。同一个实体被建成多个节点，查询结果就不完整；不同实体被错误合并，答案就可能出错。

第三个问题是关系抽取质量。很多系统在演示时可以抽出“疾病—症状”“商品—品牌”这类简单关系，但现实文本中的关系往往更复杂。比如医学中的禁忌、相互作用、适应症、并发症都有严格语义，不能只根据两个实体共现就建立关系。电商评论中也存在反讽、比较、否定和上下文省略。如果模型理解不好，就可能抽错评价标签。

第四个问题是知识更新。医学知识、药品说明、商品价格、库存、优惠、评价都在变化。

知识图谱一旦过期，就会产生错误回答。尤其是医疗和电商场景，知识图谱不能只建一次，而要有持续更新、审核、版本管理和回滚机制。

第五个问题是模板覆盖不足。传统问答（非 RAG）很依赖问题分类和 Cypher 模板。模板越多，系统越能回答复杂问题，但维护成本也越高。模板越少，系统就只能回答固定句式的问题。大模型可以缓解这个问题，但不能完全消除，因为后端数据库查询仍然需要稳定、可控、可解释。

第六个问题是模型幻觉。引入大模型以后，系统表达能力变强了，但风险也变大了。大模型可能把图谱中没有的内容说得像真的一样，尤其在医疗场景中，这是非常危险的。所以比较稳妥的架构是：大模型负责理解问题、抽取意图、组织答案；图谱负责提供事实依据；后端负责校验查询范围和结果。不能把医学事实完全交给大模型自由生成。

第七个问题是效果评估。知识图谱系统在真正评估时，要看实体识别准确率、关系抽取准确率、实体对齐准确率、问题分类准确率、Cypher 生成正确率、答案命中率、用户满意度和响应速度。医疗场景还要看安全性、可解释性和合规性。电商场景还要看转化率、点击率、召回率和推荐准确性。

所以，从教学案例到现实应用，中间还有明显差距。芝士架构这里给出的案例更多是为了说明技术链路：数据采集、实体抽取、关系抽取、Neo4j 存储、Cypher 查询、答案生成。现实中比如说，关于我们的芝士架构知识图谱的实现，我还要考虑到以下几个方面：

1. 数据库与知识图谱的同步。
2. 实体歧义的消除以及同一实体的合并。
3. 是否需要训练一个 BERT 模型？训练效果怎么样？又该如何评估训练效果？
4. 大模型如何介入参与实体的抽取和关系的定义？
5. 关系之间是否需要给大模型定一套规则（无论是模式规则还是其他规则），让它在这

些关系中进行抽取？

6. 如果因为设计的关系不好导致不够全面，当大模型遇到新的关系时，没办法很好地归类，这种问题又该怎么解决？

所以，在工程上，要想让它变得好用，实际上需要考虑更多的事情。这是我想在这个小节最后给大家说的一些内容。

21.后记

这本《芝士架构案例冲刺宝典》，我把它定位成“考前最后一公里”的工具：帮你把散落在教材、网课、笔记和真题里的高频考点重新捋顺，把容易丢分的坑提前标出来，把需要快速回忆的内容尽量用表格和速记的方式压缩到你能够在短时间内反复翻阅、反复强化的程度。考点梳理、真题拆解、套路归纳、答题模板……这些地方我确实下了不少功夫，也真心希望它能在你最焦虑、最缺方向的那段时间里，给你一点确定感。

但我也必须，这份红宝书依然有很多不完美。虽然已逾十万字，却仍然不可能覆盖所有考点、更不可能替你“兜住”软考案例那种每年都会冒出来的超纲点与新考法。你会发现书里的真题解析更集中在系统架构、质量属性等常考方向，而对嵌入式等相对小众的案例涉及较少。这不是我不想写，而是我只能优先把“最容易考、最容易丢分、最值得冲刺阶段抓住”的部分做到尽量扎实。甚至可以这么说：即使你把这里的内容背得滚瓜烂熟，上了考场，你也依然可能遇到没见过的技术名词、没想到的设问角度，让你当场难受一下。这种“难受”，很真实，也很正常。

所以我更想把这本书当作你的底盘，而不是你的天花板。底盘稳了，你才有余力去应对变化：平时的积累、对系统思维的理解、对题干信息的抓取能力、以及临场表达能力，仍然是决定你能不能走出考场时松一口气的关键。说句掏心窝的话，我希望你不仅有背出这本红宝书的实力，也能有一点点“会什么考什么”的运气；更希望你遇到新东西时，能把题干读透，把问题拆开，把你会的往评分点上靠。

最后，真的感谢你选择相信我、选择使用这本书。如果你在使用过程中发现错漏、觉得某块讲得不清楚、或者希望我补充某类题型/某个方向的案例，欢迎随时来找我。微信 Deckardcain4，我都看得到，也愿意认真回复。